# AbstractAlgebra.jl

October 13, 2026

# Contents


**Contents** ii

**I AbstractAlgebra.jl** 1
**1 Introduction** 2
**2 Features** 3
**3 Installation** 4
**4 Quick start** 5

**II Fundamental interface of AbstractAlgebra.jl** 7
**5 Type interface of AbstractAlgebra.jl** 8
5.1 Why types aren't enough . . . 8
5.2 The abstract type hierarchy in AbstractAlgebra.jl . . . 9
5.3 More complex example of parent objects . . . 9
**6 Visualization of the types of AbstractAlgebra.jl** 10
6.1 Abstract parents . . . 10
6.2 Abstract elements . . . 10
6.3 Concrete types in AbstractAlgebra.jl . . . 11
**7 Extending the interface of AbstractAlgebra.jl** 13
7.1 Elements and parents . . . 13
7.2 Special elements . . . 16
7.3 Basic manipulation . . . 17

**III Constructing mathematical objects in AbstractAlgebra.jl** 20
**8 Constructing objects in Julia** 21
**9 How we construct objects in AbstractAlgebra.jl** 22
**10 Constructing parent objects** 23
**11 List of parent object constructors** 24
**12 Parent objects with variable names** 25

**IV Rings** 29
**13 Introduction** 30
**14 Ring functionality** 31
14.1 Abstract types for rings . . . 31
14.2 Functions for types and parents of rings . . . 31
14.3 Constructors . . . 33
14.4 Basic functions . . . 33
14.5 Basic functionality for inexact rings only . . . 34
14.6 Basic functionality for commutative rings only . . . 34
14.7 Basic functionality for noncommutative rings only . . . 34
14.8 Unsafe ring operators . . . 35

14.9 Random generation . . . 38
14.10 Factorization . . . 38
14.11 Square root . . . 40
14.12 Miscellaneous . . . 41
**15 Integer ring 43**
15.1 Types and parent objects . . . 43
15.2 Integer constructors . . . 43
15.3 Basic ring functionality . . . 44
15.4 Integer functionality provided by AbstractAlgebra.jl . . . 45
**16 Total ring of fractions 48**
16.1 Generic total ring of fraction types . . . 48
16.2 Abstract types . . . 48
16.3 Total ring of fractions constructors . . . 49
16.4 Fraction constructors . . . 49
16.5 Functions for types and parents of total rings of fractions . . . 50
16.6 Total ring of fractions functions . . . 51
**17 Univariate polynomial functionality 53**
17.1 Generic univariate polynomial types . . . 53
17.2 Abstract types . . . 53
17.3 Polynomial ring constructors . . . 53
17.4 Polynomial constructors . . . 54
17.5 Similar and zero . . . 56
17.6 Functions for types and parents of polynomial rings . . . 57
17.7 Euclidean polynomial rings . . . 57
17.8 Polynomial functions . . . 59
**18 Univariate polynomials over a noncommutative ring 87**
18.1 Generic type for univariate polynomials over a noncommutative ring . . . 87
18.2 Abstract types . . . 87
18.3 Polynomial ring constructors . . . 87
18.4 Basic ring functionality . . . 89
18.5 Polynomial functionality provided by AbstractAlgebra.jl . . . 90
**19 Multivariate polynomials 99**
19.1 Abstract types . . . 99
19.2 Polynomial ring constructors . . . 99
19.3 Polynomial constructors . . . 103
19.4 Functions for types and parents of multivariate polynomial rings . . . 105
19.5 Polynomial functions . . . 105
19.6 Random generation . . . 125
**20 Universal ring 127**
20.1 Universal polynomial ring . . . 127
20.2 Constructors . . . 128
20.3 Adding variables . . . 130
20.4 Implement universal rings . . . 130
**21 Generic Laurent polynomials 132**
21.1 Generic Laurent polynomial types . . . 132
21.2 Abstract types . . . 132
21.3 Laurent polynomials ring constructor . . . 132
21.4 Basic functionality . . . 133
**22 Sparse distributed multivariate Laurent polynomials 135**
22.1 Generic multivariate Laurent polynomial types . . . 135
22.2 Abstract types . . . 135
22.3 Multivate Laurent polynomial operations . . . 135

**23 Multivariate Puiseux polynomials 138**
23.1 Generic Puiseux polynomial types . . . 138
23.2 Constructor . . . 138
23.3 Basic functionality . . . 139
23.4 Powering . . . 141
23.5 Divisibility . . . 141
23.6 Changing the coefficient ring . . . 142
23.7 Valuation . . . 142
**24 Power series 143**
24.1 Generic power series types . . . 143
24.2 Abstract types . . . 143
24.3 Series ring constructors . . . 144
24.4 Power series constructors . . . 146
24.5 Big-oh notation . . . 148
24.6 Power series models . . . 149
24.7 Functions for types and parents of series rings . . . 150
24.8 Series functions . . . 150
**25 Generic Puiseux series 163**
25.1 Types and parent objects . . . 163
25.2 Puiseux series ring constructors . . . 163
25.3 Big-oh notation . . . 164
25.4 Puiseux series implementation . . . 165
25.5 Basic ring functionality . . . 165
25.6 Puiseux series functionality provided by AbstractAlgebra.jl . . . 166
25.7 Derivative and integral . . . 169
**26 Multivariate series 173**
26.1 Generic multivariate series . . . 173
26.2 Abstract types . . . 173
26.3 Multivariate series ring constructors . . . 174
26.4 Basic ring functionality . . . 175
26.5 Power series functionality provided by AbstractAlgebra.jl . . . 177
**27 Generic residue rings 181**
27.1 Generic residue types . . . 181
27.2 Abstract types . . . 181
27.3 Residue ring constructors . . . 181
27.4 Residue constructors . . . 183
27.5 Functions for types and parents of residue rings . . . 183
27.6 Residue ring functions . . . 183
**28 Free associative algebras 187**
28.1 Generic free associative algebra types . . . 187
28.2 Free associative algebra constructors . . . 187
28.3 Free associative algebra element constructors . . . 188
28.4 Element functions . . . 189

**V Fields 195**
**29 Introduction 196**
**30 Field functionality 197**
30.1 Abstract types for rings . . . 197
30.2 Functions for types and parents of fields . . . 197
30.3 Basic functions . . . 197

**31 Generic fraction fields 198**
31.1 Generic fraction types . . . 198
31.2 Factored fraction types . . . 198
31.3 Abstract types . . . 198
31.4 Fraction field constructors . . . 198
31.5 Factored Fraction field constructors . . . 199
31.6 Fraction constructors . . . 200
31.7 Functions for types and parents of fraction fields . . . 200
31.8 Fraction field functions . . . 201
**32 Rational field 206**
32.1 Types and parent objects . . . 206
32.2 Rational constructors . . . 206
32.3 Basic field functionality . . . 207
32.4 Rational functionality provided by AbstractAlgebra.jl . . . 208
**33 Rational function fields 209**
33.1 Generic rational function field type . . . 209
33.2 Abstract types . . . 209
33.3 Rational function field constructors . . . 209
33.4 Basic rational function field functionality . . . 210
33.5 Rational function field functionality provided by AbstractAlgebra.jl . . . 211
**34 Univariate function fields 213**
34.1 Generic function field types . . . 213
34.2 Abstract types . . . 213
34.3 Function field constructors . . . 213
34.4 Basic function field functionality . . . 214
34.5 Function field functionality provided by AbstractAlgebra.jl . . . 215
**35 Finite fields 219**
35.1 Types and parent objects . . . 219
35.2 Finite field constructors . . . 219
35.3 Basic field functionality . . . 220
35.4 Basic manipulation of fields and elements . . . 220
**36 Real field 222**
36.1 Types and parent objects . . . 222
36.2 Rational constructors . . . 222
36.3 Basic field functionality . . . 223

**VI Groups 225**
**37 Permutations and Symmetric groups 226**
37.1 Permutations constructors . . . 227
37.2 Permutation interface . . . 229
37.3 Basic manipulation . . . 230
37.4 Arithmetic operators . . . 233
37.5 Coercion . . . 235
37.6 Comparison . . . 235
37.7 Misc . . . 236
**38 Partitions and Young tableaux 239**
38.1 Partitions . . . 239
38.2 Young Diagrams and Young Tableaux . . . 241
38.3 Characters of permutation groups . . . 247
38.4 Skew Diagrams . . . 252

**VII Modules** **256**
**39 Introduction** **257**
**40 Finitely presented modules** **258**
40.1 Abstract types . . . 258
40.2 Module functions . . . 258
**41 Free Modules and Vector Spaces** **265**
41.1 Generic free module and vector space types . . . 265
41.2 Abstract types . . . 265
41.3 Functionality for free modules . . . 265
**42 Submodules** **268**
42.1 Generic submodule type . . . 268
42.2 Abstract types . . . 268
42.3 Constructors . . . 268
42.4 Functionality for submodules . . . 269
**43 Quotient modules** **272**
43.1 Generic quotient module type . . . 272
43.2 Abstract types . . . 272
43.3 Constructors . . . 272
43.4 Functionality for submodules . . . 273
**44 Direct Sums** **275**
44.1 Generic direct sum type . . . 275
44.2 Abstract types . . . 275
44.3 Constructors . . . 275
44.4 Functionality for direct sums . . . 276
**45 Module Homomorphisms** **279**
45.1 Generic module homomorphism types . . . 279
45.2 Abstract types . . . 279
45.3 Generic functionality . . . 279

**VIII Ideals** **283**
**46 Ideal functionality** **284**
46.1 Generic ideal types . . . 284
46.2 Abstract types . . . 284
46.3 Ideal constructors . . . 284
46.4 Ideal functions . . . 285

**IX Matrices** **288**
**47 Introduction** **289**
**48 Working with matrices** **290**
48.1 Constructing Matrices . . . 290
48.2 Manipulating matrices . . . 302
48.3 Matrix predicates . . . 311
48.4 Matrix properties . . . 321
48.5 Matrix normal forms . . . 330
48.6 Solving Linear Systems . . . 338
**49 Matrix spaces** **341**
49.1 Creating matrix spaces . . . 341
49.2 Properties of matrix spaces . . . 341
49.3 Creating elements of a matrix space . . . 342
**50 Matrix algebras** **345**
50.1 Creating matrix algebras . . . 345

50.2 Properties of matrix algebras . . . 345
50.3 Creating elements of a matrix algebra . . . 346
50.4 Properties of matrix algebra elements . . . 348
50.5 Converting matrix algebra elements to matrices . . . 349
**51 Developer documentation 350**
51.1 Matrix implementation . . . 350

**X Maps 352**
**52 Introduction 353**
**53 Functional maps 354**
53.1 Functional map interface . . . 354
53.2 Generic functional maps . . . 355
**54 Cached maps 356**
54.1 Cached map constructors . . . 356
54.2 Functionality for cached maps . . . 357
**55 Map with inverse 359**
55.1 Map with inverse constructors . . . 359
55.2 Functionality for maps with inverses . . . 360

**XI Miscellaneous 362**
**56 Miscellaneous 363**
56.1 Printing options . . . 363
56.2 Updating the type diagrams . . . 363
56.3 Attributes . . . 363
56.4 Advanced printing . . . 368
56.5 Linear solving interface for developers . . . 375
56.6 Miscellaneous . . . 378
**57 Assertion and Verbosity Macros 379**
57.1 Verbosity macros . . . 379
57.2 Assertion macros . . . 383
57.3 Miscellaneous . . . 385

**XII Interfaces 387**
**58 Introduction 388**
**59 Ring Interface 389**
59.1 Types . . . 389
59.2 RingElement type union . . . 390
59.3 Parent object caches . . . 390
59.4 Required functions for all rings . . . 390
59.5 Required functionality for inexact rings . . . 399
59.6 Optional functionality . . . 399
59.7 Minimal example of ring implementation . . . 401
**60 Euclidean Ring Interface 408**
**61 Univariate Polynomial Ring Interface 414**
61.1 Types and parents . . . 414
61.2 Required functionality for univariate polynomials . . . 415
61.3 Optional functionality for polynomial rings . . . 418
**62 Multivariate Polynomial Ring Interface 420**
62.1 Types and parents . . . 420
62.2 Required functionality for multivariate polynomials . . . 422
62.3 Interface for sparse distributed, random access multivariates . . . 427

62.4 Optional functionality for multivariate polynomials . . . 429
**63 Series Ring Interface 431**
63.1 Types and parents . . . 431
63.2 Required functionality for series . . . 432
63.3 Optional functionality for series . . . 435
**64 Residue Ring Interface 437**
64.1 Types and parents . . . 437
64.2 Required functionality for residue rings . . . 438
**65 Field Interface 439**
65.1 Types . . . 439
65.2 FieldElement type union . . . 439
65.3 Parent object caches . . . 440
65.4 Required functions for all fields . . . 440
**66 Fraction Field Interface 442**
66.1 Types and parents . . . 442
66.2 Required functionality for fraction fields . . . 442
**67 Module Interface 444**
67.1 Abstract types . . . 444
67.2 Required functionality for modules . . . 444
**68 Ideal Interface 447**
68.1 Types and parents . . . 447
68.2 Required functionality for ideals . . . 447
68.3 Optional functionality for ideals . . . 448
**69 Matrix interface 450**
69.1 Types and parents . . . 450
69.2 Required functionality . . . 451
69.3 Dense matrix type selection . . . 452
69.4 Optional functionality . . . 452
**70 Map Interface 454**
70.1 Parent objects . . . 454
70.2 Map classes . . . 454
70.3 Implementing new map types . . . 455
70.4 Required functionality for maps . . . 455
70.5 Optional functionality for maps . . . 456
**71 Random interface 460**
**72 Linear solving interface 463**
72.1 Overview of the functionality . . . 463
72.2 Solving with several right hand sides . . . 464
72.3 Detailed documentation . . . 464

# Part I

# AbstractAlgebra.jl

## Chapter 1

# Introduction

AbstractAlgebra.jl is a computer algebra package for the Julia programming language, maintained by William Hart, Tommy Hofmann, Claus Fieker and Fredrik Johansson and other interested contributors.

- Source code

AbstractAlgebra.jl grew out of the Nemo project after a number of requests from the community for the pure Julia part of Nemo to be split off into a separate project. See the Nemo repository for more details about Nemo.

- Nemo repository

# Chapter 2

# Features

The features of AbstractAlgebra.jl include:

- Use of Julia multiprecision integers and rationals
- Finite fields (prime order, naive implementation only)
- Number fields (naive implementation only)
- Univariate polynomials
- Multivariate polynomials
- Relative and absolute power series
- Laurent series
- Fraction fields
- Residue rings, including $\mathbb{Z}/n\mathbb{Z}$
- Matrices and linear algebra

All implementations are fully recursive and generic, so that one can build matrices over polynomial rings, over a finite field, for example.

AbstractAlgebra.jl also provides a set of abstract types for Groups, Rings, Fields, Modules and elements thereof, which allow external types to be made part of the AbstractAlgebra.jl type hierarchy.

# Chapter 3

# Installation

To use AbstractAlgebra we require Julia 1.10 or higher. Please see https://julialang.org/downloads/ for instructions on how to obtain Julia for your system.

At the Julia prompt simply type

```
julia> using Pkg; Pkg.add("AbstractAlgebra")
```

# Chapter 4

# Quick start

Here are some examples of using AbstractAlgebra.jl.

This example makes use of multivariate polynomials.

```
using AbstractAlgebra

R, (x, y, z) = polynomial_ring(ZZ, [:x, :y, :z])

f = x + y + z + 1

p = f^20;

@time q = p*(p+1);
```

Here is an example using generic recursive ring constructions.

```
using AbstractAlgebra

R = GF(7)

S, y = polynomial_ring(R, :y)

T, = residue_ring(S, y^3 + 3y + 1)

U, z = polynomial_ring(T, :z)

f = (3y^2 + y + 2)*z^2 + (2*y^2 + 1)*z + 4y + 3;

g = (7y^2 - y + 7)*z^2 + (3y^2 + 1)*z + 2y + 1;

s = f^4;

t = (s + g)^4;

@time resultant(s, t)
```

Here is an example using matrices.

```
using AbstractAlgebra

R, x = polynomial_ring(ZZ, :x)

S = matrix_space(R, 10, 10)

M = rand(S, 0:3, -10:10);

@time det(M)
```

And here is an example with power series.

```
using AbstractAlgebra

R, x = QQ[:x]

S, t = power_series_ring(R, 30, :t)

u = t + O(t^100)

@time divexact((u*exp(x*u)), (exp(u)-1));
```

# Part II

# Fundamental interface of AbstractAlgebra.jl

# Chapter 5

# Type interface of AbstractAlgebra.jl

Apart from how we usually think of types in programming, we shall in this section discuss why we do not use the typical type interface.

## 5.1 Why types aren't enough

Naively, one might have expected that structures like rings in AbstractAlgebra.jl could be modeled as types and their elements as objects with the given type. But there are various reasons why this is not a good model.

Consider the ring $R = \mathbb{Z}/n\mathbb{Z}$ for a multiprecision integer $n$. If we were to model the ring $R$ as a type, then the type would somehow need to contain the modulus $n$. This is not possible in Julia, and in fact it is not desirable, since the compiler would then recompile all the associated functions every time a different modulus $n$ was used.

We could attach the modulus $n$ to the objects representing elements of the ring, rather than their type.

But now we cannot create new elements of the ring $\mathbb{Z}/n\mathbb{Z}$ given only their type, since the type no longer contains the modulus $n$.

Instead, the way we get around this in AbstractAlgebra.jl is to have special (singleton) objects that act like types, but are really just ordinary Julia objects. These objects, called *parent* objects, can contain extra information, such as the modulus $n$. In return, we associate this parent object with so called *element* objects.

In order to create new elements of $\mathbb{Z}/n\mathbb{Z}$ as above, we overload the `call` operator for the parent object.

In the following AbstractAlgebra.jl example, we create the parent object R corresponding to the ring $\mathbb{Z}/7\mathbb{Z}$. We then create a new element a of this ring by calling the parent object R.

```
R, = residue_ring(ZZ, 7)
a = R(3)
```

Here, R is the parent object, containing the modulus $7$. So this example creates the element $a = 3 \pmod 7$.

Objects known as parents which contain additional information about groups, rings, fields and modules, etc., that can't be stored in types alone.

These details are technical and can be skipped or skimmed by new users of Julia/AbstractAlgebra.jl. Types are almost never dealt with directly when scripting AbstractAlgebra.jl to do mathematical computations.

In contrast, AbstractAlgebra.jl developers will want to know how we model mathematical objects and their rings, fields, groups, etc.

## 5.2 The abstract type hierarchy in AbstractAlgebra.jl

In AbstractAlgebra.jl, we use the abstract type hierarchy in order to give structure when programming the mathematical structures. For example, abstract types in Julia can belong to one another in a hierarchy.

For example, the `Field` abstract type belongs to the `Ring` abstract type. The full hierarchy can be seen in diagrams under the section on visualisation of the abstract types.

In practice this is practical since it means that any generic function designed to work with ring objects will also work with field objects.

In AbstractAlgebra.jl we also distinguish between the elements of a field, say, and the field itself.

For example, we have an object of type `Generic.PolyRing` to model a generic polynomial ring, and elements of that polynomial ring would have type `Generic.PolyRingElem`.

For this purpose, we also have a hierarchy of abstract types, such as `FieldElem`, that the types of element objects can belong to.

## 5.3 More complex example of parent objects

Here is some code which constructs a polynomial ring over the integers, a polynomial in that ring and then does some introspection to illustrate the various relations between the objects and types.

```
julia> using AbstractAlgebra

julia> R, x = ZZ[:x]
(Univariate polynomial ring in x over integers, x)

julia> f = x^2 + 3x + 1
x^2 + 3*x + 1

julia> R isa PolyRing
true

julia> f isa PolyRingElem
true

julia> parent(f) == R
true
```

# Chapter 6

# Visualization of the types of AbstractAlgebra.jl

AbstractAlgebra.jl implements a couple of abstract types which can be extended.

## 6.1 Abstract parents

The following diagram shows a complete list of all abstract types in AbstractAlgebra.jl.

## 6.2 Abstract elements

Similarly the following diagram shows a complete list of all abstract types in AbstractAlgebra.jl.

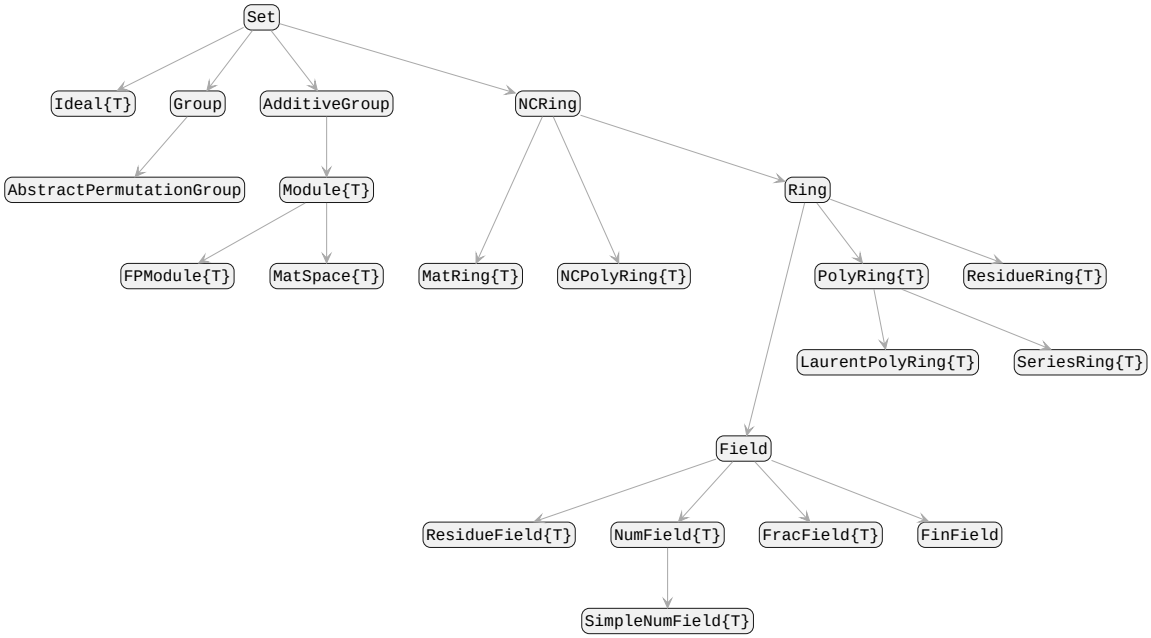



Figure 6.1: Diagram of parent types

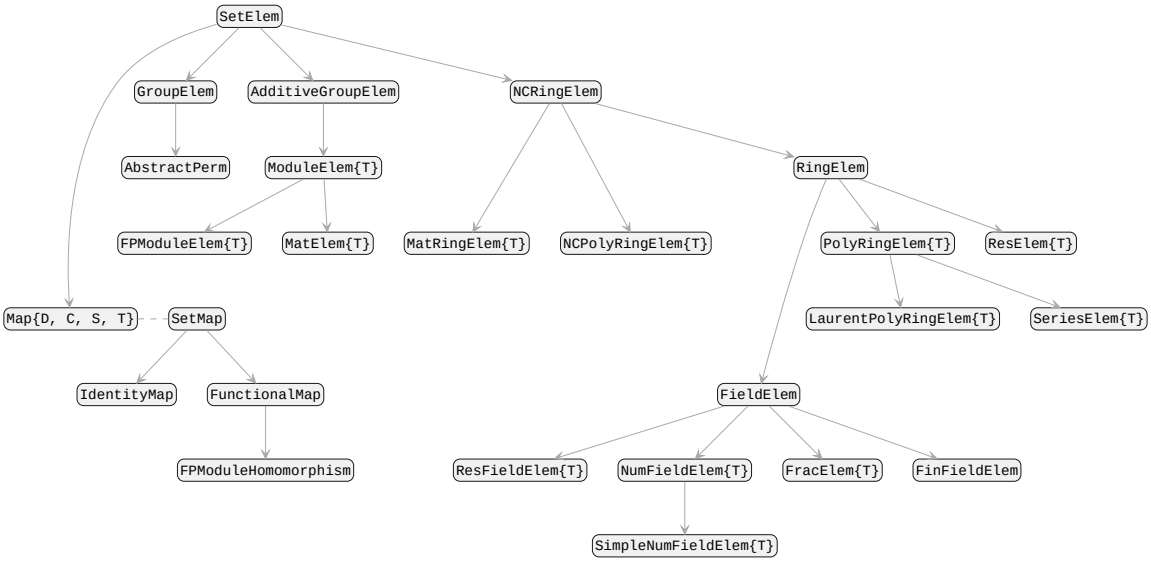



Figure 6.2: Diagram of element types

## 6.3 Concrete types in AbstractAlgebra.jl

Until now we have discussed the abstract types of AbstractAlgebra.jl. Under this subsection we will instead give some examples of concrete types in AbstractAlgebra.jl.

In parentheses we put the types of the corresponding parent objects.

- `Perm{<:Integer}` (`SymmetricGroup{<:Integer}`)
- `GFElem{<:Integer}` (`GFField{<:Integer}`)

We also think of various Julia types as though they were AbstractAlgebra.jl types:

- `BigInt` (`Integers{BigInt}`)
- `Rational{BigInt}` (`Rationals{BigInt}`)

Then there are various types for generic constructions over a base ring. They are all parameterised by a type T which is the type of the *elements* of the base ring they are defined over.

- `Generic.Poly{T}` (`Generic.PolyRing{T}`)
- `Generic.MPoly{T}` (`Generic.MPolyRing{T}`)
- `Generic.RelSeries{T}` (`Generic.RelPowerSeriesRing{T}`)
- `Generic.AbsSeries{T}` (`Generic.AbsPowerSeriesRing{T}`)
- `Generic.LaurentSeriesRingElem{T}` (`Generic.LaurentSeriesRing{T}`)
- `Generic.LaurentSeriesFieldElem{T}` (`Generic.LaurentSeriesField{T}`)

- `Generic.EuclideanRingResidueRingElem{T}` (`Generic.EuclideanRingResidueRing{T}`)
- `Generic.FracFieldElem{T}` (`Generic.FracField{T}`)
- `Generic.Mat{T}` (`MatSpace{T}`)

# Chapter 7

# Extending the interface of AbstractAlgebra.jl

In this section we will discuss on how to extend the interface of AbstractAlgebra.jl.

## 7.1 Elements and parents

Any implementation with elements and parents should implement the following interface:

`Base.parent` – Function.

```
parent(a)
```

Return parent object of given element $a$.

**Examples**

```
julia> G = SymmetricGroup(5); g = Perm([3,4,5,2,1])
(1,3,5)(2,4)

julia> parent(g) == G
true

julia> S, x = laurent_series_ring(ZZ, 3, :x)
(Laurent series ring in x over integers, x + O(x^4))

julia> parent(x) == S
true
```

source

`AbstractAlgebra.elem_type` – Function.

```
elem_type(parent)
elem_type(parent_type)
```

Given a parent object (or its type), return the type of its elements.

**Examples**

```
julia> S, x = power_series_ring(QQ, 2, :x)
(Univariate power series ring over rationals, x + O(x^3))

julia> elem_type(S) == typeof(x)
true
```

source

AbstractAlgebra.parent_type – Function.

```
parent_type(element)
parent_type(element_type)
```

Given an element (or its type), return the type of its parent object.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S = matrix_space(R, 2, 2)
Matrix space of 2 rows and 2 columns
  over univariate polynomial ring in x over integers

julia> a = rand(S, 0:1, 0:1);

julia> parent_type(a) == typeof(S)
true
```

source

### Acquiring associated elements and parents

Further, if one has a base ring, like polynomials over the integers $\mathbb{Z}[x]$, then one should implement

AbstractAlgebra.base_ring – Function.

```
base_ring(a)
```

Return the internal base ring of the given element or parent $a$.

**Examples**

```
julia> S, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> base_ring(S) == QQ
true

julia> R = GF(7)
```

```
Finite field F_7

julia> base_ring(R)
ERROR: MethodError: no method matching base_ring(::AbstractAlgebra.GFField{Int64})
```

source

AbstractAlgebra.base_ring_type – Function.

```
base_ring_type(a)
```

Return the type of the internal base ring of the given element, element type, parent or parent type $a$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> base_ring_type(R) == typeof(base_ring(R))
true

julia> base_ring_type(zero(R)) == typeof(base_ring(zero(R)))
true

julia> base_ring_type(typeof(R)) == typeof(base_ring(R))
true

julia> base_ring_type(typeof(zero(R))) == typeof(base_ring(zero(R)))
true

julia> R = GF(7)
Finite field F_7

julia> base_ring_type(R)
Union{}
```

source

If there is a well-defined notion of a coefficient ring (e.g. in the case of polynomial rings or modules), then one should implement

AbstractAlgebra.coefficient_ring – Function.

```
coefficient_ring(a)
```

Return the coefficient ring of the given element or parent $a$.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> coefficient_ring(x^2+1) == QQ
true

julia> S, (z,w) = universal_polynomial_ring(QQ, [:z,:w])
(Universal polynomial ring over Rationals,
↪  UniversalRingElem{AbstractAlgebra.Generic.MPoly{Rational{BigInt}}, Rational{BigInt}}[z, w])

julia> coefficient_ring(S) == QQ
true
```

source

AbstractAlgebra.coefficient_ring_type – Function.

```
coefficient_ring_type(a)
```

Return the type of the coefficient ring of the given element, element type, parent or parent type $a$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> coefficient_ring_type(R) == typeof(coefficient_ring(R))
true

julia> coefficient_ring_type(zero(R)) == typeof(coefficient_ring(zero(R)))
true

julia> coefficient_ring_type(typeof(R)) == typeof(coefficient_ring(R))
true

julia> coefficient_ring_type(typeof(zero(R))) == typeof(coefficient_ring(zero(R)))
true
```

source

## 7.2 Special elements

For rings, one has to extend the following methods:

Base.one – Function.

```
one(a)
```

Return the multiplicative identity in the algebraic structure of $a$, which can be either an element or parent.

**Examples**

```
julia> S = matrix_space(ZZ, 2, 2)
Matrix space of 2 rows and 2 columns
  over integers

julia> one(S)
[1   0]
[0   1]

julia> R, x = puiseux_series_field(QQ, 4, :x)
(Puiseux series field in x over rationals, x + O(x^5))

julia> one(x)
1 + O(x^4)

julia> G = GF(5)
Finite field F_5

julia> one(G)
1
```

source

Base.zero – Function.

```
zero(a)
```

Return the additive identity in the algebraic structure of $a$, which can be either an element or parent.

**Examples**

```
julia> S = matrix_ring(QQ, 2)
Matrix ring of degree 2
  over rationals

julia> zero(S)
[0//1   0//1]
[0//1   0//1]

julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> zero(x^3 + 2)
0
```

source

Groups should only extend at least one of these. The one that is required depends on if the group is additive (commutative) or multiplicative.

## 7.3 Basic manipulation

If one would like to implement a ring, these are the basic manipulation methods that all rings should extend:

Base.isone – Function.

```
isone(a)
```

Return true if $a$ is the multiplicative identity, else return false.

**Examples**

```
julia> S = matrix_space(ZZ, 2, 2); T = matrix_space(ZZ, 2, 3); U = matrix_space(ZZ, 3, 2);

julia> isone(S([1 0; 0 1]))
true

julia> isone(T([1 0 0; 0 1 0]))
false

julia> isone(U([1 0; 0 1; 0 0]))
false

julia> T, x = puiseux_series_field(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> isone(x), isone(T(1))
(false, true)
```

source

Base.iszero – Function.

```
iszero(a)
```

Return true if $a$ is the additive identity, else return false.

**Examples**

```
julia> T, x = puiseux_series_field(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> a = T(0)
O(x^10)

julia> iszero(a)
true
```

source

AbstractAlgebra.is_unit – Function.

```
is_unit(a::T) where {T <: NCRingElement}
```

Return true if $a$ is invertible, else return false.

**Examples**

```
julia> S, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> is_unit(x), is_unit(S(1)), is_unit(S(4))
(false, true, true)

julia> is_unit(ZZ(-1)), is_unit(ZZ(4))
(true, false)
```

source

With the same logic as earlier, groups only need to extend one of the methods `isone` and `iszero`.

# Part III

# Constructing mathematical objects in AbstractAlgebra.jl

# Chapter 8

# Constructing objects in Julia

In Julia, one constructs objects of a given type by calling a type constructor. This is simply a function with the same name as the type itself. For example, to construct a `BigInt` object from an `Int` in Julia, we simply call the `BigInt` constructor:

```
n = BigInt(123)
```

Note that a number literal too big to fit in an `Int` or `Int128` automatically creates a `BigInt`:

```
julia> typeof(12345678765456787654567890987654567898765678909876567890)
BigInt
```

## Chapter 9

# How we construct objects in AbstractAlgebra.jl

As we explain in Elements and parents, Julia types don't contain enough information to properly model groups, rings, fields, etc. Instead of using types to construct objects, we use special objects that we refer to as parent objects. They behave a lot like Julia types.

Consider the following simple example, to create a multiprecision integer:

```
n = ZZ(12345678765456787654567890987654567898765678909876567890)
```

Here ZZ is not a Julia type, but a callable object. However, for most purposes one can think of such a parent object as though it were a type.

# Chapter 10

# Constructing parent objects

For more complicated groups, rings, fields, etc., one first needs to construct the parent object before one can use it to construct element objects.

AbstractAlgebra.jl provides a set of functions for constructing such parent objects. For example, to create a parent object for univariate polynomials over the integers, we use the `polynomial_ring` parent object constructor.

```
R, x = polynomial_ring(ZZ, :x)
f = x^3 + 3x + 1
g = R(12)
```

In this example, R is the parent object and we use it to convert the `Int` value $12$ to an element of the polynomial ring $\mathbb{Z}[x]$.

# Chapter 11

# List of parent object constructors

For convenience, we provide a list of all the parent object constructors in AbstractAlgebra.jl and explain what mathematical domains they represent.

| Mathematics | AbstractAlgebra.jl constructor |
|---|---|
| $R = \mathbb{Z}$ | `R = ZZ` |
| $R = \mathbb{Q}$ | `R = QQ` |
| $R = \mathbb{F}_p$ | `R = GF(p)` |
| $R = \mathbb{Z}/n\mathbb{Z}$ | `R, = residue_ring(ZZ, n)` |
| $S = R[x]$ | `S, x = polynomial_ring(R, :x)` |
| $S = R[x, y]$ | `S, (x, y) = polynomial_ring(R, [:x, :y])` |
| $S = R\langle x, y\rangle$ | `S, (x, y) = free_associative_algebra(R, [:x, :y])` |
| $S = K(x)$ | `S, x = rational_function_field(K, :x)` |
| $S = K(x, y)$ | `S, (x, y) = rational_function_field(K, [:x, :y])` |
| $S = R[[x]]$ (to precision $n$) | `S, x = power_series_ring(R, n, :x)` |
| $S = R[[x, y]]$ (to precision $n$) | `S, (x, y) = power_series_ring(R, n, [:x, :y])` |
| $S = R((x))$ (to precision $n$) | `S, x = laurent_series_ring(R, n, :x)` |
| $S = K((x))$ (to precision $n$) | `S, x = laurent_series_field(K, n, :x)` |
| $S = R((x, y))$ (to precision $n$) | `S, (x, y) = laurent_polynomial_ring(R, n, [:x, :y])` |
| Puiseux polynomials in $x, y$ over $K$ | `S, (x, y) = puiseux_polynomial_ring(K, ["x", "y"])` |
| Puiseux series ring to precision $n$ | `S, x = puiseux_series_ring(R, n, :x)` |
| Puiseux series field to precision $n$ | `S, x = puiseux_series_field(K, n, :x)` |
| $S = K(x)(y)/(f)$ | `S, y = function_field(f, :y)` with $f \in K(x)[t]$ |
| $S = \mathrm{Frac}_R$ | `S = fraction_field(R)` |
| $S = R/(f)$ | `S, = residue_ring(R, f)` |
| $S = R/(f)$ (with $(f)$ maximal) | `S, = residue_field(R, f)` |
| $S = \mathrm{Mat}_{m\times n}(R)$ | `S = matrix_space(R, m, n)` |

# Chapter 12

# Parent objects with variable names

The multivariate parent object constructors (`polynomial_ring`, `power_series_ring`, `free_associative_algebra`, `laurent_polynomial_ring`, and `rational_function_field`) share a common interface for specifying the variable names, which is provided by `@varnames_interface`.

`AbstractAlgebra.@varnames_interface` – Macro.

```
@varnames_interface [M.]f(args..., varnames) macros=:yes n=n range=1:n
```

Add methods `X, vars = f(args..., varnames...)` and macro `X = @f(args..., varnames...)` to current scope.

**Created methods**

```
X, gens::Vector{T} = f(args..., varnames::Vector{Symbol})
```

Base method, called by everything else defined below. If a module `M` is specified, this is implemented as a call to `M.f`. Otherwise, a method `f` with this signature must already exist.

---

```
X, gens... = f(args..., varnames...; kv...)
X, gens... = f(args..., varnames::Tuple; kv...)
```

Compute `X` and `gens` via the base method. Then reshape `gens` into the shape defined by `varnames` according to `variable_names`.

The vararg `varnames...` method needs at least one argument to avoid confusion. Moreover a single `VarName` argument will be dispatched to use a univariate method of `f` if it exists (e.g. `polynomial_ring(R, :x)`). If you need those cases, use the `Tuple` method.

Keyword arguments are passed on to the base method.

---

```
X, x::Vector{T} = f(args..., n::Int, s::VarName = :x; kv...)
```

Shorthand for `X, x = f(args..., "$s#" => 1:n; kv...)`. The name of the argument `n` can be changed via the `n` option. The range `1:n` is given via the `range` option.

Setting `n=:no` disables creation of this method.

---

```
X = @f(args..., varnames...; kv...)
X = @f(args..., varnames::Tuple; kv...)
X = @f(args..., n::Int, s::VarName = :x; kv...)
X = @f(args..., varname::VarName; kv...)
```

These macros behave like their `f(args..., varnames; kv...)` counterparts but also introduce the indexed `varnames` into the current scope. The first version needs at least one `varnames` argument to avoid confusion. The last version calls the univariate base method if it exists (e.g. `polynomial_ring(R, varname)`).

Setting `macros=:no` disables macro creation.

**Warning**

Turning `varnames` into a vector of symbols happens by evaluating `variable_names(varnames)` in the global scope of the current module. For interactive usage in the REPL this is fine, but in general you have no access to local variables and should not use any side effects in `varnames`.

**Examples**

```
julia> f(a, s::Vector{Symbol}) = a, String.(s)
f (generic function with 1 method)

julia> AbstractAlgebra.@varnames_interface f(a, s)
@f (macro with 1 method)

julia> f
f (generic function with 5 methods)

julia> f("hello", [:x, :y, :z])
("hello", ["x", "y", "z"])

julia> f("numbered", 3)
("numbered", ["x1", "x2", "x3"])

julia> f("hello", :x => (1:1, 1:2), :y => 1:2, [:z])
("hello", ["x[1, 1]" "x[1, 2]"], ["y[1]", "y[2]"], ["z"])

julia> f("projective", ["x$i$j" for i in 0:1, j in 0:1], [:y0, :y1], [:z])
("projective", ["x00" "x01"; "x10" "x11"], ["y0", "y1"], ["z"])

julia> f("fun inputs", 'a':'g', Symbol.('x':'z', [0 1]))
("fun inputs", ["a", "b", "c", "d", "e", "f", "g"], ["x0" "x1"; "y0" "y1"; "z0" "z1"])

julia> @f("hello", "x#" => (1:1, 1:2), "y#" => (1:2), [:z])
"hello"

julia> (x11, x12, y1, y2, z)
```

```
("x11", "x12", "y1", "y2", "z")

julia> g(a, s::Vector{Symbol}; kv...) = (a, kv...), String.(s)
g (generic function with 1 method)

julia> AbstractAlgebra.@varnames_interface g(a, s)
@g (macro with 1 method)

julia> @g("parameters", [:x, :y], a=1, b=2; c=3)
("parameters", :c => 3, :a => 1, :b => 2)
```

source

AbstractAlgebra.variable_names – Function.

```
variable_names(a...) -> Vector{Symbol}
variable_names(a::Tuple) -> Vector{Symbol}
```

Create a vector of variable names from a variable name specification.

Each argument can be either an Array of VarNames, or of the form `s::VarName => iter`, or of the form `s::VarName => (iter...)`. Here `iter` is supposed to be any iterable, typically a range like `1:5`. The `:s => iter` specification is shorthand for `["s[$i]" for i in iter]`. Similarly `:s => (iter1, iter2)` is shorthand for `["s[$i,$j]" for i in iter1, j in iter2]`, and likewise for three and more iterables.

As an alternative `"s#" => iter` is shorthand for `["s$i" for i in iter]`. This also works for multiple iterators in that`"s#" => (iter1, iter2)` is shorthand for `["s$i$j" for i in iter1, j in iter2]`.

**Examples**

```
julia> AbstractAlgebra.variable_names([:x, :y])
2-element Vector{Symbol}:
 :x
 :y

julia> AbstractAlgebra.variable_names(:x => (0:0, 0:1), :y => 0:1, [:z])
5-element Vector{Symbol}:
 Symbol("x[0, 0]")
 Symbol("x[0, 1]")
 Symbol("y[0]")
 Symbol("y[1]")
 :z

julia> AbstractAlgebra.variable_names("x#" => (0:0, 0:1), "y#" => 0:1)
4-element Vector{Symbol}:
 :x00
 :x01
 :y0
 :y1

julia> AbstractAlgebra.variable_names("x#" => 9:11)
3-element Vector{Symbol}:
 :x9
```

```
 :x10
 :x11

julia> AbstractAlgebra.variable_names(["x$i$i" for i in 1:3])
3-element Vector{Symbol}:
 :x11
 :x22
 :x33

julia> AbstractAlgebra.variable_names('a':'c', ['z'])
4-element Vector{Symbol}:
 :a
 :b
 :c
 :z
```

source

`AbstractAlgebra.reshape_to_varnames` – Function.

```
reshape_to_varnames(vec::Vector{T}, varnames...) :: Tuple{Array{<:Any, T}}
reshape_to_varnames(vec::Vector{T}, varnames::Tuple) :: Tuple{Array{<:Any, T}}
```

Turn vec into the shape of varnames. Reverse flattening from variable_names.

**Examples**

```
julia> s = ([:a, :b], "x#" => (1:1, 1:2), "y#" => 1:2, [:z]);

julia> AbstractAlgebra.reshape_to_varnames(AbstractAlgebra.variable_names(s...), s...)
([:a, :b], [:x11 :x12], [:y1, :y2], [:z])

julia> R, v = polynomial_ring(ZZ, AbstractAlgebra.variable_names(s...))
(Multivariate polynomial ring in 7 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[a, b, x11, x12, y1, y2, z])

julia> (a, b), x, y, z = AbstractAlgebra.reshape_to_varnames(v, s...)
(AbstractAlgebra.Generic.MPoly{BigInt}[a, b], AbstractAlgebra.Generic.MPoly{BigInt}[x11 x12],
↪  AbstractAlgebra.Generic.MPoly{BigInt}[y1, y2], AbstractAlgebra.Generic.MPoly{BigInt}[z])

julia> R, (a, b), x, y, z = polynomial_ring(ZZ, s...)
(Multivariate polynomial ring in 7 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[a, b], AbstractAlgebra.Generic.MPoly{BigInt}[x11 x12],
↪  AbstractAlgebra.Generic.MPoly{BigInt}[y1, y2], AbstractAlgebra.Generic.MPoly{BigInt}[z])
```

source

# Part IV

# Rings

# Chapter 13

# Introduction

A rich ring hierarchy is provided, supporting both commutative and noncommutative rings.

A number of basic rings are provided, such as the integers, integers mod n and numerous fields.

A recursive rings implementation is then built on top of the basic rings via a number of generic ring constructions. These include univariate and multivariate polynomials and power series, univariate Laurent and Puiseux series, residue rings, matrix algebras, etc.

Where possible, these constructions can be built on top of one another in generic towers.

The ring hierarchy can be extended by implementing new rings to follow one or more ring interfaces. Generic functionality provided by the system is then automatically available for the new rings. These implementations can either be generic or can be specialised implementations provided by, for example, a C library.

In most cases, the interfaces consist of a set of constructors and functions that must be implemented to satisfy the interface. These are the functions that the generic code relies on being available.

# Chapter 14

# Ring functionality

AbstractAlgebra has both commutative and noncommutative rings. Together we refer to them below as rings.

## 14.1 Abstract types for rings

All commutative ring types in AbstractAlgebra belong to the `Ring` abstract type and commutative ring elements belong to the `RingElem` abstract type.

Noncommutative ring types belong to the `NCRing` abstract type and their elements to `NCRingElem`.

As Julia types cannot belong to our `RingElem` type hierarchy, we also provide the union type `RingElement` which includes `RingElem` in union with the Julia types `Integer`, `Rational` and `AbstractFloat`.

Similarly `NCRingElement` includes the Julia types just mentioned in union with `NCRingElem`.

Note that

```
Ring <: NCRing
RingElem <: NCRingElem
RingElement <: NCRingElement
```

## 14.2 Functions for types and parents of rings

```
parent_type(::Type{T}) where T <: NCRingElement
elem_type(::Type{T}) where T <: NCRing
```

Return the type of the parent (resp. element) type corresponding to the given ring element (resp. parent) type.

```
base_ring(R::NCRing)
base_ring(a::NCRingElement)
```

For generic ring constructions over a base ring (e.g. polynomials over a coefficient ring), return the parent object of that base ring.

```
parent(a::NCRingElement)
```

Return the parent of the given ring element.

AbstractAlgebra.is_domain_type – Function.

```
is_domain_type(T::Type) -> Bool
is_domain_type(x) -> Bool
```

Return true if every element of the given type is guaranteed to have an integral domain as its parent, and false if this cannot be guaranteed. For example elements of generic residue rings do not qualify, as the answer depends on the modulus, which is not part of the type.

Here T may be a ring element type or a ring type, and x may be a ring element or a ring.

**Examples**

```
julia> is_domain_type(BigInt)
true

julia> is_domain_type(ZZ)
true

julia> is_domain_type(residue_ring(ZZ, 6)[1])
false
```

**Note**

Implementors only need to define this for their ring element types; the variants taking a ring type or an instance delegate to those.

source

AbstractAlgebra.is_exact_type – Function.

```
is_exact_type(T::Type) -> Bool
is_exact_type(x) -> Bool
```

Return true if all elements of the given type are represented exactly, i.e. without approximation, and false otherwise. Integers, rationals, finite fields and polynomials and matrices over them are exact; floating point numbers, $p$-adic numbers and power series are not. As the type may be abstract or parameterised, true is only returned if every type it covers is exact.

Here T may be a ring element type or a ring type, and x may be a ring element or a ring.

**Examples**

```
julia> is_exact_type(BigInt)
true
```

```
julia> is_exact_type(ZZ)
true

julia> S, x = power_series_ring(QQ, 5, :x);

julia> is_exact_type(S), is_exact_type(x)
(false, false)
```

**Note**

Implementors only need to define this for their ring element types; the variants taking a ring type or an instance delegate to those.

source

The following function is implemented where mathematically and algorithmically possible.

`AbstractAlgebra.characteristic` – Method.

```
characteristic(R::NCRing)
```

Return the characteristic of the ring R. If the characteristic is not known, an exception is raised.

source

## 14.3 Constructors

If R is a parent object of a ring in AbstractAlgebra, it can always be used to construct certain objects in that ring.

```
(R::NCRing)() # constructs zero
(R::NCRing)(c::Integer)
(R::NCRing)(c::elem_type(R))
(R::NCRing{T})(a::T) where T <: RingElement
```

## 14.4 Basic functions

All rings in AbstractAlgebra are expected to implement basic ring operations, unary minus, binary addition, subtraction and multiplication, equality testing, powering.

In addition, the following are implemented for parents/elements just as they would be in Julia for types/objects.

```
zero(R::NCRing)
one(R::NCRing)
iszero(a::NCRingElement)
isone(a::NCRingElement)
```

In addition, the following are implemented where it is mathematically/algorithmically viable to do so.

```
is_unit(a::NCRingElement)
is_zero_divisor(a::NCRingElement)
is_zero_divisor_with_annihilator(a::NCRingElement)
```

The following standard Julia functions are also implemented for all ring elements.

```
hash(f::RingElement, h::UInt)
deepcopy_internal(a::RingElement, dict::IdDict)
show(io::IO, R::NCRing)
show(io::IO, a::NCRingElement)
```

## 14.5 Basic functionality for inexact rings only

By default, inexact ring elements in AbstractAlgebra compare equal if they are the same to the minimum precision of the two elements. However, we also provide the following more strict notion of equality, which also requires the precisions to be the same.

```
isequal(a::T, b::T) where T <: NCRingElement
```

For floating point and ball arithmetic it is sometimes useful to be able to check if two elements are approximately equal, e.g. to suppress numerical noise in comparisons. For this, the following are provided.

```
isapprox(a::T, b::T; atol::Real=sqrt(eps())) where T <: RingElement
```

Similarly, for a parameterised ring with type `MyElem{T}` over such an inexact ring we have the following.

```
isapprox(a::MyElem{T}, b::T; atol::Real=sqrt(eps())) where T <: RingElement
isapprox(a::T, b::MyElem{T}; atol::Real=sqrt(eps())) where T <: RingElement
```

These notionally perform a coercion into the parameterised ring before doing the approximate equality test.

## 14.6 Basic functionality for commutative rings only

```
divexact(a::T, b::T) where T <: RingElement
inv(a::T)
```

Return `a/b` or `1/a` respectively, where the slash here refers to the mathematical notion of division in the ring, not Julia's floating point division operator.

## 14.7 Basic functionality for noncommutative rings only

```
divexact_left(a::T, b::T) where T <: NCRingElement
divexact_right(a::T, b::T) where T <: NCRingElement
```

As per `divexact` above, except that division by `b` happens on the left or right, respectively, of `a`.

## 14.8 Unsafe ring operators

To speed up polynomial and matrix arithmetic, it sometimes makes sense to mutate values in place rather than replace them with a newly created object every time they are modified.

For this purpose, certain mutating operators are required. In order to support immutable types (struct in Julia) and systems that don't have in-place operators, all unsafe operators must return the (ostensibly) mutated value. Only the returned value is used in computations, so this lifts the requirement that the unsafe operators actually mutate the value.

Note the exclamation point is a convention, which indicates that the object may be mutated in-place.

To make use of these functions, one must be certain that no other references are held to the object being mutated, otherwise those values will also be changed!

The results of deepcopy and all arithmetic operations, including powering and division can be assumed to be new objects without other references being held, as can objects returned from constructors.

**Note**

It is important to recognise that `R(a)` where R is the ring a belongs to, does not create a new value. For this case, use `deepcopy(a)`.

`AbstractAlgebra.zero!` – Function.

```
zero!(a)
```

Return `zero(parent(a))`, possibly modifying the object a in the process.

source

`AbstractAlgebra.one!` – Function.

```
one!(a)
```

Return `one(parent(a))`, possibly modifying the object a in the process.

source

`AbstractAlgebra.add!` – Function.

```
add!(z, a, b)
add!(a, b)
```

Return `a + b`, possibly modifying the object `z` in the process. Aliasing is permitted. The two argument version is a shorthand for `add!(a, a, b)`.

source

`AbstractAlgebra.sub!` – Function.

```
sub!(z, a, b)
sub!(a, b)
```

Return `a - b`, possibly modifying the object `z` in the process. Aliasing is permitted. The two argument version is a shorthand for `sub!(a, a, b)`.

source

`AbstractAlgebra.mul!` – Function.

```
mul!(z, a, b)
mul!(a, b)
```

Return `a * b`, possibly modifying the object `z` in the process. Aliasing is permitted. The two argument version is a shorthand for `mul!(a, a, b)`.

source

`AbstractAlgebra.neg!` – Function.

```
neg!(z, a)
neg!(a)
```

Return `-a`, possibly modifying the object `z` in the process. Aliasing is permitted. The unary version is a shorthand for `neg!(a, a)`.

source

`AbstractAlgebra.inv!` – Function.

```
inv!(z, a)
inv!(a)
```

Return `AbstractAlgebra.inv(a)`, possibly modifying the object `z` in the process. Aliasing is permitted. The unary version is a shorthand for `inv!(a, a)`.

**Note**

`AbstractAlgebra.inv` and `Base.inv` differ only in their behavior on julia types like `Integer` and `Rational{Int}`. The former makes it adhere to the Ring interface.

source

`AbstractAlgebra.addmul!` – Function.

```
addmul!(z, a, b, t)
addmul!(z, a, b)
```

Return `z + a * b`, possibly modifying the objects `z` and `t` in the process.

The second version is usually a shorthand for `addmul!(z, a, b, parent(z)())`, but in some cases may be more efficient. For multiple operations in a row that use temporary storage, it is still best to use the four argument version.

source

`AbstractAlgebra.submul!` – Function.

```
submul!(z, a, b, t)
submul!(z, a, b)
```

Return `z - a * b`, possibly modifying the objects `z` and `t` in the process.

The second version is usually a shorthand for `submul!(z, a, b, parent(z)())`, but in some cases may be more efficient. For multiple operations in a row that use temporary storage, it is still best to use the four argument version.

source

`AbstractAlgebra.divexact!` – Function.

```
divexact!(z, a, b)
divexact!(a, b)
```

Return `divexact(a, b)`, possibly modifying the object `z` in the process. Aliasing is permitted. The two argument version is a shorthand for `divexact(a, a, b)`.

source

`AbstractAlgebra.div!` – Function.

```
div!(z, a, b)
div!(a, b)
```

Return `div(a, b)`, possibly modifying the object `z` in the process. Aliasing is permitted. The two argument version is a shorthand for `div(a, a, b)`.

**Note**

`AbstractAlgebra.div` and `Base.div` differ only in their behavior on julia types like `Integer` and `Rational{Int}`. The former makes it adhere to the Ring interface.

source

`AbstractAlgebra.rem!` – Function.

```
rem!(z, a, b)
rem!(a, b)
```

Return `rem(a, b)`, possibly modifying the object z in the process. Aliasing is permitted. The two argument version is a shorthand for `rem(a, a, b)`.

source

`AbstractAlgebra.mod!` – Function.

```
mod!(z, a, b)
mod!(a, b)
```

Return `mod(a, b)`, possibly modifying the object z in the process. Aliasing is permitted. The two argument version is a shorthand for `mod(a, a, b)`.

source

`AbstractAlgebra.gcd!` – Function.

```
gcd!(z, a, b)
gcd!(a, b)
```

Return `gcd(a, b)`, possibly modifying the object z in the process. Aliasing is permitted. The two argument version is a shorthand for `gcd(a, a, b)`.

source

`AbstractAlgebra.lcm!` – Function.

```
lcm!(z, a, b)
lcm!(a, b)
```

Return `lcm(a, b)`, possibly modifying the object z in the process. Aliasing is permitted. The two argument version is a shorthand for `lcm(a, a, b)`.

source

## 14.9 Random generation

The Julia random interface is implemented for all ring parents (instead of for types). The exact interface differs depending on the ring, but the parameters supplied are usually ranges, e.g. `-1:10` for the range of allowed degrees for a univariate polynomial.

```
rand(R::NCRing, v...)
```

## 14.10 Factorization

For commutative rings supporting factorization and irreducibility testing, the following optional functions may be implemented.

`AbstractAlgebra.is_irreducible` – Method.

```
is_irreducible(a::RingElement)
```

Return `true` if $a$ is irreducible, else return `false`. Zero and units are by definition never irreducible.

source

`AbstractAlgebra.is_squarefree` – Method.

```
is_squarefree(a::RingElement)
```

Return `true` if $a$ is squarefree, else return `false`. An element is squarefree if it it is not divisible by any squares except the squares of units.

source

```
factor(a::T) where T <: RingElement
factor_squarefree(a::T) where T <: RingElement
```

Return a factorization into irreducible or squarefree elements, respectively. The return is an object of type `Fac{T}`.

`AbstractAlgebra.Fac` – Type.

```
Fac{T <: RingElement}
```

Type for factored ring elements. The structure holds a unit of type T and is an iterable collection of `T => Int` pairs for the factors and exponents.

See `unit(a::Fac)`, `evaluate(a::Fac)`.

source

`AbstractAlgebra.unit` – Method.

```
unit(a::Fac{T}) -> T
```

Return the unit of the factorization.

source

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::Fac{T}) -> T
```

Multiply out the factorization into a single element.

source

`Base.getindex` – Method.

```
getindex(a::Fac, b) -> Int
```

If $b$ is a factor of $a$, the corresponding exponent is returned. Otherwise an error is thrown.

source

`Base.setindex!` – Method.

```
setindex!(a::Fac{T}, c::Int, b::T)
```

If $b$ is a factor of $a$, the corresponding entry is set to $c$.

source

## 14.11 Square root

Rings may implement functionality for detecting and computing square roots.

The exact behaviour depends on the ring. Some rings provide both operations, while others only implement `is_square`.

`AbstractAlgebra.is_square` – Function.

```
is_square(a::NCRingElement)
```

Return `true` iff `a` is the square of a value in its own ring. See also `is_square(A::MatElem)` which tests whether a matrix has square shape.

source

```
is_square(A::MatElem)
```

Return `true` iff the matrix `A` has square shape. See also `is_square(a::T) where {T <: NCRingElement}` which tests whether the given value `a` is a square in its own ring.

source

`AbstractAlgebra.sqrt` – Method.

```
sqrt(a::NCRingElem; check::Bool=true)
```

Return a square root of `a`, if it exists. By default (`check=true`), implementations should raise an exception if `a` is not a square in its ring. If `check=false`, implementations may skip this verification.

See also `is_square` and `is_square_with_sqrt`.

source

## 14.12 Miscellaneous

There are some miscellaneous functions for rings to ease up certain computations.

`AbstractAlgebra.Generic.falling_factorial` – Function.

```
falling_factorial(x::RingElement, n::Integer)
```

Return the falling factorial of $x$, i.e. $x(x-1)(x-2)\cdots(x-n+1)$. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> falling_factorial(x, 1)
x

julia> falling_factorial(x, 2)
x^2 - x

julia> falling_factorial(4, 2)
12
```

source

`AbstractAlgebra.Generic.rising_factorial` – Function.

```
rising_factorial(x::RingElement, n::Integer)
```

Return the rising factorial of $x$, i.e. $x(x+1)(x+2)\cdots(x+n-1)$. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> rising_factorial(x, 1)
x

julia> rising_factorial(x, 2)
x^2 + x

julia> rising_factorial(4, 2)
20
```

source

`AbstractAlgebra.Generic.rising_factorial2` – Function.

```
rising_factorial2(x::RingElement, n::Integer)
```

Return a tuple containing the rising factorial $x(x+1)\cdots(x+n-1)$ and its derivative. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> rising_factorial2(x, 1)
(x, 1)

julia> rising_factorial2(x, 2)
(x^2 + x, 2*x + 1)

julia> rising_factorial2(4, 2)
(20, 9)
```

source

# Chapter 15

# Integer ring

AbstractAlgebra.jl provides a module, implemented in `src/julia/Integer.jl` for making Julia `BigInts` conform to the AbstractAlgebra.jl Ring interface.

In addition to providing a parent object ZZ for Julia `BigInts`, we implement any additional functionality required by AbstractAlgebra.jl.

Because `BigInt` cannot be directly included in the AbstractAlgebra.jl abstract type hierarchy, we achieve integration of Julia `BigInts` by introducing a type union, called `RingElement`, which is a union of `RingElem` and a number of Julia types, including `BigInt`. Everywhere that `RingElem` is notionally used in AbstractAlgebra.jl, we are in fact using `RingElement`, with additional care being taken to avoid ambiguities.

The details of how this is done are technical, and we refer the reader to the implementation for details. For most intents and purposes, one can think of the Julia `BigInt` type as belonging to `RingElem`.

One other technicality is that Julia defines certain functions for `BigInt`, such as `sqrt` and `exp` differently to what AbstractAlgebra.jl requires. To get around this, we redefine these functions internally to AbstractAlgebra.jl, without redefining them for users of AbstractAlgebra.jl. This allows the internals of AbstractAlgebra.jl to function correctly, without broadcasting pirate definitions of already defined Julia functions to the world.

To access the internal definitions, one can use `AbstractAlgebra.sqrt` and `AbstractAlgebra.exp`, etc.

## 15.1 Types and parent objects

Integers have type `BigInt`, as in Julia itself. We simply supplement the functionality for this type as required for computer algebra.

The parent objects of such integers has type `Integers{BigInt}`.

For convenience, we also make `Int` a part of the AbstractAlgebra.jl type hierarchy and its parent object (accessible as `zz`) has type `Integers{Int}`. But we caution that this type is not particularly useful as a model of the integers and may not function as expected within AbstractAlgebra.jl.

## 15.2 Integer constructors

In order to construct integers in AbstractAlgebra.jl, one can first construct the integer ring itself. This is accomplished using the following constructor.

```
Integers{BigInt}()
```

This gives the unique object of type `Integers{BigInt}` representing the ring of integers in AbstractAlgebra.jl.

In practice, one simply uses ZZ which is assigned to be the return value of the above constructor. There is no need to call the constructor in practice.

Here are some examples of creating the integer ring and making use of the resulting parent object to coerce various elements into the ring.

**Examples**

```
julia> f = ZZ()
0

julia> g = ZZ(123)
123

julia> h = ZZ(BigInt(1234))
1234
```

## 15.3 Basic ring functionality

The integer ring in AbstractAlgebra.jl implements the full Ring interface and the Euclidean Ring interface.

We give some examples of such functionality.

**Examples**

```
julia> f = ZZ(12)
12

julia> h = zero(ZZ)
0

julia> k = one(ZZ)
1

julia> isone(k)
true

julia> iszero(f)
false

julia> T = parent(f)
Integers

julia> f == deepcopy(f)
true

julia> g = f + 12
24

julia> h = powermod(f, 12, ZZ(17))
4

julia> flag, q = divides(f, ZZ(3))
(true, 4)
```

## 15.4 Integer functionality provided by AbstractAlgebra.jl

The functionality below supplements that provided by Julia itself for its `BigInt` type.

### Basic functionality

#### Examples

```
julia> r = ZZ(-1)
-1

julia> is_unit(r)
true
```

### Divisibility testing

`AbstractAlgebra.is_divisible_by` – Method.

```
is_divisible_by(a::Integer, b::Integer)
```

Return `true` if $a$ is divisible by $b$, i.e. if there exists $c$ such that $a = bc$.

#### Examples

```
julia> r = ZZ(6)
6

julia> s = ZZ(3)
3

julia> is_divisible_by(r, s)
true
```

source

`AbstractAlgebra.is_associated` – Method.

```
is_associated(a::Integer, b::Integer)
```

Return `true` if $a$ and $b$ are associated, i.e. if there exists a unit $c$ such that $a = bc$. For integers, this reduces to checking if $a$ and $b$ differ by a factor of $1$ or $-1$.

source

### Square root

`AbstractAlgebra.sqrt` – Method.

```
sqrt(a::T; check::Bool=true) where T <: Integer
```

Return the square root of $a$. By default the function will throw an exception if the input is not square. If `check=false` this test is omitted.

source

`AbstractAlgebra.is_square_with_sqrt` – Method.

```
is_square_with_sqrt(a::T) where T <: RingElement
```

Return `(true, s)` if $a$ is a perfect square, where $s^2 = a$. Otherwise return `(false, ...)` where `...` is some element of `parent(a)`.

source

`AbstractAlgebra.root` – Method.

```
root(a::T, n::Int; check::Bool=true) where T <: Integer
```

Return the $n$-th root of $a$. If `check=true` the function will test if the input was a perfect $n$-th power, otherwise an exception will be raised. We require $n > 0$.

source

`AbstractAlgebra.iroot` – Method.

```
iroot(a::T, n::Int) where T <: Integer
```

Return the truncated integer part of the $n$-th root of $a$ (round towards zero). We require $n > 0$ and also $a \geq 0$ if $n$ is even.

source

`AbstractAlgebra.is_power` – Method.

```
is_power(a::T, n::Int) where T <: Integer
```

Return `true, q` if $a$ is a perfect $n$-th power with $a = q^n$. Otherwise return `false, 0`. We require $n > 0$.

source

`AbstractAlgebra.exp` – Method.

```
exp(a::T) where T <: Integer
```

Return $1$ if $a = 0$, otherwise throw an exception. This function is not generally of use to the user, but is used internally in AbstractAlgebra.jl.

source

```
exp(a::Rational{T}) where T <: Integer
```

Return $1$ if $a = 0$, otherwise throw an exception.

source

**Examples**

```
julia> d = AbstractAlgebra.sqrt(ZZ(36))
6

julia> is_square(ZZ(9))
true

julia> m = AbstractAlgebra.exp(ZZ(0))
1
```

### Coprime bases

`AbstractAlgebra.ppio` – Method.

```
ppio(a::T, b::T)
```

Return a pair $(c, d)$ such that $a = c * d$ and $c = gcd(a, b^\infty)$ if $a \neq 0$, and $c = b$, $d = 0$ if $a = 0$.

**Examples**

```
julia> c, n = ppio(ZZ(12), ZZ(26))
(4, 3)
```

source

# Chapter 16

# Total ring of fractions

AbstractAlgebra.jl provides a module, implemented in `src/generic/TotalFraction.jl`, for the total ring of fractions of a ring.

The total ring of fractions of a ring R is the localisation of R at the non-zero divisors of R, the latter being a multiplicative subset of R.

There are no restrictions on the ring except the function `is_zero_divisor` must be defined and effective for R.

In particular, we do not assume that all elements of R which are not zero divisors are units in R. This has the effect of making exact division impossible generically in the total ring of fractions of R.

This in turn limits the usefulness of the total ring of fractions as a ring in AbstractAlgebra as a great deal of generic code relies on `divexact`. Should this be a limitation, the user can define their own `divexact` function for the total ring of fractions in question.

Note that in most cases `a*inv(b)` is not a sufficient definition of `divexact(a, b)` due to the possibility that `b` is not a unit in the total ring of fractions.

It is also possible to construct a total ring of fractions of R without the `is_zero_divisor` function existing for R, but some functions such as `is_unit`, `inv`, `rand` and ad hoc arithmetic operations involving rational numbers are not available for the total ring of fractions. One must also construct fractions using the option `check=false` and it is one's own responsibility to check that the denominator is not a zero divisor.

Note that although the total ring of fractions of an integral domain R is mathematically the same thing as the fraction field of R, these will be different objects in AbstractAlgebra and have different types.

## 16.1 Generic total ring of fraction types

AbstractAlgebra.jl implements a generic type for elements of a total ring of fractions, namely`Generic.TotFrac{T}` where T is the type of elements of the base ring. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of such elements have type `Generic.TotFracRing{T}`.

## 16.2 Abstract types

The types for elements of a total ring of fractions belong directly to the abstract type `RingElem` and the type for the total ring of fractions parent object belongs directly to the abstract type `Ring`.

## 16.3 Total ring of fractions constructors

In order to construct fractions in a total ring of fractions in AbstractAlgebra.jl, one must first construct the parent object for the total ring of fractions itself. This is accomplished with the following constructor.

```
total_ring_of_fractions(R::Ring; cached::Bool = true)
```

Given a base ring R return the parent object of the total ring of fractions of $R$. By default the parent object S will depend only on R and will be cached. Setting the optional argument `cached` to `false` will prevent the parent object S from being cached.

Here are some examples of creating a total ring of fractions and making use of the resulting parent objects to coerce various elements into the ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S = total_ring_of_fractions(R)
Total ring of fractions of R

julia> f = S()
0

julia> g = S(123)
123

julia> h = S(BigInt(1234))
1234

julia> k = S(x + 1)
x + 1
```

## 16.4 Fraction constructors

One can construct fractions using the total ring of fractions parent object, as for any ring or field.

```
(R::TotFracRing)() # constructs zero
(R::TotFracRing)(c::Integer)
(R::TotFracRing)(c::elem_type(R))
(R::TotFracRing{T})(a::T) where T <: RingElement
```

Although one cannot use the double slash operator `//` to construct elements of a total ring of fractions, as no parent has been specified, one can use the double slash operator to construct elements of a total ring of fractions so long as one of the arguments to the double slash operator is already in the total ring of fractions in question.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = total_ring_of_fractions(R)
Total ring of fractions of R

julia> f = S(x + 1)
x + 1

julia> f//3
(x + 1)//3

julia> 3//f
3//(x + 1)

julia> f//x
(x + 1)//x
```

## 16.5 Functions for types and parents of total rings of fractions

Total rings of fractions in AbstractAlgebra.jl implement the Ring interface except for the `divexact` function which is not generically possible to implement.

```
base_ring(R::TotFracRing)
base_ring(a::TotFrac)
```

Return the base ring of which the total ring of fractions was constructed.

```
parent(a::TotFrac)
```

Return the total ring of fractions that the given fraction belongs to.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = total_ring_of_fractions(R)
Total ring of fractions of R

julia> f = S(x + 1)
x + 1

julia> U = base_ring(S)
Univariate polynomial ring in x over rationals

julia> V = base_ring(f)
Univariate polynomial ring in x over rationals

julia> T = parent(f)
Total ring of fractions of R
```

```
julia> m = characteristic(S)
0
```

## 16.6 Total ring of fractions functions

### Basic functions

Total rings of fractions implement the Ring interface.

```
zero(R::TotFracRing)
one(R::TotFracRing)
iszero(a::TotFrac)
isone(a::TotFrac)
```

```
inv(a::T) where T <: TotFrac
```

They also implement some of the following functions which would usually be associated with the field and fraction field interfaces.

```
is_unit(f::TotFrac)
```

```
numerator(a::TotFrac)
denominator(a::TotFrac)
```

#### Examples

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = total_ring_of_fractions(R)
Total ring of fractions of R

julia> f = S(x + 1)
x + 1

julia> g = f//(x^3 + 3x + 1)
(x + 1)//(x^3 + 3*x + 1)

julia> h = zero(S)
0

julia> k = one(S)
1

julia> isone(k)
true
```

```
julia> iszero(f)
false

julia> r = deepcopy(f)
x + 1

julia> n = numerator(g)
x + 1

julia> d = denominator(g)
x^3 + 3*x + 1
```

### Random generation

Random fractions can be generated using rand. The parameters passed after the total ring of fractions tell rand how to generate random elements of the base ring.

```
rand(R::TotFracRing, v...)
```

#### Examples

```
julia> R, = residue_ring(ZZ, 12);

julia> K = total_ring_of_fractions(R)
Total ring of fractions of R

julia> f = rand(K, 0:11)
7//5

julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S = total_ring_of_fractions(R)
Total ring of fractions of R

julia> g = rand(S, -1:3, -10:10)
(4*x + 4)//(-4*x^2 - x + 4)
```

# Chapter 17

# Univariate polynomial functionality

AbstractAlgebra.jl provides a module, implemented in `src/Poly.jl` for polynomials over any commutative ring belonging to the AbstractAlgebra abstract type hierarchy. This functionality will work for any univariate polynomial type which follows the Univariate Polynomial Ring interface.

## 17.1 Generic univariate polynomial types

AbstractAlgebra.jl provides a generic polynomial type based on Julia arrays which is implemented in `src/generic/Poly.jl`.

These generic polynomials have type `Generic.Poly{T}` where T is the type of elements of the coefficient ring. Internally they consist of a Julia array of coefficients and some additional fields for length and a parent object, etc. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of such polynomials have type `Generic.PolyRing{T}`.

The string representation of the variable of the polynomial ring and the base/coefficient ring $R$ is stored in the parent object.

## 17.2 Abstract types

All univariate polynomial element types belong to the abstract type `PolyRingElem{T}` and the polynomial ring types belong to the abstract type `PolyRing{T}`. This enables one to write generic functions that can accept any AbstractAlgebra polynomial type.

**Note**

Both the generic polynomial ring type `Generic.PolyRing{T}` and the abstract type it belongs to, `PolyRing{T}`, are called `PolyRing`. The former is a (parameterised) concrete type for a polynomial ring over a given base ring whose elements have type T. The latter is an abstract type representing all polynomial ring types in AbstractAlgebra.jl, whether generic or very specialised (e.g. supplied by a C library).

## 17.3 Polynomial ring constructors

In order to construct polynomials in AbstractAlgebra.jl, one must first construct the polynomial ring itself. This is accomplished with the following constructor.

`AbstractAlgebra.polynomial_ring` – Method.

```
polynomial_ring(R::NCRing, s::VarName = :x; cached::Bool=true)
```

Given a base ring R and symbol/string s specifying how the generator (variable) should be printed, return a tuple `S, x` representing the new polynomial ring $S = R[x]$ and the generator $x$ of the ring.

By default the parent object S depends only on R and x and will be cached. Setting the optional argument cached to `false` will prevent the parent object S from being cached.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)
```

source

A shorthand version of this function is provided: given a base ring R, we abbreviate the constructor as follows.

```
R[:x]
```

Here are some examples of creating polynomial rings and their associated generators.

**Examples**

```
julia> T, z = QQ[:z]
(Univariate polynomial ring in z over rationals, z)

julia> U, x = polynomial_ring(ZZ)
(Univariate polynomial ring in x over integers, x)
```

All of the examples here are generic polynomial rings, but specialised implementations of polynomial rings provided by external modules will also usually provide a `polynomial_ring` constructor to allow creation of their polynomial rings.

## 17.4 Polynomial constructors

Once a polynomial ring is constructed, there are various ways to construct polynomials in that ring.

The easiest way is simply using the generator returned by the `polynomial_ring` constructor and build up the polynomial using basic arithmetic.

The Julia language has special syntax for the construction of polynomials in terms of a generator, e.g. we can write 2x instead of 2*x.

A second way is to use the polynomial ring to construct a polynomial. There are the usual ways of constructing an element of a ring.

```
(R::PolyRing)() # constructs zero
(R::PolyRing)(c::Integer)
(R::PolyRing)(c::elem_type(R))
(R::PolyRing{T})(a::T) where T <: RingElement
```

The third constructor above cannot be used to coerce a polynomial from one ring to another; instead use a call like `map_coefficients(identity, f; parent=dest_ring))`, or alternatively, for *small polynomials* one can simply use evaluation: namely, `f(y)` where `y` is the generator of the destination polynomial ring. The fourth constructor creates a constant polynomial from an element of the coefficient ring.

For polynomials there is also the following more general constructor accepting an array of coefficients.

```
(S::PolyRing{T})(A::Vector{T}) where T <: RingElem
(S::PolyRing{T})(A::Vector{U}) where T <: RingElem, U <: RingElem
(S::PolyRing{T})(A::Vector{U}) where T <: RingElem, U <: Integer
```

Construct the polynomial in the ring `S` with the given array of coefficients, i.e. where `A[1]` is the constant coefficient.

A third way of constructing polynomials is to construct them directly without creating the polynomial ring.

`AbstractAlgebra.polynomial` – Function.

```
polynomial(R::Ring, arr::Vector{T}, var::VarName=:x; cached::Bool=true) where T
```

Return the polynomial over `R` in the variable `var` with coefficients `arr`, where `arr[1]` is the constant coefficient. The polynomial ring is created on the fly; setting `cached` to `false` prevents it from being cached.

**Examples**

```
julia> polynomial(ZZ, [1, 2, 3])
3*x^2 + 2*x + 1

julia> polynomial(ZZ, [1, 2, 3], :y)
3*y^2 + 2*y + 1
```

source

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x^3 + 3x + 21
x^3 + 3*x + 21

julia> g = (x + 1)*y^2 + 2x + 1
(x + 1)*y^2 + 2*x + 1
```

```
julia> R()
0

julia> S(1)
1

julia> S(y)
y

julia> S(x)
x

julia> S, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> f = S(Rational{BigInt}[2, 3, 1])
x^2 + 3*x + 2

julia> g = S(BigInt[1, 0, 4])
4*x^2 + 1

julia> h = S([4, 7, 2, 9])
9*x^3 + 2*x^2 + 7*x + 4

julia> p = polynomial(ZZ, [1, 2, 3])
3*x^2 + 2*x + 1

julia> f = polynomial(ZZ, [1, 2, 3], :y)
3*y^2 + 2*y + 1
```

## 17.5 Similar and zero

Another way of constructing polynomials is to construct one similar to an existing polynomial using either similar or zero.

```
similar(x::MyPoly{T}, R::Ring=base_ring(x)) where T <: RingElem
zero(x::MyPoly{T}, R::Ring=base_ring(x)) where T <: RingElem
```

Construct the zero polynomial with the same variable as the given polynomial with coefficients in the given ring. Both functions behave the same way for polynomials.

```
similar(x::MyPoly{T}, R::Ring, var::VarName=var(parent(x))) where T <: RingElem
similar(x::MyPoly{T}, var::VarName=var(parent(x))) where T <: RingElem
zero(x::MyPoly{T}, R::Ring, var::VarName=var(parent(x))) where T <: RingElem
zero(x::MyPoly{T}, var::VarName=var(parent(x))) where T <: RingElem
```

Construct the zero polynomial with the given variable and coefficients in the given ring, if specified, and in the coefficient ring of the given polynomial otherwise.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> f = 1 + 2x + 3x^2
3*x^2 + 2*x + 1

julia> g = similar(f)
0

julia> h = similar(f, QQ)
0

julia> k = similar(f, QQ, :y)
0
```

## 17.6 Functions for types and parents of polynomial rings

```
base_ring(R::PolyRing)
base_ring(a::PolyRingElem)
```

Return the coefficient ring of the given polynomial ring or polynomial.

```
parent(a::NCRingElement)
```

Return the polynomial ring of the given polynomial.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> U = base_ring(S)
Univariate polynomial ring in x over integers

julia> V = base_ring(y + 1)
Univariate polynomial ring in x over integers

julia> T = parent(y + 1)
Univariate polynomial ring in y over R
```

## 17.7 Euclidean polynomial rings

For polynomials over a field, the Euclidean Ring Interface is implemented.

```
mod(f::PolyRingElem, g::PolyRingElem)
divrem(f::PolyRingElem, g::PolyRingElem)
div(f::PolyRingElem, g::PolyRingElem)
```

```
mulmod(f::PolyRingElem, g::PolyRingElem, m::PolyRingElem)
powermod(f::PolyRingElem, e::Int, m::PolyRingElem)
invmod(f::PolyRingElem, m::PolyRingElem)
```

```
divides(f::PolyRingElem, g::PolyRingElem)
remove(f::PolyRingElem, p::PolyRingElem)
valuation(f::PolyRingElem, p::PolyRingElem)
```

```
gcd(f::PolyRingElem, g::PolyRingElem)
lcm(f::PolyRingElem, g::PolyRingElem)
gcdx(f::PolyRingElem, g::PolyRingElem)
gcdinv(f::PolyRingElem, g::PolyRingElem)
```

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S, = residue_ring(R, x^3 + 3x + 1);

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = (3*x^2 + x + 2)*y + x^2 + 1
(3*x^2 + x + 2)*y + x^2 + 1

julia> g = (5*x^2 + 2*x + 1)*y^2 + 2x*y + x + 1
(5*x^2 + 2*x + 1)*y^2 + 2*x*y + x + 1

julia> h = (3*x^3 + 2*x^2 + x + 7)*y^5 + 2x*y + 1
(2*x^2 - 8*x + 4)*y^5 + 2*x*y + 1

julia> invmod(f, g)
(707//3530*x^2 + 2151//1765*x + 123//3530)*y - 178//1765*x^2 - 551//3530*x + 698//1765

julia> mulmod(f, g, h)
(-30*x^2 - 43*x - 9)*y^3 + (-7*x^2 - 23*x - 7)*y^2 + (4*x^2 - 10*x - 3)*y + x^2 - 2*x

julia> powermod(f, 3, h)
(69*x^2 + 243*x + 79)*y^3 + (78*x^2 + 180*x + 63)*y^2 + (27*x^2 + 42*x + 18)*y + 3*x^2 + 3*x + 2

julia> h = mod(f, g)
(3*x^2 + x + 2)*y + x^2 + 1

julia> q, r = divrem(f, g)
```

```
(0, (3*x^2 + x + 2)*y + x^2 + 1)

julia> div(g, f)
(-5//11*x^2 + 2//11*x + 6//11)*y - 13//121*x^2 - 3//11*x - 78//121

julia> d = gcd(f*h, g*h)
y + 1//11*x^2 + 6//11

julia> k = gcdinv(f, h)
(y + 1//11*x^2 + 6//11, 0)

julia> m = lcm(f, h)
(-14*x^2 - 23*x - 2)*y - 4*x^2 - 5*x + 1

julia> flag, q = divides(g^2, g)
(true, (5*x^2 + 2*x + 1)*y^2 + 2*x*y + x + 1)

julia> valuation(3g^3, g) == 3
true

julia> val, q = remove(5g^3, g)
(3, 5)

julia> r, s, t = gcdx(g, h)
(1, 311//3530*x^2 - 2419//3530*x + 947//1765, (707//3530*x^2 + 2151//1765*x + 123//3530)*y -
↪  178//1765*x^2 - 551//3530*x + 698//1765)
```

Functions in the Euclidean Ring interface are supported over residue rings that are not fields, except that if an impossible inverse is encountered during the computation an error is thrown.

## 17.8 Polynomial functions

### Basic functionality

All basic ring functionality is provided for polynomials. The most important such functions are the following.

```
zero(R::PolyRing)
one(R::PolyRing)
iszero(a::PolyRingElem)
isone(a::PolyRingElem)
```

```
divexact(a::T, b::T) where T <: PolyRingElem
```

All functions in the polynomial interface are provided. The most important are the following.

```
var(S::PolyRing)
symbols(S::PolyRing{T}) where T <: RingElem
```

Return a symbol or length 1 array of symbols, respectively, specifying the variable of the polynomial ring. This symbol is converted to a string when printing polynomials in that ring.

In addition, the following basic functions are provided.

`AbstractAlgebra.modulus` – Method.

```
modulus(a::PolyRingElem{T}) where {T <: ResElem}
```

Return the modulus of the coefficients of the given polynomial.

source

`AbstractAlgebra.leading_coefficient` – Method.

```
leading_coefficient(a::PolynomialElem)
```

Return the leading coefficient of the given polynomial. This will be the nonzero coefficient of the term with highest degree unless the polynomial in the zero polynomial, in which case a zero coefficient is returned.

source

```
leading_coefficient(p::MPolyRingElem)
```

Return the leading coefficient of the polynomial $p$.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y]; internal_ordering=:deglex);

julia> p = 2*x*y + 3*y^3 + 1
3*y^3 + 2*x*y + 1

julia> leading_coefficient(p)
3
```

source

`AbstractAlgebra.trailing_coefficient` – Method.

```
trailing_coefficient(a::PolynomialElem)
```

Return the trailing coefficient of the given polynomial. This will be the nonzero coefficient of the term with lowest degree unless the polynomial is the zero polynomial, in which case a zero coefficient is returned.

source

```
trailing_coefficient(p::MPolyRingElem)
```

Return the trailing coefficient of the polynomial $p$, i.e. the coefficient of the last nonzero term.

source

`AbstractAlgebra.constant_coefficient` – Method.

```
constant_coefficient(a::PolynomialElem)
```

Return the constant coefficient of the given polynomial. If the polynomial is the zero polynomial, the function will return zero.

source

`AbstractAlgebra.set_coefficient!` – Method.

```
set_coefficient!(c::PolynomialElem{T}, n::Int, a::T) where T <: RingElement
set_coefficient!(c::PolynomialElem{T}, n::Int, a::U) where {T <: RingElement, U <: Integer}
```

Set the coefficient of degree $n$ to $a$.

source

`AbstractAlgebra.tail` – Method.

```
tail(a::PolynomialElem)
```

Return the tail of the given polynomial, i.e. the polynomial without its leading term (if any).

source

`AbstractAlgebra.gen` – Method.

```
gen(a::MPolyRing{T}, i::Int) where {T <: RingElement}
```

Return the $i$-th generator (variable) of the given polynomial ring.

source

```
gen(R::AbsPowerSeriesRing{T}) where T <: RingElement
```

Return the generator of the power series ring, i.e. $x + O(x^n)$ where $n$ is the precision of the power series ring $R$.

source

```
gen(S::AbsSimpleFunctionField{T, U}) where {T <: FieldElement, U <: PolyRingElem{T}}
```

Return the generator of the function field returned by the function field constructor.

source

`AbstractAlgebra.is_gen` – Method.

```
is_gen(a::PolynomialElem)
```

Return `true` if the given polynomial is the constant generator of its polynomial ring, otherwise return `false`.

source

```
is_gen(x::MPoly{T}) where {T <: RingElement}
```

Return `true` if the given polynomial is a generator (variable) of the polynomial ring it belongs to.

source

`AbstractAlgebra.is_monic` – Method.

```
is_monic(a::PolynomialElem)
```

Return `true` if the given polynomial is monic, i.e. has leading coefficient equal to one, otherwise return `false`.

source

`Base.length` – Method.

```
length(a::PolynomialElem)
```

Return the length of the polynomial. The length of a univariate polynomial is defined to be the number of coefficients in its dense representation, including zero coefficients. Thus naturally the zero polynomial has length zero and additionally for nonzero polynomials the length is one more than the degree. (Note that the leading coefficient will always be nonzero.)

source

`AbstractAlgebra.degree` – Method.

```
degree(a::PolynomialElem)
```

Return the degree of the given polynomial. This is defined to be one less than the length, even for constant polynomials.

source

`AbstractAlgebra.is_monomial` – Method.

```
is_monomial(a::PolynomialElem)
```

Return `true` if the given polynomial is a monomial.

source

```
is_monomial(x::MPolyRingElem)
```

Return `true` if the given polynomial has precisely one term whose coefficient is one.

source

`AbstractAlgebra.is_monomial_recursive` – Method.

```
is_monomial_recursive(a::PolynomialElem)
```

Return `true` if the given polynomial is a monomial. This function is recursive, with all scalar types returning true.

source

`AbstractAlgebra.is_term` – Method.

```
is_term(a::PolynomialElem)
```

Return `true` if the given polynomial has one term.

source

```
is_term(x::MPolyRingElem)
```

Return `true` if the given polynomial has precisely one term.

source

`AbstractAlgebra.is_term_recursive` – Method.

```
is_term_recursive(a::PolynomialElem)
```

Return `true` if the given polynomial has one term. This function is recursive, with all scalar types returning true.

source

`AbstractAlgebra.is_constant` – Method.

```
is_constant(a::PolynomialElem)
```

Return `true` if a is a degree zero polynomial or the zero polynomial, i.e. a constant polynomial.

source

`AbstractAlgebra.is_separable` – Method.

```
is_separable(f::PolyRingElem) -> Bool
```

Return whether for a polynomial $f$ over a ring $R$ the quotient ring $R[x]/(f)$ is separable over $R$. If $R$ is a field, this is equivalent to $f$ having no repeated roots in an algebraic closure, or to $f$ being coprime to the formal derivative $f'$.

source

`AbstractAlgebra.is_unit` – Method.

```
is_unit(f::T) where {T <: PolyRingElem}
```

Return `true` if the given polynomial over a commutative ring is invertible, otherwise `false`.

**Examples**

```
julia> Z, x = ZZ[:x]
(Univariate polynomial ring in x over integers, x)

julia> t = x^2+1
x^2 + 1

julia> is_unit(t)
false

julia> f = Z(1)
1

julia> is_unit(f)
true
```

source

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> T, z = polynomial_ring(QQ, :z)
(Univariate polynomial ring in z over rationals, z)

julia> U, = residue_ring(ZZ, 17);

julia> V, w = polynomial_ring(U, :w)
(Univariate polynomial ring in w over U, w)

julia> var(R)
:x
```

```
julia> symbols(R)
1-element Vector{Symbol}:
 :x

julia> a = zero(S)
0

julia> b = one(S)
1

julia> isone(b)
true

julia> c = BigInt(1)//2*z^2 + BigInt(1)//3
1//2*z^2 + 1//3

julia> d = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> f = leading_coefficient(d)
x

julia> y = gen(S)
y

julia> g = is_gen(w)
true

julia> divexact((2x + 1)*(x + 1), (x + 1))
2*x + 1

julia> m = is_unit(b)
true

julia> n = degree(d)
2

julia> r = modulus(w)
17

julia> is_term(2y^2)
true

julia> is_monomial(y^2)
true

julia> is_monomial_recursive(x*y^2)
true

julia> is_monomial(x*y^2)
false

julia> S, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)
```

```
julia> f = x^3 + 3x + 1
x^3 + 3*x + 1

julia> g = S(BigInt[1, 2, 0, 1, 0, 0, 0]);

julia> n = length(f)
4

julia> c = coeff(f, 1)
3

julia> g = set_coefficient!(g, 2, ZZ(11))
x^3 + 11*x^2 + 2*x + 1

julia> g = set_coefficient!(g, 7, ZZ(4))
4*x^7 + x^3 + 11*x^2 + 2*x + 1
```

### Iterators

An iterator is provided to return the coefficients of a univariate polynomial. The iterator is called `coefficients` and allows iteration over the coefficients, starting with the term of degree zero (if there is one). Note that coefficients of each degree are given, even if they are zero. This is best illustrated by example.

#### Examples

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> f = x^2 + 2
x^2 + 2

julia> C = collect(coefficients(f))
3-element Vector{BigInt}:
 2
 0
 1

julia> for c in coefficients(f)
          println(c)
       end
2
0
1
```

### Truncation

`Base.truncate` – Method.

```
truncate(a::PolynomialElem, n::Int)
```

Return $a$ truncated to $n$ terms, i.e. the remainder upon division by $x^n$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> truncate(f, 1)
3
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> truncate(f, 1)
[3 0; 0 3]
```

source

AbstractAlgebra.mullow – Method.

```
mullow(a::PolyRingElem{T}, b::PolyRingElem{T}, n::Int) where T <: RingElement
```

Return $a \times b$ truncated to $n$ terms.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = (x + 1)*y + (x^3 + 2x + 2)
```

```
(x + 1)*y + x^3 + 2*x + 2

julia> mullow(f, g, 4)
(x^2 + x)*y^3 + (x^4 + 3*x^2 + 4*x + 1)*y^2 + (x^4 + x^3 + 2*x^2 + 7*x + 5)*y + 3*x^3 + 6*x + 6
```

source

### Reversal

Base.reverse – Method.

```
reverse(x::PolynomialElem, len::Int)
```

Return the reverse of the polynomial $x$, thought of as a polynomial of the given length (the polynomial will be notionally truncated or padded with zeroes before the leading term if necessary to match the specified length). The resulting polynomial is normalised. If `len` is negative we throw a `DomainError()`.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = reverse(f, 7)
3*y^6 + (x + 1)*y^5 + x*y^4

julia> h = reverse(f)
3*y^2 + (x + 1)*y + x
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> g = reverse(f, 7)
[3 0; 0 3]*y^6 + (x + 1)*y^5 + x*y^4
```

```
julia> h = reverse(f)
[3 0; 0 3]*y^2 + (x + 1)*y + x
```

source

Base.reverse – Method.

```
reverse(x::PolynomialElem)
```

Return the reverse of the polynomial $x$, i.e. the leading coefficient of $x$ becomes the constant coefficient of the result, etc. The resulting polynomial is normalised.

source

### Shifting

AbstractAlgebra.shift_left – Method.

```
shift_left(f::PolynomialElem, n::Int)
```

Return the polynomial $f$ shifted left by $n$ terms, i.e. multiplied by $x^n$, where $x$ is the variable of the ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = shift_left(f, 7)
x*y^9 + (x + 1)*y^8 + 3*y^7
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> g = shift_left(f, 7)
x*y^9 + (x + 1)*y^8 + [3 0; 0 3]*y^7
```

source

AbstractAlgebra.shift_right – Method.

```
shift_right(f::PolynomialElem, n::Int)
```

Return the polynomial $f$ shifted right by $n$ terms, i.e. divided by $x^n$, where $x$ is the variable of the ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> shift_right(f, 2)
x
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> shift_right(f, 2)
x
```

source

### Inflation and deflation

AbstractAlgebra.deflation – Method.

```
deflation(p::PolyRingElem)
```

Return a tuple `(shift, defl)` where `shift` is the exponent of the trailing term of $p$ and `defl` is the gcd of the distance between the exponents of the nonzero terms of $p$. If $p = 0$, both `shift` and `defl` will be zero.

source

`AbstractAlgebra.inflate` – Method.

```
inflate(f::PolyRingElem, shift::Int64, n::Int64) -> PolyRingElem
```

Given a polynomial $f$ in $x$, return $f(x^n) * x^j$, i.e. multiply all exponents by $n$ and shift $f$ left by $j$.

source

`AbstractAlgebra.inflate` – Method.

```
inflate(f::PolyRingElem, n::Int64) -> PolyRingElem
```

Given a polynomial $f$ in $x$, return $f(x^n)$, i.e. multiply all exponents by $n$.

source

`AbstractAlgebra.deflate` – Method.

```
deflate(f::PolyRingElem, shift::Int64, n::Int64) -> PolyRingElem
```

Given a polynomial $g$ in $x^n$ such that `f = g(x)*x^{shift}`, write $f$ as a polynomial in $x$, i.e. divide all exponents of $g$ by $n$.

source

`AbstractAlgebra.deflate` – Method.

```
deflate(f::PolyRingElem, n::Int64) -> PolyRingElem
```

Given a polynomial $f$ in $x^n$, write it as a polynomial in $x$, i.e. divide all exponents by $n$.

source

`AbstractAlgebra.deflate` – Method.

```
deflate(x::PolyRingElem) -> PolyRingElem, Int
```

Deflate the polynomial $f$ maximally, i.e. find the largest $n$ s.th. $f$ can be deflated by $n$, i.e. $f$ is actually a polynomial in $x^n$. Return $g, n$ where $g$ is the deflation of $f$.

source

### Square root

Methods for `is_square` and `sqrt` are provided for inputs of type `PolyRingElem`.

**Examples**

```
R, x = polynomial_ring(ZZ, :x)
g = x^2+6*x+1
sqrt(g^2)
```

### Change of base ring

`AbstractAlgebra.change_base_ring` – Method.

```
change_base_ring(R::Ring, p::PolyRingElem{<: RingElement}; parent::PolyRing)
```

Return the polynomial obtained by coercing the non-zero coefficients of `p` into `R`.

If the optional `parent` keyword is provided, the polynomial will be an element of `parent`. The caching of the parent object can be controlled via the `cached` keyword argument.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> g = x^3 + 6*x + 1
x^3 + 6*x + 1

julia> change_base_ring(GF(2), g)
x^3 + 1
```

source

`AbstractAlgebra.change_coefficient_ring` – Method.

```
change_coefficient_ring(R::Ring, p::PolyRingElem{<: RingElement}; parent::PolyRing)
```

Return the polynomial obtained by coercing the non-zero coefficients of `p` into `R`.

If the optional `parent` keyword is provided, the polynomial will be an element of `parent`. The caching of the parent object can be controlled via the `cached` keyword argument.

source

`AbstractAlgebra.map_coefficients` – Method.

```
map_coefficients(f, p::PolyRingElem{<: RingElement}; cached::Bool=true, parent::PolyRing)
```

Transform the polynomial `p` by applying `f` on each non-zero coefficient.

If the optional `parent` keyword is provided, the polynomial will be an element of `parent`. The caching of the parent object can be controlled via the `cached` keyword argument.

source

### Pseudodivision

Given two polynomials $a, b$, pseudodivision computes polynomials $q$ and $r$ with length$(r)$ < length$(b)$ such that $L^d a = bq + r$, where $d =$ length$(a)-$ length$(b) + 1$ and $L$ is the leading coefficient of $b$.

We call $q$ the pseudoquotient and $r$ the pseudoremainder.

`AbstractAlgebra.pseudorem` – Method.

```
pseudorem(f::PolyRingElem{T}, g::PolyRingElem{T}) where T <: RingElement
```

Return the pseudoremainder of $f$ divided by $g$. If $g = 0$ we throw a `DivideError()`.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = (x + 1)*y + (x^3 + 2x + 2)
(x + 1)*y + x^3 + 2*x + 2

julia> h = pseudorem(f, g)
x^7 + 3*x^5 + 2*x^4 + x^3 + 5*x^2 + 4*x + 1
```

source

`AbstractAlgebra.pseudodivrem` – Method.

```
pseudodivrem(f::PolyRingElem{T}, g::PolyRingElem{T}) where T <: RingElement
```

Return a tuple $(q, r)$ consisting of the pseudoquotient and pseudoremainder of $f$ divided by $g$. If $g = 0$ we throw a `DivideError()`.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = (x + 1)*y + (x^3 + 2x + 2)
(x + 1)*y + x^3 + 2*x + 2
```

```
julia> pseudodivrem(f, g)
((x^2 + x)*y - x^4 - x^2 + 1, x^7 + 3*x^5 + 2*x^4 + x^3 + 5*x^2 + 4*x + 1)
```

source

### Content and primitive part

`AbstractAlgebra.content` – Method.

```
content(a::PolyRingElem)
```

Return the content of $a$, i.e. the greatest common divisor of its coefficients.

source

`AbstractAlgebra.primpart` – Method.

```
primpart(a::PolyRingElem)
```

Return the primitive part of $a$, i.e. the polynomial divided by its content.

source

#### Examples

```
R, x = polynomial_ring(ZZ, :x)
S, y = polynomial_ring(R, :y)

k = x*y^2 + (x + 1)*y + 3

n = content(k)
p = primpart(k*(x^2 + 1))
```

### Evaluation, composition and substitution

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::PolyRingElem, b)
```

Evaluate the polynomial expression $a$ at the value $b$ and return the result.

source

`AbstractAlgebra.compose` – Method.

```
compose(f::PolyRingElem, g::PolyRingElem; inner)
```

Compose the polynomial $a$ with the polynomial $b$ and return the result.

- If `inner = :right`, then `f(g)` is returned.
- If `inner = :left`, then `g(f)` is returned.

source

`AbstractAlgebra.subst` – Method.

```
subst(f::PolyRingElem{T}, a::Any) where T <: RingElement
```

Evaluate the polynomial $f$ at $a$. Note that $a$ can be anything, whether a ring element or not.

source

We also overload the functional notation so that the polynomial $f$ can be evaluated at $a$ by writing $f(a)$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = (x + 1)*y + (x^3 + 2x + 2)
(x + 1)*y + x^3 + 2*x + 2

julia> M = R[x + 1 2x; x - 3 2x - 1]
[x + 1       2*x]
[x - 3   2*x - 1]

julia> k = evaluate(f, 3)
12*x + 6

julia> m = evaluate(f, x^2 + 2x + 1)
x^5 + 4*x^4 + 7*x^3 + 7*x^2 + 4*x + 4

julia> n = compose(f, g; inner = :second)
(x^3 + 2*x^2 + x)*y^2 + (2*x^5 + 2*x^4 + 4*x^3 + 9*x^2 + 6*x + 1)*y + x^7 + 4*x^5 + 5*x^4 + 5*x^3 +
↪  10*x^2 + 8*x + 5

julia> p = subst(f, M)
[3*x^3 - 3*x^2 + 3*x + 4       6*x^3 + 2*x^2 + 2*x]
[3*x^3 - 8*x^2 - 2*x - 3   6*x^3 - 8*x^2 + 2*x + 2]

julia> q = f(M)
```

```
[3*x^3 - 3*x^2 + 3*x + 4       6*x^3 + 2*x^2 + 2*x]
[3*x^3 - 8*x^2 - 2*x - 3   6*x^3 - 8*x^2 + 2*x + 2]

julia> r = f(23)
552*x + 26
```

### Derivative and integral

`AbstractAlgebra.derivative` – Method.

```
derivative(a::PolynomialElem)
```

Return the derivative of the polynomial $a$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> derivative(f)
2*x*y + x + 1
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> derivative(f)
[2 0; 0 2]*x*y + x + 1
```

source

```
derivative(f::AbsPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

source

```
derivative(f::RelPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> derivative(f)
1 + 9*x^2 + O(x^9)
```

source

```
derivative(f::MPolyRingElem{T}, j::Int) where {T <: RingElement}
```

Return the partial derivative of f with respect to $j$-th variable of the polynomial ring.

source

```
derivative(f::MPolyRingElem{T}, x::MPolyRingElem{T}) where T <: RingElement
```

Return the partial derivative of f with respect to x. The value x must be a generator of the polynomial ring of f.

**Examples**

```
julia> R, (x, y) = AbstractAlgebra.polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x*y + x + y + 1
x*y + x + y + 1

julia> derivative(f, x)
y + 1

julia> derivative(f, y)
x + 1

julia> derivative(f, 1)
y + 1

julia> derivative(f, 2)
x + 1
```

source

```
derivative(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the derivative of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> f = x^(5//3) + x^(7//3) + x^(11//3)
x^(5//3) + x^(7//3) + x^(11//3) + O(x^5)

julia> derivative(f)
5//3*x^(2//3) + 7//3*x^(4//3) + 11//3*x^(8//3) + O(x^4)
```

source

`AbstractAlgebra.integral` – Method.

```
integral(x::PolyRingElem{T}) where {T <: Union{ResElem, FieldElement}}
```

Return the integral of the polynomial $x$.

**Examples**

```
julia> T, z = polynomial_ring(QQ, :z)
(Univariate polynomial ring in z over rationals, z)

julia> U, = residue_ring(T, z^3 + 3z + 1);

julia> V, w = polynomial_ring(U, :w)
(Univariate polynomial ring in w over U, w)

julia> integral((z^2 + 2z + 1)*w^2 + (z + 1)*w - 2z + 4)
(1//3*z^2 + 2//3*z + 1//3)*w^3 + (1//2*z + 1//2)*w^2 + (-2*z + 4)*w
```

source

```
integral(f::AbsPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

source

```
integral(f::RelPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> integral(f)
2*x + 1//2*x^2 + 3//4*x^4 + O(x^11)
```

source

```
integral(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the integral of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> integral(x^(5//3) + x^(7//3) + x^(11//3))
3//8*x^(8//3) + 3//10*x^(10//3) + 3//14*x^(14//3) + O(x^6)
```

source

### Resultant and discriminant

`AbstractAlgebra.sylvester_matrix` – Method.

```
sylvester_matrix(p::PolyRingElem, q::PolyRingElem)
```

Return the sylvester matrix of the given polynomials.

source

`AbstractAlgebra.resultant` – Method.

```
resultant(p::PolyRingElem{T}, q::PolyRingElem{T}) where T <: RingElement
```

Return the resultant of the given polynomials.

source

`AbstractAlgebra.resx` – Method.

```
resx(a::PolyRingElem{T}, b::PolyRingElem{T}) where T <: RingElement
```

Return a tuple $(r, s, t)$ such that $r$ is the resultant of $a$ and $b$ and such that $r = a \times s + b \times t$.

source

AbstractAlgebra.discriminant – Method.

```
discriminant(a::PolyRingElem)
```

Return the discriminant of the given polynomial.

source

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = 3x*y^2 + (x + 1)*y + 3
3*x*y^2 + (x + 1)*y + 3

julia> g = 6(x + 1)*y + (x^3 + 2x + 2)
(6*x + 6)*y + x^3 + 2*x + 2

julia> S = sylvester_matrix(f, g)
[    3*x           x + 1               3]
[6*x + 6   x^3 + 2*x + 2               0]
[      0         6*x + 6   x^3 + 2*x + 2]

julia> h = resultant(f, g)
3*x^7 + 6*x^5 - 6*x^3 + 96*x^2 + 192*x + 96

julia> k = discriminant(f)
x^2 - 34*x + 1
```

### Newton representation

AbstractAlgebra.monomial_to_newton! – Method.

```
monomial_to_newton!(P::Vector{T}, roots::Vector{T}) where T <: RingElement
```

Converts a polynomial $p$, given as an array of coefficients, in-place from its coefficients given in the standard monomial basis to the Newton basis for the roots $r_0, r_1, \ldots, r_{n-2}$. In other words, this determines output coefficients $c_i$ such that $c_0 + c_1(x - r_0) + c_2(x - r_0)(x - r_1) + \ldots + c_{n-1}(x - r_0)(x - r_1)\cdots(x - r_{n-2})$ is equal to the input polynomial.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
```

```
(Univariate polynomial ring in y over R, y)

julia> f = 3x*y^2 + (x + 1)*y + 3
3*x*y^2 + (x + 1)*y + 3

julia> g = deepcopy(f)
3*x*y^2 + (x + 1)*y + 3

julia> roots = [R(1), R(2), R(3)]
3-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 1
 2
 3

julia> monomial_to_newton!(g.coeffs, roots)

julia> g
3*x*y^2 + (10*x + 1)*y + 4*x + 4
```

source

AbstractAlgebra.newton_to_monomial! – Method.

```
newton_to_monomial!(P::Vector{T}, roots::Vector{T}) where T <: RingElement
```

Converts a polynomial $p$, given as an array of coefficients, in-place from its coefficients given in the Newton basis for the roots $r_0, r_1, \ldots, r_{n-2}$ to the standard monomial basis. In other words, this evaluates $c_0 + c_1(x - r_0) + c_2(x - r_0)(x - r_1) + \ldots + c_{n-1}(x - r_0)(x - r_1)\cdots(x - r_{n-2})$ where $c_i$ are the input coefficients given by $p$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> c = [4x + 4, 10x + 1, 3x];

julia> newton_to_monomial!(c, [R(1), R(2), R(3)])

julia> c
3-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 3
 x + 1
 3*x
```

source

### Roots

AbstractAlgebra.Generic.roots – Method.

```
roots(f::PolyRingElem)
```

Returns the roots of the polynomial f in the base ring of f as an array.

source

AbstractAlgebra.Generic.roots – Method.

```
roots(R::Field, f::PolyRingElem)
```

Returns the roots of the polynomial f in the field R as an array.

source

### Interpolation

AbstractAlgebra.interpolate – Method.

```
interpolate(S::PolyRing, x::Vector{T}, y::Vector{T}) where T <: RingElement
```

Given two arrays of values $xs$ and $ys$ of the same length $n$, find the polynomial $f$ in the polynomial ring $R$ of length at most $n$ such that $f$ has the value $ys$ at the points $xs$. The values in the arrays $xs$ and $ys$ must belong to the base ring of the polynomial ring $R$. If no such polynomial exists, an exception is raised.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> xs = [R(1), R(2), R(3), R(4)]
4-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 1
 2
 3
 4

julia> ys = [R(1), R(4), R(9), R(16)]
4-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 1
 4
 9
 16

julia> f = interpolate(S, xs, ys)
y^2
```

source

### Power sums

AbstractAlgebra.polynomial_to_power_sums – Method.

```
polynomial_to_power_sums(f::PolyRingElem{T}, n::Int=degree(f)) where T <: RingElement ->
↪  Vector{T}
```

Uses Newton (or Newton-Girard) formulas to compute the first $n$ sums of powers of the roots of $f$ from the coefficients of $f$, starting with the sum of (first powers of) the roots. The input polynomial must be monic, at least degree $1$ and have nonzero constant coefficient.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> f = x^4 - 2*x^3 + 10*x^2 + 7*x - 5
x^4 - 2*x^3 + 10*x^2 + 7*x - 5

julia> V = polynomial_to_power_sums(f)
4-element Vector{BigInt}:
   2
 -16
 -73
  20
```

source

AbstractAlgebra.power_sums_to_polynomial – Method.

```
power_sums_to_polynomial(P::Vector{T};
                parent::PolyRing{T}=poly_ring(parent(P[1])) where T <: RingElement ->
                ↪  PolyRingElem{T}
```

Uses the Newton (or Newton-Girard) identities to obtain the polynomial with given sums of powers of roots. The list must be nonempty and contain `degree(f)` entries where $f$ is the polynomial to be recovered. The list must start with the sum of first powers of the roots.

**Examples**

```
julia> power_sums_to_polynomial(BigInt[2, -16, -73, 20])
x^4 - 2*x^3 + 10*x^2 + 7*x - 5
```

source

### Special functions

The following special functions can be computed for any polynomial ring. Typically one uses the generator $x$ of a polynomial ring to get the respective special polynomials expressed in terms of that generator.

AbstractAlgebra.chebyshev_t – Method.

```
chebyshev_t(n::Int, x::PolyRingElem)
```

Return the Chebyshev polynomial of the first kind $T_n(x)$, defined by $T_n(x) = \cos(n \cos^{-1}(x))$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = chebyshev_t(20, y)
524288*y^20 - 2621440*y^18 + 5570560*y^16 - 6553600*y^14 + 4659200*y^12 - 2050048*y^10 +
↪  549120*y^8 - 84480*y^6 + 6600*y^4 - 200*y^2 + 1
```

source

`AbstractAlgebra.chebyshev_u` – Method.

```
chebyshev_u(n::Int, x::PolyRingElem)
```

Return the Chebyshev polynomial of the first kind $U_n(x)$, defined by $(n + 1)U_n(x) = T'_{n+1}(x)$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> chebyshev_u(15, y)
32768*y^15 - 114688*y^13 + 159744*y^11 - 112640*y^9 + 42240*y^7 - 8064*y^5 + 672*y^3 - 16*y
```

source

### Random generation

One may generate random polynomials with degrees in a given range. Additional parameters are used to construct coefficients as elements of the coefficient ring.

```
rand(R::PolyRing, deg_range::AbstractUnitRange{Int}, v...)
rand(R::PolyRing, deg::Int, v...)
```

**Examples**

```
R, x = polynomial_ring(ZZ, :x)
f = rand(R, -1:3, -10:10)

S, y = polynomial_ring(GF(7), :y)
g = rand(S, 2:2)

U, z = polynomial_ring(R, :z)
h = rand(U, 3:3, -1:2, -10:10)
```

### Graeffe transform

`AbstractAlgebra.graeffe_transform` – Method.

```
graeffe_transform(f::PolyRingElem)
```

Return a polynomial whose roots are the squares of the roots of the given polynomial. Also known as Dandelin-Graeffe transform.

**Examples**

```
julia> ZZx, x = polynomial_ring(ZZ, :x);

julia> graeffe_transform(x^2-1)
x^2 - 2*x + 1

julia> graeffe_transform(x^3-x-1)
x^3 - 2*x^2 + x - 1
```

source

### Ring homomorphisms

`AbstractAlgebra.hom` – Method.

```
hom(R::AbstractAlgebra.PolyRing, S::NCRing, [coeff_map,] image)
```

Given a homomorphism `coeff_map` from `C` to `S`, where `C` is the coefficient ring of `R`, and given an element `image` of `S`, return the homomorphism from `R` to `S` whose restriction to `C` is `coeff_map`, and which sends the generator of `R` to `image`.

If no coefficient map is entered, invoke a canonical homomorphism of `C` to `S`, if such a homomorphism exists, and throw an error, otherwise.

**Examples**

```
julia> Zx, x = ZZ[:x];

julia> F = hom(Zx, Zx, x + 1);
```

```
julia> F(x^2)
x^2 + 2*x + 1

julia> Fp = GF(3); Fpy, y = Fp[:y];

julia> G = hom(Zx, Fpy, c -> Fp(c), y^3);

julia> G(5*x + 1)
2*y^3 + 1
```

source

# Chapter 18

# Univariate polynomials over a noncommutative ring

AbstractAlgebra.jl provides a module, implemented in `src/NCPoly.jl` for univariate polynomials over any noncommutative ring in the AbstractAlgebra type hierarchy.

## 18.1 Generic type for univariate polynomials over a noncommutative ring

AbstractAlgebra.jl implements a generic univariate polynomial type over noncommutative rings in `src/generic/NCPoly.jl`.

These generic polynomials have type `Generic.NCPoly{T}` where T is the type of elements of the coefficient ring. Internally they consist of a Julia array of coefficients and some additional fields for length and a parent object, etc. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of such polynomials have type `Generic.NCPolyRing{T}`.

The string representation of the variable of the polynomial ring and the base/coefficient ring $R$ is stored in the parent object.

## 18.2 Abstract types

The polynomial element types belong to the abstract type `NCPolyRingElem{T}` and the polynomial ring types belong to the abstract type `NCPolyRing{T}`. This enables one to write generic functions that can accept any AbstractAlgebra polynomial type.

**Note**

Note that both the generic polynomial ring type `Generic.NCPolyRing{T}` and the abstract type it belongs to, `NCPolyRing{T}` are both called `NCPolyRing`. The former is a (parameterised) concrete type for a polynomial ring over a given base ring whose elements have type T. The latter is an abstract type representing all polynomial ring types in AbstractAlgebra.jl, whether generic or very specialised (e.g. supplied by a C library).

## 18.3 Polynomial ring constructors

In order to construct polynomials in AbstractAlgebra.jl, one must first construct the polynomial ring itself. This is accomplished with the following constructor.

`AbstractAlgebra.polynomial_ring` – Method.

```
polynomial_ring(R::NCRing, s::VarName = :x; cached::Bool=true)
```

Given a base ring R and symbol/string s specifying how the generator (variable) should be printed, return a tuple `S, x` representing the new polynomial ring $S = R[x]$ and the generator $x$ of the ring.

By default the parent object S depends only on R and x and will be cached. Setting the optional argument cached to `false` will prevent the parent object S from being cached.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)
```

source

A shorthand version of this function is provided: given a base ring R, we abbreviate the constructor as follows.

```
R[:x]
```

Here are some examples of creating polynomial rings and making use of the resulting parent objects to coerce various elements into the polynomial ring.

**Examples**

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> U, z = R[:z]
(Univariate polynomial ring in z over matrix ring, z)

julia> f = S()
0

julia> g = S(123)
[123 0; 0 123]

julia> h = T(BigInt(1234))
[1234 0; 0 1234]

julia> k = T(x + 1)
x + 1
```

```
julia> m = U(z + 1)
z + 1
```

All of the examples here are generic polynomial rings, but specialised implementations of polynomial rings provided by external modules will also usually provide a `polynomial_ring` constructor to allow creation of their polynomial rings.

## 18.4 Basic ring functionality

Once a polynomial ring is constructed, there are various ways to construct polynomials in that ring.

The easiest way is simply using the generator returned by the `polynomial_ring` constructor and build up the polynomial using basic arithmetic, as described in the Ring interface.

The Julia language also has special syntax for the construction of polynomials in terms of a generator, e.g. we can write 2x instead of 2*x.

The polynomial rings in AbstractAlgebra.jl implement the full Ring interface. Of course the entire Univariate Polynomial Ring interface is also implemented.

We give some examples of such functionality.

**Examples**

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x^3 + 3x + 21
x^3 + [3 0; 0 3]*x + [21 0; 0 21]

julia> g = (x + 1)*y^2 + 2x + 1
(x + 1)*y^2 + [2 0; 0 2]*x + 1

julia> h = zero(T)
0

julia> k = one(S)
1

julia> isone(k)
true

julia> iszero(f)
false

julia> n = length(g)
3
```

```
julia> U = base_ring(T)
Univariate polynomial ring in x over matrix ring

julia> V = base_ring(y + 1)
Univariate polynomial ring in x over matrix ring

julia> v = var(T)
:y

julia> U = parent(y + 1)
Univariate polynomial ring in y over S

julia> g == deepcopy(g)
true
```

## 18.5 Polynomial functionality provided by AbstractAlgebra.jl

The functionality listed below is automatically provided by AbstractAlgebra.jl for any polynomial module that implements the full Univariate Polynomial Ring interface over a noncommutative ring. This includes AbstractAlgebra.jl's own generic polynomial rings.

But if a C library provides all the functionality documented in the Univariate Polynomial Ring interface over a noncommutative ring, then all the functions described here will also be automatically supplied by AbstractAlgebra.jl for that polynomial type.

Of course, modules are free to provide specific implementations of the functions described here, that override the generic implementation.

### Basic functionality

`AbstractAlgebra.leading_coefficient` – Method.

```
leading_coefficient(a::PolynomialElem)
```

Return the leading coefficient of the given polynomial. This will be the nonzero coefficient of the term with highest degree unless the polynomial in the zero polynomial, in which case a zero coefficient is returned.

source

`AbstractAlgebra.trailing_coefficient` – Method.

```
trailing_coefficient(a::PolynomialElem)
```

Return the trailing coefficient of the given polynomial. This will be the nonzero coefficient of the term with lowest degree unless the polynomial is the zero polynomial, in which case a zero coefficient is returned.

source

`AbstractAlgebra.gen` – Method.

```
gen(R::NCPolyRing)
```

Return the generator of the given polynomial ring.

source

AbstractAlgebra.is_gen – Method.

```
is_gen(a::PolynomialElem)
```

Return `true` if the given polynomial is the constant generator of its polynomial ring, otherwise return `false`.

source

AbstractAlgebra.is_monomial – Method.

```
is_monomial(a::PolynomialElem)
```

Return `true` if the given polynomial is a monomial.

source

AbstractAlgebra.is_term – Method.

```
is_term(a::PolynomialElem)
```

Return `true` if the given polynomial has one term.

source

**Examples**

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> a = zero(T)
0

julia> b = one(T)
1

julia> c = BigInt(1)*y^2 + BigInt(1)
y^2 + 1
```

```
julia> d = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> f = leading_coefficient(d)
x

julia> y = gen(T)
y

julia> g = is_gen(y)
true

julia> n = degree(d)
2

julia> is_term(2y^2)
true

julia> is_monomial(y^2)
true
```

### Truncation

Base.truncate – Method.

```
truncate(a::PolynomialElem, n::Int)
```

Return $a$ truncated to $n$ terms, i.e. the remainder upon division by $x^n$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> truncate(f, 1)
3
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
```

```
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> truncate(f, 1)
[3 0; 0 3]
```

source

AbstractAlgebra.mullow – Method.

```
mullow(a::NCPolyRingElem{T}, b::NCPolyRingElem{T}, n::Int) where T <: NCRingElem
```

Return $a \times b$ truncated to $n$ terms.

**Examples**

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> g = (x + 1)*y + (x^3 + 2x + 2)
(x + 1)*y + x^3 + [2 0; 0 2]*x + [2 0; 0 2]

julia> mullow(f, g, 4)
(x^2 + x)*y^3 + (x^4 + [3 0; 0 3]*x^2 + [4 0; 0 4]*x + 1)*y^2 + (x^4 + x^3 + [2 0; 0 2]*x^2 + [7
↪  0; 0 7]*x + [5 0; 0 5])*y + [3 0; 0 3]*x^3 + [6 0; 0 6]*x + [6 0; 0 6]
```

source

### Reversal

Base.reverse – Method.

```
reverse(x::PolynomialElem, len::Int)
```

Return the reverse of the polynomial $x$, thought of as a polynomial of the given length (the polynomial will be notionally truncated or padded with zeroes before the leading term if necessary to match the specified length). The resulting polynomial is normalised. If `len` is negative we throw a `DomainError()`.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = reverse(f, 7)
3*y^6 + (x + 1)*y^5 + x*y^4

julia> h = reverse(f)
3*y^2 + (x + 1)*y + x
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> g = reverse(f, 7)
[3 0; 0 3]*y^6 + (x + 1)*y^5 + x*y^4

julia> h = reverse(f)
[3 0; 0 3]*y^2 + (x + 1)*y + x
```

source

`Base.reverse` – Method.

```
reverse(x::PolynomialElem)
```

Return the reverse of the polynomial $x$, i.e. the leading coefficient of $x$ becomes the constant coefficient of the result, etc. The resulting polynomial is normalised.

source

### Shifting

AbstractAlgebra.shift_left – Method.

```
shift_left(f::PolynomialElem, n::Int)
```

Return the polynomial $f$ shifted left by $n$ terms, i.e. multiplied by $x^n$, where $x$ is the variable of the ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> g = shift_left(f, 7)
x*y^9 + (x + 1)*y^8 + 3*y^7
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> g = shift_left(f, 7)
x*y^9 + (x + 1)*y^8 + [3 0; 0 3]*y^7
```

source

AbstractAlgebra.shift_right – Method.

```
shift_right(f::PolynomialElem, n::Int)
```

Return the polynomial $f$ shifted right by $n$ terms, i.e. divided by $x^n$, where $x$ is the variable of the ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> shift_right(f, 2)
x
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> shift_right(f, 2)
x
```

source

### Evaluation

AbstractAlgebra.evaluate – Method.

```
evaluate(a::NCPolyRingElem, b::NCRingElement)
```

Evaluate the polynomial $a$ at the value $b$ and return the result.

source

We also overload the functional notation so that the polynomial $f$ can be evaluated at $a$ by writing $f(a)$.

#### Examples

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)
```

```
julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)


julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> k = evaluate(f, 3)
[12 0; 0 12]*x + [6 0; 0 6]

julia> m = evaluate(f, x^2 + 2x + 1)
x^5 + [4 0; 0 4]*x^4 + [7 0; 0 7]*x^3 + [7 0; 0 7]*x^2 + [4 0; 0 4]*x + [4 0; 0 4]

julia> r = f(23)
[552 0; 0 552]*x + [26 0; 0 26]
```

### Derivative

`AbstractAlgebra.derivative` – Method.

```
derivative(a::PolynomialElem)
```

Return the derivative of the polynomial $a$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> f = x*y^2 + (x + 1)*y + 3
x*y^2 + (x + 1)*y + 3

julia> derivative(f)
2*x*y + x + 1
```

```
julia> R = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> S, x = polynomial_ring(R, :x)
(Univariate polynomial ring in x over matrix ring, x)

julia> T, y = polynomial_ring(S, :y)
(Univariate polynomial ring in y over S, y)

julia> f = x*y^2 + (x + 1)*y + 3
```

```
x*y^2 + (x + 1)*y + [3 0; 0 3]

julia> derivative(f)
[2 0; 0 2]*x*y + x + 1
```

source

# Chapter 19

# Multivariate polynomials

AbstractAlgebra.jl provides sparse distributed multivariate polynomials over any commutative ring belonging to the AbstractAlgebra abstract type hierarchy. This is the implementation that should be used for most applications of multivariate polynomials.

## 19.1 Abstract types

The polynomial element types belong to the abstract type `MPolyRingElem{T}` and the polynomial ring types belong to the abstract type `MPolyRing{T}`.

**Note**

Note that both the generic polynomial ring type `Generic.MPolyRing{T}` and the abstract type it belongs to, `MPolyRing{T}` are both called `MPolyRing`. The former is a (parameterised) concrete type for a polynomial ring over a given base ring whose elements have type T. The latter is an abstract type representing all multivariate polynomial ring types in AbstractAlgebra.jl, whether generic or very specialised (e.g. supplied by a C library).

## 19.2 Polynomial ring constructors

In order to construct multivariate polynomials in AbstractAlgebra.jl, one must first construct the polynomial ring itself. This is accomplished with the following constructors.

`AbstractAlgebra.polynomial_ring` – Method.

```
polynomial_ring(R::Ring, varnames::Vector{Symbol}; cached=true, internal_ordering=:lex)
```

Given a coefficient ring R and variable names, say `varnames = [:x1, :x2, ...]`, return a tuple `S, [x1, x2, ...]` of the polynomial ring $S = R[x1, x2, \ldots]$ and its generators $x1, x2, \ldots$.

By default (`cached=true`), the output S will be cached, i.e. if `polynomial_ring` is invoked again with the same arguments, the same (*identical*) ring is returned. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

The monomial ordering used for the internal storage of polynomials in S can be set with `internal_ordering` and must be one of `:lex`, `:deglex` or `:degrevlex`.

See also: `polynomial_ring(::Ring, ::Vararg)`, `@polynomial_ring`.

**Examples**

```
julia> S, generators = polynomial_ring(ZZ, [:x, :y, :z])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y, z])
```

source

`AbstractAlgebra.polynomial_ring` – Method.

```
polynomial_ring(R::Ring, varnames...; cached=true, internal_ordering=:lex)
polynomial_ring(R::Ring, varnames::Tuple; cached=true, internal_ordering=:lex)
```

Like `polynomial_ring(::Ring, ::Vector{Symbol})` with more ways to give `varnames` as specified in `variable_names`.

Return a tuple `S, generators...` with `generators[i]` corresponding to `varnames[i]`.

**Note**

In the first method, `varnames` must not be empty, and if it consists of only one name, the univariate `polynomial_ring(R::NCRing, s::VarName)` method is called instead.

**Examples**

```
julia> S, (a, b, c) = polynomial_ring(ZZ, [:a, :b, :c])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[a, b, c])

julia> S, x, y = polynomial_ring(ZZ, :x => (1:2, 1:2), :y => 1:3);

julia> S
Multivariate polynomial ring in 7 variables x[1, 1], x[2, 1], x[1, 2], x[2, 2], ..., y[3]
  over integers

julia> x
2×2 Matrix{AbstractAlgebra.Generic.MPoly{BigInt}}:
 x[1, 1]  x[1, 2]
 x[2, 1]  x[2, 2]

julia> y
3-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 y[1]
 y[2]
 y[3]
```

source

`AbstractAlgebra.polynomial_ring` – Method.

```
polynomial_ring(R::Ring, varnames...; cached=true, internal_ordering=:lex)
polynomial_ring(R::Ring, varnames::Tuple; cached=true, internal_ordering=:lex)
```

Like `polynomial_ring(::Ring, ::Vector{Symbol})` with more ways to give `varnames` as specified in `variable_names`.

Return a tuple `S, generators...` with `generators[i]` corresponding to `varnames[i]`.

**Note**

In the first method, `varnames` must not be empty, and if it consists of only one name, the univariate `polynomial_ring(R::NCRing, s::VarName)` method is called instead.

**Examples**

```
julia> S, (a, b, c) = polynomial_ring(ZZ, [:a, :b, :c])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[a, b, c])

julia> S, x, y = polynomial_ring(ZZ, :x => (1:2, 1:2), :y => 1:3);

julia> S
Multivariate polynomial ring in 7 variables x[1, 1], x[2, 1], x[1, 2], x[2, 2], ..., y[3]
  over integers

julia> x
2×2 Matrix{AbstractAlgebra.Generic.MPoly{BigInt}}:
 x[1, 1]  x[1, 2]
 x[2, 1]  x[2, 2]

julia> y
3-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 y[1]
 y[2]
 y[3]
```

source

```
polynomial_ring(R::Ring, n::Int, s::Symbol=:x; cached=true, internal_ordering=:lex)
```

Same as `polynomial_ring(::Ring, ["s$i" for i in 1:n])`.

**Examples**

```
julia> S, x = polynomial_ring(ZZ, 3)
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x1, x2, x3])
```

source

`AbstractAlgebra.@polynomial_ring` – Macro.

```
@polynomial_ring(R::Ring, varnames...; cached=true, internal_ordering=:lex)
```

Return polynomial ring from `polynomial_ring(::Ring, ::Vararg)` and introduce the generators into the current scope.

**Examples**

```
julia> S = @polynomial_ring(ZZ, "x#" => (1:2, 1:2), "y#" => 1:3)
Multivariate polynomial ring in 7 variables x11, x21, x12, x22, ..., y3
  over integers

julia> x11, x21, x12, x22
(x11, x21, x12, x22)

julia> y1, y2, y3
(y1, y2, y3)

julia> (S, [x11 x12; x21 x22], [y1, y2, y3]) == polynomial_ring(ZZ, "x#" => (1:2, 1:2), "y#" =>
↪  1:3)
true
```

source

Like for univariate polynomials, a shorthand constructor is provided when the number of generators is greater than 1: given a base ring R, we abbreviate the constructor as follows:

```
R[:x, :y, ...]
```

In addition to that, it is also possible to construct univariate polynomial rings over other univariate polynomial rings in a similar fashion:

```
R[:x][:y]...
```

Here are some examples of creating multivariate polynomial rings and making use of the resulting parent objects to coerce various elements into the polynomial ring.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y]; internal_ordering=:deglex)
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> T, (z, t) = QQ[:z, :t]
(Multivariate polynomial ring in 2 variables over rationals,
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}[z, t])

julia> f = R()
0

julia> g = R(123)
```

```
123

julia> h = R(BigInt(1234))
1234

julia> k = R(x + 1)
x + 1

julia> m = R(x + y + 1)
x + y + 1

julia> derivative(k, 1)
1

julia> derivative(k, 2)
0

julia> R, x = polynomial_ring(ZZ, 10); R
Multivariate polynomial ring in 10 variables x1, x2, x3, x4, ..., x10
  over integers

julia> T, (z, t) = QQ[:z][:t]
(Univariate polynomial ring in t over univariate polynomial ring,
↪  AbstractAlgebra.Generic.Poly{AbstractAlgebra.Generic.Poly{Rational{BigInt}}}[z, t])
```

## 19.3 Polynomial constructors

Multivariate polynomials can be constructed from the generators in the usual way using arithmetic operations.

Also, all of the standard ring element constructors may be used to construct multivariate polynomials.

```
(R::MPolyRing{T})() where T <: RingElement
(R::MPolyRing{T})(c::Integer) where T <: RingElement
(R::MPolyRing{T})(a::elem_type(R)) where T <: RingElement
(R::MPolyRing{T})(a::T) where T <: RingElement
```

For more efficient construction of multivariate polynomial, one can use the `MPoly` build context, where terms (coefficient followed by an exponent vector) are pushed onto a context one at a time and then the polynomial constructed from those terms in one go using the `finish` function.

`AbstractAlgebra.Generic.MPolyBuildCtx` – Method.

```
MPolyBuildCtx(R::MPolyRing)
```

Return a build context for creating polynomials in the given ring.

source

`AbstractAlgebra.Generic.push_term!` – Method.

```
push_term!(M::MPolyBuildCtx, c::RingElem, v::Vector{Int})
```

Add the term with coefficient c and exponent vector v to the polynomial under construction in the build context M.

source

AbstractAlgebra.Generic.finish – Method.

```
finish(M::MPolyBuildCtx)
```

Finish construction of the polynomial, sort the terms, remove duplicate and zero terms and return the created polynomial.

source

Note that the finish function resets the build context so that it can be used to construct multiple polynomials..

When a multivariate polynomial type has a representation that allows constant time access (e.g. it is represented internally by arrays), the following additional constructor is available. It takes and array of coefficients and and array of exponent vectors.

```
(S::MPolyRing{T})(A::Vector{T}, m::Vector{Vector{Int}}) where T <: RingElem
```

Create the polynomial in the given ring with nonzero coefficients specified by the elements of $A$ and corresponding exponent vectors given by the elements of $m$.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> C = MPolyBuildCtx(R)
Builder for an element of R

julia> push_term!(C, ZZ(3), [1, 2]);


julia> push_term!(C, ZZ(2), [1, 1]);


julia> push_term!(C, ZZ(4), [0, 0]);


julia> f = finish(C)
3*x*y^2 + 2*x*y + 4

julia> push_term!(C, ZZ(4), [1, 1]);


julia> f = finish(C)
```

```
4*x*y

julia> S, (x, y) = polynomial_ring(QQ, [:x, :y])
(Multivariate polynomial ring in 2 variables over rationals,
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}[x, y])

julia> f = S(Rational{BigInt}[2, 3, 1], [[3, 2], [1, 0], [0, 1]])
2*x^3*y^2 + 3*x + y
```

## 19.4 Functions for types and parents of multivariate polynomial rings

```
base_ring(R::MPolyRing)
base_ring(a::MPolyRingElem)
```

Return the coefficient ring of the given polynomial ring or polynomial, respectively.

```
parent(a::MPolyRingElem)
```

Return the polynomial ring of the given polynomial.

## 19.5 Polynomial functions

### Basic manipulation

All the standard ring functions are available, including the following.

```
zero(R::MPolyRing)
one(R::MPolyRing)
iszero(a::MPolyRingElem)
isone(a::MPolyRingElem)
```

```
divexact(a::T, b::T) where T <: MPolyRingElem
```

All basic functions from the Multivariate Polynomial interface are provided.

```
symbols(S::MPolyRing)
number_of_variables(f::MPolyRing)
gens(S::MPolyRing)
gen(S::MPolyRing, i::Int)
```

```
internal_ordering(S::MPolyRing{T})
```

Note that the currently supported orderings are `:lex`, `:deglex` and `:degrevlex`.

```
length(f::MPolyRingElem)
degrees(f::MPolyRingElem)
total_degree(f::MPolyRingElem)
```

```
is_gen(x::MPolyRingElem)
```

```
divexact(f::T, g::T) where T <: MPolyRingElem
```

For multivariate polynomial types that allow constant time access to coefficients, the following are also available, allowing access to the given coefficient, monomial or term. Terms are numbered from the most significant first.

```
coeff(f::MPolyRingElem, n::Int)
coeff(a::MPolyRingElem, exps::Vector{Int})
```

Access a coefficient by term number or exponent vector.

```
monomial(f::MPolyRingElem, n::Int)
monomial!(m::T, f::T, n::Int) where T <: MPolyRingElem
```

The second version writes the result into a preexisting polynomial object to save an allocation.

```
term(f::MPolyRingElem, n::Int)
```

```
exponent(f::MyMPolyRingElem, i::Int, j::Int)
```

Return the exponent of the $j$-th variable in the $i$-th term of the polynomial $f$.

```
exponent_vector(a::MPolyRingElem, i::Int)
```

```
setcoeff!(a::MPolyRingElem{T}, exps::Vector{Int}, c::T) where T <: RingElement
```

Although multivariate polynomial rings are not usually Euclidean, the following functions from the Euclidean interface are often provided.

```
divides(f::T, g::T) where T <: MPolyRingElem
remove(f::T, g::T) where T <: MPolyRingElem
valuation(f::T, g::T) where T <: MPolyRingElem
```

```
divrem(f::T, g::T) where T <: MPolyRingElem
div(f::T, g::T) where T <: MPolyRingElem
```

Compute a tuple $(q, r)$ such that $f = qg+r$, where the coefficients of terms of $r$ whose monomials are divisible by the leading monomial of $g$ are reduced modulo the leading coefficient of $g$ (according to the Euclidean function on the coefficients). The `divrem` version returns both quotient and remainder whilst the `div` version only returns the quotient.

Note that the result of these functions depend on the ordering of the polynomial ring.

```
gcd(f::T, g::T) where T <: MPolyRingElem
```

The following functionality is also provided for all multivariate polynomials.

`AbstractAlgebra.is_univariate` – Method.

```
is_univariate(R::MPolyRing)
```

Returns `true` if $R$ is a univariate polynomial ring, i.e. has exactly one variable, and `false` otherwise.

source

`AbstractAlgebra.var_indices` – Method.

```
var_indices(p::MPolyRingElem{T}) where {T <: RingElement}
```

Return the indices of the variables actually occurring in $p$.

source

`AbstractAlgebra.vars` – Method.

```
vars(p::MPolyRingElem{T}) where {T <: RingElement}
```

Return the variables actually occurring in $p$.

source

`AbstractAlgebra.var_index` – Method.

```
var_index(x::MPolyRingElem{T}) where {T <: RingElement}
```

Return the index of the given variable $x$. If $x$ is not a variable in a multivariate polynomial ring, an exception is raised.

source

`AbstractAlgebra.degree` – Method.

```
degree(f::MPolyRingElem{T}, i::Int) where T <: RingElement
```

Return the degree of the polynomial $f$ in terms of the i-th variable.

source

AbstractAlgebra.degree – Method.

```
degree(f::MPolyRingElem{T}, x::MPolyRingElem{T}) where T <: RingElement
```

Return the degree of the polynomial $f$ in terms of the variable $x$.

source

AbstractAlgebra.degrees – Method.

```
degrees(f::MPolyRingElem{T}) where T <: RingElement
```

Return an array of the degrees of the polynomial $f$ in terms of each variable.

source

AbstractAlgebra.is_constant – Method.

```
is_constant(x::MPolyRingElem{T}) where T <: RingElement
```

Return `true` if x is a degree zero polynomial or the zero polynomial, i.e. a constant polynomial.

source

AbstractAlgebra.is_term – Method.

```
is_term(x::MPolyRingElem)
```

Return `true` if the given polynomial has precisely one term.

source

AbstractAlgebra.is_monomial – Method.

```
is_monomial(x::MPolyRingElem)
```

Return `true` if the given polynomial has precisely one term whose coefficient is one.

source

AbstractAlgebra.is_univariate – Method.

```
is_univariate(p::MPolyRingElem)
```

Returns `true` if $p$ is a univariate polynomial, i.e. involves at most one variable (thus constant polynomials are considered univariate), and `false` otherwise. The result depends on the terms of the polynomial, not simply on the number of variables in the polynomial ring.

source

`AbstractAlgebra.coeff` – Method.

```
coeff(f::MPolyRingElem{T}, m::MPolyRingElem{T}) where T <: RingElement
```

Return the coefficient of the monomial $m$ of the polynomial $f$. If there is no such monomial, zero is returned.

source

`AbstractAlgebra.monomial` – Method.

```
monomial(R::MPolyRing, exps::Vector{Int})
```

Return the monomial in R corresponding to the given exponent vector.

source

`AbstractAlgebra.term` – Method.

```
term(R::MPolyRing, c::RingElement, exps::Vector{Int})
```

Return the term in R corresponding to the given exponent vector with coefficient c.

source

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x^2 + 2x + 1
x^2 + 2*x + 1

julia> V = vars(f)
1-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 x

julia> var_index(y) == 2
true

julia> degree(f, x) == 2
true
```

```
julia> degree(f, 2) == 0
true

julia> d = degrees(f)
2-element Vector{Int64}:
 2
 0

julia> is_constant(R(1))
true

julia> is_term(2x)
true

julia> is_monomial(y)
true

julia> is_unit(R(1))
true

julia> S, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x^3*y + 3x*y^2 + 1
x^3*y + 3*x*y^2 + 1

julia> c1 = coeff(f, 1)
1

julia> c2 = coeff(f, x^3*y)
1

julia> m = monomial(f, 2)
x*y^2

julia> e1 = exponent(f, 1, 1)
3

julia> v1 = exponent_vector(f, 1)
2-element Vector{Int64}:
 3
 1

julia> t1 = term(f, 1)
x^3*y

julia> setcoeff!(f, [3, 1], 12)
12*x^3*y + 3*x*y^2 + 1

julia> S, (x, y) = polynomial_ring(QQ, [:x, :y]; internal_ordering=:deglex)
(Multivariate polynomial ring in 2 variables over rationals,
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}[x, y])
```

```
julia> V = symbols(S)
2-element Vector{Symbol}:
 :x
 :y

julia> X = gens(S)
2-element Vector{AbstractAlgebra.Generic.MPoly{Rational{BigInt}}}:
 x
 y

julia> ord = internal_ordering(S)
:deglex

julia> S, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x^3*y + 3x*y^2 + 1
x^3*y + 3*x*y^2 + 1

julia> n = length(f)
3

julia> is_gen(y)
true

julia> number_of_variables(S) == 2
true

julia> d = total_degree(f)
4

julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = 2x^2*y + 2x + y + 1
2*x^2*y + 2*x + y + 1

julia> g = x^2*y^2 + 1
x^2*y^2 + 1

julia> flag, q = divides(f*g, f)
(true, x^2*y^2 + 1)

julia> d = divexact(f*g, f)
x^2*y^2 + 1

julia> v, q = remove(f*g^3, g)
(3, 2*x^2*y + 2*x + y + 1)

julia> n = valuation(f*g^3, g)
3

julia> R, (x, y) = polynomial_ring(QQ, [:x, :y])
```

```
(Multivariate polynomial ring in 2 variables over rationals,
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}[x, y])

julia> f = 3x^2*y^2 + 2x + 1
3*x^2*y^2 + 2*x + 1

julia> f1 = divexact(f, 5)
3//5*x^2*y^2 + 2//5*x + 1//5

julia> f2 = divexact(f, QQ(2, 3))
9//2*x^2*y^2 + 3*x + 3//2
```

### Square root

Over rings for which an exact square root is available, it is possible to take the square root of a polynomial or test whether it is a square.

```
sqrt(f::MPolyRingElem, check::Bool=true)
is_square(::MPolyRingElem)
```

#### Examples

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = -4*x^5*y^4 + 5*x^5*y^3 + 4*x^4 - x^3*y^4
-4*x^5*y^4 + 5*x^5*y^3 + 4*x^4 - x^3*y^4

julia> sqrt(f^2)
4*x^5*y^4 - 5*x^5*y^3 - 4*x^4 + x^3*y^4

julia> is_square(f)
false
```

### Iterators

The following iterators are provided for multivariate polynomials.

```
coefficients(p::MPoly)
monomials(p::MPoly)
terms(p::MPoly)
exponent_vectors(a::MPoly)
```

#### Examples

```
julia> S, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])
```

```
julia> f = x^3*y + 3x*y^2 + 1
x^3*y + 3*x*y^2 + 1

julia> C = collect(coefficients(f))
3-element Vector{BigInt}:
 1
 3
 1

julia> M = collect(monomials(f))
3-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 x^3*y
 x*y^2
 1

julia> T = collect(terms(f))
3-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 x^3*y
 3*x*y^2
 1

julia> V = collect(exponent_vectors(f))
3-element Vector{Vector{Int64}}:
 [3, 1]
 [1, 2]
 [0, 0]
```

### Changing base (coefficient) rings

In order to substitute the variables of a polynomial $f$ over a ring $T$ by elements in a $T$-algebra $S$, you first have to change the base ring of $f$ using the following function, where $g$ is a function representing the structure homomorphism of the $T$-algebra $S$.

`AbstractAlgebra.change_base_ring` – Method.

```
change_base_ring(R::Ring, p::MPolyRingElem{<: RingElement}; parent::MPolyRing,
↪  cached::Bool=true)
```

Return the polynomial obtained by coercing the non-zero coefficients of p into R.

If the optional `parent` keyword is provided, the polynomial will be an element of `parent`. The caching of the parent object can be controlled via the `cached` keyword argument.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> fz = x^2*y^2 + x + 1
x^2*y^2 + x + 1
```

```
julia> fq = change_base_ring(QQ, fz)
x^2*y^2 + x + 1
```

source

AbstractAlgebra.change_coefficient_ring – Method.

```
change_coefficient_ring(R::Ring, p::MPolyRingElem{<: RingElement}; parent::MPolyRing,
↪  cached::Bool=true)
```

Return the polynomial obtained by coercing the non-zero coefficients of p into R.

If the optional parent keyword is provided, the polynomial will be an element of parent. The caching of the parent object can be controlled via the cached keyword argument.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> change_coefficient_ring(QQ, x^2*y^2 + x + 1)
x^2*y^2 + x + 1
```

source

AbstractAlgebra.map_coefficients – Method.

```
map_coefficients(f, p::MPolyRingElem{<: RingElement}; parent::MPolyRing)
```

Transform the polynomial p by applying f on each non-zero coefficient.

If the optional parent keyword is provided, the polynomial will be an element of parent. The caching of the parent object can be controlled via the cached keyword argument.

source

In case a specific parent ring is constructed, it can also be passed to the function.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> S,  = polynomial_ring(QQ, [:x, :y])
(Multivariate polynomial ring in 2 variables over rationals,
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}[x, y])

julia> fz = x^5 + y^3 + 1
x^5 + y^3 + 1
```

```
julia> fq = change_base_ring(QQ, fz, parent=S)
x^5 + y^3 + 1
```

### Multivariate coefficients

In order to return the "coefficient" (as a multivariate polynomial in the same ring), of a given monomial (in which some of the variables may not appear and others may be required to appear to exponent zero), we can use the following function.

`AbstractAlgebra.coeff` – Method.

```
coeff(a::MPolyRingElem{T}, vars::Vector{Int}, exps::Vector{Int}) where T <: RingElement
```

Return the "coefficient" of $a$ (as a multivariate polynomial in the same ring) of the monomial consisting of the product of the variables of the given indices raised to the given exponents (note that not all variables need to appear and the exponents can be zero). E.g. `coeff(f, [1, 3], [0, 2])` returns the coefficient of $x^0 * z^2$ in the polynomial $f$ (assuming variables $x, y, z$ in that order).

**Examples**

```
julia> R, (x, y, z) = polynomial_ring(ZZ, [:x, :y, :z])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y, z])

julia> f = x^4*y^2*z^2 - 2x^4*y*z^2 + 4x^4*z^2 + 2x^2*y^2 + x + 1
x^4*y^2*z^2 - 2*x^4*y*z^2 + 4*x^4*z^2 + 2*x^2*y^2 + x + 1

julia> coeff(f, [1, 3], [4, 2]) == coeff(f, [x, z], [4, 2])
true
```

source

`AbstractAlgebra.coeff` – Method.

```
coeff(a::T, vars::Vector{T}, exps::Vector{Int}) where T <: MPolyRingElem
```

Return the "coefficient" of $a$ (as a multivariate polynomial in the same ring) of the monomial consisting of the product of the given variables to the given exponents (note that not all variables need to appear and the exponents can be zero). E.g. `coeff(f, [x, z], [0, 2])` returns the coefficient of $x^0 * z^2$ in the polynomial $f$.

source

### Inflation/deflation

`AbstractAlgebra.deflation` – Method.

```
deflation(f::MPolyRingElem{T}) where T <: RingElement
```

Compute deflation parameters for the exponents of the polynomial $f$. This is a pair of arrays of integers, the first array of which (the shift) gives the minimum exponent for each variable of the polynomial, and the second of which (the deflation) gives the gcds of all the exponents after subtracting the shift, again per variable. This functionality is used by gcd (and can be used by factorisation algorithms).

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x^7*y^8 + 3*x^4*y^8 - x^4*y^2 + 5x*y^5 - x*y^2
x^7*y^8 + 3*x^4*y^8 - x^4*y^2 + 5*x*y^5 - x*y^2

julia> shift, defl = deflation(f)
([1, 2], [3, 3])
```

source

AbstractAlgebra.deflate – Method.

```
deflate(f::MPolyRingElem{T}, shift::Vector{Int}, defl::Vector{Int}) where T <: RingElement
```

Return a polynomial with the same coefficients as $f$ but whose exponents have been reduced by the given shifts (supplied as an array of shifts, one for each variable), then deflated (divided) by the given exponents (again supplied as an array of deflation factors, one for each variable). The algorithm automatically replaces a deflation of $0$ by $1$, to avoid division by $0$.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> deflate(x^7*y^8 + 3*x^4*y^8 - x^4*y^2 + 5x*y^5 - x*y^2, [1, 2], [3, 3])
x^2*y^2 + 3*x*y^2 - x + 5*y - 1
```

source

AbstractAlgebra.deflate – Method.

```
deflate(f::MPolyRingElem{T}, defl::Vector{Int}) where T <: RingElement
```

Return a polynomial with the same coefficients as $f$ but whose exponents have been deflated (divided) by the given exponents (supplied as an array of deflation factors, one for each variable).

The algorithm automatically replaces a deflation of $0$ by $1$, to avoid division by $0$.

source

AbstractAlgebra.deflate – Method.

```
deflate(f::MPolyRingElem{T}, defl::Vector{Int}) where T <: RingElement
```

Return a polynomial with the same coefficients as $f$ but whose exponents have been deflated maximally, i.e. with each exponent divide by the largest integer which divides the degrees of all exponents of that variable in $f$.

source

AbstractAlgebra.deflate – Method.

```
deflate(f::MPolyRingElem, vars::Vector{Int}, shift::Vector{Int}, defl::Vector{Int})
```

Return a polynomial with the same coefficients as $f$ but where exponents of some variables (supplied as an array of variable indices) have been reduced by the given shifts (supplied as an array of shifts), then deflated (divided) by the given exponents (again supplied as an array of deflation factors). The algorithm automatically replaces a deflation of $0$ by $1$, to avoid division by $0$.

source

AbstractAlgebra.deflate – Method.

```
deflate(f::T, vars::Vector{T}, shift::Vector{Int}, defl::Vector{Int}) where T <: MPolyRingElem
```

Return a polynomial with the same coefficients as $f$ but where the exponents of the given variables have been reduced by the given shifts (supplied as an array of shifts), then deflated (divided) by the given exponents (again supplied as an array of deflation factors). The algorithm automatically replaces a deflation of $0$ by $1$, to avoid division by $0$.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> g = (x + y + 1)^2
x^2 + 2*x*y + 2*x + y^2 + 2*y + 1

julia> g0 = coeff(g, [y], [0])
x^2 + 2*x + 1

julia> deflate(g - g0, [y], [1], [1])
2*x + y + 2
```

source

AbstractAlgebra.inflate – Method.

```
inflate(f::MPolyRingElem{T}, shift::Vector{Int}, defl::Vector{Int}) where T <: RingElement
```

Return a polynomial with the same coefficients as $f$ but whose exponents have been inflated (multiplied) by the given deflation exponents (supplied as an array of inflation factors, one for each variable) and then increased by the given shifts (again supplied as an array of shifts, one for each variable).

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> inflate(x^2*y^2 + 3*x*y^2 - x + 5*y - 1, [1, 2], [3, 3])
x^7*y^8 + 3*x^4*y^8 - x^4*y^2 + 5*x*y^5 - x*y^2
```

source

AbstractAlgebra.inflate – Method.

```
inflate(f::MPolyRingElem{T}, defl::Vector{Int}) where T <: RingElement
```

Return a polynomial with the same coefficients as $f$ but whose exponents have been inflated (multiplied) by the given deflation exponents (supplied as an array of inflation factors, one for each variable).

source

AbstractAlgebra.inflate – Method.

```
inflate(f::MPolyRingElem, vars::Vector{Int}, shift::Vector{Int}, defl::Vector{Int})
```

Return a polynomial with the same coefficients as $f$ but where exponents of some variables (supplied as an array of variable indices) have been inflated (multiplied) by the given deflation exponents (supplied as an array of inflation factors) and then increased by the given shifts (again supplied as an array of shifts).

source

AbstractAlgebra.inflate – Method.

```
inflate(f::T, vars::Vector{T}, shift::Vector{Int}, defl::Vector{Int}) where T <: MPolyRingElem
```

Return a polynomial with the same coefficients as $f$ but where the exponents of the given variables have been inflated (multiplied) by the given deflation exponents (supplied as an array of inflation factors) and then increased by the given shifts (again supplied as an array of shifts).

source

### Conversions

AbstractAlgebra.to_univariate – Method.

```
to_univariate(R::PolyRing{T}, p::MPolyRingElem{T}) where T <: RingElement
```

Assuming the polynomial $p$ is actually a univariate polynomial, convert the polynomial to a univariate polynomial in the given univariate polynomial ring $R$. An exception is raised if the polynomial $p$ involves more than one variable.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> S, z = polynomial_ring(ZZ, :z)
(Univariate polynomial ring in z over integers, z)

julia> f = 2x^5 + 3x^4 - 2x^2 - 1
2*x^5 + 3*x^4 - 2*x^2 - 1

julia> g = to_univariate(S, f)
2*z^5 + 3*z^4 - 2*z^2 - 1
```

source

### Evaluation

The following function allows evaluation of a polynomial at all its variables. The result is always in the ring that a product of a coefficient and one of the values belongs to, i.e. if all the values are in the coefficient ring, the result of the evaluation will be too.

AbstractAlgebra.evaluate – Method.

```
evaluate(a::MPolyRingElem{T}, vals::Vector{U}) where {T <: RingElement, U}
```

Evaluate the polynomial expression by substituting in the array of values for each of the variables. The evaluation will succeed if multiplication is defined between elements of the coefficient ring of $a$ and elements of the supplied vector.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = 2x^2*y^2 + 3x + y + 1
2*x^2*y^2 + 3*x + y + 1

julia> evaluate(f, BigInt[1, 2])
```

```
14

julia> evaluate(f, [QQ(1), QQ(2)])
14//1

julia> evaluate(f, [1, 2])
14

julia> f(1, 2) == 14
true

julia> evaluate(f, [x + y, 2y - x])
2*x^4 - 4*x^3*y - 6*x^2*y^2 + 8*x*y^3 + 2*x + 8*y^4 + 5*y + 1

julia> f(x + y, 2y - x)
2*x^4 - 4*x^3*y - 6*x^2*y^2 + 8*x*y^3 + 2*x + 8*y^4 + 5*y + 1

julia> R, (x, y, z) = polynomial_ring(ZZ, [:x, :y, :z])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y, z])

julia> f = x^2*y^2 + 2x*z + 3y*z + z + 1
x^2*y^2 + 2*x*z + 3*y*z + z + 1

julia> evaluate(f, [1, 3], [3, 4])
9*y^2 + 12*y + 29

julia> evaluate(f, [x, z], [3, 4])
9*y^2 + 12*y + 29

julia> evaluate(f, [1, 2], [x + z, x - z])
x^4 - 2*x^2*z^2 + 5*x*z + z^4 - z^2 + z + 1

julia> S = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> M1 = S([1 2; 3 4])
[1   2]
[3   4]

julia> M2 = S([2 3; 1 -1])
[2    3]
[1   -1]

julia> M3 = S([-1 1; 1 1])
[-1   1]
[ 1   1]

julia> evaluate(f, [M1, M2, M3])
[ 64    83]
[124   149]
```

source

The following functions allow evaluation of a polynomial at some of its variables. Note that the result will be a product of values and an element of the polynomial ring, i.e. even if all the values are in the coefficient ring and all variables are given values, the result will be a constant polynomial, not a coefficient.

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::MPolyRingElem{T}, vars::Vector{Int}, vals::Vector{U}) where {T <: RingElement, U <:
↪  RingElement}
```

Evaluate the polynomial expression by substituting in the supplied values in the array `vals` for the corresponding variables with indices given by the array `vars`. The evaluation will succeed if multiplication is defined between elements of the coefficient ring of $a$ and elements of `vals`.

source

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::S, vars::Vector{S}, vals::Vector{U}) where {S <: MPolyRingElem{T}, U <: RingElement}
↪  where T <: RingElement
```

Evaluate the polynomial expression by substituting in the supplied values in the array `vals` for the corresponding variables (supplied as polynomials) given by the array `vars`. The evaluation will succeed if multiplication is defined between elements of the coefficient ring of $a$ and elements of `vals`.

source

The following function allows evaluation of a polynomial at values in a not necessarily commutative ring, e.g. elements of a matrix algebra.

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::MPolyRingElem{T}, vals::Vector{U}) where {T <: RingElement, U}
```

Evaluate the polynomial expression by substituting in the array of values for each of the variables. The evaluation will succeed if multiplication is defined between elements of the coefficient ring of $a$ and elements of the supplied vector.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = 2x^2*y^2 + 3x + y + 1
2*x^2*y^2 + 3*x + y + 1

julia> evaluate(f, BigInt[1, 2])
14

julia> evaluate(f, [QQ(1), QQ(2)])
14//1
```

```
julia> evaluate(f, [1, 2])
14

julia> f(1, 2) == 14
true

julia> evaluate(f, [x + y, 2y - x])
2*x^4 - 4*x^3*y - 6*x^2*y^2 + 8*x*y^3 + 2*x + 8*y^4 + 5*y + 1

julia> f(x + y, 2y - x)
2*x^4 - 4*x^3*y - 6*x^2*y^2 + 8*x*y^3 + 2*x + 8*y^4 + 5*y + 1

julia> R, (x, y, z) = polynomial_ring(ZZ, [:x, :y, :z])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y, z])

julia> f = x^2*y^2 + 2x*z + 3y*z + z + 1
x^2*y^2 + 2*x*z + 3*y*z + z + 1

julia> evaluate(f, [1, 3], [3, 4])
9*y^2 + 12*y + 29

julia> evaluate(f, [x, z], [3, 4])
9*y^2 + 12*y + 29

julia> evaluate(f, [1, 2], [x + z, x - z])
x^4 - 2*x^2*z^2 + 5*x*z + z^4 - z^2 + z + 1

julia> S = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> M1 = S([1 2; 3 4])
[1   2]
[3   4]

julia> M2 = S([2 3; 1 -1])
[2    3]
[1   -1]

julia> M3 = S([-1 1; 1 1])
[-1   1]
[ 1   1]

julia> evaluate(f, [M1, M2, M3])
[ 64    83]
[124   149]
```

source

### Leading and constant coefficients, leading monomials and leading terms

The leading and trailing coefficient, constant coefficient, leading monomial and leading term of a polynomial p are returned by the following functions:

`AbstractAlgebra.leading_coefficient` – Method.

```
leading_coefficient(p::MPolyRingElem)
```

Return the leading coefficient of the polynomial $p$.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y]; internal_ordering=:deglex);

julia> p = 2*x*y + 3*y^3 + 1
3*y^3 + 2*x*y + 1

julia> leading_coefficient(p)
3
```

source

`AbstractAlgebra.trailing_coefficient` – Method.

```
trailing_coefficient(p::MPolyRingElem)
```

Return the trailing coefficient of the polynomial $p$, i.e. the coefficient of the last nonzero term.

source

`AbstractAlgebra.leading_monomial` – Method.

```
leading_monomial(p::MPolyRingElem)
```

Return the leading monomial of $p$. This function throws an `ArgumentError` if $p$ is zero.

source

`AbstractAlgebra.leading_term` – Method.

```
leading_term(p::MPolyRingElem)
```

Return the leading term of the polynomial `p`. This function throws an `ArgumentError` if $p$ is zero.

source

`AbstractAlgebra.constant_coefficient` – Method.

```
constant_coefficient(p::MPolyRingElem)
```

Return the constant coefficient of the polynomial $p$ or zero if it doesn't have one.

source

AbstractAlgebra.tail – Method.

```
tail(p::MPolyRingElem)
```

Return the tail of the polynomial $p$, i.e. the polynomial without its leading term (if any).

source

### Least common multiple, greatest common divisor

The greatest common divisor of two polynomials a and b is returned by

Base.gcd – Method.

```
gcd(a::MPoly{T}, a::MPoly{T}) where {T <: RingElement}
```

Return the greatest common divisor of a and b in parent(a).

source

Note that this functionality is currently only provided for AbstractAlgebra generic polynomials. It is not automatically provided for all multivariate rings that implement the multivariate interface.

However, if such a gcd is provided, the least common multiple of two polynomials a and b is returned by

Base.lcm – Method.

```
lcm(a::AbstractAlgebra.MPolyRingElem{T}, a::AbstractAlgebra.MPolyRingElem{T}) where {T <:
↪  RingElement}
```

Return the least common multiple of a and b in parent(a).

**Examples**

```
julia> R,(x,y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> a = x*y + 2*y
x*y + 2*y

julia> b = x^3*y + y
x^3*y + y

julia> lcm(a,b)
x^4*y + 2*x^3*y + x*y + 2*y

julia> lcm(a,b) == a * b // gcd(a,b)
true
```

source

### Derivations

`AbstractAlgebra.derivative` – Method.

```
derivative(f::MPolyRingElem{T}, x::MPolyRingElem{T}) where T <: RingElement
```

Return the partial derivative of `f` with respect to `x`. The value `x` must be a generator of the polynomial ring of `f`.

**Examples**

```
julia> R, (x, y) = AbstractAlgebra.polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x*y + x + y + 1
x*y + x + y + 1

julia> derivative(f, x)
y + 1

julia> derivative(f, y)
x + 1

julia> derivative(f, 1)
y + 1

julia> derivative(f, 2)
x + 1
```

source

### Homogeneous polynomials

`AbstractAlgebra.is_homogeneous` – Method.

```
is_homogeneous(x::MPolyRingElem)
```

Return `true` if the given polynomial is homogeneous with respect to the standard grading and `false` otherwise. Here by standard grading we mean that all variables of the polynomial ring are graded with weight 1.

source

## 19.6 Random generation

Random multivariate polynomials in a given ring can be constructed by passing a range of degrees for the variables and a range on the number of terms. Additional parameters are used to generate the coefficients of the polynomial.

Note that zero coefficients may currently be generated, leading to less than the requested number of terms.

```
rand(R::MPolyRing, exp_range::AbstractUnitRange{Int}, term_range::AbstractUnitRange{Int}, v...)
```

### Examples

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = rand(R, -1:2, 3:5, -10:10)
4*x^4*y^4

julia> S, (s, t) = polynomial_ring(GF(7), [:x, :y])
(Multivariate polynomial ring in 2 variables over finite field F_7,
↪  AbstractAlgebra.Generic.MPoly{AbstractAlgebra.GFElem{Int64}}[x, y])

julia> g = rand(S, -1:2, 3:5)
4*x^3*y^4
```

# Chapter 20

# Universal ring

AbstractAlgebra provides a module, implemented in `src/UniversalRing.jl` for universal rings. They "universalize" rings which have a notion of generators (or variables) and coefficients. The most important example of such a ring is a multivariate polynomial ring. In contrast to the usual multivariate polynomial ring, its universal pendant allows for variables to be added at any time.

Another example is the ring of multivariate Laurent polynomials.

The type of the universal ring is `UniversalRing{T, U}` where `T` is the type of the elements of the underlying ring and `U` is the type of the coefficients. In the case of multivariate polynomials `T` would be a subtype of `MPolyRingElem` and `U` would be the type of the coefficients of the polynomials, for example `BigInt`.

## 20.1 Universal polynomial ring

To compensate for the fact that the number of variables may change, many of the functions relax their restrictions on exponent vectors. For example, if one creates a polynomial when the ring only has two variables, each exponent vector would consist of two integers. Later, when the ring has more variable, these exponent vectors will still be accepted. The exponent vectors are simply padded out to the full number of variables behind the scenes.

The universal polynomial ring behaves exactly like a multivariate polynomial ring with the few differences noted above.

The only functionality not implemented is the ability to do `divrem` by an ideal of polynomials.

The universal polynomial ring is very useful for doing symbolic manipulation. However, it is important to understand that AbstractAlgebra is not a symbolic system and the performance of the universal polynomial ring will closely match that of a multivariate polynomial ring with the same number of variables.

The disadvantage of this approach to symbolic manipulation is that some manipulations that would be offered by a symbolic system are not available, as variables are not identified by their names alone in AbstractAlgebra, as would be the case symbolically, but by objects.

The most powerful symbolic tools we offer are the generalized evaluation functions, the multivariate coefficient functionality, the ability to change coefficient ring and to map coefficients according to a supplied function and the ability to convert a multivariate which happens to have just one variable into a dense univariate polynomial.

Further facilities may be added in future to ease symbolic manipulations.

## 20.2 Constructors

In order to construct universal polynomials in AbstractAlgebra, one must first construct the universal polynomial ring itself. This is unique given a coefficient ring.

The universal polynomial ring over a given coefficient ring R is constructed with one of the following constructor functions.

`AbstractAlgebra.universal_polynomial_ring` – Function.

```
universal_polynomial_ring(R::Ring, varnames::Vector{Symbol}; cached::Bool=true,
↪  internal_ordering::Symbol=:lex)
universal_polynomial_ring(R::Ring; cached::Bool=true, internal_ordering::Symbol=:lex)
```

Given a coefficient ring R and variable names, say `varnames = [:x1, :x2, ...]`, return a tuple `S, [x1, x2, ...]` of the universal polynomial ring `S = R[x1, x2, \dots]` and its generators `x1, x2, \dots`.

If `varnames` is omitted, return an object representing the universal polynomial ring `S = R[\ldots]` with no variables in it initially.

By default (`cached=true`), the output S will be cached, i.e. if `universal_polynomial_ring` is invoked again with the same coefficient ring R and monomial ordering `internal_ordering`, the same (*identical*) ring is returned. Keep in mind that this ring might already have more variables than supplied in `varnames`. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

**Examples**

```
julia> S, (x,y) = universal_polynomial_ring(ZZ, [:x,:y])
(Universal polynomial ring over Integers,
↪  UniversalRingElem{AbstractAlgebra.Generic.MPoly{BigInt}, BigInt}[x, y])

julia> z = gen(S, :z)
z

julia> x*y - z
x*y - z

julia> S = universal_polynomial_ring(ZZ)
Universal polynomial ring over Integers

julia> x = gen(S, :x)
x

julia> y, z = gens(S, [:y, :z])
2-element Vector{UniversalRingElem{AbstractAlgebra.Generic.MPoly{BigInt}, BigInt}}:
 y
 z

julia> x*y - z
x*y - z
```

source

`AbstractAlgebra.@universal_polynomial_ring` – Macro.

```
@universal_polynomial_ring(R::Ring, varnames...; cached=true, internal_ordering=:lex)
```

Return the ring from `universal_polynomial_ring` and introduce the generators into the current scope.

**Examples**

```
julia> S = @universal_polynomial_ring(ZZ, [:m, :n])
Universal polynomial ring over Integers

julia> m*n
m*n
```

source

Similarly, on can construct universal Laurent polynomial rings.

`AbstractAlgebra.universal_laurent_polynomial_ring` – Function.

```
universal_laurent_polynomial_ring(R::Ring, varnames::Vector{Symbol}; cached::Bool=true)
universal_laurent_polynomial_ring(R::Ring; cached::Bool=true)
```

Given a coefficient ring R and variable names, say `varnames = [:x1, :x2, ...]`, return a tuple `S, [x1, x2, ...]` of the universal Laurent polynomial ring `S = R[x1, x2, \dots]` and its generators `x1, x2, \dots`.

If `varnames` is omitted, return an object representing the universal Laurent polynomial ring `S = R[\ldots]` with no variables in it initially.

By default (`cached=true`), the output S will be cached, i.e. if `universal_laurent_polynomial_ring` is invoked again with the same coefficient ring R, the same (*identical*) ring is returned. Keep in mind that this ring might already have more variables than supplied in `varnames`. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

**Examples**

```
julia> S, (x,y) = universal_laurent_polynomial_ring(ZZ, [:x,:y])
(Universal Laurent polynomial ring over Integers,
↪  UniversalRingElem{AbstractAlgebra.Generic.LaurentMPolyWrap{BigInt,
↪  AbstractAlgebra.Generic.MPoly{BigInt}, AbstractAlgebra.Generic.LaurentMPolyWrapRing{BigInt,
↪  AbstractAlgebra.Generic.MPolyRing{BigInt}}}, BigInt}[x, y])

julia> z = gen(S, :z)
z

julia> x*y - z
x*y - z

julia> S = universal_laurent_polynomial_ring(ZZ)
Universal Laurent polynomial ring over Integers

julia> x = gen(S, :x)
x
```

```
julia> y, z = gens(S, [:y, :z])
2-element Vector{UniversalRingElem{AbstractAlgebra.Generic.LaurentMPolyWrap{BigInt,
↪  AbstractAlgebra.Generic.MPoly{BigInt}, AbstractAlgebra.Generic.LaurentMPolyWrapRing{BigInt,
↪  AbstractAlgebra.Generic.MPolyRing{BigInt}}}, BigInt}}:
 y
 z

julia> x^(-1)*y - z
-z + x^-1*y
```

source

## 20.3 Adding variables

There are two ways to add variables to a universal ring S.

```
gen(S::UniversalRing, var::VarName)
gens(S::UniversalRing, vars::Vector{VarName})
```

### Examples

```
julia> S = universal_polynomial_ring(ZZ)
Universal polynomial ring over Integers

julia> x = gen(S, :x)
x

julia> number_of_generators(S)
1

julia> y, z = gens(S, [:y, :z])
2-element Vector{UniversalRingElem{AbstractAlgebra.Generic.MPoly{BigInt}, BigInt}}:
 y
 z

julia> number_of_generators(S)
3
```

## 20.4 Implement universal rings

To be able to use a ring as a base ring of an universal ring one has to implement a few methods. In addition to the variable and coefficient related methods like gen, gens, symbols, coefficient_ring there are the two internal methods _add_gens and _upgrade used to extend the number of variables.

AbstractAlgebra._add_gens – Function.

```
_add_gens(R::MPolyRing, varnames::Vector{Symbol})
```

Return a new uncached multivariate polynomial ring which has the same properties as R but `varnames` as additional generators.

source

```
_add_gens(R::LaurentMPolyWrapRing, varnames::Vector{Symbol})
```

Return a new uncached multivariate Laurent polynomial ring which has the same properties as R but `varnames` as additional generators.

source

`AbstractAlgebra._upgrade` – Function.

```
_upgrade(p::MPolyRingElem{T}, R::MPolyRing{T}) where {T}
```

Return an element of R which is obtained from `p` by mapping the $i$-th variable of `parent(p)` to the $i$-th variable of R. For this to work, R needs to have at least as many variables as `parent(p)`.

source

```
_upgrade(p::LaurentMPolyWrap{T}, R::LaurentMPolyRingWrap{T}) where {T}
```

Return an element of R which is obtained from `p` by mapping the $i$-th variable of `parent(p)` to the $i$-th variable of R. For this to work, R needs to have at least as many variables as `parent(p)`.

source

### Hashing

To be able to use subtypes of `UniversalRingElem` efficiently as keys of a dictionary or similar applications, it is necessary to implement a custom `hash` function. The results should not depend on the number of variables currently in the parent. An example implementation can be found in `src/UnivPoly.jl`

# Chapter 21

# Generic Laurent polynomials

Laurent polynomials are similar to polynomials but can have terms of negative degrees, and form a ring denoted by $R[x, x^{-1}]$ where R is the coefficient ring.

## 21.1 Generic Laurent polynomial types

AbstractAlgebra.jl provides a generic implementation of Laurent polynomials, built in terms of regular polynomials in the file `src/generic/LaurentPoly.jl`.

The type `LaurentPolyWrap{T, ...} <: LaurentPolyRingElem{T}` implements generic Laurent polynomials by wrapping regular polynomials: a Laurent polynomial `l` wraps a polynomial `p` and an integer `n` such that $l = x^{-n} * p$.

The corresponding parent type is `LaurentPolyWrapRing{T, ...} <: LaurentPolyRing{T}`.

## 21.2 Abstract types

Two abstract types `LaurentPolyRingElem{T}` and `LaurentPolyRing{T}` are defined to represent Laurent polynomials and rings thereof, parameterized on a base ring `T`.

## 21.3 Laurent polynomials ring constructor

In order to instantiate Laurent polynomials, one must first construct the parent ring:

`AbstractAlgebra.laurent_polynomial_ring` – Function.

```
laurent_polynomial_ring(R::Ring, s::VarName)
```

Given a base ring `R` and string `s` specifying how the generator (variable) should be printed, return a tuple `S, x` representing the new Laurent polynomial ring $S = R[x, 1/x]$ and the generator $x$ of the ring.

**Examples**

```
julia> R, x = laurent_polynomial_ring(ZZ, :x)
(Univariate Laurent Polynomial Ring in x over Integers, x)

julia> 2x^-3 + x^2
x^2 + 2*x^-3
```

```
julia> rand(R, -3:3, -9:9)
-3*x^2 - 8*x + 4 + 3*x^-1 - 6*x^-2 + 9*x^-3
```

source

```
laurent_polynomial_ring(R::Ring, varnames...; cached::Bool = true)
```

Given a base ring R and variable names `varnames...`, say `:x, :y, :z`, return a tuple `S, x, y, z` representing the new ring $S = R[x, 1/x, y, 1/y, z, 1/z]$ and the generators $x, y, z$ of the ring.

By default (`cached=true`), the output S will be cached, i.e. if `laurent_polynomial_ring` is invoked again with the same arguments, the same (*identical*) ring is returned. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

For information about the many ways to specify `varnames...` refer to `polynomial_ring` or the specification in `AbstractAlgebra.@varnames_interface`.

**Examples**

```
julia> S, (x, y) = laurent_polynomial_ring(ZZ, [:x, :y])
(Multivariate Laurent polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.LaurentMPolyWrap{BigInt, AbstractAlgebra.Generic.MPoly{BigInt},
↪  AbstractAlgebra.Generic.LaurentMPolyWrapRing{BigInt,
↪  AbstractAlgebra.Generic.MPolyRing{BigInt}}}[x, y])

julia> (x + y)*x^-1
1 + x^-1*y
```

source

## 21.4 Basic functionality

Laurent polynomials implement the ring interface, and some methods from the polynomial interface, for example:

```
julia> R, x = laurent_polynomial_ring(ZZ, :x)
(Univariate Laurent polynomial ring in x over integers, x)

julia> var(R)
:x

julia> symbols(R)
1-element Vector{Symbol}:
 :x

julia> number_of_variables(R)
1

julia> f = x^-2 + 2x
2*x + x^-2
```

```
julia> coeff.(f, -2:2)
5-element Vector{BigInt}:
 1
 0
 0
 2
 0

julia> set_coefficient!(f, 3, ZZ(5))
5*x^3 + 2*x + x^-2

julia> is_gen(f)
false

julia> shift_left(f,2)
5*x^5 + 2*x^3 + 1

julia> map_coefficients(x->2x, f)
10*x^3 + 4*x + 2*x^-2

julia> change_base_ring(RealField, f)
5.0*x^3 + 2.0*x + x^-2

julia> leading_coefficient(f), trailing_coefficient(f)
(5, 1)

julia> constant_coefficient(f), is_constant(f)
(0, false)
```

# Chapter 22

# Sparse distributed multivariate Laurent polynomials

Every element of the multivariate Laurent polynomial ring $R[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}]$ can be presented as a sum of products of powers of the $x_i$ where the power can be *any* integer. Therefore, the interface for sparse multivarate polynomials carries over with the additional feature that exponents can be negative.

## 22.1 Generic multivariate Laurent polynomial types

AbstractAlgebra.jl provides a generic implementation of multivariate Laurent polynomials, built in terms of regular multivariate polynomials, in the file `src/generic/LaurentMPoly.jl`.

The type `LaurentMPolyWrap{T, ...} <: LaurentMPolyRingElem{T}` implements generic multivariate Laurent polynomials by wrapping regular polynomials: a Laurent polynomial `l` wraps a polynomial `p` and a vector of integers $n_i$ such that $l = \prod_i x_i^{n_i} * p$. The representation is said to be normalized when each $n_i$ is as large as possible (or zero when `l` is zero), but the representation of a given element is not required to be normalized internally.

The corresponding parent type is `LaurentMPolyWrapRing{T, ...} <: LaurentMPolyRing{T}`.

## 22.2 Abstract types

Two abstract types `LaurentMPolyRingElem{T}` and `LaurentMPolyRing{T}` are defined to represent Laurent polynomials and rings thereof, parameterized on a base ring `T`.

## 22.3 Multivate Laurent polynomial operations

Since, from the point of view of the interface, Laurent polynomials are simply regular polynomials with possibly negative exponents, the following functions from the polynomial interface are completely analogous. As with regular polynomials, an implementation must provide access to the elements as a sum of individual terms *in some order*. This order currently cannot be specified in the constructor.

`AbstractAlgebra.laurent_polynomial_ring` – Method.

```
laurent_polynomial_ring(R::Ring, varnames...; cached::Bool = true)
```

Given a base ring `R` and variable names `varnames...`, say `:x, :y, :z`, return a tuple `S, x, y, z` representing the new ring $S = R[x, 1/x, y, 1/y, z, 1/z]$ and the generators $x, y, z$ of the ring.

By default (`cached=true`), the output S will be cached, i.e. if `laurent_polynomial_ring` is invoked again with the same arguments, the same (*identical*) ring is returned. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

For information about the many ways to specify `varnames...` refer to `polynomial_ring` or the specification in `AbstractAlgebra.@varnames_interface`.

**Examples**

```
julia> S, (x, y) = laurent_polynomial_ring(ZZ, [:x, :y])
(Multivariate Laurent polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.LaurentMPolyWrap{BigInt, AbstractAlgebra.Generic.MPoly{BigInt},
↪  AbstractAlgebra.Generic.LaurentMPolyWrapRing{BigInt,
↪  AbstractAlgebra.Generic.MPolyRing{BigInt}}}[x, y])

julia> (x + y)*x^-1
1 + x^-1*y
```

source

`AbstractAlgebra.@laurent_polynomial_ring` – Macro.

```
@laurent_polynomial_ring(R::Ring, varnames...; cached=true)
```

Return the ring from `laurent_polynomial_ring` and introduce the generators into the current scope.

**Examples**

```
julia> S = @laurent_polynomial_ring(ZZ, [:u, :v])
Multivariate Laurent polynomial ring in 2 variables u, v
  over integers

julia> u*v^-1
u*v^-1
```

source

```
(S::LaurentMPolyRing{T})(A::Vector{T}, m::Vector{Vector{Int}})
```

```
MPolyBuildCtx(R::LaurentMPolyRing)
push_term!(M::LaurentMPolyBuildCtx, c::RingElem, v::Vector{Int})
finish(M::LaurentMPolyBuildCtx)
```

```
symbols(S::LaurentMPolyRing)
number_of_variables(f::LaurentMPolyRing)
gens(S::LaurentMPolyRing)
gen(S::LaurentMPolyRing, i::Int)
is_gen(x::LaurentMPolyRingElem)
var_index(p::LaurentMPolyRingElem)
length(f::LaurentMPolyRingElem)
```

```
coefficients(p::LaurentMPolyRingElem)
monomials(p::LaurentMPolyRingElem)
terms(p::LaurentMPolyRingElem)
exponent_vectors(p::LaurentMPolyRingElem)
leading_coefficient(p::LaurentMPolyRingElem)
leading_monomial(p::LaurentMPolyRingElem)
leading_term(p::LaurentMPolyRingElem)
leading_exponent_vector(p::LaurentMPolyRingElem)
constant_coefficient(p::LaurentMPolyRingElem)
is_constant(p::LaurentMPolyRingElem)
```

```
change_base_ring(::Ring, p::LaurentMPolyRingElem)
change_coefficient_ring(::Ring, p::LaurentMPolyRingElem)
map_coefficients(::Any, p::LaurentMPolyRingElem)
```

```
evaluate(p::LaurentMPolyRingElem, ::Vector)
```

```
derivative(p::LaurentMPolyRingElem, x::LaurentMPolyRingElem)
derivative(p::LaurentMPolyRingElem, i::Int)
```

```
rand(R::LaurentMPolyRingElem, length_range::AbstractUnitRange{Int},
↪  exp_range::AbstractUnitRange{Int}, v...)
```

The choice of canonical unit for Laurent polynomials includes the product $\prod_i x_i^{n_i}$ from the normalized representation. In particular, this means that the output of gcd will not have any negative exponents.

```
julia> R, (x, y) = laurent_polynomial_ring(ZZ, [:x, :y]);

julia> canonical_unit(2*x^-5 - 3*x + 4*y^-4 + 5*y^2)
-x^-5*y^-4

julia> gcd(x^-3 - y^3, x^-2 - y^2)
x*y - 1
```

# Chapter 23

# Multivariate Puiseux polynomials

A Puiseux polynomial over a field $K$ is a finite sum of terms $cx_1^{e_1} \cdots x_n^{e_n}$ with $c \in K$ and rational exponents $e_i$, which may be negative. Puiseux polynomials form a ring containing the Laurent polynomial ring $K[x_1^{\pm 1}, \ldots, x_n^{\pm 1}]$.

## 23.1 Generic Puiseux polynomial types

AbstractAlgebra.jl provides a generic implementation of multivariate Puiseux polynomials in the file `src/generic/PuiseuxMPoly.j` built on top of the generic multivariate Laurent polynomials.

The element type is `PuiseuxMPolyRingElem{T} <: RingElem` and the parent type is `PuiseuxMPolyRing{T} <: Ring`, parameterized on the coefficient type T.

A Puiseux polynomial $f$ is stored as a pair $(g, d)$ of a Laurent polynomial $g$ and a positive integer $d$, its *scale*, with

$$f(x_1, \ldots, x_n) = g(x_1^{1/d}, \ldots, x_n^{1/d}).$$

The representation is normalized: $d$ is coprime to the greatest common divisor of all exponents of $g$, so $d$ is the least common denominator of the exponents of $f$. Every Puiseux polynomial has a unique normalized representation, which makes comparing elements cheap. Arithmetic rescales both operands to a common scale, operates on the underlying Laurent polynomials and normalizes the result.

## 23.2 Constructor

`AbstractAlgebra.puiseux_polynomial_ring` – Method.

```
puiseux_polynomial_ring(K::Field, variableName::Vector{String})
```

Return a tuple `(R, x)` consisting of the ring R of Puiseux polynomials over K in the given variables and the vector x of its generators.

The elements of R are finite sums of terms $cx_1^{e_1} \cdots x_n^{e_n}$ with $c \in K$ and rational, possibly negative, exponents $e_i$.

**Examples**

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> R
Puiseux polynomial ring in 2 variables x, y
  over rationals

julia> f = x^(1//2) + 2*y^(-1//3)
x^(1//2) + 2*y^(-1//3)

julia> f^2
x + 4*x^(1//2)*y^(-1//3) + 4*y^(-2//3)
```

source

Elements can be created from the generators by the usual arithmetic operations, and constants can be coerced into the ring by calling it:

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> R(3)
3

julia> 3*x^(1//2)*y^(-2//3) - 1
3*x^(1//2)*y^(-2//3) - 1
```

## 23.3 Basic functionality

Puiseux polynomial rings implement the Ring interface. In addition, the following functions are available.

```
base_ring(R::PuiseuxMPolyRing)
coefficient_ring(R::PuiseuxMPolyRing)
symbols(R::PuiseuxMPolyRing)
number_of_variables(R::PuiseuxMPolyRing)
gens(R::PuiseuxMPolyRing)
is_univariate(R::PuiseuxMPolyRing)
```

Here `base_ring` returns the underlying Laurent polynomial ring, and `coefficient_ring` returns the field $K$.

```
length(f::PuiseuxMPolyRingElem)
coefficients(f::PuiseuxMPolyRingElem)
monomials(f::PuiseuxMPolyRingElem)
exponent_vectors(f::PuiseuxMPolyRingElem)
leading_coefficient(f::PuiseuxMPolyRingElem)
is_gen(f::PuiseuxMPolyRingElem)
is_monomial(f::PuiseuxMPolyRingElem)
is_term(f::PuiseuxMPolyRingElem)
is_unit(f::PuiseuxMPolyRingElem)
canonical_unit(f::PuiseuxMPolyRingElem)
```

The exponent vectors have entries of type `Rational{Int}`. The units are the nonzero terms, i.e. the nonzero constant multiples of monomials. As for Laurent polynomials, the canonical unit of `f` is the canonical unit of its leading coefficient times $x_1^{m_1} \cdots x_n^{m_n}$, where $m_i$ is the smallest exponent of $x_i$ in `f`.

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> f = x^(1//2) + 2*y^(1//3);

julia> collect(coefficients(f))
2-element Vector{Rational{BigInt}}:
 1
 2

julia> exponent_vectors(f)
2-element Vector{Vector{Rational{Int64}}}:
 [1//2, 0]
 [0, 1//3]

julia> is_unit(f), is_unit(2*x^(1//2))
(false, true)

julia> canonical_unit(3*x*y + 2*x^(1//2))
3*x^(1//2)
```

The normalized representation can be inspected with the following functions.

`AbstractAlgebra.Generic.poly` – Method.

```
poly(f::PuiseuxMPolyRingElem)
```

Return the Laurent polynomial $g$ in `base_ring(parent(f))` with $f(x_1, \ldots, x_n) = g(x_1^{1/d}, \ldots, x_n^{1/d})$, where $d$ is `scale(f)`.

**Examples**

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> poly(x^(1//2) + 2*y^(1//3))
x^3 + 2*y^2
```

source

`AbstractAlgebra.Generic.scale` – Method.

```
scale(f::PuiseuxMPolyRingElem)
```

Return the least common denominator $d > 0$ of the exponents of `f`, so that `f` is the Laurent polynomial `poly(f)` evaluated at $x_1^{1/d}, \ldots, x_n^{1/d}$.

**Examples**

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> scale(x^(1//2) + 2*y^(1//3))
6
```

source

## 23.4 Powering

```
^(f::PuiseuxMPolyRingElem, n::Integer)
^(f::PuiseuxMPolyRingElem, q::Rational)
```

Any Puiseux polynomial can be raised to a nonnegative integer power. A negative integer power requires `f` to be a monomial times a constant, and a non-integer rational power requires `f` to be a monomial with coefficient one.

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> (x^(1//2) + y)^2
x + 2*x^(1//2)*y + y^2

julia> (x*y^2)^(1//3)
x^(1//3)*y^(2//3)

julia> (x + y)^(1//2)
ERROR: ArgumentError: only monomials can be exponentiated to non-integer powers
[...]
```

## 23.5 Divisibility

```
divexact(f::PuiseuxMPolyRingElem, g::PuiseuxMPolyRingElem; check::Bool = true)
divexact(f::PuiseuxMPolyRingElem, a::RingElement; check::Bool = true)
divides(f::PuiseuxMPolyRingElem, g::PuiseuxMPolyRingElem)
```

`divexact` returns the quotient of `f` by a nonzero Puiseux polynomial or constant `g`, provided the division is exact. `divides(f, g)` returns a tuple `(flag, q)` where `flag` is `true` if $f = qg$ for some Puiseux polynomial $q$; otherwise `q` is zero.

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> f = x^(1//2) + 2*y^(1//3);

julia> divexact(f^2, f)
x^(1//2) + 2*y^(1//3)

julia> divexact(f, x^(1//6))
x^(1//3) + 2*x^(-1//6)*y^(1//3)
```

```
julia> divides(f^2, f)
(true, x^(1//2) + 2*y^(1//3))

julia> divides(x, f)
(false, 0)
```

## 23.6 Changing the coefficient ring

```
change_base_ring(K::Ring, f::PuiseuxMPolyRingElem; cached::Bool = true, parent::PuiseuxMPolyRing)
map_coefficients(g, f::PuiseuxMPolyRingElem; cached::Bool = true, parent::PuiseuxMPolyRing)
```

Return the Puiseux polynomial obtained from `f` by coercing each coefficient into K, respectively by applying `g` to each coefficient. Unless `parent` is given, the result lies in the Puiseux polynomial ring over K, respectively over the parent of the images of `g`, in the same variables as `parent(f)`.

```
julia> R, (x, y) = puiseux_polynomial_ring(QQ, ["x", "y"]);

julia> map_coefficients(c -> 2*c, x^(1//2) + 3*y^(1//3))
2*x^(1//2) + 6*y^(1//3)
```

## 23.7 Valuation

`AbstractAlgebra.valuation` – Method.

```
valuation(f::PuiseuxMPolyRingElem)
```

Return the smallest exponent occurring in the univariate Puiseux polynomial `f`, or `PosInf()` if `f` is zero.

**Examples**

```
julia> R, (t,) = puiseux_polynomial_ring(QQ, ["t"]);

julia> valuation(t^(-2//3) + t^(1//2))
-2//3
```

source

# Chapter 24

# Power series

AbstractAlgebra.jl allows the creation of capped relative and absolute power series over any computable commutative ring $R$.

Capped relative power series are power series of the form $a_j x^j + a_{j+1}x^{j+1} + \cdots + a_{k-1}x^{k-1} + O(x^k)$ where $a_j \in R$ and the relative precision $k - j$ is at most equal to some specified precision $n$.

Capped absolute power series are power series of the form $a_j x^j + a_{j+1}x^{j+1} + \cdots + a_{n-1}x^{n-1} + O(x^n)$ where $j \geq 0$, $a_j \in R$ and the precision $n$ is fixed.

There are two implementations of relative series: relative power series, implemented in `src/RelSeries.jl` for which $j > 0$ in the above description, and Laurent series where $j$ can be negative, implemented in `src/Laurent.jl`. Note that there are two implementations for Laurent series, one over rings and one over fields, though in practice most of the implementation uses the same code in both cases.

There is a single implementation of absolute series: absolute power series, implemented in `src/AbsSeries.jl`.

## 24.1 Generic power series types

AbstractAlgebra.jl provides generic series types implemented in `src/generic/AbsSeries.jl`, `src/generic/RelSeries.jl` and `src/generic/LaurentSeries.jl` which implement the Series interface.

These generic series have types `Generic.RelSeries{T}`, `Generic.AbsSeries{T}`, `Generic.LaurentSeriesRingElem{T}` and `Generic.LaurentSeriesFieldElem{T}`. See the file `src/generic/GenericTypes.jl` for details.

The parent objects have types `Generic.AbsPowerSeriesRing{T}` and `Generic.RelPowerSeriesRing{T}` and `Generic.LaurentSeriesRing{T}` respectively.

The default precision, string representation of the variable and base ring $R$ of a generic power series are stored in its parent object.

## 24.2 Abstract types

Relative power series elements belong to the abstract type `RelPowerSeriesRingElem`.

Laurent series elements belong directly to either `RingElem` or `FieldElem` since it is more useful to be able to distinguish whether they belong to a ring or field than it is to distinguish that they are relative series.

Absolute power series elements belong to `AbsPowerSeriesRingElem`.

The parent types for relative and absolute power series, `Generic.RelPowerSeriesRing{T}` and `Generic.AbsPowerSeriesRing{T}` respectively, belong to `SeriesRing{T}`.

The parent types of Laurent series belong directly to `Ring` and `Field` respectively.

## 24.3 Series ring constructors

In order to construct series in AbstractAlgebra.jl, one must first construct the ring itself. This is accomplished with any of the following constructors.

`AbstractAlgebra.power_series_ring` – Method.

```
power_series_ring(R::Ring, prec::Int, s::VarName; cached::Bool=true,
↪  model::Symbol=:capped_relative)
```

Return a tuple $(S, x)$ consisting of the parent object S of a power series ring over the given base ring and a generator x for the power series ring. The maximum precision of power series in the ring is set to `prec`. If the model is set to `:capped_relative` this is taken as a maximum relative precision, and if it is set to `:capped_absolute` this is take to be a maximum absolute precision. The supplied string `s` specifies the way the generator of the power series ring will be printed. By default, the parent object S will be cached so that supplying the same base ring, string and precision in future will return the same parent object and generator. If caching of the parent object is not required, `cached` can be set to `false`.

**Examples**

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> S, y = power_series_ring(ZZ, 10, :y; model=:capped_absolute)
(Univariate power series ring over integers, y + O(y^10))
```

source

`AbstractAlgebra.@power_series_ring` – Macro.

```
@power_series_ring(R::Ring, prec::Int, varnames...; cached=true)
```

Return the series ring from `power_series_ring` and introduce the generators into the current scope. A single variable name gives the univariate ring of `power_series_ring(::Ring, ::Int, ::VarName)`.

**Examples**

```
julia> S = @power_series_ring(ZZ, 3, [:a, :b])
Multivariate power series ring in 2 variables a, b
  over integers

julia> a*b
b*a + O(b^3) + O(a^3)
```

source

`AbstractAlgebra.laurent_series_ring` – Function.

```
laurent_series_ring(R::Ring, prec::Int, s::VarName; cached::Bool=true)
laurent_series_field(R::Field, prec::Int, s::VarName; cached::Bool=true)
```

Return a tuple $(S, x)$ consisting of the parent object S of a Laurent series ring over the given base ring and a generator x for the Laurent series ring. The maximum precision of the series in the ring is set to `prec`. This is taken as a maximum relative precision. The supplied string s specifies the way the generator of the Laurent series ring will be printed. By default, the parent object S will be cached so that supplying the same base ring, string and precision in future will return the same parent object and generator. If caching of the parent object is not required, `cached` can be set to `false`.

source

`AbstractAlgebra.laurent_series_field` – Function.

```
laurent_series_field(R::Field, prec::Int, s::VarName; cached::Bool=true)
```

See `laurent_series_ring`.

source

It is also possible to construct absolute and relative power series with a default variable. These are lightweight constructors and should be used in generic algorithms wherever possible when creating series rings where the symbol does not matter.

`AbstractAlgebra.AbsPowerSeriesRing` – Function.

```
AbsPowerSeriesRing(R::Ring, prec::Int)
RelPowerSeriesRing(R::Ring, prec::Int)
```

Return the absolute resp. relative power series ring over R with precision cap `prec` and the default variable x. Only the ring is returned, not a generator, and the ring is not cached.

These are lightweight constructors meant for generic algorithms that need a series ring whose variable name does not matter.

source

`AbstractAlgebra.RelPowerSeriesRing` – Function.

```
RelPowerSeriesRing(R::Ring, prec::Int)
```

See `AbsPowerSeriesRing(::Ring, ::Int)`.

source

Here are some examples of constructing various kinds of series rings and coercing various elements into those rings.

**Examples**

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> S, y = power_series_ring(ZZ, 10, :y; model=:capped_absolute)
(Univariate power series ring over integers, y + O(y^10))

julia> T, z = laurent_series_ring(ZZ, 10, :z)
(Laurent series ring in z over integers, z + O(z^11))

julia> U, w = laurent_series_field(QQ, 10, :w)
(Laurent series field in w over rationals, w + O(w^11))

julia> f = R()
O(x^10)

julia> g = S(123)
123 + O(y^10)

julia> h = U(BigInt(1234))
1234 + O(w^10)

julia> k = T(z + 1)
1 + z + O(z^10)

julia> V = AbsPowerSeriesRing(ZZ, 10)
Univariate power series ring in x with precision 10
  over integers
```

## 24.4 Power series constructors

Series can be constructed using arithmetic operators using the generator of the series. Also see the big-oh notation below for specifying the precision.

All of the standard ring constructors can also be used to construct power series.

```
(R::SeriesRing)() # constructs zero
(R::SeriesRing)(c::Integer)
(R::SeriesRing)(c::elem_type(R))
(R::SeriesRing{T})(a::T) where T <: RingElement
```

In addition, the following constructors that are specific to power series are provided. They take an array of coefficients, a length, precision and valuation. Coefficients will be coerced into the coefficient ring if they are not already in that ring.

For relative series we have:

```
(S::SeriesRing{T})(A::Vector{T}, len::Int, prec::Int, val::Int) where T <: RingElem
(S::SeriesRing{T})(A::Vector{U}, len::Int, prec::Int, val::Int) where {T <: RingElem, U <:
↪  RingElem}
(S::SeriesRing{T})(A::Vector{U}, len::Int, prec::Int, val::Int) where {T <: RingElem, U <: Integer}
```

And for absolute series:

```
(S::SeriesRing{T})(A::Vector{T}, len::Int, prec::Int) where T <: RingElem
```

**Examples**

```
julia> S, x = power_series_ring(QQ, 10, :x; model=:capped_absolute)
(Univariate power series ring over rationals, x + O(x^10))

julia> S(Rational{BigInt}[0, 2, 3, 1], 4, 6)
2*x + 3*x^2 + x^3 + O(x^6)
```

It is also possible to create series directly without having to create the corresponding series ring.

`AbstractAlgebra.abs_series` – Function.

```
abs_series(R::Ring, arr::Vector{T}, len::Int, prec::Int, var::VarName=:x;
↪  max_precision::Int=prec, cached::Bool=true) where T
```

Return the absolute power series over R in the variable `var` whose first `len` coefficients are given by the first `len` entries of `arr`: `arr[i]` is the coefficient of `var^(i - 1)`. The input must satisfy `0 <= len <= length(arr)` and `prec >= len`, where `prec` is the absolute precision. The series ring is created on the fly with maximum absolute precision `max_precision`; setting `cached` to `false` prevents it from being cached.

**Examples**

```
julia> abs_series(ZZ, [1, 2, 3], 3, 5, :y)
1 + 2*y + 3*y^2 + O(y^5)

julia> abs_series(ZZ, [1, 2, 3], 1, 6; cached=false)
1 + O(x^6)

julia> abs_series(ZZ, [1, 2, 3], 3, 5; max_precision=10)
1 + 2*x + 3*x^2 + O(x^5)

julia> abs_series(ZZ, [], 0, 6)
O(x^6)
```

source

`AbstractAlgebra.rel_series` – Function.

```
rel_series(R::Ring, arr::Vector{T}, len::Int, prec::Int, val::Int, var::VarName=:x;
↪  max_precision::Int=prec, cached::Bool=true) where T
```

Return the relative power series over R in the variable `var` of valuation `val` whose first `len` coefficients are given by the first `len` entries of `arr`: `arr[i]` is the coefficient of `var^(val + i - 1)`. The input must satisfy `0 <= len <= length(arr)` and `prec >= val + len`, where `prec` is the absolute precision. The series ring is created on the fly with maximum relative precision `max_precision`; setting `cached` to `false` prevents it from being cached.

**Examples**

```
julia> rel_series(ZZ, [1, 2, 3], 3, 7, 4)
x^4 + 2*x^5 + 3*x^6 + O(x^7)

julia> rel_series(ZZ, BigInt[], 0, 3, 1)
O(x^3)
```

source

AbstractAlgebra.Generic.laurent_series – Function.

```
laurent_series(R::Ring, arr::Vector{T}, len::Int, prec::Int, val::Int, scale::Int,
↪  var::VarName=:x; max_precision::Int=prec, cached::Bool=true) where T
laurent_series(R::Field, arr::Vector{T}, len::Int, prec::Int, val::Int, scale::Int,
↪  var::VarName=:x; max_precision::Int=prec, cached::Bool=true) where T
```

Return the Laurent series over R in the variable `var` of valuation `val` whose first `len` coefficients are given by the first `len` entries of `arr`. The entry `arr[i]` is the coefficient of `var^(val + (i - 1)*scale)`, so consecutive coefficients are `scale` exponents apart. The input must satisfy `0 <= len <= length(arr)`, `scale > 0`, and `prec >= val + (len - 1)*scale + 1`, where `prec` is the absolute precision. The Laurent series parent is created on the fly with maximum relative precision `max_precision`. It is a field when R is a field and a ring otherwise. Setting `cached` to `false` prevents the parent from being cached.

**Examples**

```
julia> laurent_series(ZZ, [1, 2, 3], 3, 5, 0, 2)
1 + 2*x^2 + 3*x^4 + O(x^5)

julia> laurent_series(ZZ, [1, 2, 3], 3, 5, 0, 2; max_precision=10)
1 + 2*x^2 + 3*x^4 + O(x^5)
```

source

## 24.5 Big-oh notation

Series elements can be given a precision using the big-oh notation. This is provided by a function of the following form, (or something equivalent for Laurent series):

```
O(x::SeriesElem)
```

**Examples**

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> S, y = laurent_series_ring(ZZ, 10, :y)
(Laurent series ring in y over integers, y + O(y^11))

julia> f = 1 + 2x + O(x^5)
1 + 2*x + O(x^5)
```

```
julia> g = 2y + 7y^2 + O(y^7)
2*y + 7*y^2 + O(y^7)
```

What is happening here in practice is that `O(x^n)` is creating the series `0 + O(x^n)` and the rules for addition of series dictate that if this is added to a series of greater precision, then the lower of the two precisions must be used.

Of course it may be that the precision of the series that `O(x^n)` is added to is already lower than `n`, in which case adding `O(x^n)` has no effect. This is the case if the default precision is too low, since `x` on its own has the default precision.

## 24.6 Power series models

Capped relative power series have their maximum relative precision capped at some value `prec_max`. This means that if the leading term of a nonzero power series element is $c_a x^a$ and the precision is $b$ then the power series is of the form $c_a x^a + c_{a+1}x^{a+1} + \ldots + O(x^{a+b})$.

The zero power series is simply taken to be $0 + O(x^b)$.

The capped relative model has the advantage that power series are stable multiplicatively. In other words, for nonzero power series $f$ and $g$ we have that `divexact(f*g), g) == f`.

However, capped relative power series are not additively stable, i.e. we do not always have $(f + g) - g = f$.

Similar comments apply to Laurent series.

On the other hand, capped absolute power series have their absolute precision capped. This means that if the leading term of a nonzero power series element is $c_a x^a$ and the precision is $b$ then the power series is of the form $c_a x^a + c_{a+1}x^{a+1} + \ldots + O(x^b)$.

Capped absolute series are additively stable, but not necessarily multiplicatively stable.

For all models, the maximum precision is also used as a default precision in the case of coercing coefficients into the ring and for any computation where the result could mathematically be given to infinite precision.

In all models we say that two power series are equal if they agree up to the minimum **absolute** precision of the two power series.

Thus, for example, $x^5 + O(x^{10}) == 0 + O(x^5)$, since the minimum absolute precision is $5$.

During computations, it is possible for power series to lose relative precision due to cancellation. For example if $f = x^3 + x^5 + O(x^8)$ and $g = x^3 + x^6 + O(x^8)$ then $f - g = x^5 - x^6 + O(x^8)$ which now has relative precision $3$ instead of relative precision $5$.

Amongst other things, this means that equality is not transitive. For example $x^6 + O(x^{11}) == 0 + O(x^5)$ and $x^7 + O(x^{12}) == 0 + O(x^5)$ but $x^6 + O(x^{11}) \neq x^7 + O(x^{12})$.

Sometimes it is necessary to compare power series not just for arithmetic equality, as above, but to see if they have precisely the same precision and terms. For this purpose we introduce the `isequal` function.

For example, if $f = x^2 + O(x^7)$ and $g = x^2 + O(x^8)$ and $h = 0 + O(x^2)$ then $f == g$, $f == h$ and $g == h$, but `isequal(f, g)`, `isequal(f, h)` and `isequal(g, h)` would all return `false`. However, if $k = x^2 + O(x^7)$ then `isequal(f, k)` would return `true`.

There are further difficulties if we construct polynomial over power series. For example, consider the polynomial in $y$ over the power series ring in $x$ over the rationals. Normalisation of such polynomials is problematic. For instance, what is the leading coefficient of $(0 + O(x^{10}))y + (1 + O(x^{10}))$?

If one takes it to be $(0 + O(x^{10}))$ then some functions may not terminate due to the fact that algorithms may require the degree of polynomials to decrease with each iteration. Instead, the degree may remain constant and simply accumulate leading terms which are arithmetically zero but not identically zero.

On the other hand, when constructing power series over other power series, if we simply throw away terms which are arithmetically equal to zero, our computations may have different output depending on the order in which the power series are added!

One should be aware of these difficulties when working with power series. Power series, as represented on a computer, simply don't satisfy the axioms of a ring. They must be used with care in order to approximate operations in a mathematical power series ring.

Simply increasing the precision will not necessarily give a "more correct" answer and some computations may not even terminate due to the presence of arithmetic zeroes!

An absolute power series ring over a ring $R$ with precision $p$ behaves very much like the quotient $R[x]/(x^p)$ of the polynomial ring over $R$. Therefore one can often treat absolute power series rings as though they were rings. However, this depends on all series being given a precision equal to the specified maximum precision and not a lower precision.

## 24.7 Functions for types and parents of series rings

```
base_ring(R::SeriesRing)
base_ring(a::SeriesElem)
```

Return the coefficient ring of the given series ring or series.

```
parent(a::SeriesElem)
```

Return the parent of the given series.

## 24.8 Series functions

Unless otherwise noted, the functions below are available for all series models, including Laurent series. We denote this by using the abstract type `RelPowerSeriesRingElem`, even though absolute series and Laurent series types do not belong to this abstract type.

### Basic functionality

Series implement the Ring Interface

```
zero(R::SeriesRing)
one(R::SeriesRing)
iszero(a::SeriesElem)
isone(a::SeriesElem)
```

```
divexact(a::T, b::T) where T <: SeriesElem
inv(a::SeriesElem)
```

Series also implement the Series Interface, the most important basic functions being the following.

```
var(S::SeriesRing)
```

Return a symbol for the variable of the given series ring.

```
max_precision(S::SeriesRing)
```

Return the precision cap of the given series ring.

```
precision(f::SeriesElem)
valuation(f::SeriesElem)
```

```
gen(R::SeriesRing)
```

The following functions are also provided for all series.

```
coeff(a::SeriesElem, n::Int)
```

Return the degree $n$ coefficient of the given power series. Note coefficients are numbered from $n = 0$ for the constant coefficient. If $n$ exceeds the current precision of the power series, the function returns a zero coefficient.

For power series types, $n$ must be non-negative. Laurent series do not have this restriction.

`AbstractAlgebra.modulus` – Method.

```
modulus(a::SeriesElem{T}) where {T <: ResElem}
```

Return the modulus of the coefficients of the given power series.

source

`AbstractAlgebra.is_gen` – Method.

```
is_gen(a::RelPowerSeriesRingElem)
```

Return `true` if the given power series is arithmetically equal to the generator of its power series ring to its current precision, otherwise return `false`.

source

**Examples**

```
julia> S, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> f = 1 + 3x + x^3 + O(x^10)
1 + 3*x + x^3 + O(x^10)

julia> g = 1 + 2x + x^2 + O(x^10)
1 + 2*x + x^2 + O(x^10)

julia> h = zero(S)
O(x^10)

julia> k = one(S)
1 + O(x^10)

julia> isone(k)
true

julia> iszero(f)
false

julia> n = pol_length(f)
4

julia> c = polcoeff(f, 3)
1

julia> U = base_ring(S)
Integers

julia> v = var(S)
:x

julia> max_precision(S) == 10
true

julia> T = parent(x + 1)
Univariate power series ring in x with precision 10
  over integers

julia> g == deepcopy(g)
true

julia> t = divexact(2g, 2)
1 + 2*x + x^2 + O(x^10)

julia> p = precision(f)
10

julia> R, t = power_series_ring(QQ, 10, :t)
(Univariate power series ring over rationals, t + O(t^11))

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))
```

```
julia> a = O(x^4)
O(x^4)

julia> b = (t + 3)*x + (t^2 + 1)*x^2 + O(x^4)
(3 + t + O(t^10))*x + (1 + t^2 + O(t^10))*x^2 + O(x^4)

julia> k = is_gen(gen(R))
true

julia> m = is_unit(-1 + x + 2x^2)
true

julia> n = valuation(a)
4

julia> p = valuation(b)
1

julia> c = coeff(b, 2)
1 + t^2 + O(t^10)

julia> S, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> f = 1 + 3x + x^3 + O(x^5)
1 + 3*x + x^3 + O(x^5)

julia> g = S(BigInt[1, 2, 0, 1, 0, 0, 0], 4, 10, 3);

julia> set_length!(g, 3)
x^3 + 2*x^4 + O(x^10)

julia> g = setcoeff!(g, 2, BigInt(11))
x^3 + 2*x^4 + 11*x^5 + O(x^10)

julia> fit!(g, 8)

julia> g = setcoeff!(g, 7, BigInt(4))
x^3 + 2*x^4 + 11*x^5 + O(x^10)
```

### Change base ring

`AbstractAlgebra.map_coefficients` – Method.

```
map_coefficients(f, p::SeriesElem{<: RingElement}; cached::Bool=true, parent::PolyRing)
```

Transform the series `p` by applying `f` on each non-zero coefficient.

If the optional `parent` keyword is provided, the polynomial will be an element of `parent`. The caching of the parent object can be controlled via the `cached` keyword argument.

**Examples**

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> f = 4*x^6 + x^7 + 9*x^8 + 16*x^9 + 25*x^10 + O(x^11)
4*x^6 + x^7 + 9*x^8 + 16*x^9 + 25*x^10 + O(x^11)

julia> map_coefficients(AbstractAlgebra.sqrt, f)
2*x^6 + x^7 + 3*x^8 + 4*x^9 + 5*x^10 + O(x^11)
```

source

AbstractAlgebra.change_base_ring – Method.

```
change_base_ring(R::Ring, p::SeriesElem{<: RingElement}; parent::PolyRing)
```

Return the series obtained by coercing the non-zero coefficients of p into R.

If the optional parent keyword is provided, the series will be an element of parent. The caching of the parent object can be controlled via the cached keyword argument.

**Examples**

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> f = 4*x^6 + x^7 + 9*x^8 + 16*x^9 + 25*x^10 + O(x^11)
4*x^6 + x^7 + 9*x^8 + 16*x^9 + 25*x^10 + O(x^11)

julia> change_base_ring(QQ, f)
4*x^6 + x^7 + 9*x^8 + 16*x^9 + 25*x^10 + O(x^11)
```

source

### Shifting

AbstractAlgebra.shift_left – Method.

```
shift_left(x::RelPowerSeriesRingElem{T}, n::Int) where T <: RingElement
```

Return the power series $x$ shifted left by $n$ terms, i.e. multiplied by $t^n$, where $t$ is the variable of the ring.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))

julia> a = 2x + x^3
```

```
2*x + x^3 + O(x^31)

julia> b = O(x^4)
O(x^4)

julia> f = shift_left(a, 2)
2*x^3 + x^5 + O(x^33)

julia> g = shift_left(b, 2)
O(x^6)
```

source

AbstractAlgebra.shift_right – Method.

```
shift_right(x::RelPowerSeriesRingElem{T}, n::Int) where T <: RingElement
```

Return the power series $x$ shifted right by $n$ terms, i.e. divided by $t^n$, where $t$ is the variable of the ring.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))

julia> shift_right(1 + x + 2x^2 + O(x^5), 1)
1 + 2*x + O(x^4)

julia> shift_right(2x + x^3 + O(x^4), 3)
1 + O(x^1)
```

source

### Truncation

Base.truncate – Method.

```
truncate(a::RelPowerSeriesRingElem{T}, n::Int) where T <: RingElement
```

Return $a$ truncated to (absolute) precision $n$.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))
```

```
julia> a = 2x + x^3
2*x + x^3 + O(x^31)

julia> b = O(x^4)
O(x^4)

julia> c = 1 + x + 2x^2 + O(x^5)
1 + x + 2*x^2 + O(x^5)

julia> d = 2x + x^3 + O(x^4)
2*x + x^3 + O(x^4)

julia> f = truncate(a, 3)
2*x + O(x^3)

julia> g = truncate(b, 2)
O(x^2)

julia> h = truncate(c, 7)
1 + x + 2*x^2 + O(x^5)

julia> k = truncate(d, 5)
2*x + x^3 + O(x^4)
```

source

### Division

Base.inv – Method.

```
Base.inv(a::RelPowerSeriesRingElem)
```

Return the inverse of the power series $a$, i.e. $1/a$.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))

julia> a = 1 + x + 2x^2 + O(x^5)
1 + x + 2*x^2 + O(x^5)

julia> b = S(-1)
-1 + O(x^30)

julia> c = inv(a)
1 - x - x^2 + 3*x^3 - x^4 + O(x^5)
```

```
julia> d = inv(b)
-1 + O(x^30)
```

source

### Composition

AbstractAlgebra.compose – Method.

```
compose(f::RelPowerSeriesRingElem, g::RelPowerSeriesRingElem; inner)
```

Compose the series $a$ with the series $b$ and return the result.

- If inner = :second, then f(g) is returned and g must have positive valuation.
- If inner = :first, then g(f) is returned and f must have positive valuation.

source

Note that subst can be used instead of compose, however the provided functionality is the same. General series substitution is not well-defined.

### Derivative and integral

AbstractAlgebra.derivative – Method.

```
derivative(f::AbsPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

source

```
derivative(f::RelPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> derivative(f)
1 + 9*x^2 + O(x^9)
```

source

```
derivative(f::MPolyRingElem{T}, j::Int) where {T <: RingElement}
```

Return the partial derivative of f with respect to $j$-th variable of the polynomial ring.

source

```
derivative(f::MPolyRingElem{T}, x::MPolyRingElem{T}) where T <: RingElement
```

Return the partial derivative of f with respect to x. The value x must be a generator of the polynomial ring of f.

**Examples**

```
julia> R, (x, y) = AbstractAlgebra.polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x*y + x + y + 1
x*y + x + y + 1

julia> derivative(f, x)
y + 1

julia> derivative(f, y)
x + 1

julia> derivative(f, 1)
y + 1

julia> derivative(f, 2)
x + 1
```

source

```
derivative(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the derivative of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> f = x^(5//3) + x^(7//3) + x^(11//3)
x^(5//3) + x^(7//3) + x^(11//3) + O(x^5)

julia> derivative(f)
5//3*x^(2//3) + 7//3*x^(4//3) + 11//3*x^(8//3) + O(x^4)
```

source

AbstractAlgebra.integral – Method.

```
integral(f::AbsPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

source

```
integral(f::RelPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> integral(f)
2*x + 1//2*x^2 + 3//4*x^4 + O(x^11)
```

source

```
integral(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the integral of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> integral(x^(5//3) + x^(7//3) + x^(11//3))
3//8*x^(8//3) + 3//10*x^(10//3) + 3//14*x^(14//3) + O(x^6)
```

source

### Special functions

Base.log – Method.

```
log(a::SeriesElem{T}) where T <: FieldElement
```

Return the logarithm of the power series $a$.

source

```
log(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the logarithm of the given Puiseux series $a$.

**Examples**

```
julia> T, z = puiseux_series_ring(QQ, 30, :z)
(Puiseux series field in z over rationals, z + O(z^31))

julia> log(1 + z + 3z^2 + O(z^5))
z + 5//2*z^2 - 8//3*z^3 - 7//4*z^4 + O(z^5)
```

source

Base.exp – Method.

```
exp(a::AbsPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::RelPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.LaurentSeriesElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the exponential of the given Puiseux series $a$.

**Examples**

```
julia> T, z = puiseux_series_ring(QQ, 30, :z)
(Puiseux series field in z over rationals, z + O(z^31))

julia> exp(z + 2z^2 + 5z^3 + O(z^5))
1 + z + 5//2*z^2 + 43//6*z^3 + 193//24*z^4 + O(z^5)
```

source

Methods for `is_square` and `sqrt` are provided for inputs of type `RelPowerSeriesRingElem`.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S, x = power_series_ring(R, 30, :x)
(Univariate power series ring over R, x + O(x^31))

julia> T, z = power_series_ring(QQ, 30, :z)
(Univariate power series ring over rationals, z + O(z^31))

julia> a = 1 + z + 3z^2 + O(z^5)
1 + z + 3*z^2 + O(z^5)

julia> b = z + 2z^2 + 5z^3 + O(z^5)
z + 2*z^2 + 5*z^3 + O(z^5)

julia> c = exp(x + O(x^40))
1 + x + 1//2*x^2 + 1//6*x^3 + 1//24*x^4 + 1//120*x^5 + 1//720*x^6 + 1//5040*x^7 + 1//40320*x^8 +
↪  1//362880*x^9 + 1//3628800*x^10 + 1//39916800*x^11 + 1//479001600*x^12 + 1//6227020800*x^13 +
↪  1//87178291200*x^14 + 1//1307674368000*x^15 + 1//20922789888000*x^16 + 1//355687428096000*x^17
↪  + 1//6402373705728000*x^18 + 1//121645100408832000*x^19 + 1//2432902008176640000*x^20 +
↪  1//51090942171709440000*x^21 + 1//1124000727777607680000*x^22 + 1//25852016738884976640000*x^23
↪  + 1//620448401733239439360000*x^24 + 1//15511210043330985984000000*x^25 +
↪  1//403291461126605635584000000*x^26 + 1//10888869450418352160768000000*x^27 +
↪  1//304888344611713860501504000000*x^28 + 1//8841761993739701954543616000000*x^29 +
↪  1//265252859812191058636308480000000*x^30 + O(x^31)

julia> d = divexact(x, exp(x + O(x^40)) - 1)
1 - 1//2*x + 1//12*x^2 - 1//720*x^4 + 1//30240*x^6 - 1//1209600*x^8 + 1//47900160*x^10 -
↪  691//1307674368000*x^12 + 1//74724249600*x^14 - 3617//10670622842880000*x^16 +
↪  43867//51090942171709440000*x^18 - 174611//802857662698291200000*x^20 +
↪  77683//1410110003939180544000*x^22 - 236364091//1693824136731743669452800000*x^24 +
↪  657931//1861345205199718318080000000*x^26 - 3392780147//3789326568745586551947264000000*x^28 +
↪  O(x^29)

julia> f = exp(b)
1 + z + 5//2*z^2 + 43//6*z^3 + 193//24*z^4 + O(z^5)

julia> log(exp(b)) == b
true

julia> h = sqrt(a)
1 + 1//2*z + 11//8*z^2 - 11//16*z^3 - 77//128*z^4 + O(z^5)
```

### Random generation

Random series can be constructed using the rand function. A range of possible valuations is provided. The maximum precision of the ring is used as a bound on the precision. Other parameters are used to construct random coefficients.

```
rand(R::SeriesRing, val_range::AbstractUnitRange{Int}, v...)
```

### Examples

```
julia> R, x = power_series_ring(ZZ, 10, :x)
(Univariate power series ring over integers, x + O(x^11))

julia> f = rand(R, 3:5, -10:10)
3*x^4 - x^5 + 4*x^7 + 4*x^8 - 7*x^9 + 2*x^10 + 4*x^11 - x^12 - 4*x^13 + O(x^14)
```

# Chapter 25

# Generic Puiseux series

AbstractAlgebra.jl allows the creation of Puiseux series over any computable commutative ring $R$.

Puiseux series are power series of the form $a_j x^{j/m} + a_{j+1}x^{(j+1)/m} + \cdots + a_{k-1}x^{(k-1)/m} + O(x^{k/m})$ for some integer $m > 0$ where $i \geq 0$, $a_i \in R$ and the relative precision $k - j$ is at most equal to some specified precision $n$.

The generic Puiseux series module is implemented in `src/generic/PuiseuxSeries.jl`.

As well as implementing the Series Ring interface, the Puiseux series module in AbstractAlgebra.jl implements the generic algorithms described below.

All of the generic functionality is part of the `Generic` submodule of AbstractAlgebra.jl. This is exported by default so that it is not necessary to qualify function names.

## 25.1 Types and parent objects

The types of generic Puiseux series implemented by AbstractAlgebra.jl are `Generic.PuiseuxSeriesRingElem{T}` and `Generic.PuiseuxSeriesFieldElem{T}`.

Both series element types belong to the union type `Generic.PuiseuxSeriesElem`.

Puiseux series elements belong directly to either `RingElem` or `FieldElem` since it is more useful to be able to distinguish whether they belong to a ring or field than it is to distinguish that they are Puiseux series.

The parent types for Puiseux series, `Generic.PuiseuxSeriesRing{T}` and `Generic.PuiseuxSeriesField{T}` respectively, belong to `Ring` and `Field` respectively.

The default precision, string representation of the variable and base ring $R$ of a generic Puiseux series are stored in its parent object.

## 25.2 Puiseux series ring constructors

In order to construct Puiseux series in AbstractAlgebra.jl, one must first construct the ring itself. This is accomplished with any of the following constructors.

`AbstractAlgebra.puiseux_series_ring` – Function.

```
puiseux_series_ring(R::Ring, prec::Int, s::VarName; cached::Bool=true)
puiseux_series_field(R::Field, prec::Int, s::VarName; cached::Bool=true)
```

Return a tuple $(S, x)$ consisting of the parent object S of a Puiseux series ring over the given base ring and a generator x for the Puiseux series ring. The maximum precision of the series in the ring is set to `prec`. This is taken as a maximum relative precision of the underlying Laurent series that are used to implement the Puiseux series in the ring. The supplied string s specifies the way the generator of the Puiseux series ring will be printed. By default, the parent object S will be cached so that supplying the same base ring, string and precision in future will return the same parent object and generator. If caching of the parent object is not required, cached can be set to `false`.

**Examples**

```
julia> R, x = puiseux_series_ring(ZZ, 10, :x)
(Puiseux series ring in x over integers, x + O(x^11))

julia> S, y = puiseux_series_field(QQ, 10, :y)
(Puiseux series field in y over rationals, y + O(y^11))

julia> R()
O(x^10)

julia> S(123)
123 + O(y^10)

julia> S(y + 1)
1 + y + O(y^10)
```

source

`AbstractAlgebra.puiseux_series_field` – Function.

```
puiseux_series_field(R::Field, prec::Int, s::VarName; cached::Bool=true)
```

See `puiseux_series_ring`.

source

## 25.3 Big-oh notation

Series elements can be given a precision using the big-oh notation. This is provided by a function of the following form, (or something equivalent for Laurent series):

```
O(x::SeriesElem)
```

**Examples**

```
julia> R, x = puiseux_series_ring(ZZ, 10, :x)
(Puiseux series ring in x over integers, x + O(x^11))

julia> f = 1 + 2x + O(x^5)
1 + 2*x + O(x^5)

julia> g = 2x^(1//3) + 7x^(2//3) + O(x^(7//3))
2*x^(1//3) + 7*x^(2//3) + O(x^(7//3))
```

What is happening here in practice is that `O(x^n)` is creating the series `0 + O(x^n)` and the rules for addition of series dictate that if this is added to a series of greater precision, then the lower of the two precisions must be used.

Of course it may be that the precision of the series that `O(x^n)` is added to is already lower than `n`, in which case adding `O(x^n)` has no effect. This is the case if the default precision is too low, since `x` on its own has the default precision.

## 25.4 Puiseux series implementation

Puiseux series have their maximum relative precision capped at some value `prec_max`. This refers to the internal Laurent series used to store the Puiseux series, i.e. the series without denominators in the exponents.

The Puiseux series type stores such a Laurent series and a `scale` or denominator for the exponents. For example, $f(x) = 1 + x^{1/3} + 2x^{2/3} + O(x^{7/3})$ would be stored as a Laurent series $1 + x + 2x^2 + O(x^7)$ and a scale of $3$..

The maximum precision is also used as a default (Laurent) precision in the case of coercing coefficients into the ring and for any computation where the result could mathematically be given to infinite precision.

In all models we say that two Puiseux series are equal if they agree up to the minimum **absolute** precision of the two power series.

Thus, for example, $x^5 + O(x^{10}) == 0 + O(x^5)$, since the minimum absolute precision is $5$.

Sometimes it is necessary to compare Puiseux series not just for arithmetic equality, as above, but to see if they have precisely the same precision and terms. For this purpose we introduce the `isequal` function.

For example, if $f = x^2 + O(x^7)$ and $g = x^2 + O(x^8)$ and $h = 0 + O(x^2)$ then $f == g$, $f == h$ and $g == h$, but `isequal(f, g)`, `isequal(f, h)` and `isequal(g, h)` would all return `false`. However, if $k = x^2 + O(x^7)$ then `isequal(f, k)` would return `true`.

There are a number of technicalities that must be observed when working with Puiseux series. As these are the same as for the other series rings in AbstractAlgebra.jl, we refer the reader to the documentation of series rings for information about these issues.

## 25.5 Basic ring functionality

All Puiseux series provide the functionality described in the Ring and Series Ring interfaces with the exception of the `pol_length` and `polcoeff` functions. Naturally the `set_precision!`, `set_valuation!` and `coeff` functions can take a rational exponent.

**Examples**

```
julia> S, x = puiseux_series_ring(ZZ, 10, :x)
(Puiseux series ring in x over integers, x + O(x^11))

julia> f = 1 + 3x + x^3 + O(x^10)
1 + 3*x + x^3 + O(x^10)

julia> g = 1 + 2x^(1//3) + x^(2//3) + O(x^(7//3))
1 + 2*x^(1//3) + x^(2//3) + O(x^(7//3))

julia> h = zero(S)
O(x^10)

julia> k = one(S)
```

```
1 + O(x^10)

julia> isone(k)
true

julia> iszero(f)
false

julia> coeff(g, 1//3)
2

julia> U = base_ring(S)
Integers

julia> v = var(S)
:x

julia> T = parent(x + 1)
Puiseux series ring in x over integers

julia> g == deepcopy(g)
true

julia> t = divexact(2g, 2)
1 + 2*x^(1//3) + x^(2//3) + O(x^(7//3))

julia> p = precision(f)
10//1
```

## 25.6 Puiseux series functionality provided by AbstractAlgebra.jl

The functionality below is automatically provided by AbstractAlgebra.jl for any Puiseux series.

Of course, modules are encouraged to provide specific implementations of the functions described here, that override the generic implementation.

### Basic functionality

```
coeff(a::Generic.PuiseuxSeriesElem, n::Int)
```

```
coeff(a::Generic.PuiseuxSeriesElem, n::Rational{Int})
```

Return the coefficient of the term of exponent $n$ of the given power series. If $n$ exceeds the current precision of the power series or does not correspond to a nonzero term of the Puiseux series, the function returns a zero coefficient.

`AbstractAlgebra.modulus` – Method.

```
modulus(a::Generic.PuiseuxSeriesElem{T}) where {T <: ResElem}
```

Return the modulus of the coefficients of the given Puiseux series.

source

AbstractAlgebra.is_gen – Method.

```
is_gen(a::Generic.PuiseuxSeriesElem)
```

Return `true` if the given Puiseux series is arithmetically equal to the generator of its Puiseux series ring to its current precision, otherwise return `false`.

source

**Examples**

```
julia> R, t = puiseux_series_ring(QQ, 10, :t)
(Puiseux series field in t over rationals, t + O(t^11))

julia> S, x = puiseux_series_ring(R, 30, :x)
(Puiseux series field in x over R, x + O(x^31))

julia> a = O(x^4)
O(x^4)

julia> b = (t + 3)*x + (t^2 + 1)*x^2 + O(x^4)
(3 + t + O(t^10))*x + (1 + t^2 + O(t^10))*x^2 + O(x^4)

julia> k = is_gen(gen(R))
true

julia> m = is_unit(-1 + x^(1//3) + 2x^2)
true

julia> n = valuation(a)
4//1

julia> p = valuation(b)
1//1

julia> c = coeff(b, 2)
1 + t^2 + O(t^10)
```

## Division

Base.inv – Method.

```
inv(A::MatElem{T}) where {T <: RingElement}
```

Given an invertible $n \times n$ matrix $A$ over a ring, return the $n \times n$ matrix $X$ such that $MX = I_n$, where $I_n$ is the $n \times n$ identity matrix.

If $A$ is not invertible over the base ring, an exception is raised.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> inv(M)
[-5//3    2//3    1//1]
[ 4//3   -1//3   -2//1]
[ 0//1    0//1    1//1]
```

source

```
Base.inv(a::PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the inverse of the power series $a$, i.e. $1/a$, if it exists. Otherwise an exception is raised.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 30, :x)
(Puiseux series field in x over rationals, x + O(x^31))

julia> a = 1 + x + 2x^2 + O(x^5)
1 + x + 2*x^2 + O(x^5)

julia> b = R(-1)
-1 + O(x^30)

julia> c = inv(a)
1 - x - x^2 + 3*x^3 - x^4 + O(x^5)

julia> d = inv(b)
-1 + O(x^30)
```

source

```
inv(a::LocalizedEuclideanRingElem{T}, checked::Bool = true)  where {T <: RingElem}
```

Returns the inverse element of $a$ if $a$ is a unit. If 'checked = false' the invertibility of $a$ is not checked and the corresponding inverse element of the Fraction Field is returned.

source

## 25.7 Derivative and integral

`AbstractAlgebra.derivative` – Method.

```
derivative(f::AbsPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

source

```
derivative(f::RelPowerSeriesRingElem{T})
```

Return the derivative of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> derivative(f)
1 + 9*x^2 + O(x^9)
```

source

```
derivative(f::MPolyRingElem{T}, j::Int) where {T <: RingElement}
```

Return the partial derivative of `f` with respect to $j$-th variable of the polynomial ring.

source

```
derivative(f::MPolyRingElem{T}, x::MPolyRingElem{T}) where T <: RingElement
```

Return the partial derivative of `f` with respect to `x`. The value `x` must be a generator of the polynomial ring of `f`.

**Examples**

```
julia> R, (x, y) = AbstractAlgebra.polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> f = x*y + x + y + 1
x*y + x + y + 1

julia> derivative(f, x)
y + 1

julia> derivative(f, y)
x + 1
```

```
julia> derivative(f, 1)
y + 1

julia> derivative(f, 2)
x + 1
```

source

```
derivative(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the derivative of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> f = x^(5//3) + x^(7//3) + x^(11//3)
x^(5//3) + x^(7//3) + x^(11//3) + O(x^5)

julia> derivative(f)
5//3*x^(2//3) + 7//3*x^(4//3) + 11//3*x^(8//3) + O(x^4)
```

source

`AbstractAlgebra.integral` – Method.

```
integral(f::AbsPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

source

```
integral(f::RelPowerSeriesRingElem{T})
```

Return the integral of the power series $f$.

```
julia> R, x = power_series_ring(QQ, 10, :x)
(Univariate power series ring in x over Rationals, x + O(x^11))

julia> f = 2 + x + 3x^3
2 + x + 3*x^3 + O(x^10)

julia> integral(f)
2*x + 1//2*x^2 + 3//4*x^4 + O(x^11)
```

source

```
integral(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the integral of the given Puiseux series $a$.

**Examples**

```
julia> R, x = puiseux_series_ring(QQ, 10, :x)
(Puiseux series field in x over rationals, x + O(x^11))

julia> integral(x^(5//3) + x^(7//3) + x^(11//3))
3//8*x^(8//3) + 3//10*x^(10//3) + 3//14*x^(14//3) + O(x^6)
```

source

### Special functions

Base.log – Method.

```
log(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the logarithm of the given Puiseux series $a$.

**Examples**

```
julia> T, z = puiseux_series_ring(QQ, 30, :z)
(Puiseux series field in z over rationals, z + O(z^31))

julia> log(1 + z + 3z^2 + O(z^5))
z + 5//2*z^2 - 8//3*z^3 - 7//4*z^4 + O(z^5)
```

source

Base.exp – Method.

```
exp(a::AbsPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::RelPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.LaurentSeriesElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the exponential of the given Puiseux series $a$.

**Examples**

```
julia> T, z = puiseux_series_ring(QQ, 30, :z)
(Puiseux series field in z over rationals, z + O(z^31))

julia> exp(z + 2z^2 + 5z^3 + O(z^5))
1 + z + 5//2*z^2 + 43//6*z^3 + 193//24*z^4 + O(z^5)
```

source

Methods for `is_square` and `sqrt` are provided for inputs of type `PuiseuxSeriesElem`.

# Chapter 26

# Multivariate series

AbstractAlgebra.jl provide multivariate series over a commutative ring.

Series with capped absolute precision are provided with and without weights.

For the unweighted case precision in each variable can be set per series, but is capped at some maximum precision which is set when defining the ring.

For the weighted case, a single precision is set on the ring only. Terms are truncated at that precision (after applying weights).

## 26.1 Generic multivariate series

Generic multivariate series over a commutative ring, `AbsMSeries{T}` is implemented in `src/generic/AbsMSeries.jl`.

Such series are capped absolute series and have type `Generic.AbsMSeries{T}` where T is the type of elements of the coefficient ring.

Internally they consist of a multivariate polynomial. For unweighted series they also contain a vector of precisions, one for each variable.

For weighted series weights and a precision are stored on the ring only. The vector of precisions in the series objects is ignored.

See the file `src/generic/GenericTypes.jl` for details of the type.

The series are implemented in terms of multivariate polynomials which are used internally to keep track of the coefficients of the series.

Only lex ordering is provided at present both weighted and unweighted, though series print in reverse order to what multivariate polynomials would print, i.e. least significant term first, as would be expected for series.

Parent objects of such series have type `Generic.AbsMSeriesRing{T}`.

The symbol representation of the variables and the multivariate polynomial ring is stored in the parent object.

## 26.2 Abstract types

Multivariate series element types belong to the abstract type `MSeriesElem{T}` and the multivariate series ring types belong to the abstract type `MSeriesRing{T}`. This enables one to write generic functions that can accept any AbstractAlgebra multivariate series type.

## 26.3 Multivariate series ring constructors

In order to construct multivariate series in AbstractAlgebra.jl, one must first construct the series ring itself. This is accomplished with the following constructors.

`AbstractAlgebra.power_series_ring` – Method.

```
power_series_ring(R::Ring, prec::Vector{Int}, varnames::Vector{Symbol}; cached::Bool=true)
power_series_ring(R::Ring, prec::Int, varnames::Vector{Symbol}; weights::Vector{Int},
↪  cached::Bool=true)
power_series_ring(R::Ring, weights::Vector{Int}, prec::Int, varnames::Vector{Symbol};
↪  cached::Bool=true)
```

Given a base ring R and variable names, say `varnames = [:x, :y, ...]`, return a tuple `S, [x, y, ...]` of the multivariate power series ring $S = R[[x, y, \dots]]$ and its generators.

Precision is absolute and capped per variable by the matching entry of `prec`; a single `Int` caps every variable by the same value. Given `weights`, the cap is instead a single bound `prec` on the total weighted degree, where the $i$-th variable has weight `weights[i]`.

By default (`cached=true`), the output S will be cached, i.e. if `power_series_ring` is invoked again with the same arguments, the same (*identical*) ring is returned. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

**Examples**

```
julia> R, (x, y) = power_series_ring(ZZ, [2, 3], [:x, :y])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(y^3)
↪  + O(x^2), y + O(y^3) + O(x^2)])

julia> x + y + O(y^2)
y + x + O(y^2) + O(x^2)

julia> power_series_ring(ZZ, [2, 3], 10, [:u, :v])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[u + O(10),
↪  v + O(10)])
```

source

Here are some examples of creating multivariate series rings and making use of the resulting parent objects to coerce various elements into the series ring.

Note that one can also use the function call `O(x^n)` with unweighted series to specify the precision in the variable x of a given series expression should be precision n.

**Note**

It is not possible to use `x^0` in the `O()` function, since there is no distinction between `x^0` and `y^0` as far as the system is concerned. If one wishes to set the precision of a variable to precision `0`, one must use the `set_precision!` function described below.

If one wants a series with the same precision in all variables, one can use `O(R, n)` where R is the series ring and n is the desired precision.

If all the precisions are to be the same, the vector of integers for the precisions can be replaced by a single integer in the constructor.

**Examples**

```
julia> R, (x, y) = power_series_ring(ZZ, [2, 3], [:x, :y])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(y^3) +
↪  O(x^2), y + O(y^3) + O(x^2)])

julia> f = R()
O(y^3) + O(x^2)

julia> g = R(123)
123 + O(y^3) + O(x^2)

julia> h = R(BigInt(1234))
1234 + O(y^3) + O(x^2)

julia> k = R(x + 1)
1 + x + O(y^3) + O(x^2)

julia> m = x + y + O(y^2)
y + x + O(y^2) + O(x^2)

julia> R, (x, y) = power_series_ring(ZZ, 3, [:x, :y])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(y^3) +
↪  O(x^3), y + O(y^3) + O(x^3)])

julia> n = x + y + O(R, 2)
y + x + O(y^2) + O(x^2)

julia> R, (x, y) = power_series_ring(ZZ, [2, 3], 10, [:x, :y])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(10), y
↪  + O(10)])

julia> R()
O(10)

julia> R(x)
x + O(10)
```

## 26.4 Basic ring functionality

Once a multivariate series ring is constructed, there are various ways to construct series in that ring.

The easiest way is simply using the generators returned by the `power_series_ring` constructor and build up the power series using basic arithmetic, as described in the Ring interface.

The power series rings in AbstractAlgebra.jl implement the full Ring interface.

We give some examples of such functionality.

**Note**

The divexact function can currently only divide by unit series (i.e. whose constant coefficient is invertible).

**Examples**

```
julia> R, (x,) = power_series_ring(ZZ, [5], [:x])
(Multivariate power series ring in 1 variable over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(x^5)])

julia> f = x^3 + 3x + 21
21 + 3*x + x^3 + O(x^5)

julia> h = zero(R)
O(x^5)

julia> k = one(R)
1 + O(x^5)

julia> isone(k)
true

julia> iszero(f)
false

julia> n = length(f)
3

julia> U = base_ring(R)
Integers

julia> v = symbols(R)
1-element Vector{Symbol}:
 :x

julia> T = parent(x + 1)
Multivariate power series ring in 1 variable x
  over integers

julia> f == deepcopy(f)
true

julia> t = divexact(f*x, 1 + x)
21*x - 18*x^2 + 18*x^3 - 17*x^4 + O(x^5)

julia> R, (x, y) = power_series_ring(ZZ, [2, 3], 10, [:x, :y])
(Multivariate power series ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.AbsMSeries{BigInt, AbstractAlgebra.Generic.MPoly{BigInt}}[x + O(10), y
↪  + O(10)])

julia> f = 3x^2*y + 1
1 + 3*y*x^2 + O(10)

julia> one(R)
1 + O(10)
```

## 26.5 Power series functionality provided by AbstractAlgebra.jl

The functionality listed below is automatically provided by AbstractAlgebra.jl for absolute series over any commutative ring.

### Basic functionality

The following are provided for weighted and unweighted series:

`AbstractAlgebra.number_of_variables` – Method.

```
number_of_variables(R::AbsMSeriesRing)
```

Return the number of variables in the series ring.

source

`AbstractAlgebra.symbols` – Method.

```
symbols(R::MSeriesRing)
```

Return a vector of symbols, one for each of the variables of the series ring $R$.

source

`Base.precision` – Method.

```
precision(a::AbsMSeries)
```

Return a vector of precisions, one for each variable in the series ring. If the ring is weighted the weighted precision is returned instead.

source

`AbstractAlgebra.coeff` – Method.

```
coeff(a::AbsMSeries, n::Int)
```

Return the coefficient of the $n$-th nonzero term of the series (or zero if there are fewer than $n$ nonzero terms). Terms are numbered from the least significant term, i.e. the first term displayed when the series is printed.

source

`AbstractAlgebra.gen` – Method.

```
gen(R::AbsMSeriesRing, i::Int)
```

Return the $i$-th generator (variable) of the series ring $R$. Numbering starts from $1$ for the most significant variable.

source

AbstractAlgebra.gens – Method.

```
gens(R::AbsMSeriesRing)
```

Return a vector of the generators (variables) of the series ring $R$, starting with the most significant.

source

AbstractAlgebra.is_gen – Method.

```
is_gen(a::AbsMSeries)
```

Return true if the series $a$ is a generator of its parent series ring.

source

AbstractAlgebra.is_unit – Method.

```
is_unit(a::AbsMSeries)
```

Return `true` if the series is a unit in its series ring, i.e. if its constant term is a unit in the base ring.

source

Base.length – Method.

```
length(a::AbsMSeries)
```

Return the number of nonzero terms in the series $a$.

source

The following are only available for unweighted series.

AbstractAlgebra.max_precision – Method.

```
max_precision(R::AbsMSeriesRing)
```

Return a vector of precision caps, one for each variable in the ring. Arithmetic operations will be performed to precisions not exceeding these values.

source

AbstractAlgebra.valuation – Method.

```
valuation(a::AbsMSeries)
```

Return the valuation of $a$ as a vector of integers, one for each variable.

source

### Iteration

`AbstractAlgebra.coefficients` – Method.

```
coefficients(a::AbsMSeries)
```

Return an array of the nonzero coefficients of the series, in the order they would be displayed, i.e. least significant term first.

source

`AbstractAlgebra.exponent_vectors` – Method.

```
exponent_vectors(a::AbsMSeries)
```

Return an array of the exponent vectors of the nonzero terms of the series, in the order they would be displayed, i.e. least significant term first.

source

### Truncation

`Base.truncate` – Method.

```
truncate(a::AbstractAlgebra.AbsMSeries, prec::Vector{Int})
```

Return $a$ truncated to (absolute) precisions given by the vector `prec`.

source

`Base.truncate` – Method.

```
truncate(a::AbstractAlgebra.AbsMSeries, prec::Int)
```

Return $a$ truncated to precision `prec`. This either truncates by weight in the weighted cases or truncates each variable to precision `prec` in the unweighted case.

source

### Exact division

`AbstractAlgebra.divexact` – Method.

```
divexact(x::AbsMSeries{T}, y::AbsMSeries{T}; check::Bool=true) where T <: RingElement
```

Return the exact quotient of the series $x$ by the series $y$. This function currently assumes $y$ is an invertible series.

source

### Evaluation

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::U, vars::Vector{Int}, vals::Vector{U}) where {T <: RingElement, U <: AbsMSeries{T}}
```

Evaluate the series expression by substituting in the supplied values in the array `vals` for the corresponding variables with indices given by the array `vars`. The values must be in the same ring as $a$.

source

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::U, vars::Vector{U}, vals::Vector{U}) where {T <: RingElement, U <: AbsMSeries{T}}
```

Evaluate the series expression by substituting in the supplied values in the array `vals` for the corresponding variables given by the array `vars`. The values must be in the same ring as $a$.

source

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::U, vals::Vector{U}) where {T <: RingElement, U <: AbsMSeries{T}}
```

Evaluate the series expression by substituting in the supplied values in the array `vals` for the variables the series ring to which $a$ belongs. The values must be in the same ring as $a$.

source

### Random generation

`Base.rand` – Method.

```
rand(S::MSeriesRing, term_range::AbstractUnitRange{Int}, v...)
```

Return a random element of the series ring $S$ with number of terms in the range given by `term_range` and where coefficients of the series are randomly generated in the base ring using the data given by `v`. The exponents of the variable in the terms will be less than the precision caps for the Ring $S$ when it was created.

source

# Chapter 27

# Generic residue rings

AbstractAlgebra.jl provides modules, implemented in `src/Residue.jl` and `src/residue_field` for residue rings and fields, respectively, over any Euclidean domain (in practice most of the functionality is provided for GCD domains that provide a meaningful GCD function) belonging to the AbstractAlgebra.jl abstract type hierarchy.

## 27.1 Generic residue types

AbstractAlgebra.jl implements generic residue rings of Euclidean rings with type `EuclideanRingResidueRingElem{T}` or in the case of residue rings that are known to be fields, `EuclideanRingResidueFieldElem{T}`, where T is the type of elements of the base ring. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of generic residue ring elements have type `EuclideanRingResidueRing{T}` and those of residue fields have type `EuclideanRingResidueField{T}`.

The defining modulus of the residue ring is stored in the parent object.

## 27.2 Abstract types

All residue element types belong to the abstract type `ResElem{T}` or `ResFieldElem{T}` in the case of residue fields, and the residue ring types belong to the abstract type `ResidueRing{T}` or `ResidueField{T}` respectively. This enables one to write generic functions that can accept any AbstractAlgebra residue type.

## 27.3 Residue ring constructors

In order to construct residues in AbstractAlgebra.jl, one must first construct the residue ring itself. This is accomplished with one of the following constructors.

```
residue_ring(R::Ring, m::RingElem; cached::Bool = true)
```

```
residue_field(R::Ring, m::RingElem; cached::Bool = true)
```

Given a base ring R and residue $m$ contained in this ring, return the parent object of the residue ring $R/(m)$ together with the canonical projection. By default the parent object S will depend only on R and `m` and will be cached. Setting the optional argument cached to `false` will prevent the parent object S from being cached.

The `residue_field` constructor does the same thing as the `residue_ring` constructor, but the resulting object has type belonging to `Field` rather than `Ring`, so it can be used anywhere a field is expected in AbstractAlgebra.jl. No check is made for maximality of the ideal generated by $m$.

There are also the following for constructing residue rings and fields.

`AbstractAlgebra.quo` – Method.

```
quo(R::Ring, a::RingElement; cached::Bool = true)
```

Returns `S, f` where `S = residue_ring(R, a)` and `f` is the projection map from `R` to `S`. This map is supplied as a map with section where the section is the lift of an element of the residue field back to the ring `R`.

source

`AbstractAlgebra.quo` – Method.

```
quo(::Type{Field}, R::Ring, a::RingElement; cached::Bool = true)
```

Returns `S, f` where `S = residue_field(R, a)` and `f` is the projection map from `R` to `S`. This map is supplied as a map with section where the section is the lift of an element of the residue field back to the ring `R`.

source

Here are some examples of creating residue rings and making use of the resulting parent objects to coerce various elements into the residue ring.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S, = residue_ring(R, x^3 + 3x + 1);

julia> f = S()
0

julia> g = S(123)
123

julia> h = S(BigInt(1234))
1234

julia> k = S(x + 1)
x + 1

julia> U, f = quo(R, x^3 + 3x + 1)
(Residue ring of R modulo x^3 + 3*x + 1, Map: R -> S)

julia> U === S
true
```

All of the examples here are generic residue rings, but specialised implementations of residue rings provided by external modules will also usually provide a `residue_ring` constructor to allow creation of their residue rings.

## 27.4 Residue constructors

One can use the parent objects of a residue ring to construct residues, as per any ring.

```
(R::ResidueRing)() # constructs zero
(R::ResidueRing)(c::Integer)
(R::ResidueRing)(c::elem_type(R))
(R::ResidueRing{T})(a::T) where T <: RingElement
```

## 27.5 Functions for types and parents of residue rings

```
base_ring(R::ResidueRing)
base_ring(a::ResElem)
```

Return the base ring over which the ring was constructed.

```
parent(a::ResElem)
```

Return the parent of the given residue.

## 27.6 Residue ring functions

### Basic functionality

Residue rings implement the Ring interface.

```
zero(R::NCRing)
one(R::NCRing)
iszero(a::NCRingElement)
isone(a::NCRingElement)
```

```
divexact(a::T, b::T) where T <: RingElement
inv(a::T)
```

The Residue Ring interface is also implemented.

```
modulus(S::ResidueRing)
```

```
data(f::ResElem)
lift(f::ResElem)
```

Return a lift of the residue to the base ring.

The following functions are also provided for residues.

`AbstractAlgebra.modulus` – Method.

```
modulus(R::ResElem)
```

Return the modulus $a$ of the residue ring $S = R/(a)$ that the supplied residue $r$ belongs to.

source

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S, = residue_ring(R, x^3 + 3x + 1);

julia> f = S(x + 1)
x + 1

julia> h = zero(S)
0

julia> k = one(S)
1

julia> isone(k)
true

julia> iszero(f)
false

julia> is_unit(f)
true

julia> m = modulus(S)
x^3 + 3*x + 1

julia> d = data(f)
x + 1

julia> U = base_ring(S)
Univariate polynomial ring in x over rationals

julia> V = base_ring(f)
Univariate polynomial ring in x over rationals

julia> T = parent(f)
Residue ring of R modulo x^3 + 3*x + 1

julia> f == deepcopy(f)
true
```

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)
```

### Inversion

Base.inv – Method.

```
Base.inv(a::ResElem)
```

Return the inverse of the element $a$ in the residue ring. If an impossible inverse is encountered, an exception is raised.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S, = residue_ring(R, x^3 + 3x + 1);

julia> f = S(x + 1)
x + 1

julia> g = inv(f)
1//3*x^2 - 1//3*x + 4//3
```

source

### Greatest common divisor

Base.gcd – Method.

```
gcd(a::ResElem{T}, b::ResElem{T}) where {T <: RingElement}
```

Return a greatest common divisor of $a$ and $b$ if one exists. This is done by taking the greatest common divisor of the data associated with the supplied residues and taking its greatest common divisor with the modulus.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S, = residue_ring(R, x^3 + 3x + 1);

julia> f = S(x + 1)
x + 1

julia> g = S(x^2 + 2x + 1)
x^2 + 2*x + 1
```

```
julia> h = gcd(f, g)
1
```

source

### Square Root

Methods for `is_square` and `sqrt` are provided for inputs of type `ResFieldElem`.

#### Examples

```
julia> R = residue_field(ZZ, 733)
Residue field of Integers modulo 733

julia> a = R(86)
86

julia> is_square(a)
true

julia> sqrt(a)
532
```

### Random generation

Random residues can be generated using `rand`. The parameters after the residue ring are used to generate elements of the base ring.

```
rand(R::ResidueRing, v...)
```

#### Examples

```
julia> R, = residue_ring(ZZ, 7);

julia> f = rand(R, 0:6)
4

julia> S, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> g = rand(S, 2:2, -10:10)
-1//4*x^2 - 2//7*x + 1
```

# Chapter 28

# Free associative algebras

AbstractAlgebra.jl provides a module, implemented in `src/FreeAssociativeAlgebra.jl` for free associative algebras over any commutative ring belonging to the AbstractAlgebra abstract type hierarchy.

## 28.1 Generic free associative algebra types

AbstractAlgebra provides a generic type `Generic.FreeAssociativeAlgebraElem{T}` where T is the type of elements of the coefficient ring. The elements are implemented using a Julia array of coefficients and a vector of vectors of `Int`s for the monomial words. Parent objects of such elements have type `Generic.FreeAssociativeAlgebra{T}`.

The element types belong to the abstract type `NCRingElem`, and the algebra types belong to the abstract type `NCRing`.

The following basic functions are implemented.

```
base_ring(R::FreeAssociativeAlgebra)
base_ring(a::FreeAssociativeAlgebraElem)
parent(a::FreeAssociativeAlgebraElem)
```

## 28.2 Free associative algebra constructors

`AbstractAlgebra.free_associative_algebra` – Method.

```
free_associative_algebra(R::Ring, varnames::Vector{Symbol}; cached::Bool=true)
```

Given a coefficient ring R and variable names, say `varnames = [:x1, :x2, ...]`, return a tuple `S, [x1, x2, ...]` of the free associative algebra $S = R\langle x1, x2, \dots \rangle$ and its generators $x1, x2, \dots$.

By default (`cached=true`), the output S will be cached, i.e. if `free_associative_algebra` is invoked again with the same arguments, the same (*identical*) algebra is returned. Setting `cached` to `false` ensures a distinct new algebra is returned, and will also prevent it from being cached.

For the many ways to specify `varnames` refer to `polynomial_ring` or `AbstractAlgebra.@varnames_interface`.

**Examples**

```
julia> R, (x, y) = free_associative_algebra(ZZ, [:x, :y])
(Free associative algebra on 2 indeterminates over integers,
↪  AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}[x, y])

julia> (x + y + 1)^2
x^2 + x*y + y*x + y^2 + 2*x + 2*y + 1

julia> (x*y*x*x)^4
x*y*x^3*y*x^3*y*x^3*y*x^2
```

source

AbstractAlgebra.free_associative_algebra – Method.

```
free_associative_algebra(R::Ring, n::Int, s::VarName=:x; cached::Bool=true)
```

Same as free_associative_algebra(::Ring, ["s$i" for i in 1:n]).

**Examples**

```
julia> S, x = free_associative_algebra(ZZ, 3)
(Free associative algebra on 3 indeterminates over integers,
↪  AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}[x1, x2, x3])
```

source

AbstractAlgebra.@free_associative_algebra – Macro.

```
@free_associative_algebra(R::Ring, varnames...; cached=true)
```

Return the algebra from free_associative_algebra(::Ring, ::Vector{Symbol}) and introduce the generators into the current scope.

**Examples**

```
julia> S = @free_associative_algebra(ZZ, "z#" => 1:3)
Free associative algebra on 3 indeterminates z1, z2, z3
  over integers

julia> z1*z2*z3
z1*z2*z3
```

source

## 28.3 Free associative algebra element constructors

Elements of a free associative algebra can be constructed from the generators in the usual way using arithmetic operations. Also, all of the standard ring element constructors may be used. Finally, the MPolyBuildCtx is overloaded to work with coefficients and monomial words and not exponent vectors.

**Examples**

```
julia> R, (x, y, z) = free_associative_algebra(ZZ, [:x, :y, :z])
(Free associative algebra on 3 indeterminates over integers,
↪  AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}[x, y, z])

julia> B = MPolyBuildCtx(R)
Builder for an element of R

julia> push_term!(B, ZZ(1), [1,2,3,1]); push_term!(B, ZZ(2), [3,3,1]); finish(B)
x*y*z*x + 2*z^2*x

julia> push_term!(B, ZZ(3), [3,3,3]); push_term!(B, ZZ(4), Int[]); finish(B)
3*z^3 + 4

julia> [gen(R, 2), R(9)]
2-element Vector{AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}}:
 y
 9
```

## 28.4 Element functions

### Basic manipulation

The standard ring functions are available. The following functions from the multivariate polynomial interface are provided.

```
symbols(S::FreeAssociativeAlgebra)
number_of_variables(f::FreeAssociativeAlgebra)
gens(S::FreeAssociativeAlgebra)
gen(S::FreeAssociativeAlgebra, i::Int)
is_gen(x::FreeAssociativeAlgebraElem)
total_degree(a::FreeAssociativeAlgebraElem)
length(f::FreeAssociativeAlgebraElem)
```

As with multivariate polynomials, an implementation must provide access to the elements as a sum of individual terms *in some order*. The `length` function provides the number of such terms, and the following functions provide the first such term.

```
leading_coefficient(a::FreeAssociativeAlgebraElem)
leading_monomial(a::FreeAssociativeAlgebraElem)
leading_term(a::FreeAssociativeAlgebraElem)
```

`AbstractAlgebra.Generic.leading_exponent_word` – Function.

```
leading_exponent_word(a::FreeAssociativeAlgebraElem)
```

Return the exponent word of the leading term of a, i.e. the vector of variable indices spelling out its leading monomial. Throw an error if a is zero.

source

For types that allow constant time access to coefficients, the following are also available, allowing access to the given coefficient, monomial or term. Terms are numbered from the most significant first.

```
coeff(f::FreeAssociativeAlgebraElem, n::Int)
monomial(f::FreeAssociativeAlgebraElem, n::Int)
term(f::FreeAssociativeAlgebraElem, n::Int)
```

In contrast with the interface for multivariable polynomials, the function `exponent_vector` is replaced by `exponent_word`

`AbstractAlgebra.Generic.exponent_word` – Method.

```
exponent_word(a::FreeAssociativeAlgebraElem{T}, i::Int) where T <: RingElement
```

Return a vector of variable indices corresponding to the monomial of the $i$-th term of $a$. Term numbering begins at $1$, and the variable indices are given in the order of the variables for the ring.

**Examples**

```
julia> R, (x, y, z) = free_associative_algebra(ZZ, [:x, :y, :z])
(Free associative algebra on 3 indeterminates over integers,
↪  AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}[x, y, z])

julia> map(total_degree, (R(0), R(1), -x^2*y^2*z^2*x + z*y))
(-1, 0, 7)

julia> leading_term(-x^2*y^2*z^2*x + z*y)
-x^2*y^2*z^2*x

julia> leading_monomial(-x^2*y^2*z^2*x + z*y)
x^2*y^2*z^2*x

julia> leading_coefficient(-x^2*y^2*z^2*x + z*y)
-1

julia> exponent_word(-x^2*y^2*z^2*x + z*y, 1)
7-element Vector{Int64}:
 1
 1
 2
 2
 3
 3
 1
```

source

`AbstractAlgebra.evaluate` – Method.

```
evaluate(a::FreeAssociativeAlgebraElem{T}, vals::Vector{U}) where {T <: RingElement, U <:
↪  NCRingElem}
```

Evaluate a by substituting in the array of values for each of the variables. The evaluation will succeed if multiplication is defined between elements of the coefficient ring of a and elements of `vals`.

The syntax `a(vals...)` is also supported.

**Examples**

```
julia> R, (x, y) = free_associative_algebra(ZZ, ["x", "y"]);

julia> f = x*y - y*x
x*y - y*x

julia> S = matrix_ring(ZZ, 2);

julia> m1 = S([1 2; 3 4])
[1   2]
[3   4]

julia> m2 = S([0 1; 1 0])
[0   1]
[1   0]

julia> evaluate(f, [m1, m2])
[-1   -3]
[ 3    1]

julia> m1*m2 - m2*m1 == evaluate(f, [m1, m2])
true

julia> m1*m2 - m2*m1 == f(m1, m2)
true
```

source

### Iterators

The following iterators are provided for elements of a free associative algebra, with `exponent_words` providing the analogous functionality that `exponent_vectors` provides for multivariate polynomials.

```
terms(p::FreeAssociativeAlgebraElem)
coefficients(p::FreeAssociativeAlgebraElem)
monomials(p::FreeAssociativeAlgebraElem)
```

`AbstractAlgebra.exponent_words` – Method.

```
exponent_words(a::FreeAssociativeAlgebraElem{T}; inplace::Bool = false) where T <: RingElement
```

Return an iterator for the exponent words of the given polynomial. To retrieve an array of the exponent words, use `collect(exponent_words(a))`.

If `inplace` is `true`, the elements of the iterator may share their memory. This means that an element returned by the iterator may be overwritten 'in place' in the next iteration step. This may result in significantly fewer memory allocations. However, using the in-place version is only meaningful, if just one

element of the iterator is needed at any time. For example, calling `collect` on this iterator will not give useful results.

**Examples**

```
julia> R, (a, b, c) = free_associative_algebra(ZZ, [:a, :b, :c])
(Free associative algebra on 3 indeterminates over integers,
↪  AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}[a, b, c])

julia> collect(terms(3*b*a*c - b + c + 2))
4-element Vector{AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}}:
 3*b*a*c
 -b
 c
 2

julia> collect(coefficients(3*b*a*c - b + c + 2))
4-element Vector{BigInt}:
  3
 -1
  1
  2

julia> collect(monomials(3*b*a*c - b + c + 2))
4-element Vector{AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{BigInt}}:
 b*a*c
 b
 c
 1

julia> collect(exponent_words(3*b*a*c - b + c + 2))
4-element Vector{Vector{Int64}}:
 [2, 1, 3]
 [2]
 [3]
 []
```

source

### Groebner bases

The function `groebner_basis` provides the computation of a Groebner basis of an ideal, given a set of generators of that ideal. Since such a Groebner basis is not necessarily finite, one can additionally pass a `reduction_bound` to the function, to only compute a partial Groebner basis.

`AbstractAlgebra.Generic.groebner_basis` – Method.

```
groebner_basis(g::Vector{FreeAssociativeAlgebraElem{T}}, reduction_bound::Int = typemax(Int),
↪  remove_redundancies::Bool = false;
↪  obstruction_free_set::Vector{FreeAssociativeAlgebraElem{T}})
```

Compute a Groebner basis for the ideal generated by `g`. Stop when `reduction_bound` many non-zero entries have been added to the Groebner basis. If the computation stops due to the bound being exceeded,

the result is in general not an actual Groebner basis, just a subset of one. However, whenever the normal form with respect to this incomplete Groebner basis is `0`, it will also be `0` with respect to the full Groebner basis.

If `remove_redundancies` is set to true, some redundant obstructions will be removed during the computation, which might save time, however in practice it seems to inflate the running time regularly.

One can provide the keyword argument `obstruction_free_set` with a precomputed partial Groebner basis. It is assumed (but not checked) that all entries of `obstruction_free_set` are indeed contained in the ideal generated by `g`.

**Examples**

```
julia> R, (a, b) = free_associative_algebra(QQ, ["a", "b"]);

julia> f1 = a^2 -1;

julia> f2 = b^3 -1;

julia> f3 = a*b*a*b^2*a*b*a - b;

julia> g = AbstractAlgebra.groebner_basis([(a*b*a*b^2)^2 - 1]; obstruction_free_set =
↪  [f1,f2,f3])
12-element Vector{AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{Rational{BigInt}}}:
 a^2 - 1
 b^3 - 1
 a*b*a*b^2*a*b*a - b
 a*b*a*b^2*a*b*a*b^2 - 1
 -b*a*b^2*a*b*a*b^2 + a
 -b*a*b^2*a*b*a + a*b
 -b*a*b^2*a*b + a*b*a
 -a*b^2*a*b + b^2*a*b*a
 b^2*a*b*a*b^2 - a*b^2*a
 -a*b*a*b^2 + b*a*b^2*a
 b^2*a*b*a*b*a*b - a*b*a*b*a
 -a*b*a*b*a*b + b*a*b*a*b*a
```

source

`AbstractAlgebra.Generic.normal_form` – Method.

```
normal_form(f::FreeAssociativeAlgebraElem{T}, g::Vector{FreeAssociativeAlgebraElem{T}},
↪  aut::AhoCorasickAutomaton)
```

Assuming `g` is a Groebner basis and `aut` an Aho-Corasick automaton for the elements of `g`, compute the normal form of `f` with respect to `g`

source

`AbstractAlgebra.Generic.interreduce!` – Method.

```
interreduce!(g::Vector{FreeAssociativeAlgebraElem{T}}) where T
```

Interreduce a given Groebner basis with itself, i.e. compute the normal form of each element of g with respect to the rest of the elements and discard elements with normal form $0$ and duplicates.

source

The implementation uses a non-commutative version of the Buchberger algorithm as described in

Xingqiang Xiu, Non-commutative Gröbner Bases and Applications, PhD thesis, 2012.

**Examples**

```
julia> R = @free_associative_algebra(GF(2), [:x, :y, :u, :v, :t, :s])
Free associative algebra on 6 indeterminates x, y, u, v, t, s
  over finite field F_2

julia> g = Generic.groebner_basis([u*(x*y)^3 + u*(x*y)^2 + u + v, (y*x)^3*t + (y*x)^2*t + t + s])
5-element
↪  Vector{AbstractAlgebra.Generic.FreeAssociativeAlgebraElem{AbstractAlgebra.GFElem{Int64}}}:
 u*x*y*x*y*x*y + u*x*y*x*y + u + v
 y*x*y*x*y*x*t + y*x*y*x*t + t + s
 u*x*s + v*x*t
 u*x*y*x*s + v*x*y*x*t
 u*x*y*x*y*x*s + v*x*y*x*y*x*t
```

In order to check whether a given element of the algebra is in the ideal generated by a Groebner basis g, one can compute its normal form.

```
julia> R = @free_associative_algebra(GF(2), [:x, :y, :u, :v, :t, :s]);

julia> g = Generic.groebner_basis([u*(x*y)^3 + u*(x*y)^2 + u + v, (y*x)^3*t + (y*x)^2*t + t + s]);

julia> normal_form(u*(x*y)^3*s*t + u*(x*y)^2*s*t +u*s*t + v*s*t, g)
0
```

# Part V

# Fields

# Chapter 29

# Introduction

A number of basic fields are provided, such as the rationals, finite fields, the real field, etc.

Various generic field constructions can then be made recursively on top of these basic fields. For example, fraction fields, residue fields, function fields, etc.

From the point of view of the system, all fields are rings and whether an object is a ring/field or an element thereof can be determined at the type level. There are abstract types for all field and for all field element types.

The field hierarchy can be extended by implementing new fields to follow one or more field interfaces, including the interface that all fields must follow. Once an interface is satisfied, all the corresponding generic functionality will work over the new field.

Implementations of new fields can either be generic or can be specialised implementations provided by, for example, a C library.

# Chapter 30

# Field functionality

## 30.1 Abstract types for rings

All field types in AbstractAlgebra belong to the `Field` abstract type and field elements belong to the `FieldElem` abstract type.

As Julia types cannot belong to our `FieldElem` type hierarchy, we also provide the union type `FieldElement` which includes `FieldElem` in union with the Julia types `Rational` and `AbstractFloat`.

Note that

```
Field <: Ring
FieldElem <: RingElem
FieldElement <: RingElement
```

Of course all `Ring` functionality is available for AbstractAlgebra fields and their elements.

## 30.2 Functions for types and parents of fields

```
characteristic(R::MyParent)
```

Return the characteristic of the field. If the characteristic is not known, an exception is raised.

## 30.3 Basic functions

```
is_unit(f::MyElem)
```

Return `true` if the given element is invertible, i.e. nonzero in the field.

# Chapter 31

# Generic fraction fields

AbstractAlgebra.jl provides a module, implemented in `src/Fraction.jl` for fraction fields over any gcd domain belonging to the AbstractAlgebra.jl abstract type hierarchy.

## 31.1 Generic fraction types

AbstractAlgebra.jl implements a generic fraction type `Generic.FracFieldElem{T}` where T is the type of elements of the base ring. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of such fraction elements have type `Generic.FracField{T}`.

## 31.2 Factored fraction types

AbstractAlgebra.jl also implements a fraction type `Generic.FactoredFracFieldElem{T}` with parent objects of such fractions having type `Generic.FactoredFracField{T}`. As opposed to the fractions of type `Generic.FracFieldElem{T}`, which are just a numerator and denominator, these fractions are maintained in factored form as much as possible.

## 31.3 Abstract types

All fraction element types belong to the abstract type `FracElem{T}` and the fraction field types belong to the abstract type `FracField{T}`. This enables one to write generic functions that can accept any AbstractAlgebra fraction type.

**Note**

Both the generic fraction field type `Generic.FracField{T}` and the abstract type it belongs to, `FracField{T}` are both called `FracField`. The former is a (parameterised) concrete type for a fraction field over a given base ring whose elements have type T. The latter is an abstract type representing all fraction field types in AbstractAlgebra.jl, whether generic or very specialised (e.g. supplied by a C library).

## 31.4 Fraction field constructors

In order to construct fractions in AbstractAlgebra.jl, one can first construct the fraction field itself. This is accomplished with the following constructor.

```
fraction_field(R::Ring; cached::Bool = true)
```

Given a base ring R return the parent object of the fraction field of $R$. By default the parent object S will depend only on R and will be cached. Setting the optional argument `cached` to `false` will prevent the parent object S from being cached.

Here are some examples of creating fraction fields and making use of the resulting parent objects to coerce various elements into the fraction field.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over integers

julia> f = S()
0

julia> g = S(123)
123

julia> h = S(BigInt(1234))
1234

julia> k = S(x + 1)
x + 1
```

## 31.5 Factored Fraction field constructors

The corresponding factored field uses the following constructor.

```
factored_fraction_field(R::Ring; cached::Bool = true)
```

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y])
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> S = factored_fraction_field(R)
Factored fraction field of Multivariate polynomial ring in 2 variables over integers

julia> (X, Y) = (S(x), S(y))
(x, y)

julia> f = X^6*(X+Y)^2*(X^2+Y)^3*(X+2*Y)^-3*(X+3*Y)^-4
x^6*(x + y)^2*(x^2 + y)^3/((x + 2*y)^3*(x + 3*y)^4)
```

```
julia> numerator(f)
x^14 + 2*x^13*y + x^12*y^2 + 3*x^12*y + 6*x^11*y^2 + 3*x^10*y^3 + 3*x^10*y^2 + 6*x^9*y^3 +
↪  3*x^8*y^4 + x^8*y^3 + 2*x^7*y^4 + x^6*y^5

julia> denominator(f)
x^7 + 18*x^6*y + 138*x^5*y^2 + 584*x^4*y^3 + 1473*x^3*y^4 + 2214*x^2*y^5 + 1836*x*y^6 + 648*y^7

julia> derivative(f, x)
x^5*(x + y)*(x^2 + y)^2*(7*x^5 + 58*x^4*y + 127*x^3*y^2 + x^3*y + 72*x^2*y^3 + 22*x^2*y^2 +
↪  61*x*y^3 + 36*y^4)//((x + 2*y)^4*(x + 3*y)^5)
```

## 31.6 Fraction constructors

One can construct fractions using the fraction field parent object, as for any ring or field.

```
(R::FracField)() # constructs zero
(R::FracField)(c::Integer)
(R::FracField)(c::elem_type(R))
(R::FracField{T})(a::T) where T <: RingElement
```

One may also use the Julia double slash operator to construct elements of the fraction field without constructing the fraction field parent first.

```
//(x::T, y::T) where T <: RingElement
```

### Examples

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over rationals

julia> f = S(x + 1)
x + 1

julia> g = (x^2 + x + 1)//(x^3 + 3x + 1)
(x^2 + x + 1)//(x^3 + 3*x + 1)

julia> x//f
x//(x + 1)

julia> f//x
(x + 1)//x
```

## 31.7 Functions for types and parents of fraction fields

Fraction fields in AbstractAlgebra.jl implement the Ring interface.

```
base_ring(R::FracField)
base_ring(a::FracElem)
```

Return the base ring of which the fraction field was constructed.

```
parent(a::FracElem)
```

Return the fraction field of the given fraction.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over rationals

julia> f = S(x + 1)
x + 1

julia> U = base_ring(S)
Univariate polynomial ring in x over rationals

julia> V = base_ring(f)
Univariate polynomial ring in x over rationals

julia> T = parent(f)
Fraction field
  of univariate polynomial ring in x over rationals

julia> m = characteristic(S)
0
```

## 31.8 Fraction field functions

### Basic functions

Fraction fields implement the Ring interface.

```
zero(R::FracField)
one(R::FracField)
iszero(a::FracElem)
isone(a::FracElem)
```

```
inv(a::T) where T <: FracElem
```

They also implement the field interface.

```
is_unit(f::FracElem)
```

And they implement the fraction field interface.

```
numerator(a::FracElem)
denominator(a::FracElem)
```

### Examples

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over rationals

julia> f = S(x + 1)
x + 1

julia> g = (x^2 + x + 1)//(x^3 + 3x + 1)
(x^2 + x + 1)//(x^3 + 3*x + 1)

julia> h = zero(S)
0

julia> k = one(S)
1

julia> isone(k)
true

julia> iszero(f)
false

julia> r = deepcopy(f)
x + 1

julia> n = numerator(g)
x^2 + x + 1

julia> d = denominator(g)
x^3 + 3*x + 1
```

### Greatest common divisor

Base.gcd – Method.

```
gcd(a::FracElem{T}, b::FracElem{T}) where {T <: RingElem}
```

Return a greatest common divisor of $a$ and $b$ if one exists. N.B: we define the GCD of $a/b$ and $c/d$ to be gcd$(ad, bc)/bd$, reduced to lowest terms. This requires the existence of a greatest common divisor function for the base ring.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> f = (x + 1)//(x^3 + 3x + 1)
(x + 1)//(x^3 + 3*x + 1)

julia> g = (x^2 + 2x + 1)//(x^2 + x + 1)
(x^2 + 2*x + 1)//(x^2 + x + 1)

julia> h = gcd(f, g)
(x + 1)//(x^5 + x^4 + 4*x^3 + 4*x^2 + 4*x + 1)
```

source

### Square root

Methods for `is_square` and `sqrt` are provided for inputs of type `FracElem`.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over rationals

julia> a = (21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)
(21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)

julia> sqrt(a^2)
(21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)

julia> is_square(a^2)
true
```

### Remove and valuation

When working over a Euclidean domain, it is convenient to extend valuations to the fraction field. To facilitate this, we define the following functions.

`AbstractAlgebra.remove` – Method.

```
remove(z::FracElem{T}, p::T) where {T <: RingElement}
```

Return the tuple $n, x$ such that $z = p^n x$ where $x$ has valuation $0$ at $p$.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> f = (x + 1)//(x^3 + 3x + 1)
(x + 1)//(x^3 + 3*x + 1)

julia> g = (x^2 + 1)//(x^2 + x + 1)
(x^2 + 1)//(x^2 + x + 1)

julia> v, q = remove(f^3*g, x + 1)
(3, (x^2 + 1)//(x^11 + x^10 + 10*x^9 + 12*x^8 + 39*x^7 + 48*x^6 + 75*x^5 + 75*x^4 + 66*x^3 +
↪  37*x^2 + 10*x + 1))

julia> v = valuation(f^3*g, x + 1)
3

julia> v = valuation(q, x + 1)
0
```

source

`AbstractAlgebra.valuation` – Method.

```
valuation(z::FracElem{T}, p::T) where {T <: RingElement}
```

Return the valuation of $z$ at $p$.

source

### Random generation

Random fractions can be generated using `rand`. The parameters passed after the fraction field tell `rand` how to generate random elements of the base ring.

```
rand(R::FracField, v...)
```

**Examples**

```
julia> K = fraction_field(ZZ)
Rationals

julia> f = rand(K, -10:10)
-1//3

julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> S = fraction_field(R)
Fraction field
  of univariate polynomial ring in x over integers
```

```
julia> g = rand(S, -1:3, -10:10)
(-4*x - 4)//(4*x^2 + x - 4)
```

### Extra functionality for factored fractions

The `Generic.FactoredFracFieldElem{T}` type implements an interface similar to that of the `Fac{T}` type for iterating over the terms in the factorisation. There is also the function `push_term!(a, b, e)` for efficiently performing `a *= b^e`, and the function `normalise` returns relatively prime terms.

#### Examples

```
julia> F = factored_fraction_field(ZZ)
Factored fraction field of Integers

julia> f = F(-1)
-1

julia> push_term!(f, 10, 10)
-10^10

julia> push_term!(f, 42, -8)
-10^10/42^8

julia> normalise(f)
-5^10*2^2/21^8

julia> unit(f)
-1

julia> collect(f)
2-element Vector{Tuple{BigInt, Int64}}:
 (10, 10)
 (42, -8)
```

# Chapter 32

# Rational field

AbstractAlgebra.jl provides a module, implemented in `src/julia/Rational.jl` for making Julia `Rational{BigInt}`s conform to the AbstractAlgebra.jl Field interface.

In addition to providing a parent object `QQ` for Julia `Rational{BigInt}`s, we implement any additional functionality required by AbstractAlgebra.jl.

Because `Rational{BigInt}` cannot be directly included in the AbstractAlgebra.jl abstract type hierarchy, we achieve integration of Julia `Rational{BigInt}`s by introducing a type union, called `FieldElement`, which is a union of `FieldElem` and a number of Julia types, including `Rational{BigInt}`. Everywhere that `FieldElem` is notionally used in AbstractAlgebra.jl, we are in fact using `FieldElement`, with additional care being taken to avoid ambiguities.

The details of how this is done are technical, and we refer the reader to the implementation for details. For most intents and purposes, one can think of the Julia `Rational{BigInt}` type as belonging to `FieldElem`.

One other technicality is that Julia defines certain functions for `Rational{BigInt}`, such as `sqrt` and `exp` differently to what AbstractAlgebra.jl requires. To get around this, we redefine these functions internally to AbstractAlgebra.jl, without redefining them for users of AbstractAlgebra.jl. This allows the internals of AbstractAlgebra.jl to function correctly, without broadcasting pirate definitions of already defined Julia functions to the world.

To access the internal definitions, one can use `AbstractAlgebra.sqrt` and `AbstractAlgebra.exp`, etc.

## 32.1 Types and parent objects

Rationals have type `Rational{BigInt}`, as in Julia itself. We simply supplement the functionality for this type as required for computer algebra.

The parent objects of such integers has type `Rationals{BigInt}`.

For convenience, we also make `Rational{Int}` a part of the AbstractAlgebra.jl type hierarchy and its parent object (accessible as `qq`) has type `Rationals{Int}`. But we caution that this type is not particularly useful as a model of the rationals and may not function as expected within AbstractAlgebra.jl.

## 32.2 Rational constructors

In order to construct rationals in AbstractAlgebra.jl, one can first construct the rational field itself. This is accomplished using either of the following constructors.

```
fraction_field(R::Integers{BigInt})
```

```
Rationals{BigInt}()
```

This gives the unique object of type `Rationals{BigInt}` representing the field of rationals in AbstractAlgebra.jl.

In practice, one simply uses `QQ` which is assigned to be the return value of the above constructor. There is no need to call the constructor in practice.

Here are some examples of creating the rational field and making use of the resulting parent object to coerce various elements into the field.

**Examples**

```
julia> f = QQ()
0//1

julia> g = QQ(123)
123//1

julia> h = QQ(BigInt(1234))
1234//1

julia> k = QQ(BigInt(12), BigInt(7))
12//7

julia> QQ == fraction_field(ZZ)
true
```

## 32.3 Basic field functionality

The rational field in AbstractAlgebra.jl implements the full Field and Fraction Field interfaces.

We give some examples of such functionality.

**Examples**

```
julia> f = QQ(12, 7)
12//7

julia> h = zero(QQ)
0//1

julia> k = one(QQ)
1//1

julia> isone(k)
true

julia> iszero(f)
false
```

```
julia> U = base_ring(QQ)
Integers

julia> V = base_ring(f)
Integers

julia> T = parent(f)
Rationals

julia> f == deepcopy(f)
true

julia> g = f + 12
96//7

julia> r = ZZ(12)//ZZ(7)
12//7

julia> n = numerator(r)
12
```

## 32.4 Rational functionality provided by AbstractAlgebra.jl

The functionality below supplements that provided by Julia itself for its `Rational{BigInt}` type.

### Square and n-th root

The functions `sqrt`, `is_square`, `is_square_with_sqrt` are all provided, as are `root` and `is_power`.

**Examples**

```
julia> d = AbstractAlgebra.sqrt(ZZ(36)//ZZ(25))
6//5

julia> is_square(ZZ(9)//ZZ(16))
true

julia> root(ZZ(27)//64, 3)
3//4
```

# Chapter 33

# Rational function fields

AbstractAlgebra.jl provides a module, implemented in `src/generic/RationalFunctionField.jl` for rational function fields $k(x)$ or `k[x_1, x_2, \ldots, x_n]` over a field $k$.

## 33.1 Generic rational function field type

Rational functions have type `Generic.RationalFunctionFieldElem{T, U}` where `T` is the type of elements of the coefficient field $k$ and `U` is the type of polynomials (either univariate or multivariate) over that field. See the file `src/generic/GenericTypes.jl` for details.

Parent objects corresponding to the rational function field $k$ have type `Generic.RationalFunctionField{T, U}`.

## 33.2 Abstract types

The rational function types belong to the abstract type `Field` and the rational function field types belong to the abstract type `FieldElem`.

## 33.3 Rational function field constructors

In order to construct rational functions in AbstractAlgebra.jl, one can first construct the function field itself. This is accomplished with one of the following constructors.

`AbstractAlgebra.rational_function_field` – Function.

```
rational_function_field(k::Field, s::VarName=:t; cached::Bool=true)
rational_function_field(k::Field, varnames::Vector{Symbol}; cached::Bool=true)
rational_function_field(k::Field, n::Int, s::VarName=:x; cached::Bool=true)
```

Given a coefficient field `k` and variable names, return a tuple `(S, x)` consisting of the rational function field $S = k(x, \ldots)$ over `k` and its generator(s) `x`. The numbered form uses the names `Symbol.(s, 1:n)`.

By default (`cached=true`), the output `S` will be cached, i.e. if `rational_function_field` is invoked again with the same arguments, the same (*identical*) field is returned. Setting `cached` to `false` ensures a distinct new field is returned, and will also prevent it from being cached.

For the many ways to specify `varnames` refer to `polynomial_ring` or `AbstractAlgebra.@varnames_interface`.

**Examples**

```
julia> S, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> S(123)
123

julia> S(numerator(x + 1, false), numerator(x + 2, false))
(x + 1)//(x + 2)

julia> R, (x, y) = rational_function_field(QQ, [:x, :y])
(Rational function field over rationals,
↪  AbstractAlgebra.Generic.RationalFunctionFieldElem{Rational{BigInt},
↪  AbstractAlgebra.Generic.MPoly{Rational{BigInt}}}[x, y])

julia> (x + y)//y^2
(x + y)//y^2
```

source

AbstractAlgebra.@rational_function_field – Macro.

```
@rational_function_field(k::Field, varnames...; cached=true)
```

Return the field from rational_function_field and introduce the generators into the current scope.

**Examples**

```
julia> S = @rational_function_field(QQ, [:s, :t])
Rational function field
  over rationals

julia> (s + t)//t
(s + t)//t
```

source

## 33.4 Basic rational function field functionality

Fraction fields in AbstractAlgebra.jl implement the full Field interface and the entire fraction field interface.

We give some examples of such functionality.

**Examples**

```
julia> S, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> f = S(x + 1)
x + 1

julia> g = (x^2 + x + 1)//(x^3 + 3x + 1)
(x^2 + x + 1)//(x^3 + 3*x + 1)
```

```
julia> h = zero(S)
0

julia> k = one(S)
1

julia> isone(k)
true

julia> iszero(f)
false

julia> m = characteristic(S)
0

julia> U = base_ring(S)
Rationals

julia> V = base_ring(f)
Rationals

julia> T = parent(f)
Rational function field
  over rationals

julia> r = deepcopy(f)
x + 1

julia> n = numerator(g)
x^2 + x + 1

julia> d = denominator(g)
x^3 + 3*x + 1
```

Note that numerator and denominator are returned as elements of a polynomial ring whose variable is printed the same way as that of the generator of the rational function field.

## 33.5 Rational function field functionality provided by AbstractAlgebra.jl

The following functionality is provided for rational function fields.

### Greatest common divisor

Base.gcd – Method.

```
gcd(a::RationalFunctionFieldElem{T, U}, b::RationalFunctionFieldElem{T, U}) where {T <:
↪  FieldElement, U <: Union{PolyRingElem, MPolyRingElem}}
```

Return a greatest common divisor of $a$ and $b$ if one exists. N.B: we define the GCD of $a/b$ and $c/d$ to be gcd$(ad, bc)/bd$, reduced to lowest terms.

**Examples**

```
julia> R, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> f = (x + 1)//(x^3 + 3x + 1)
(x + 1)//(x^3 + 3*x + 1)

julia> g = (x^2 + 2x + 1)//(x^2 + x + 1)
(x^2 + 2*x + 1)//(x^2 + x + 1)

julia> h = gcd(f, g)
(x + 1)//(x^5 + x^4 + 4*x^3 + 4*x^2 + 4*x + 1)
```

source

### Square root

Methods for `is_square` and `sqrt` are provided for inputs of type `RationalFunctionFieldElem`.

**Examples**

```
julia> R, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> a = (21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)
(21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)

julia> sqrt(a^2)
(21//4*x^6 - 15*x^5 + 27//14*x^4 + 9//20*x^3 + 3//7*x + 9//10)//(x + 3)

julia> is_square(a^2)
true
```

# Chapter 34

# Univariate function fields

Univariate function fields in AbstractAlgebra are algebraic extensions $K/k(x)$ of a rational function field $k(x)$ over a field $k$.

These are implemented in a module implemented in `src/generic/FunctionField.jl`.

## 34.1 Generic function field types

Function field objects $K/k(x)$ in AbstractAlgebra have type `Generic.AbsSimpleFunctionField{T, U}` where `T` is the type of elements of the field `k` and `U` is the type of the univariate polynomials over that field.

Corresponding function field elements have type `Generic.AbsSimpleFunctionFieldElem{T, U}`. See the file `src/generic/GenericTypes.jl` for details.

## 34.2 Abstract types

Function field types belong to the abstract type `Field` and their elements to the abstract type `FieldElem`.

## 34.3 Function field constructors

In order to construct function fields in AbstractAlgebra.jl, one first constructs the rational function field they are an extension of, then supplies a polynomial over this field to the following constructor:

`AbstractAlgebra.Generic.function_field` – Function.

```
function_field(p::PolyRingElem{<:RationalFunctionFieldElem}, s::VarName; cached::Bool=true)
```

Given an irreducible polynomial `p` over a rational function field $k(x)$ and a variable name `s`, return a tuple `(S, z)` consisting of the function field $S = k(x)[z]/(p)$ and its generator `z`, which is printed as `s`.

By default (`cached=true`), the output `S` will be cached, i.e. if `function_field` is invoked again with the same arguments, the same (*identical*) field is returned. Setting `cached` to `false` ensures a distinct new field is returned, and will also prevent it from being cached.

**Examples**

```
julia> R1, x1 = rational_function_field(QQ, :x1);  # characteristic 0
```

```
julia> U1, z1 = R1[:z1];

julia> f = (x1^2 + 1)//(x1 + 1)*z1^3 + 4*z1 + 1//(x1 + 1)
(x1^2 + 1)//(x1 + 1)*z1^3 + 4*z1 + 1//(x1 + 1)

julia> S1, y1 = function_field(f, :y1)
(Function Field over rationals with defining polynomial (x1^2 + 1)*y1^3 + (4*x1 + 4)*y1 + 1, y1)

julia> S1((x1 + 1)//(x1 + 2))
(x1 + 1)//(x1 + 2)

julia> R2, x2 = rational_function_field(GF(23), :x2);  # positive characteristic

julia> U2, z2 = R2[:z2];

julia> S2, y2 = function_field(z2^2 + 3z2 + 1, :y2)
(Function Field over finite field F_23 with defining polynomial y2^2 + 3*y2 + 1, y2)

julia> S2(R2(5))
5
```

source

## 34.4 Basic function field functionality

Function fields implement the full Ring and Field interfaces. We give some examples of such functionality.

**Examples**

```
julia> R, x = rational_function_field(GF(23), :x) # characteristic p
(Rational function field over finite field F_23, x)

julia> U, z = R[:z]
(Univariate polynomial ring in z over R, z)

julia> g = z^2 + 3z + 1
z^2 + 3*z + 1

julia> S, y = function_field(g, :y)
(Function Field over finite field F_23 with defining polynomial y^2 + 3*y + 1, y)

julia> f = (x + 1)*y + 1
(x + 1)*y + 1

julia> base_ring(f)
Rational function field
  over finite field F_23

julia> f^2
(20*x^2 + 19*x + 22)*y + 22*x^2 + 21*x

julia> f*inv(f)
1
```

## 34.5 Function field functionality provided by AbstractAlgebra.jl

The following functionality is provided for function fields.

### Basic manipulation

`AbstractAlgebra.Generic.base_field` – Method.

```
base_field(R::AbsSimpleFunctionField)
```

Return the rational function field that the field R is an extension of. Synonymous with `base_ring`.

source

`AbstractAlgebra.var` – Method.

```
var(R::AbsSimpleFunctionField)
```

Return the variable name of the generator of the function field R as a symbol.

source

`AbstractAlgebra.Generic.defining_polynomial` – Method.

```
defining_polynomial(R::AbsSimpleFunctionField)
modulus(R::AbsSimpleFunctionField)
```

Return the original polynomial that was used to define the function field R.

source

`Base.numerator` – Method.

```
Base.numerator(R::AbsSimpleFunctionField{T, U}, canonicalise::Bool=true) where {T <:
↪  FieldElement, U <: PolyRingElem{T}}
Base.denominator(R::AbsSimpleFunctionField{T, U}, canonicalise::Bool=true) where {T <:
↪  FieldElement, U <: PolyRingElem{T}}
```

Thinking of elements of the rational function field as fractions, put the defining polynomial of the function field over a common denominator and return the numerator/denominator respectively. Note that the resulting polynomials belong to a different ring than the original defining polynomial. The `canonicalise` is ignored, but exists for compatibility with the Generic interface.

source

`Base.numerator` – Method.

```
Base.numerator(a::AbsSimpleFunctionFieldElem{T, U}, canonicalise::Bool=true) where {T <:
↪  FieldElement, U <: PolyRingElem{T}}
Base.denominator(a::AbsSimpleFunctionFieldElem{T, U}, canonicalise::Bool=true) where {T <:
↪  FieldElement, U <: PolyRingElem{T}}
```

Return the numerator and denominator of the function field element a. Note that elements are stored in fraction free form so that the denominator is a common denominator for the coefficients of the element a. If `canonicalise` is set to `true` the fraction is first canonicalised.

source

`AbstractAlgebra.degree` – Method.

```
degree(S::AbsSimpleFunctionField)
```

Return the degree of the defining polynomial of the function field, i.e. the degree of the extension that the function field makes of the underlying rational function field.

source

`AbstractAlgebra.gen` – Method.

```
gen(S::AbsSimpleFunctionField{T, U}) where {T <: FieldElement, U <: PolyRingElem{T}}
```

Return the generator of the function field returned by the function field constructor.

source

`AbstractAlgebra.is_gen` – Method.

```
is_gen(a::AbsSimpleFunctionFieldElem)
```

Return `true` if a is the generator of the function field returned by the function field constructor.

source

`AbstractAlgebra.coeff` – Method.

```
coeff(a::AbsSimpleFunctionFieldElem, n::Int)
```

Return the degree n coefficient of the element a in its polynomial representation in terms of the generator of the function field. The coefficient is returned as an element of the underlying rational function field.

source

`AbstractAlgebra.Generic.num_coeff` – Method.

```
num_coeff(a::AbsSimpleFunctionFieldElem, n::Int)
```

Return the degree n coefficient of the numerator of the element a (in its polynomial representation in terms of the generator of the function field, rationalised as per `numerator/denominator` described above). The coefficient will be an polynomial over the `base_ring` of the underlying rational function field.

source

**Examples**

```
julia> R, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> U, z = R[:z]
(Univariate polynomial ring in z over R, z)

julia> g = z^2 + 3*(x + 1)//(x + 2)*z + 1
z^2 + (3*x + 3)//(x + 2)*z + 1

julia> S, y = function_field(g, :y)
(Function Field over rationals with defining polynomial (x + 2)*y^2 + (3*x + 3)*y + x + 2, y)

julia> base_field(S)
Rational function field
  over rationals

julia> var(S)
:y

julia> characteristic(S)
0

julia> defining_polynomial(S)
z^2 + (3*x + 3)//(x + 2)*z + 1

julia> numerator(S)
(x + 2)*y^2 + (3*x + 3)*y + x + 2

julia> denominator(S)
x + 2

julia> a = (x + 1)//(x^2 + 1)*y + 3x + 2
((x + 1)*y + 3*x^3 + 2*x^2 + 3*x + 2)//(x^2 + 1)

julia> numerator(a, false)
(x + 1)*y + 3*x^3 + 2*x^2 + 3*x + 2

julia> denominator(a, false)
x^2 + 1

julia> degree(S)
2

julia> gen(S)
```

```
y

julia> is_gen(y)
true

julia> coeff(a, 1)
(x + 1)//(x^2 + 1)

julia> num_coeff(a, 1)
x + 1
```

### Trace and norm

`LinearAlgebra.norm` – Method.

```
norm(a::AbsSimpleFunctionFieldElem)
```

Return the absolute norm of a as an element of the underlying rational function field.

source

```
julia> R, x = rational_function_field(QQ, :x)
(Rational function field over rationals, x)

julia> U, z = R[:z]
(Univariate polynomial ring in z over R, z)

julia> g = z^2 + 3*(x + 1)//(x + 2)*z + 1
z^2 + (3*x + 3)//(x + 2)*z + 1

julia> S, y = function_field(g, :y)
(Function Field over rationals with defining polynomial (x + 2)*y^2 + (3*x + 3)*y + x + 2, y)

julia> f = (-3*x - 5//3)//(x - 2)*y + (x^3 + 1//9*x^2 + 5)//(x - 2)
((-3*x - 5//3)*y + x^3 + 1//9*x^2 + 5)//(x - 2)

julia> norm(f)
(x^7 + 20//9*x^6 + 766//81*x^5 + 2027//81*x^4 + 110//3*x^3 + 682//9*x^2 + 1060//9*x + 725//9)//(x^3
↪  - 2*x^2 - 4*x + 8)

julia> tr(f)
(2*x^4 + 38//9*x^3 + 85//9*x^2 + 24*x + 25)//(x^2 - 4)
```

# Chapter 35

# Finite fields

AbstractAlgebra.jl provides a module, implemented in `src/julia/GF.jl` for finite fields. The module is a naive implementation that supports only fields of degree $1$ (prime fields). They are modelled as $\mathbb{Z}/p\mathbb{Z}$ for $p$ a prime.

## 35.1 Types and parent objects

Finite fields have type `GFField{T}` where T is either `Int` or `BigInt`.

Elements of such a finite field have type `GFElem{T}`.

## 35.2 Finite field constructors

In order to construct finite fields in AbstractAlgebra.jl, one must first construct the field itself. This is accomplished with the following constructors.

`AbstractAlgebra.GF` – Method.

```
GF(p::T; check::Bool=true) where T <: Integer
```

Return the finite field $\mathbb{F}_p$, where $p$ is a prime. By default, the integer $p$ is checked with a probabilistic algorithm for primality. When `check == false`, no check is made, but the behaviour of the resulting object is undefined if $p$ is composite.

source

Here are some examples of creating a finite field and making use of the resulting parent object to coerce various elements into the field.

**Examples**

```
julia> F = GF(13)
Finite field F_13

julia> g = F(3)
3

julia> h = F(g)
3
```

```
julia> GF(4)
ERROR: DomainError with 4:
Characteristic is not prime in GF(p)
Stacktrace:
[...]
```

## 35.3 Basic field functionality

The finite field module in AbstractAlgebra.jl implements the full Field interface.

We give some examples of such functionality.

**Examples**

```
julia> F = GF(13)
Finite field F_13

julia> f = F(7)
7

julia> h = zero(F)
0

julia> k = one(F)
1

julia> isone(k)
true

julia> iszero(h)
true

julia> T = parent(h)
Finite field F_13

julia> h == deepcopy(h)
true

julia> h = h + 2
2

julia> m = inv(k)
1
```

## 35.4 Basic manipulation of fields and elements

`AbstractAlgebra.data` – Method.

```
data(R::GFElem)
```

Return the internal data used to represent the finite field element. This coincides with `lift` except where the internal data ids a machine integer.

source

`AbstractAlgebra.lift` – Method.

```
lift(R::GFElem)
```

Lift the finite field element to the integers. The result will be a multiprecision integer regardless of how the field element is represented internally.

source

`AbstractAlgebra.gen` – Method.

```
gen(R::GFField{T}) where T <: Integer
```

Return a generator of the field. Currently this returns 1.

source

`AbstractAlgebra.order` – Method.

```
order(R::GFField)
```

Return the order, i.e. the number of element in the given finite field.

source

`AbstractAlgebra.degree` – Method.

```
degree(R::GFField)
```

Return the degree of the given finite field.

source

**Examples**

```
julia> F = GF(13)
Finite field F_13

julia> d = degree(F)
1

julia> n = order(F)
13

julia> g = gen(F)
1
```

# Chapter 36

# Real field

AbstractAlgebra.jl provides a module, implemented in `src/julia/Float.jl` for making Julia `BigFloats` conform to the AbstractAlgebra.jl Field interface.

In addition to providing a parent object `RealField` for Julia `BigFloats`, we implement any additional functionality required by AbstractAlgebra.jl.

Because `BigFloat` cannot be directly included in the AbstractAlgebra.jl abstract type hierarchy, we achieve integration of Julia `BigFloats` by introducing a type union, called `FieldElement`, which is a union of `FieldElem` and a number of Julia types, including `BigFloat`. Everywhere that `FieldElem` is notionally used in AbstractAlgebra.jl, we are in fact using `FieldElement`, with additional care being taken to avoid ambiguities.

The details of how this is done are technical, and we refer the reader to the implementation for details. For most intents and purposes, one can think of the Julia `BigFloat` type as belonging to `FieldElem`.

## 36.1 Types and parent objects

Reals have type `BigFloat`, as in Julia itself. We simply supplement the functionality for this type as required for computer algebra.

The parent objects of such integers has type `Floats{BigFloat}`.

For convenience, we also make `Float64` a part of the AbstractAlgebra.jl type hierarchy and its parent object (accessible as `RDF`) has type `Floats{Float64}`.

## 36.2 Rational constructors

In order to construct reals in AbstractAlgebra.jl, one can first construct the real field itself. This is accomplished using the following constructor.

```
Floats{BigFloat}()
```

This gives the unique object of type `Floats{BigFloat}` representing the field of reals in AbstractAlgebra.jl.

In practice, one simply uses `RealField` which is assigned to be the return value of the above constructor. There is no need to call the constructor in practice.

Here are some examples of creating the real field and making use of the resulting parent object to coerce various elements into the field.

**Examples**

```
julia> RR = RealField
Floats

julia> f = RR()
0.0

julia> g = RR(123)
123.0

julia> h = RR(BigInt(1234))
1234.0

julia> k = RR(12//7)
1.714285714285714285714285714285714285714285714285714285714285714285714285714291

julia> m = RR(2.3)
2.29999999999999982236431605997495353221893310546875
```

## 36.3 Basic field functionality

The real field in AbstractAlgebra.jl implements the full Field interface.

We give some examples of such functionality.

**Examples**

```
julia> RR = RealField
Floats

julia> f = RR(12//7)
1.714285714285714285714285714285714285714285714285714285714285714285714285714291

julia> h = zero(RR)
0.0

julia> k = one(RR)
1.0

julia> isone(k)
true

julia> iszero(f)
false

julia> base_ring_type(RR)
Union{}

julia> T = parent(f)
Floats

julia> f == deepcopy(f)
true
```

```
julia> g = f + 12
13.714285714285714285
```

# Part VI

# Groups

# Chapter 37

# Permutations and Symmetric groups

AbstractAlgebra.jl provides rudimentary native support for permutation groups (implemented in `src/generic/PermGroups.jl`). All functionality of permutations is accessible in the `Generic` submodule.

Permutations are represented internally via vector of integers, wrapped in type `Perm{T}`, where `T<:Integer` carries the information on the type of elements of a permutation. Symmetric groups are singleton parent objects of type `SymmetricGroup{T}` and are used mostly to store the length of a permutation, since it is not included in the permutation type.

Symmetric groups are created using the `SymmetricGroup` (inner) constructor.

Both `SymmetricGroup` and `Perm` and can be parametrized by any type `T<:Integer` . By default the parameter is the `Int`-type native to the systems architecture. However, if you are sure that your permutations are small enough to fit into smaller integer type (such as `Int32`, `UInt16`, or even `Int8`), you may choose to change the parametrizing type accordingly. In practice this may result in decreased memory footprint (when storing multiple permutations) and noticeable faster performance, if your workload is heavy in operations on permutations, which e.g. does not fit into cache of your cpu.

All the permutation group types belong to the `Group` abstract type and the corresponding permutation element types belong to the `GroupElem` abstract type.

`AbstractAlgebra.Generic.setpermstyle` – Function.

```
setpermstyle(format::Symbol)
```

Select the style in which permutations are displayed (in the REPL or in general as strings). This can be either

- `:array` - as vector of integers whose $n$-th position represents the value at $n$), or
- `:cycles` - as, more familiar for mathematicians, decomposition into disjoint cycles, where the value at $n$ is represented by the entry immediately following $n$ in a cycle (the default).

The difference is purely esthetical.

**Examples**

```
julia> setpermstyle(:array)
:array
```

```
julia> Perm([2,3,1,5,4])
[2, 3, 1, 5, 4]

julia> setpermstyle(:cycles)
:cycles

julia> Perm([2,3,1,5,4])
(1,2,3)(4,5)
```

source

## 37.1 Permutations constructors

There are several methods to construct permutations in AbstractAlgebra.jl.

- The easiest way is to directly call to the `Perm` (inner) constructor:

`AbstractAlgebra.Perm` – Type.

```
Perm{T<:Integer}
```

The type of permutations. Fieldnames:

- `d::Vector{T}` - vector representing the permutation
- `modified::Bool` - bit to check the validity of cycle decomposition
- `cycles::CycleDec{T}` - (cached) cycle decomposition

A permutation $p$ consists of a vector (`p.d`) of $n$ integers from $1$ to $n$. If the $i$-th entry of the vector is $j$, this corresponds to $p$ sending $i \to j$. The cycle decomposition (`p.cycles`) is computed on demand and should never be accessed directly. Use `cycles(p)` instead.

There are two inner constructors of `Perm`:

- `Perm(n::T)` constructs the trivial `Perm{T}`-permutation of length $n$.
- `Perm(v::AbstractVector{<:Integer} [,check=true])` constructs a permutation represented by `v`. By default `Perm` constructor checks if the vector constitutes a valid permutation. To skip the check call `Perm(v, false)`.

**Examples**

```
julia> Perm([1,2,3])
()

julia> g = Perm(Int32[2,3,1])
(1,2,3)

julia> typeof(g)
Perm{Int32}
```

source

Since the parent object can be reconstructed from the permutation itself, you can work with permutations without explicitly constructing the parent object.

- The other way is to first construct the permutation group they belong to. This is accomplished with the inner constructor `SymmetricGroup(n::Integer)` which constructs the permutation group on $n$ symbols and returns the parent object representing the group.

`AbstractAlgebra.Generic.SymmetricGroup` – Type.

```
SymmetricGroup{T<:Integer}
```

The full symmetric group singleton type. `SymmetricGroup(n)` constructs the full symmetric group $S_n$ on $n$-symbols. The type of elements of the group is inferred from the type of `n`.

**Examples**

```
julia> G = SymmetricGroup(5)
Full symmetric group over 5 elements

julia> elem_type(G)
Perm{Int64}

julia> H = SymmetricGroup(UInt16(5))
Full symmetric group over 5 elements

julia> elem_type(H)
Perm{UInt16}
```

source

A vector of integers can be then coerced to a permutation by calling a parent permutation group on it. The advantage is that the vector is automatically converted to the integer type fixed at the creation of the parent object.

**Examples:**

```
julia> G = SymmetricGroup(BigInt(5)); p = G([2,3,1,5,4])
(1,2,3)(4,5)

julia> typeof(p)
Perm{BigInt}

julia> H = SymmetricGroup(UInt16(5)); r = H([2,3,1,5,4])
(1,2,3)(4,5)

julia> typeof(r)
Perm{UInt16}

julia> one(H)
()
```

By default the coercion checks for non-unique values in the vector, but this can be switched off with `G([2,3,1,5,4], false)`.

- Finally there is a `perm"..."` string macro to construct a permutation from a string input.

`AbstractAlgebra.Generic.@perm_str` – Macro.

```
perm"..."
```

String macro to parse disjoint cycles into `Perm{Int}`.

Strings for the output of GAP could be copied directly into `perm"..."`. Cycles of length $1$ are not necessary, but can be included. A permutation of the minimal support is constructed, i.e. the maximal $n$ in the decomposition determines the parent group $S_n$.

**Examples**

```
julia> p = perm"(1,3)(2,4)"
(1,3)(2,4)

julia> typeof(p)
Perm{Int64}

julia> parent(p) == SymmetricGroup(4)
true

julia> p = perm"(1,3)(2,4)(10)"
(1,3)(2,4)

julia> parent(p) == SymmetricGroup(10)
true
```

source

## 37.2 Permutation interface

The following basic functionality is provided by the default permutation group implementation in AbstractAlgebra.jl, to support construction of other generic constructions over permutation groups. Any custom permutation group implementation in AbstractAlgebra.jl should provide the group element arithmetic and comparison.

A custom implementation also needs to implement `hash(::Perm, ::UInt)` and (possibly) `deepcopy_internal(::Perm, ::IdDict)`.

**Note**

Permutation group elements are mutable and so returning shallow copies is not sufficient.

```
getindex(a::Perm, n::Integer)
```

Allow access to entry $n$ of the given permutation via the syntax `a[n]`. Note that entries are $1$-indexed.

```
setindex!(a::Perm, d::Integer, n::Integer)
```

Set the $n$-th entry of the given permutation to $d$. This allows Julia to provide the syntax `a[n] = d` for setting entries of a permutation. Entries are $1$-indexed.

**Note**

Using `setindex!` invalidates the cycle decomposition cached in a permutation, which will be computed the next time it is needed.

Given the parent object `G` for a permutation group, the following coercion functions are provided to coerce various arguments into the permutation group. Developers provide these by overloading the permutation group parent objects.

```
one(G)
```

Return the identity permutation.

```
G(A::Vector{<:Integer})
```

Return the permutation whose entries are given by the elements of the supplied vector.

```
G(p::Perm)
```

Take a permutation that is already in the permutation group and simply return it. A copy of the original is not made if not necessary.

## 37.3 Basic manipulation

Numerous functions are provided to manipulate permutation group elements.

`AbstractAlgebra.Generic.cycles` – Method.

```
cycles(g::Perm)
```

Decompose permutation `g` into disjoint cycles.

Return a `CycleDec` object which iterates over disjoint cycles of `g`. The ordering of cycles is not guaranteed, and the order within each cycle is computed up to a cyclic permutation. The cycle decomposition is cached in `g` and used in future computation of `permtype`, `parity`, `sign`, `order` and `^` (powering).

**Examples**

```
julia> g = Perm([3,4,5,2,1,6])
(1,3,5)(2,4)

julia> collect(cycles(g))
3-element Vector{Vector{Int64}}:
 [1, 3, 5]
 [2, 4]
 [6]
```

source

Cycle structure is cached in a permutation, since once available, it provides a convenient shortcut in many other algorithms.

`AbstractAlgebra.Generic.parity` – Method.

```
parity(g::Perm)
```

Return the parity of the given permutation, i.e. the parity of the number of transpositions in any decomposition of g into transpositions.

`parity` returns $1$ if the number is odd and $0$ otherwise. `parity` uses cycle decomposition of g if already available, but will not compute it on demand. Since cycle structure is cached in g you may call `cycles(g)` before calling `parity`.

**Examples**

```
julia> g = Perm([3,4,1,2,5])
(1,3)(2,4)

julia> parity(g)
0

julia> g = Perm([3,4,5,2,1,6])
(1,3,5)(2,4)

julia> parity(g)
1
```

source

`Base.sign` – Method.

```
sign(g::Perm)
```

Return the sign of a permutation.

`sign` returns $1$ if g is even and $-1$ if g is odd. `sign` represents the homomorphism from the permutation group to the unit group of $\mathbb{Z}$ whose kernel is the alternating group.

**Examples**

```
julia> g = Perm([3,4,1,2,5])
(1,3)(2,4)

julia> sign(g)
1

julia> g = Perm([3,4,5,2,1,6])
(1,3,5)(2,4)

julia> sign(g)
-1
```

source

AbstractAlgebra.Generic.permtype – Method.

```
permtype(g::Perm)
```

Return the type of permutation g, i.e. lengths of disjoint cycles in cycle decomposition of g.

The lengths are sorted in decreasing order by default. permtype(g) fully determines the conjugacy class of g.

**Examples**

```
julia> g = Perm([3,4,5,2,1,6])
(1,3,5)(2,4)

julia> permtype(g)
3-element Vector{Int64}:
 3
 2
 1

julia> e = one(g)
()

julia> permtype(e)
6-element Vector{Int64}:
 1
 1
 1
 1
 1
 1
```

source

Note that even an Int64 can be easily overflowed when computing with symmetric groups. Thus, by default, order returns (always correct) BigInts. If you are sure that the computation will not overflow, you may use order(::Type{T}, ...) to perform computations with machine integers. Julia's standard promotion rules apply for the returned value.

Since SymmetricGroup implements the iterator protocol, you may iterate over all permutations via a simple loop:

```
for p in SymmetricGroup(n)
   ...
end
```

Iteration over all permutations in reasonable time, (i.e. in terms of minutes) is possible when $n13$.

You may also use the non-allocating Generic.elements! function for $n14$ (or even $15$ if you are patient enough), which is an order of magnitude faster.

AbstractAlgebra.Generic.elements! – Method.

```
Generic.elements!(G::SymmetricGroup)
```

Return an unsafe iterator over all permutations in `G`. Only one permutation is allocated and then modified in-place using the non-recursive Heaps algorithm.

Note: you need to explicitly copy permutations intended to be stored or modified.

**Examples**

```
julia> elts = Generic.elements!(SymmetricGroup(5));


julia> length(elts)
120

julia> for p in Generic.elements!(SymmetricGroup(3))
         println(p)
       end
()
(1,2)
(1,3,2)
(2,3)
(1,2,3)
(1,3)

julia> A = collect(Generic.elements!(SymmetricGroup(3))); A
6-element Vector{Perm{Int64}}:
 (1,3)
 (1,3)
 (1,3)
 (1,3)
 (1,3)
 (1,3)

julia> unique(A)
1-element Vector{Perm{Int64}}:
 (1,3)
```

source

However, since all permutations yielded by `elements!` are aliased (modified "in-place"), `collect(Generic.elements!(Symmetri` returns a vector of identical permutations.

**Note**

If you intend to use or store elements yielded by `elements!` you need to **deepcopy** them explicitly.

## 37.4 Arithmetic operators

`Base.:*` – Method.

```
*(g::Perm, h::Perm)
```

Return the composition $hg$ of two permutations.

This corresponds to the action of permutation group on the set `[1..n]` **on the right** and follows the convention of GAP.

If g and h are parametrized by different types, the result is promoted accordingly.

**Examples**

```
julia> Perm([2,3,1,4])*Perm([1,3,4,2]) # (1,2,3)*(2,3,4)
(1,3)(2,4)
```

source

Base.:^ – Method.

```
^(g::Perm, n::Integer)
```

Return the $n$-th power of a permutation g.

By default g^n is computed by cycle decomposition of g if `n > 3`. `Generic.power_by_squaring` provides a different method for powering which may or may not be faster, depending on the particular case. Due to caching of the cycle structure, repeated powering of g will be faster with the default method.

**Examples**

```
julia> g = Perm([2,3,4,5,1])
(1,2,3,4,5)

julia> g^3
(1,4,2,5,3)

julia> g^5
()
```

source

Base.inv – Method.

```
Base.inv(g::Perm)
```

Return the inverse of the given permutation, i.e. the permutation $g^{-1}$ such that $gg^{-1} = g^{-1}g$ is the identity permutation.

source

Permutations parametrized by different types can be multiplied, and follow the standard julia integer promotion rules:

```
g = rand(SymmetricGroup(Int8(5)));
h = rand(SymmetricGroup(UInt32(5)));
typeof(g*h)

# output
Perm{UInt32}
```

## 37.5 Coercion

The following coercions are available for `G::SymmetricGroup` parent objects. Each of the methods perform basic sanity checks on the input which can be switched off by the second argument.

**Examples**

```
(G::SymmetricGroup)(::AbstractVector{<:Integer}[, check=true])
```

Turn a vector of integers into a permutation (performing conversion, if necessary).

```
(G::SymmetricGroup)(::Perm[, check=true])
```

Coerce a permutation `p` into group `G` (performing the conversion, if necessary). If `p` is already an element of `G` no copy is performed.

```
(G::SymmetricGroup)(::String[, check=true])
```

Parse the string input e.g. copied from the output of GAP. The method uses the same logic as the `perm"..."` macro. The string is sanitized and checked for disjoint cycles. Both `string(p::Perm)` (if `setpermstyle(:cycles)`) and `string(cycles(p::Perm))` are valid input for this method.

```
(G::SymmetricGroup{T})(::CycleDec{T}[, check=true]) where T
```

Turn a cycle decomposition object into a permutation.

## 37.6 Comparison

`Base.:(==)` – Method.

```
==(g::Perm, h::Perm)
```

Return `true` if permutations are equal, otherwise return `false`.

Permutations parametrized by different integer types are considered equal if they define the same permutation in the abstract permutation group.

**Examples**

```
julia> g = Perm(Int8[2,3,1])
(1,2,3)

julia> h = perm"(3,1,2)"
(1,2,3)

julia> g == h
true
```

source

`Base.:(==)` – Method.

```
==(G::SymmetricGroup, H::SymmetricGroup)
```

Return `true` if permutation groups are equal, otherwise return `false`.

Permutation groups on the same number of letters, but parametrized by different integer types are considered different.

**Examples**

```
julia> G = SymmetricGroup(UInt(5))
Permutation group over 5 elements

julia> H = SymmetricGroup(5)
Permutation group over 5 elements

julia> G == H
false
```

source

## 37.7 Misc

`Base.rand` – Method.

```
rand([rng=Random.default_rng(),] G::SymmetricGroup)
```

Return a random permutation from `G`.

source

`AbstractAlgebra.Generic.matrix_repr` – Method.

```
matrix_repr(a::Perm{T}) -> Matrix{T}
matrix_repr(M::Type{<:Matrix}, a::Perm)
matrix_repr(M::Type{<:SparseMatrixCSC}, a::Perm)
```

Return the permutation matrix representing a via the natural embedding of the permutation group into the general linear group over $\mathbb{Z}$.

The result type can be chosen via `M`; if `M` has no element type, the integer type of `a` is used. The sparse variant requires `SparseArrays` to be loaded.

**Examples**

```
julia> p = Perm([2,3,1])
(1,2,3)

julia> matrix_repr(p)
3×3 Matrix{Int64}:
 0  1  0
 0  0  1
 1  0  0

julia> using SparseArrays

julia> matrix_repr(SparseMatrixCSC, p)
3×3 SparseMatrixCSC{Int64, Int64} with 3 stored entries:
 ·  1  ·
 ·  ·  1
 1  ·  ·
```

source

```
matrix_repr(Y::YoungTableau{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, Y::YoungTableau) -> M
```

Construct an integer matrix representing the tableau; equivalent to `M(Y)`.

**Examples**

```
julia> y = YoungTableau([4,3,1]);

julia> matrix_repr(y)
3×4 Matrix{Int64}:
 1  2  3  4
 5  6  7  0
 8  0  0  0
```

source

```
matrix_repr(xi::SkewDiagram{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, xi::SkewDiagram) -> M
```

Return a matrix A representing the diagram xi, i.e. A[i,j] == 1 if and only if (i,j) is in xi.lam but not in xi.mu; equivalent to M(xi).

source

AbstractAlgebra.Generic.emb – Method.

```
emb(G::SymmetricGroup, V::Vector{Int}, check::Bool=true)
```

Return the natural embedding of a permutation group into G as the subgroup permuting points indexed by V.

**Examples**

```
julia> p = Perm([2,3,1])
(1,2,3)

julia> f = Generic.emb(SymmetricGroup(5), [3,2,5]);


julia> f(p)
(2,5,3)
```

source

AbstractAlgebra.Generic.emb! – Method.

```
emb!(result::Perm, p::Perm, V)
```

Embed permutation p into permutation result on the indices given by V.

This corresponds to the natural embedding of $S_k$ into $S_n$ as the subgroup permuting points indexed by V.

**Examples**

```
julia> p = Perm([2,1,4,3])
(1,2)(3,4)

julia> Generic.emb!(Perm(collect(1:5)), p, [3,1,4,5])
(1,3)(4,5)
```

source

# Chapter 38

# Partitions and Young tableaux

AbstractAlgebra.jl provides basic support for computations with Young tableaux, skew diagrams and the characters of permutation groups (implemented `src/generic/YoungTabs.jl`). All functionality of permutations is accessible in the `Generic` submodule.

## 38.1 Partitions

The basic underlying object for those concepts is `Partition` of a number $n$, i.e. a sequence of positive integers $n_1, \ldots, n_k$ which sum to $n$. Partitions in AbstractAlgebra.jl are represented internally by non-increasing `Vectors` of `Ints`. Partitions are printed using the standard notation, i.e. $9 = 4 + 2 + 1 + 1 + 1$ is shown as $4_1 2_1 1_3$ with the subscript indicating the count of a summand in the partition.

`AbstractAlgebra.Generic.Partition` – Type.

```
Partition(part::Vector{<:Integer}[, check::Bool=true]) <: AbstractVector{Int}
```

Represent integer partition in the non-increasing order.

`part` will be sorted, if necessary. Checks for validity of input can be skipped by calling the (inner) constructor with `false` as the second argument.

Functionally `Partition` is a thin wrapper over `Vector{Int}`.

Fieldnames:

- `n::Int` - the partitioned number
- `part::Vector{Int}` - a non-increasing sequence of summands of `n`.

**Examples**

```
julia> p = Partition([4,2,1,1,1])
4₁2₁1₃

julia> p.n == sum(p.part)
true
```

source

### Array interface

`Partition` is a concrete (immutable) subtype of `AbstractVector{Integer}` and implements the standard Array interface.

`Base.size` – Method.

```
size(p::Partition)
```

Return the size of the vector which represents the partition.

**Examples**

```
julia> p = Partition([4,3,1]); size(p)
(3,)
```

source

`Base.getindex` – Method.

```
getindex(p::Partition, i::Integer)
```

Return the `i`-th part (in non-increasing order) of the partition.

source

These functions work on the level of `p.part` vector.

One can easily iterate over all partitions of $n$ using the `Generic.partitions` function.

`AbstractAlgebra.Generic.partitions` – Function.

```
partitions(n::Integer)
```

Return the vector of all permutations of `n`. For an unsafe generator version see `partitions!`.

**Examples**

```
julia> Generic.partitions(5)
7-element Vector{AbstractAlgebra.Generic.Partition{Int64}}:
 1₅
 2₁1₃
 3₁1₂
 2₂1₁
 4₁1₁
 3₁2₁
 5₁
```

source

You may also have a look at JuLie.jl package for more utilities related to partitions.

The number of all partitions can be computed by the hidden function `_numpart`. Much faster implementation is available in Nemo.jl.

`AbstractAlgebra.Generic._numpart` – Function.

```
_numpart(n::Integer)
```

Return the number of all distinct integer partitions of `n`. The function uses Euler pentagonal number theorem for recursive formula. For more details see OEIS sequence A000041. Note that `_numpart(0) = 1` by convention.

source

Since `Partition` is a subtype of `AbstractVector` generic functions which operate on vectors should work in general. However the meaning of `conj` has been changed to agree with the traditional understanding of conjugation of `Partitions`:

`Base.conj` – Method.

```
conj(part::Partition)
```

Return the conjugated partition of `part`, i.e. the partition corresponding to the Young diagram of `part` reflected through the main diagonal.

**Examples**

```
julia> p = Partition([4,2,1,1,1])
4₁2₁1₃

julia> conj(p)
5₁2₁1₂
```

source

`Base.conj` – Method.

```
conj(part::Partition, v::Vector)
```

Return the conjugated partition of `part` together with permuted vector `v`.

source

## 38.2 Young Diagrams and Young Tableaux

Mathematically speaking Young diagram is a diagram which consists of rows of square boxes such that the number of boxes in each row is no less than the number of boxes in the previous row. For example partition $4_1 3_2 1$ represents the following diagram.

```
┌──┬──┬──┬──┐
│  │  │  │  │
├──┼──┼──┼──┴┐
│  │  │  │   │
├──┼──┼──┤
│  │  │  │
├──┼──┼──┘
│  │
└──┘
```

Young Tableau is formally a bijection between the set of boxes of a Young Diagram and the set $\{1, \ldots, n\}$. If a bijection is increasing along rows and columns of the diagram it is referred to as **standard**. For example

```
┌──┬──┬──┬──┐
│ 1│ 2│ 3│ 4│
├──┼──┼──┼──┴┐
│ 5│ 6│ 7│   │
├──┼──┼──┤
│ 8│ 9│10│
├──┼──┼──┘
│11│
└──┘
```

is a standard Young tableau of $4_1 3_2 1$ where the bijection assigns consecutive natural numbers to consecutive (row-major) cells.

### Constructors

In AbstractAlgebra.jl Young tableau are implemented as essentially row-major sparse matrices, i.e. `YoungTableau <: AbstractMatrix{Int}` but only the defining `Partition` and the (row-major) fill-vector is stored.

`AbstractAlgebra.Generic.YoungTableau` – Type.

```
YoungTableau(part::Partition[, fill::Vector{Int}=collect(1:sum(part))])  <: AbstractMatrix{Int}
```

Return the Young tableaux of partition `part`, filled linearly by `fill` vector. Note that `fill` vector is in **row-major** format.

Fields:

- `part` - the partition defining Young diagram
- `fill` - the row-major fill vector: the entries of the diagram.

#### Examples

```
julia> p = Partition([4,3,1]); y = YoungTableau(p)
┌──┬──┬──┬──┐
│ 1│ 2│ 3│ 4│
├──┼──┼──┼──┴┐
│ 5│ 6│ 7│   │
```

```
├──┼──┘└──┘
│ 8 │
└──┘

julia> y.part
4₁3₁1₁

julia> y.fill
8-element Vector{Int64}:
 1
 2
 3
 4
 5
 6
 7
 8
```

source

For convenience there exists an alternative constructor of YoungTableau, which accepts a vector of integers and constructs Partition internally.

```
YoungTableau(p::Vector{Integer}[, fill=collect(1:sum(p))])
```

### Array interface

To make YoungTableaux array-like we implement the following functions:

Base.size – Method.

```
size(Y::YoungTableau)
```

Return size of the smallest array containing Y, i.e. the tuple of the number of rows and the number of columns of Y.

**Examples**

```
julia> y = YoungTableau([4,3,1]); size(y)
(3, 4)
```

source

Base.getindex – Method.

```
getindex(Y::YoungTableau, n::Integer)
```

Return the column-major linear index into the size(Y)-array. If a box is outside of the array return 0.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> y[1]
1

julia> y[2]
5

julia> y[4]
2

julia> y[6]
0
```

source

Also the double-indexing corresponds to `(row, column)` access to an abstract array.

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> y[1,2]
2

julia> y[2,3]
7

julia> y[3,2]
0
```

Functions defined for `AbstractArray` type based on those (e.g. `length`) should work. Again, as in the case of `Partition` the meaning of `conj` is altered to reflect the usual meaning for Young tableaux:

`Base.conj` – Method.

```
conj(Y::YoungTableau)
```

Return the conjugated tableau, i.e. the tableau reflected through the main diagonal.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┴┐
│ 5 │ 6 │ 7 │
├───┼───┼───┴┐
│ 8 │
└───┘

julia> conj(y)
┌───┬───┬───┐
│ 1 │ 5 │ 8 │
├───┼───┼───┴┐
│ 2 │ 6 │
├───┼───┴┐
│ 3 │ 7 │
├───┼───┴┐
│ 4 │
└───┘
```

source

### Pretty-printing

Similarly to permutations we have two methods of displaying Young Diagrams:

`AbstractAlgebra.Generic.setyoungtabstyle` – Function.

```
setyoungtabstyle(format::Symbol)
```

Select the style in which Young tableaux are displayed (in REPL or in general as string). This can be either

- `:array` - as matrices of integers, or
- `:diagram` - as filled Young diagrams (the default).

The difference is purely esthetical.

**Examples**

```
julia> Generic.setyoungtabstyle(:array)
:array

julia> p = Partition([4,3,1]); YoungTableau(p)
 1  2  3  4
 5  6  7
 8

julia> Generic.setyoungtabstyle(:diagram)
:diagram

julia> YoungTableau(p)
┌───┬───┬───┬───┐
```

```
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘
```

source

### Ulitility functions

`AbstractAlgebra.Generic.matrix_repr` – Method.

```
matrix_repr(a::Perm{T}) -> Matrix{T}
matrix_repr(M::Type{<:Matrix}, a::Perm)
matrix_repr(M::Type{<:SparseMatrixCSC}, a::Perm)
```

Return the permutation matrix representing a via the natural embedding of the permutation group into the general linear group over $\mathbb{Z}$.

The result type can be chosen via `M`; if `M` has no element type, the integer type of a is used. The sparse variant requires `SparseArrays` to be loaded.

**Examples**

```
julia> p = Perm([2,3,1])
(1,2,3)

julia> matrix_repr(p)
3×3 Matrix{Int64}:
 0  1  0
 0  0  1
 1  0  0

julia> using SparseArrays

julia> matrix_repr(SparseMatrixCSC, p)
3×3 SparseMatrixCSC{Int64, Int64} with 3 stored entries:
 ·  1  ·
 ·  ·  1
 1  ·  ·
```

source

```
matrix_repr(Y::YoungTableau{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, Y::YoungTableau) -> M
```

Construct an integer matrix representing the tableau; equivalent to `M(Y)`.

**Examples**

```
julia> y = YoungTableau([4,3,1]);

julia> matrix_repr(y)
3×4 Matrix{Int64}:
 1  2  3  4
 5  6  7  0
 8  0  0  0
```

source

```
matrix_repr(xi::SkewDiagram{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, xi::SkewDiagram) -> M
```

Return a matrix A representing the diagram `xi`, i.e. `A[i,j] == 1` if and only if `(i,j)` is in `xi.lam` but not in `xi.mu`; equivalent to `M(xi)`.

source

`Base.fill!` – Method.

```
fill!(Y::YoungTableaux, V::Vector{<:Integer})
```

Replace the fill vector `Y.fill` by `V`. No check if the resulting tableau is standard (i.e. increasing along rows and columns) is performed.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> fill!(y, [2:9...])
┌───┬───┬───┬───┐
│ 2 │ 3 │ 4 │ 5 │
├───┼───┼───┼───┘
│ 6 │ 7 │ 8 │
├───┼───┴───┘
│ 9 │
└───┘
```

source

## 38.3 Characters of permutation groups

Irreducible characters (at least over field of characteristic $0$) of the full group of permutations $S_n$ correspond via Specht modules to partitions of $n$.

AbstractAlgebra.Generic.character – Method.

```
character(lambda::Partition)
```

Return the $\lambda$-th irreducible character of permutation group on `sum(lambda)` symbols. The returned character function is of the following signature:

```
chi(p::Perm[, check::Bool=true]) -> BigInt
```

The function checks (if `p` belongs to the appropriate group) can be switched off by calling `chi(p, false)`. The values computed by $\chi$ are cached in look-up table.

The computation follows the Murnaghan-Nakayama formula: $\chi_\lambda(\sigma) = \sum_{\text{rimhook } \xi \subset \lambda} (-1)^{ll(\lambda \setminus \xi)} \chi_{\lambda \setminus \xi}(\tilde{\sigma})$ where $\lambda \setminus \xi$ denotes the skew diagram of $\lambda$ with $\xi$ removed, $ll$ denotes the leg-length (i.e. number of rows - 1) and $\tilde{\sigma}$ is permutation obtained from $\sigma$ by the removal of the longest cycle.

For more details see e.g. Chapter 2.8 of *Group Theory and Physics* by S.Sternberg.

**Examples**

```
julia> G = SymmetricGroup(4)
Full symmetric group over 4 elements

julia> chi = character(Partition([3,1])); # character of the regular representation


julia> chi(one(G))
3

julia> chi(perm"(1,3)(2,4)")
-1
```

source

AbstractAlgebra.Generic.character – Method.

```
character(lambda::Partition, p::Perm, check::Bool=true) -> BigInt
```

Return the value of `lambda`-th irreducible character of the permutation group on permutation `p`.

source

AbstractAlgebra.Generic.character – Method.

```
character(lambda::Partition, mu::Partition, check::Bool=true) -> BigInt
```

Return the value of `lambda`-th irreducible character on the conjugacy class represented by partition `mu`.

source

The values computed by characters are cached in an internal dictionary `Dict{Tuple{BitVector,Vector{Int}}, BigInt}`. Note that all of the above functions return `BigInts`. If you are sure that the computations do not overflow, variants of the last two functions using `Int` are available:

```
character(::Type{Int}, lambda::Partition, p::Perm[, check::Bool=true])
character(::Type{Int}, lambda::Partition, mu::Partition[, check::Bool=true])
```

The dimension $\dim \lambda$ of the irreducible module corresponding to partition $\lambda$ can be computed using Hook length formula

`AbstractAlgebra.Generic.rowlength` – Function.

```
rowlength(Y::YoungTableau, i, j)
```

Return the row length of Y at box `(i,j)`, i.e. the number of boxes in the `i`-th row of the diagram of Y located to the right of the `(i,j)`-th box.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> Generic.rowlength(y, 1,2)
2

julia> Generic.rowlength(y, 2,3)
0

julia> Generic.rowlength(y, 3,3)
0
```

source

`AbstractAlgebra.Generic.collength` – Function.

```
collength(Y::YoungTableau, i, j)
```

Return the column length of Y at box `(i,j)`, i.e. the number of boxes in the `j`-th column of the diagram of Y located below of the `(i,j)`-th box.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> Generic.collength(y, 1,1)
2

julia> Generic.collength(y, 1,3)
1

julia> Generic.collength(y, 2,4)
0
```

source

AbstractAlgebra.Generic.hooklength – Function.

```
hooklength(Y::YoungTableau, i, j)
```

Return the hook-length of an element in Y at position (i,j), i.e the number of cells in the i-th row to the right of (i,j)-th box, plus the number of cells in the j-th column below the (i,j)-th box, plus 1.

Return 0 for (i,j) not in the tableau Y.

**Examples**

```
julia> y = YoungTableau([4,3,1])
┌───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │
├───┼───┼───┼───┘
│ 5 │ 6 │ 7 │
├───┼───┴───┘
│ 8 │
└───┘

julia> hooklength(y, 1,1)
6

julia> hooklength(y, 1,3)
3

julia> hooklength(y, 2,4)
0
```

source

AbstractAlgebra.Generic.dim – Method.

```
dim(Y::YoungTableau) -> BigInt
```

Return the dimension (using hook-length formula) of the irreducible representation of permutation group $S_n$ associated the partition `Y.part`.

Since the computation overflows easily `BigInt` is returned. You may perform the computation of the dimension in different type by calling `dim(Int, Y)`.

**Examples**

```
julia> dim(YoungTableau([4,3,1]))
70

julia> dim(YoungTableau([3,1])) # the regular representation of S_4
3
```

source

The character associated with `Y.part` can also be used to compute the dimension, but as it is expected the Murnaghan-Nakayama is much slower even though (due to caching) consecutive calls are fast:

```
julia> λ = Partition(collect(12:-1:1))
12₁11₁10₁9₁8₁7₁6₁5₁4₁3₁2₁1₁

julia> @time dim(YoungTableau(λ))
  0.224430 seconds (155.77 k allocations: 7.990 MiB)
907959013273274765688008132453133022298362218754867200 0

julia> @time dim(YoungTableau(λ))
  0.000038 seconds (335 allocations: 10.734 KiB)
907959013273274765688008132453133022298362218754867200 0

julia> G = SymmetricGroup(sum(λ))
Full symmetric group over 78 elements

julia> @time character(λ, one(G))
  0.000046 seconds (115 allocations: 16.391 KiB)
907959013273274765688008132453133022298362218754867200 0

julia> @time character(λ, one(G))
  0.001439 seconds (195 allocations: 24.453 KiB)
907959013273274765688008132453133022298362218754867200 0
```

### Low-level functions and characters

As mentioned above `character` functions use the Murnaghan-Nakayama rule for evaluation. The implementation follows

> Dan Bernstein, The computational complexity of rules for the character table of $S_n$ *Journal of Symbolic Computation*, **37** (6), 2004, p. 727-748,

implementing the following functions. For precise definitions and meaning please consult the paper cited.

`AbstractAlgebra.Generic.partitionseq` – Function.

```
partitionseq(lambda::Partition)
```

Return a sequence (as `BitVector`) of `false`s and `true`s constructed from `lambda`: tracing the lower contour of the Young Diagram associated to `lambda` from left to right a `true` is inserted for every horizontal and `false` for every vertical step. The sequence always starts with `true` and ends with `false`.

source

```
partitionseq(seq::BitVector)
```

Return the essential part of the sequence `seq`, i.e. a subsequence starting at first `true` and ending at last `false`.

source

`AbstractAlgebra.Generic.is_rimhook` – Method.

```
is_rimhook(R::BitVector, idx::Integer, len::Integer)
```

`R[idx:idx+len]` forms a rim hook in the Young Diagram of partition corresponding to R iff `R[idx] == true` and `R[idx+len] == false`.

source

`AbstractAlgebra.Generic.MN1inner` – Function.

```
MN1inner(R::BitVector, mu::Partition, t::Integer, charvals)
```

Return the value of $\lambda$-th irreducible character on conjugacy class of permutations represented by partition `mu`, where R is the (binary) partition sequence representing $\lambda$. Values already computed are stored in `charvals::Dict{Tuple{BitVector,Vector{Int}}, Int}`. This is an implementation (with slight modifications) of the Murnaghan-Nakayama formula as described in

```
Dan Bernstein,
"The computational complexity of rules for the character table of Sn"
_Journal of Symbolic Computation_, 37(6), 2004, p. 727-748.
```

source

## 38.4 Skew Diagrams

Skew diagrams are formally differences of two Young diagrams. Given $\lambda$ and $\mu$, two partitions of $n+m$ and $m$ (respectively). Suppose that each of cells of $\mu$ is a cell of $\lambda$ (i.e. parts of $\mu$ are no greater than the corresponding parts of $\lambda$). Then the skew diagram denoted by $\lambda/\mu$ is the set theoretic difference the of sets of boxes, i.e. is a diagram with exactly $n$ boxes:

`AbstractAlgebra.Generic.SkewDiagram` – Type.

```
SkewDiagram(lambda::Partition, mu::Partition) <: AbstractMatrix{Int}
```

Implements a skew diagram, i.e. a difference of two Young diagrams represented by partitions `lambda` and `mu`. (below dots symbolise the removed entries)

**Examples**

```
julia> l = Partition([4,3,2])
4₁3₁2₁

julia> m = Partition([3,1,1])
3₁1₂

julia> xi = SkewDiagram(l,m)
3×4 AbstractAlgebra.Generic.SkewDiagram{Int64}:
 ·  ·  ·  1
 ·  1  1
 ·  1
```

source

SkewDiagram implements array interface with the following functions:

Base.size – Method.

```
size(xi::SkewDiagram)
```

Return the size of array where `xi` is minimally contained. See `size(Y::YoungTableau)` for more details.

source

Base.in – Method.

```
in(t::Tuple{Integer,Integer}, xi::SkewDiagram)
```

Check if box at position `(i,j)` belongs to the skew diagram `xi`.

source

Base.getindex – Method.

```
getindex(xi::SkewDiagram, n::Integer)
```

Return 1 if linear index `n` corresponds to (column-major) entry in `xi.lam` which is not contained in `xi.mu`. Otherwise return `0`.

source

The support for skew diagrams is very rudimentary. The following functions are available:

`AbstractAlgebra.Generic.is_rimhook` – Method.

```
is_rimhook(xi::SkewDiagram)
```

Check if `xi` represents a rim-hook diagram, i.e. its diagram is edge-connected and contains no $2 \times 2$ squares.

source

`AbstractAlgebra.Generic.leglength` – Function.

```
leglength(xi::SkewDiagram[, check::Bool=true])
```

Compute the leglength of a rim-hook `xi`, i.e. the number of rows with non-zero entries minus one. If `check` is `false` function will not check whether `xi` is actually a rim-hook.

source

`AbstractAlgebra.Generic.matrix_repr` – Method.

```
matrix_repr(a::Perm{T}) -> Matrix{T}
matrix_repr(M::Type{<:Matrix}, a::Perm)
matrix_repr(M::Type{<:SparseMatrixCSC}, a::Perm)
```

Return the permutation matrix representing a via the natural embedding of the permutation group into the general linear group over $\mathbb{Z}$.

The result type can be chosen via `M`; if `M` has no element type, the integer type of `a` is used. The sparse variant requires `SparseArrays` to be loaded.

**Examples**

```
julia> p = Perm([2,3,1])
(1,2,3)

julia> matrix_repr(p)
3×3 Matrix{Int64}:
 0  1  0
 0  0  1
 1  0  0

julia> using SparseArrays

julia> matrix_repr(SparseMatrixCSC, p)
3×3 SparseMatrixCSC{Int64, Int64} with 3 stored entries:
 ⋅  1  ⋅
 ⋅  ⋅  1
 1  ⋅  ⋅
```

source

```
matrix_repr(Y::YoungTableau{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, Y::YoungTableau) -> M
```

Construct an integer matrix representing the tableau; equivalent to `M(Y)`.

**Examples**

```
julia> y = YoungTableau([4,3,1]);

julia> matrix_repr(y)
3×4 Matrix{Int64}:
 1  2  3  4
 5  6  7  0
 8  0  0  0
```

source

```
matrix_repr(xi::SkewDiagram{T}) -> Matrix{T}
matrix_repr(M::Type{<:AbstractMatrix}, xi::SkewDiagram) -> M
```

Return a matrix A representing the diagram `xi`, i.e. `A[i,j] == 1` if and only if `(i,j)` is in `xi.lam` but not in `xi.mu`; equivalent to `M(xi)`.

source

# Part VII

# Modules

# Chapter 39

# Introduction

As with many generic constructions in AbstractAlgebra, the modules that are provided in AbstractAlgebra itself work over a Euclidean domain. Moreover, they are limited to finitely presented modules.

Free modules and vector spaces are provided over Euclidean domains and fields respectively and then sub-module, quotient module and direct sum module constructions are possible recursively over these.

It's also possible to compute an invariant decomposition using the Smith Normal Form.

The system also provides module homomorphisms and isomorphisms, building on top of the map interface.

As for rings and fields, modules follow an interface which other modules are expected to follow. However, very little generic functionality is provided automatically once this interface is implemented by a new module type.

The purpose of the module interface is simply to encourage uniformity in the module interfaces of systems that build on AbstractAlgebra. Of course modules are so diverse that this is a very loosely defined interface to accommodate the diversity of possible representations and implementations.

# Chapter 40

# Finitely presented modules

AbstractAlgebra allows the construction of finitely presented modules (i.e. with finitely many generators and relations), starting from free modules.

The generic code provided by AbstractAlgebra will only work for modules over euclidean domains.

Free modules can be built over both commutative and noncommutative rings. Other types of module are restricted to fields and euclidean rings.

## 40.1 Abstract types

AbstractAlgebra provides two abstract types for finitely presented modules and their elements:

- `FPModule{T}` is the abstract type for finitely presented module parent

types

- `FPModuleElem{T}` is the abstract type for finitely presented module

element types

Note that the abstract types are parameterised. The type T should usually be the type of elements of the ring the module is over.

## 40.2 Module functions

All finitely presented modules over a Euclidean domain implement the following functions.

### Basic functions

```
zero(M::FPModule)
```

```
iszero(m::FPModuleElem{T}) where T <: RingElement
```

Return `true` if the given module element is zero.

```
number_of_generators(M::FPModule{T}) where T <: RingElement
```

Return the number of generators of the module $M$ in its current representation.

```
gen(M::FPModule{T}, i::Int) where T <: RingElement
```

Return the $i$-th generator (indexed from $1$) of the module $M$.

```
gens(M::FPModule{T}) where T <: RingElement
```

Return a Julia array of the generators of the module $M$.

```
relations(M::FPModule{T}) where T <: RingElement
```

Return a Julia vector of all the relations between the generators of `M`. Each relation is given as an AbstractAlgebra row matrix.

`AbstractAlgebra.vector_space_dim` – Function.

```
vector_space_dim(M::FPModule)
```

Return the dimension of the given vector space over `base_ring(M)`. This method is only supported, when `base_ring(M)` is a field.

**Examples**

```
julia> M = free_module(QQ, 2)
Vector space of dimension 2 over rationals

julia> vector_space_dim(M)
2

julia> m = M([1, 2])
(1//1, 2//1)

julia> N, = sub(M, [m])
(Subspace over rationals with 1 generator and no relations, Hom: N -> M)

julia> vector_space_dim(N)
1

julia> Q, = quo(M, N)
(Quotient space over rationals with 1 generator and no relations, Hom: M -> Q)

julia> vector_space_dim(Q)
1
```

source

`AbstractAlgebra.Generic.dim` – Method.

```
dim(M::FPModule{T}) where T <: FieldElement
```

Return the dimension of the given vector space `M` over `base_ring(M)`. This is an alias for `vector_space_dim`.

source

```
rank(M::FPModule{T}) where T <: FieldElement
```

Return the rank of the module `M`. For a finitely presented module over a field this is the same as `dim(M)`. For a free module over an arbitrary ring see the section on free modules. For all other modules `rank` is not implemented and throws an error.

**Examples**

```
julia> M = free_module(QQ, 2)
Vector space of dimension 2 over rationals

julia> n = number_of_generators(M)
2

julia> G = gens(M)
2-element Vector{AbstractAlgebra.Generic.FreeModuleElem{Rational{BigInt}}}:
 (1//1, 0//1)
 (0//1, 1//1)

julia> R = relations(M)
AbstractAlgebra.Generic.MatSpaceElem{Rational{BigInt}}[]

julia> g1 = gen(M, 1)
(1//1, 0//1)

julia> !iszero(g1)
true

julia> M = free_module(QQ, 2)
Vector space of dimension 2 over rationals

julia> z = zero(M)
(0//1, 0//1)

julia> iszero(z)
true
```

### Element constructors

We can construct elements of a module $M$ by specifying linear combinations of the generators of $M$. This is done by passing a vector of ring elements.

```
(M::FPModule{T})(v::Vector{T}) where T <: RingElement
```

Construct the element of the module $M$ corresponding to $\sum_i g[i]v[i]$ where $g[i]$ are the generators of the module $M$. The resulting element will lie in the module $M$.

### Coercions

Given a module $M$ and an element $n$ of a module $N$, it is possible to coerce $n$ into $M$ using the notation $M(n)$ in certain circumstances.

In particular the element $n$ will be automatically coerced along any canonical injection of a submodule map and along any canonical projection of a quotient map. There must be a path from $N$ to $M$ along such maps.

#### Examples

```
F = free_module(ZZ, 3)

S1, f = sub(F, [rand(F, -10:10)])

S, g = sub(F, [rand(F, -10:10)])
Q, h = quo(F, S)

m = rand(S1, -10:10)
n = Q(m)
```

### Arithmetic operators

Elements of a module can be added, subtracted or multiplied by an element of the ring the module is defined over and compared for equality.

In the case of a noncommutative ring, both left and right scalar multiplication are defined.

### Basic manipulation

```
zero(M::FPModule)
```

#### Examples

```
julia> M = free_module(QQ, 2)
Vector space of dimension 2 over rationals

julia> z = zero(M)
(0//1, 0//1)
```

### Element indexing

`Base.getindex` – Method.

```
getindex(a::Fac, b) -> Int
```

If $b$ is a factor of $a$, the corresponding exponent is returned. Otherwise an error is thrown.

source

`AbstractAlgebra.coordinates` – Method.

```
coordinates(v::FPModuleElem{T}, i::Int) where T <: RingElement
```

Return the coordinates of the module element $v$ as a `Vector{T}`.

source

### Module comparison

`Base.:(==)` – Method.

```
==(M::FPModule{T}, N::FPModule{T}) where T <: RingElement
```

Return `true` if the modules are (constructed to be) the same module elementwise. This is not object equality and it is not isomorphism. In fact, each method of constructing modules (submodules, quotient modules, products, etc.) must extend this notion of equality to the modules they create.

**Examples**

```
julia> M = free_module(QQ, 2)
Vector space of dimension 2 over rationals

julia> M == M
true
```

source

### Isomorphism

`AbstractAlgebra.is_isomorphic` – Method.

```
is_isomorphic(M::FPModule{T}, N::FPModule{T}) where T <: RingElement
```

Return `true` if the modules $M$ and $N$ are isomorphic.

source

**Note**

Note that this function relies on the Smith normal form over the base ring of the modules being able to be made unique. This is true for Euclidean domains for which `divrem` has a fixed choice of quotient and remainder, but it will not in general be true for Euclidean rings that are not domains.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> m1 = rand(M, -10:10)
(3, -1, 0)

julia> m2 = rand(M, -10:10)
(4, 4, -7)

julia> S, f = sub(M, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S -> M)

julia> I, g = image(f)
(Submodule over integers with 2 generators and no relations, Hom: I -> M)

julia> is_isomorphic(S, I)
true
```

### Invariant Factor Decomposition

For modules over a euclidean domain one can take the invariant factor decomposition to determine the structure of the module. The invariant factors are unique up to multiplication by a unit, and even unique if a `canonical_unit` is available for the ring that canonicalises elements.

`AbstractAlgebra.snf` – Method.

```
snf(m::FPModule{T}) where T <: RingElement
```

Return a pair `M, f` consisting of the invariant factor decomposition $M$ of the module `m` and a module homomorphism (isomorphisms) $f : M \to m$. The module `M` is itself a module which can be manipulated as any other module in the system.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> m1 = rand(M, -10:10)
(3, -1, 0)

julia> m2 = rand(M, -10:10)
(4, 4, -7)

julia> S, f = sub(M, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S -> M)

julia> Q, g = quo(M, S)
(Quotient module over integers with 2 generators and relations:
[16 -21], Hom: M -> Q)
```

```
julia> I, f = snf(Q)
(Invariant factor decomposed module over integers with invariant factors BigInt[0], Hom: I -> Q)
```

source

AbstractAlgebra.invariant_factors – Method.

```
invariant_factors(m::FPModule{T}) where T <: RingElement
```

Return a vector of the invariant factors of the module $M$.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> m1 = rand(M, -10:10)
(3, -1, 0)

julia> m2 = rand(M, -10:10)
(4, 4, -7)

julia> S, f = sub(M, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S -> M)

julia> Q, g = quo(M, S)
(Quotient module over integers with 2 generators and relations:
[16 -21], Hom: M -> Q)

julia> invariant_factors(Q)
1-element Vector{BigInt}:
 0
```

source

# Chapter 41

# Free Modules and Vector Spaces

AbstractAlgebra allows the construction of free modules of any rank over any Euclidean ring and the vector space of any dimension over a field. By default the system considers the free module of a given rank over a given ring or vector space of given dimension over a field to be unique.

## 41.1 Generic free module and vector space types

AbstractAlgebra provides generic types for free modules and vector spaces, via the type `FreeModule{T}` for free modules, where T is the type of the elements of the ring $R$ over which the module is built.

Elements of a free module have type `FreeModuleElem{T}`.

Vector spaces are simply free modules over a field.

The implementation of generic free modules can be found in `src/generic/FreeModule.jl`.

The free module of a given rank over a given ring is made unique on the system by caching them (unless an optional cache parameter is set to `false`).

See `src/generic/GenericTypes.jl` for an example of how to implement such a cache (which usually makes use of a dictionary).

## 41.2 Abstract types

The type `FreeModule{T}` belongs to `FPModule{T}` and `FreeModuleElem{T}` to `FPModuleElem{T}`. Here the FP prefix stands for finitely presented.

## 41.3 Functionality for free modules

As well as implementing the entire module interface, free modules provide the following functionality.

### Constructors

`AbstractAlgebra.free_module` – Method.

```
free_module(R::NCRing, rank::Int; cached::Bool = true)
```

Return the free module over the ring $R$ with the given rank.

source

AbstractAlgebra.vector_space – Method.

```
vector_space(R::Field, dim::Int; cached::Bool = true)
```

Return the vector space over the field $R$ with the given dimension.

source

Construct the free module/vector space of given rank/dimension.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals
```

### Basic manipulation

LinearAlgebra.rank – Method.

```
rank(M::FreeModule{T}) where T <: NCRingElement
```

Return the rank of the given free module.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> rank(M)
3
```

source

AbstractAlgebra.vector_space_dim – Method.

```
vector_space_dim(M::FreeModule{T}) where T <: FieldElement
dim(M::FreeModule{T}) where T <: FieldElement
```

Return the dimension of the given vector space.

**Examples**

```
julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> dim(V)
2
```

source

AbstractAlgebra.Generic.basis – Method.

```
basis(M::FreeModule)
```

Return the standard basis of the given free module, i.e. its generators.

**Examples**

```
julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> basis(V)
2-element Vector{AbstractAlgebra.Generic.FreeModuleElem{Rational{BigInt}}}:
 (1//1, 0//1)
 (0//1, 1//1)
```

source

# Chapter 42

# Submodules

AbstractAlgebra allows the construction of submodules/subvector spaces of AbstractAlgebra modules over euclidean domains. These are given as the submodule generated by a finite list of elements in the original module.

We define two submodules to be equal if they are (transitively) submodules of the same module $M$ and their generators generate the same set of elements.

## 42.1 Generic submodule type

AbstractAlgebra implements a generic submodule type `Generic.Submodule{T}` where T is the element type of the base ring in `src/generic/Submodule.jl`. See `src/generic/GenericTypes.jl` for more details of the type definition.

Elements of a generic submodule have type `Generic.SubmoduleElem{T}`.

## 42.2 Abstract types

Submodule types belong to the abstract type `FPModule{T}` and their elements to `FPModuleElem{T}`.

## 42.3 Constructors

`AbstractAlgebra.sub` – Method.

```
sub(m::FPModule{T}, gens::Vector{<:FPModuleElem{T}}) where T <: RingElement
```

Return the submodule of the module `m` generated by the given generators, given as elements of `m`.

source

`AbstractAlgebra.sub` – Method.

```
sub(m::Module{T}, subs::Vector{<:Generic.Submodule{T}}) where T <: RingElement
```

Return the submodule S of the module `m` generated by the union of the given submodules of $m$, and a map which is the canonical injection from S to `m`.

source

Note that the preimage of the canonical injection can be obtained using the preimage function described in the section on module homomorphisms. As the canonical injection is injective, this is unique.

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers

julia> m = M([ZZ(1), ZZ(2)])
(1, 2)

julia> n = M([ZZ(2), ZZ(-1)])
(2, -1)

julia> N, f = sub(M, [m, n])
(Submodule over integers with 2 generators and no relations, Hom: N -> M)

julia> v = N([ZZ(3), ZZ(4)])
(3, 4)

julia> v2 = f(v)
(3, 26)

julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> m = V([QQ(1), QQ(2)])
(1//1, 2//1)

julia> n = V([QQ(2), QQ(-1)])
(2//1, -1//1)

julia> N, f = sub(V, [m, n])
(Subspace over rationals with 2 generators and no relations, Hom: N -> V)
```

## 42.4 Functionality for submodules

In addition to the Module interface, AbstractAlgebra submodules implement the following functionality.

### Basic manipulation

`AbstractAlgebra.Generic.supermodule` – Method.

```
supermodule(M::Submodule{T}) where T <: RingElement
```

Return the module that this module is a submodule of.

source

`AbstractAlgebra.Generic.is_submodule` – Method.

```
is_submodule(M::AbstractAlgebra.FPModule{T}, N::AbstractAlgebra.FPModule{T}) where T <:
↪  RingElement
```

Return `true` if $N$ was constructed as a submodule of $M$. The relation is taken transitively (i.e. subsubmodules are submodules for the purposes of this relation, etc). The module $M$ is also considered a submodule of itself for this relation.

source

`AbstractAlgebra.Generic.is_compatible` – Method.

```
is_compatible(M::AbstractAlgebra.FPModule{T}, N::AbstractAlgebra.FPModule{T}) where T <:
↪  RingElement
```

Return `true, P` if the given modules are compatible, i.e. that they are (transitively) submodules of the same module, P. Otherwise return `false, M`.

source

`AbstractAlgebra.vector_space_dim` – Method.

```
vector_space_dim(N::Submodule{T}) where T <: FieldElement
dim(N::Submodule{T}) where T <: FieldElement
rank(N::Submodule{T}) where T <: FieldElement
```

Return the dimension of the given vector subspace.

source

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers

julia> m = M([ZZ(2), ZZ(3)])
(2, 3)

julia> n = M([ZZ(1), ZZ(4)])
(1, 4)

julia> N1, = sub(M, [m, n])
(Submodule over integers with 2 generators and no relations, Hom: N1 -> M)

julia> N2, = sub(M, [m])
(Submodule over integers with 1 generator and no relations, Hom: N2 -> M)

julia> supermodule(N1) == M
true

julia> is_compatible(N1, N2)
(true, Free module of rank 2 over integers)
```

```
julia> is_submodule(N1, M)
false

julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> m = V([QQ(2), QQ(3)])
(2//1, 3//1)

julia> N, = sub(V, [m])
(Subspace over rationals with 1 generator and no relations, Hom: N -> V)

julia> dim(V)
2

julia> dim(N)
1
```

### Intersection

Base.intersect – Method.

```
intersect(M::FPModule{T}, N::FPModule{T}) where T <: RingElement
```

Return the intersection of the modules $M$ as a submodule of $M$. Note that $M$ and $N$ must be (constructed as) submodules (transitively) of some common module $P$.

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers

julia> m = M([ZZ(2), ZZ(3)])
(2, 3)

julia> n = M([ZZ(1), ZZ(4)])
(1, 4)

julia> N1 = sub(M, [m, n])
(Submodule over integers with 2 generators and no relations, Hom: submodule over integers with 2
↪  generators and no relations -> M)

julia> N2 = sub(M, [m])
(Submodule over integers with 1 generator and no relations, Hom: submodule over integers with 1
↪  generator and no relations -> M)

julia> I = intersect(N1, N2)
Any[]
```

source

# Chapter 43

# Quotient modules

AbstractAlgebra allows the construction of quotient modules/spaces of AbstractAlgebra modules over euclidean domains. These are given as the quotient of a module by a submodule of that module.

We define two quotient modules to be equal if they are quotients of the same module $M$ by two equal submodules.

## 43.1 Generic quotient module type

AbstractAlgebra implements the generic quotient module type `Generic.QuotientModule{T}` where `T` is the element type of the base ring, in `src/generic/QuotientModule.jl`.

Elements of generic quotient modules have type `Generic.QuotientModuleElem{T}`.

## 43.2 Abstract types

Quotient module types belong to the `FPModule{T}` abstract type and their elements to `FPModuleElem{T}`.

## 43.3 Constructors

`AbstractAlgebra.quo` – Method.

```
quo(m::FPModule{T}, subm::FPModule{T}) where T <: RingElement
```

Return the quotient `M` of the module `m` by the module `subm` (which must have been (transitively) constructed as a submodule of `m` or be `m` itself) along with the canonical quotient map from `m` to `M`.

source

Note that a preimage of the canonical projection can be obtained using the preimage function described in the section on module homomorphisms. Note that a preimage element of the canonical projection is not unique and has no special properties.

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers
```

```
julia> m = M([ZZ(1), ZZ(2)])
(1, 2)

julia> N, f = sub(M, [m])
(Submodule over integers with 1 generator and no relations, Hom: N -> M)

julia> Q, g = quo(M, N)
(Quotient module over integers with 1 generator and no relations, Hom: M -> Q)

julia> p = M([ZZ(3), ZZ(1)])
(3, 1)

julia> v2 = g(p)
(-5)

julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> m = V([QQ(1), QQ(2)])
(1//1, 2//1)

julia> N, f = sub(V, [m])
(Subspace over rationals with 1 generator and no relations, Hom: N -> V)

julia> Q, g = quo(V, N)
(Quotient space over rationals with 1 generator and no relations, Hom: V -> Q)
```

## 43.4 Functionality for submodules

In addition to the Module interface, AbstractAlgebra submodules implement the following functionality.

### Basic manipulation

`AbstractAlgebra.Generic.supermodule` – Method.

```
supermodule(M::QuotientModule{T}) where T <: RingElement
```

Return the module that this module is a quotient of.

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers

julia> m = M([ZZ(2), ZZ(3)])
(2, 3)

julia> N, g = sub(M, [m])
(Submodule over integers with 1 generator and no relations, Hom: N -> M)
```

```
julia> Q, h = quo(M, N)
(Quotient module over integers with 2 generators and relations:
[2 3], Hom: M -> Q)

julia> supermodule(Q) == M
true
```

source

AbstractAlgebra.vector_space_dim – Method.

```
vector_space_dim(N::QuotientModule{T}) where T <: FieldElement
dim(N::QuotientModule{T}) where T <: FieldElement
rank(N::QuotientModule{T}) where T <: FieldElement
```

Return the dimension of the given vector quotient space.

**Examples**

```
julia> V = vector_space(QQ, 2)
Vector space of dimension 2 over rationals

julia> N, f = sub(V, [V([QQ(1), QQ(2)])])
(Subspace over rationals with 1 generator and no relations, Hom: N -> V)

julia> Q, g = quo(V, N)
(Quotient space over rationals with 1 generator and no relations, Hom: V -> Q)

julia> dim(Q)
1
```

source

# Chapter 44

# Direct Sums

AbstractAlgebra allows the construction of the external direct sum of any nonempty vector of finitely presented modules.

Note that external direct sums are considered equal iff they are the same object.

## 44.1 Generic direct sum type

AbstractAlgebra provides a generic direct sum type `Generic.DirectSumModule{T}` where `T` is the element type of the base ring. The implementation is in `src/generic/DirectSum.jl`

Elements of direct sum modules have type `Generic.DirectSumModuleElem{T}`.

## 44.2 Abstract types

Direct sum module types belong to the abstract type `FPModule{T}` and their elements to `FPModuleElem{T}`.

## 44.3 Constructors

`AbstractAlgebra.direct_sum` – Function.

```
direct_sum(m::Vector{<:FPModule{T}}) where T <: RingElement
direct_sum(vals::FPModule{T}...) where T <: RingElement
```

Return a tuple $M, f, g$ consisting of $M$ the direct sum of the modules `m` (supplied as a vector of modules), a vector $f$ of the injections of the $m[i]$ into $M$ and a vector $g$ of the projections from $M$ onto the $m[i]$.

**Examples**

```
julia> F = free_module(ZZ, 5)
Free module of rank 5 over integers

julia> m1 = F(BigInt[4, 7, 8, 2, 6])
(4, 7, 8, 2, 6)

julia> m2 = F(BigInt[9, 7, -2, 2, -4])
(9, 7, -2, 2, -4)
```

```
julia> S1, f1 = sub(F, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S1 -> F)

julia> m1 = F(BigInt[3, 1, 7, 7, -7])
(3, 1, 7, 7, -7)

julia> m2 = F(BigInt[-8, 6, 10, -1, 1])
(-8, 6, 10, -1, 1)

julia> S2, f2 = sub(F, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S2 -> F)

julia> m1 = F(BigInt[2, 4, 2, -3, -10])
(2, 4, 2, -3, -10)

julia> m2 = F(BigInt[5, 7, -6, 9, -5])
(5, 7, -6, 9, -5)

julia> S3, f3 = sub(F, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S3 -> F)

julia> D, f = direct_sum(S1, S2, S3)
(DirectSumModule over integers, AbstractAlgebra.Generic.ModuleHomomorphism{BigInt}[Hom: S1 -> D,
↪  Hom: S2 -> D, Hom: S3 -> D], AbstractAlgebra.Generic.ModuleHomomorphism{BigInt}[Hom: D ->
↪  S1, Hom: D -> S2, Hom: D -> S3])
```

source

## 44.4 Functionality for direct sums

In addition to the Module interface, AbstractAlgebra direct sums implement the following functionality.

### Basic manipulation

`AbstractAlgebra.Generic.summands` – Method.

```
summands(M::DirectSumModule{T}) where T <: RingElement
```

Return the modules that this module is a direct sum of.

**Examples**

```
julia> F = free_module(ZZ, 5)
Free module of rank 5 over integers

julia> m1 = F(BigInt[4, 7, 8, 2, 6])
(4, 7, 8, 2, 6)

julia> m2 = F(BigInt[9, 7, -2, 2, -4])
(9, 7, -2, 2, -4)

julia> S1, f1 = sub(F, [m1, m2])
```

```
(Submodule over integers with 2 generators and no relations, Hom: S1 -> F)

julia> m1 = F(BigInt[3, 1, 7, 7, -7])
(3, 1, 7, 7, -7)

julia> m2 = F(BigInt[-8, 6, 10, -1, 1])
(-8, 6, 10, -1, 1)

julia> S2, f2 = sub(F, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S2 -> F)

julia> m1 = F(BigInt[2, 4, 2, -3, -10])
(2, 4, 2, -3, -10)

julia> m2 = F(BigInt[5, 7, -6, 9, -5])
(5, 7, -6, 9, -5)

julia> S3, f3 = sub(F, [m1, m2])
(Submodule over integers with 2 generators and no relations, Hom: S3 -> F)

julia> D, f = direct_sum(S1, S2, S3)
(DirectSumModule over integers, AbstractAlgebra.Generic.ModuleHomomorphism{BigInt}[Hom: S1 -> D,
↪  Hom: S2 -> D, Hom: S3 -> D], AbstractAlgebra.Generic.ModuleHomomorphism{BigInt}[Hom: D ->
↪  S1, Hom: D -> S2, Hom: D -> S3])

julia> summands(D)
3-element Vector{AbstractAlgebra.Generic.Submodule{BigInt}}:
 Submodule over integers with 2 generators and no relations
 Submodule over integers with 2 generators and no relations
 Submodule over integers with 2 generators and no relations
```

source

```
(D::DirectSumModule{T}(::Vector{<:FPModuleElem{T}}) where T <: RingElement
```

Given a vector (or $1$-dim array) of module elements, where the $i$-th entry has to be an element of the $i$-summand of $D$, create the corresponding element in $D$.

**Examples**

```
julia> N = free_module(QQ, 1);

julia> M = free_module(QQ, 2);

julia> D, _ = direct_sum(M, N, M);

julia> D([gen(M, 1), gen(N, 1), gen(M, 2)])
(1//1, 0//1, 1//1, 0//1, 1//1)
```

### Special Homomorphisms

Due to the special structure as direct sums, homomorphisms can be created by specifying homomorphisms for all summands. In case of the codmain being a direct sum as well, any homomorphism may be thought of as a matrix containing maps from the $i$-th source summand to the $j$-th target module:

```
ModuleHomomorphism(D::DirectSumModule{T}, S::DirectSumModule{T}, m::Matrix{Any}) where T <:
↪  RingElement
```

Given a matrix $m$ such that the $(i, j)$-th entry is either $0$ (`Int(0)`) or a `ModuleHomomorphism` from the $i$-th summand of $D$ to the $j$-th summand of $S$, construct the corresponding homomorphism.

```
ModuleHomomorphism(D::DirectSumModule{T}, S::FPModuleElem{T}, m::Vector{ModuleHomomorphism})
```

Given an array $a$ of `ModuleHomomorphism` such that $a_i$, the $i$-th entry of $a$ is a `ModuleHomomorphism` from the $i$-th summand of `D` into `S`, construct the direct sum of the components.

Given a matrix $m$ such that the $(i, j)$-th entry is either $0$ (`Int(0)`) or a `ModuleHomomorphism` from the $i$-th summand of $D$ to the $j$-th summand of $S$, construct the corresponding homomorphism.

#### Examples

```
julia> N = free_module(QQ, 2);

julia> D, _ = direct_sum(N, N);

julia> p = ModuleHomomorphism(N, N, [3,4] .* basis(N));

julia> q = ModuleHomomorphism(N, N, [5,7] .* basis(N));

julia> phi = ModuleHomomorphism(D, D, [p 0; 0 q])
Module homomorphism
  from DirectSumModule over rationals
  to DirectSumModule over rationals

julia> r = ModuleHomomorphism(N, D, [2,3] .* gens(D)[1:2])
Module homomorphism
  from vector space of dimension 2 over rationals
  to DirectSumModule over rationals

julia> psi = ModuleHomomorphism(D, D, [r, r])
Module homomorphism
  from DirectSumModule over rationals
  to DirectSumModule over rationals
```

# Chapter 45

# Module Homomorphisms

Abstract Algebra provides homomorphisms of finitely presented modules.

## 45.1 Generic module homomorphism types

AbstractAlgebra defines two module homomorphism types, namely `Generic.ModuleHomomorphism` and `Generic.ModuleIsomorph`
Functionality for these is implemented in `src/generic/ModuleHomomorphism.jl`.

## 45.2 Abstract types

The `Generic.ModuleHomomorphism` and `Generic.ModuleIsomorphism` types inherit from `Map(FPModuleHomomorphism)`.

## 45.3 Generic functionality

The following generic functionality is provided for module homomorphisms.

### Constructors

Homomorphisms of AbstractAlgebra modules, $f : R^s \to R^t$, can be represented by $s \times t$ matrices over $R$.

`AbstractAlgebra.ModuleHomomorphism` – Method.

```
ModuleHomomorphism(M1::FPModule{T},
                   M2::FPModule{T}, m::MatElem{T}) where T <: RingElement
```

Create the homomorphism $f : M_1 \to M_2$ represented by the matrix $m$.

**Examples**

```
julia> M = free_module(ZZ, 2)
Free module of rank 2 over integers

julia> f = ModuleHomomorphism(M, M, matrix(ZZ, 2, 2, [1, 2, 3, 4]))
Module homomorphism
  from free module of rank 2 over integers
  to free module of rank 2 over integers
```

```
julia> m = M([ZZ(1), ZZ(2)])
(1, 2)

julia> f(m)
(7, 10)
```

source

AbstractAlgebra.ModuleIsomorphism – Method.

```
ModuleIsomorphism(M1::FPModule{T}, M2::FPModule{T}, A::MatElem{T},
                  minv::MatElem{T}) where T <: RingElement
```

Create the isomorphism $f : M_1 \to M_2$ represented by the matrix $A$. The inverse morphism is automatically computed.

source

They can also be created by giving images (in the codomain) of the generators of the domain:

```
ModuleHomomorphism(M1::FPModule{T}, M2::FPModule{T}, v::Vector{<:FPModuleElem{T}}) where T <:
↪  RingElement
```

### Kernels

AbstractAlgebra.kernel – Method.

```
kernel(f::ModuleHomomorphism{T}) where T <: RingElement
```

Return a pair K, g consisting of the kernel object $K$ of the given module homomorphism $f$ (as a submodule of its domain) and the canonical injection from the kernel into the domain of $f$.

**Examples**

```
julia> M = free_module(ZZ, 3)
Free module of rank 3 over integers

julia> m = M([ZZ(1), ZZ(2), ZZ(3)])
(1, 2, 3)

julia> S, f = sub(M, [m])
(Submodule over integers with 1 generator and no relations, Hom: S -> M)

julia> Q, g = quo(M, S)
(Quotient module over integers with 2 generators and no relations, Hom: M -> Q)

julia> kernel(g)
(Submodule over integers with 1 generator and no relations, Hom: submodule over integers with 1
↪  generator and no relations -> M)
```

source

### Images

`AbstractAlgebra.image` – Method.

```
image(f::Map(FPModuleHomomorphism))
```

Return a pair `I, g` consisting of the image object $I$ of the given module homomorphism $f$ (as a submodule of its codomain) and the canonical injection from the image into the codomain of $f$

source

```
M = free_module(ZZ, 3)

m = M([ZZ(1), ZZ(2), ZZ(3)])

S, f = sub(M, [m])
Q, g = quo(M, S)
K, k = kernel(g)

image(compose(k, g))
```

### Preimages

`AbstractAlgebra.preimage` – Method.

```
preimage(f::Map(FPModuleHomomorphism),
         v::FPModuleElem{T}) where T <: RingElement
```

Return a preimage of $v$ under the homomorphism $f$, i.e. an element of the domain of $f$ that maps to $v$ under $f$. Note that this has no special mathematical properties. It is an element of the set theoretical preimage of the map $f$ as a map of sets, if one exists. The preimage is neither unique nor chosen in a canonical way in general. When no such element exists, an exception is raised.

source

`AbstractAlgebra.has_preimage_with_preimage` – Method.

```
has_preimage_with_preimage(f::Map(FPModuleHomomorphism),
  v::FPModuleElem{T}) where T <: RingElement
```

Check if $v$ has a preimage under the homomorphism $f$. If it does, return a tuple `(true, y)` for `y` in `domain(f)` such that $f(y) = x$ holds, otherwise, return `(false, z)` where `z` is any element of type `elem_type(domain(f))`.

source

```
M = free_module(ZZ, 3)

m = M([ZZ(1), ZZ(2), ZZ(3)])

S, f = sub(M, [m])
Q, g = quo(M, S)

m = rand(M, -10:10)
n = g(m)

p = preimage(g, n)
```

### Inverses

Module isomorphisms can be cheaply inverted.

Base.inv – Method.

```
inv(a::LocalizedEuclideanRingElem{T}, checked::Bool = true)  where {T <: RingElem}
```

Returns the inverse element of $a$ if $a$ is a unit. If 'checked = false' the invertibility of $a$ is not checked and the corresponding inverse element of the Fraction Field is returned.

source

```
M = free_module(ZZ, 2)
N = matrix(ZZ, 2, 2, BigInt[1, 0, 0, 1])
f = ModuleIsomorphism(M, M, N)

g = inv(f)
```

# Part VIII

# Ideals

# Chapter 46

# Ideal functionality

AbstractAlgebra.jl provides a module, implemented in `src/generic/Ideal.jl` for ideals of a Euclidean domain (assuming the existence of a gcdx function) or of a univariate or multivariate polynomial ring over the integers. Univariate and multivariate polynomial rings over other domains (other than fields) are not supported at this time.

**Info**

A more complete implementation for ideals defined over other rings is provided by Hecke and Oscar.

## 46.1 Generic ideal types

AbstractAlgebra.jl provides a generic ideal type based on Julia arrays which is implemented in `src/generic/Ideal.jl`.

These generic ideals have type `Generic.Ideal{T}` where T is the type of elements of the ring the ideals belong to. Internally they consist of a Julia array of generators and some additional fields for a parent object, etc. See the file `src/generic/GenericTypes.jl` for details.

Parent objects of ideals have type `Generic.IdealSet{T}`.

## 46.2 Abstract types

All ideal types belong to the abstract type `Ideal{T}` and their parents belong to the abstract type `Set`. This enables one to write generic functions that can accept any AbstractAlgebra ideal type.

**Note**

Both the generic ideal type `Generic.Ideal{T}` and the abstract type it belongs to, `Ideal{T}`, are called `Ideal`. The former is a (parameterised) concrete type for an ideal in the ring whose elements have type T. The latter is an abstract type representing all ideal types in AbstractAlgebra.jl, whether generic or very specialised (e.g. supplied by a C library).

## 46.3 Ideal constructors

One may construct ideals in AbstractAlgebra.jl with the following constructor.

```
Generic.Ideal(R::Ring, V::Vector{T}) where T <: RingElement
```

Given a set of elements V in the ring R, construct the ideal of R generated by the elements V. Note that V may be arbitrary, e.g. it can contain duplicates, zero entries or be empty.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y]; internal_ordering=:degrevlex)
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> V = [3*x^2*y - 3*y^2, 9*x^2*y + 7*x*y]
2-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 3*x^2*y - 3*y^2
 9*x^2*y + 7*x*y

julia> I = Generic.Ideal(R, V)
AbstractAlgebra.Generic.Ideal{AbstractAlgebra.Generic.MPoly{BigInt}}(Multivariate polynomial ring
↪  in 2 variables over integers, AbstractAlgebra.Generic.MPoly{BigInt}[7*x*y + 9*y^2, 243*y^3 -
↪  147*y^2, x*y^2 + 36*y^3 - 21*y^2, x^2*y + 162*y^3 - 99*y^2])

julia> W = map(ZZ, [2, 5, 7])
3-element Vector{BigInt}:
 2
 5
 7

julia> J = Generic.Ideal(ZZ, W)
AbstractAlgebra.Generic.Ideal{BigInt}(Integers, BigInt[1])
```

## 46.4 Ideal functions

### Basic functionality

`AbstractAlgebra.gens` – Method.

```
gens(I::Ideal{T}) where T <: RingElement
```

Return a list of generators of the ideal I in reduced form and canonicalised.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> V = [1 + 2x^2 + 3x^3, 5x^4 + 1, 2x - 1]
3-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 3*x^3 + 2*x^2 + 1
 5*x^4 + 1
 2*x - 1

julia> I = Generic.Ideal(R, V)
AbstractAlgebra.Generic.Ideal{AbstractAlgebra.Generic.Poly{BigInt}}(Univariate polynomial ring
↪  in x over integers, AbstractAlgebra.Generic.Poly{BigInt}[3, x + 1])
```

```
julia> gens(I)
2-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 3
 x + 1
```

source

### Arithmetic of Ideals

Ideals support addition, multiplication, scalar multiplication and equality testing of ideals.

### Containment

`AbstractAlgebra.is_subset` – Method.

```
Base.issubset(I::T, J::T) where {T <: Ideal}
```

Return `true` if the ideal `I` is a subset of the ideal `J`. An exception is thrown if the ideals are not defined over the same base ring.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, :x)
(Univariate polynomial ring in x over integers, x)

julia> V = [1 + 2x^2 + 3x^3, 5x^4 + 1, 2x - 1]
3-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 3*x^3 + 2*x^2 + 1
 5*x^4 + 1
 2*x - 1

julia> W = [1 + 2x^2 + 3x^3, 5x^4 + 1]
2-element Vector{AbstractAlgebra.Generic.Poly{BigInt}}:
 3*x^3 + 2*x^2 + 1
 5*x^4 + 1

julia> I = Generic.Ideal(R, V)
AbstractAlgebra.Generic.Ideal{AbstractAlgebra.Generic.Poly{BigInt}}(Univariate polynomial ring
↪  in x over integers, AbstractAlgebra.Generic.Poly{BigInt}[3, x + 1])

julia> J = Generic.Ideal(R, W)
AbstractAlgebra.Generic.Ideal{AbstractAlgebra.Generic.Poly{BigInt}}(Univariate polynomial ring
↪  in x over integers, AbstractAlgebra.Generic.Poly{BigInt}[282, 3*x + 255, x^2 + 107])

julia> is_subset(I, J)
false

julia> is_subset(J, I)
true

julia> intersect(I, J) == J
true
```

source

Base.intersect – Method.

```
intersect(I::Ideal{T}, J::Ideal{T}) where T <: RingElement
```

Return the intersection of the ideals I and J.

source

### Normal form

For ideal of polynomial rings it is possible to return the normal form of a polynomial with respect to an ideal.

AbstractAlgebra.Generic.normal_form – Method.

```
normal_form(p::U, I::Ideal{U}) where {U}
```

Return the normal form of the polynomial p with respect to the ideal I.

**Examples**

```
julia> R, (x, y) = polynomial_ring(ZZ, [:x, :y]; internal_ordering=:degrevlex)
(Multivariate polynomial ring in 2 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y])

julia> V = [3*x^2*y - 3*y^2, 9*x^2*y + 7*x*y]
2-element Vector{AbstractAlgebra.Generic.MPoly{BigInt}}:
 3*x^2*y - 3*y^2
 9*x^2*y + 7*x*y

julia> I = Generic.Ideal(R, V)
AbstractAlgebra.Generic.Ideal{AbstractAlgebra.Generic.MPoly{BigInt}}(Multivariate polynomial
↪  ring in 2 variables over integers, AbstractAlgebra.Generic.MPoly{BigInt}[7*x*y + 9*y^2,
↪  243*y^3 - 147*y^2, x*y^2 + 36*y^3 - 21*y^2, x^2*y + 162*y^3 - 99*y^2])


julia> normal_form(30x^5*y + 2x + 1, I)
135*y^4 + 138*y^3 - 147*y^2 + 2*x + 1
```

source

# Part IX

# Matrices

# Chapter 47

# Introduction

AbstractAlgebra provides exact linear algebra over a wide range of rings, including the integers, finite fields, polynomial rings and residue rings. Unlike Julia's standard linear algebra functionality, matrices are not restricted to numerical coefficient domains.

Matrices in AbstractAlgebra belong to parent objects. Rectangular $m \times n$ matrices are elements of matrix spaces, while square $n \times n$ matrices may additionally be constructed as elements of matrix algebras, where matrix multiplication gives them the structure of a ring.

The matrix functionality includes standard arithmetic, linear solving, elementary row and column operations, determinants, inverses, kernels, characteristic and minimal polynomials, and various decompositions and canonical forms. Additional specialised algorithms are available over suitable base rings, such as Hermite and Smith normal forms over Euclidean domains and Popov forms over polynomial rings.

The following sections describe how to construct matrices, access and modify their entries, compute with them, and use the parent objects associated to matrices.

# Chapter 48

# Working with matrices

## 48.1 Constructing Matrices

There are several functions for creating matrices – in each case you need to indicate the ring to which the matrix elements belong, unless this is already unambiguously indicated by the ring to which the arguments belong. Here we present several of the basic functions for constructing matrices.

### Why Julia matrices are not sufficient

Unfortunately, Julia's matrices cannot be used in our context due to two independent problems:

- In empty matrices (0 rows or columns) all that is known about a Julia matrix is the type of its entries,

however for advanced algebraic types, this information is insufficient to create elements, hence `zero(T)` or friends cannot work.

- Many Julia functions (e.g. `det`) assume that all types used embed into the real or complex numbers. For

instance, in Julia `det(ones(Int, (1,1))) == 1.0`, so the fact that the determinant is exactly the integer 1 is lost. Furthermore, more general rings cannot be embedded into the reals at all.

For this reason, the matrix constructors below must accept (or infer) the ring in which all entries of the matrix reside.

### Constructing matrices from entries

`AbstractAlgebra.matrix` – Method.

```
matrix(R::NCRing, entries::AbstractMatrix{T}) where {T}
```

Return the matrix over the ring `R` with entries as in the Julia `AbstractMatrix` `entries`. All entries of `entries` must be coercible into `R`.

**Examples**

```
julia> matrix(GF(3), [1 2 ; 3 4])
[1   2]
[0   1]

julia> matrix(ZZ, BigInt[3 1 2; 2 0 1])
[3   1   2]
[2   0   1]
```

source

AbstractAlgebra.matrix – Method.

```
matrix(R::NCRing, r::Int, c::Int, entries::AbstractVecOrMat{T}) where {T}
```

Return the r by c matrix over the ring R from entries.

If entries is a vector, its entries are read row-wise, so the $(i, j)$ entry is given by entries[c*(i - 1) + j]. All entries must be coercible into R.

If entries is a matrix, this is equivalent to matrix(R, entries).

**Examples**

```
julia> matrix(ZZ, 3, 2, BigInt[3, 1, 2, 2, 0, 1])
[3   1]
[2   2]
[0   1]
```

source

Several other signatures are supported:

```
matrix(entries::AbstractMatrix{T}) where {T<:NCRingElement}
matrix(entries::AbstractVector{T}) where {T<:NCRingElement}
matrix(entries::AbstractVector{<:AbstractVector{T}}) where {T<:NCRingElement}
```

In each signature, entries supplies the matrix entries. Nested vectors can also be used to construct matrices over a specified base ring:

```
matrix(R::NCRing, entries::AbstractVector{<:AbstractVector})
```

### Constructing matrices from existing matrices

#### Changing the base ring

Matrices can be converted to another base ring using change_base_ring.

AbstractAlgebra.change_base_ring – Method.

```
change_base_ring(R::NCRing, A::MatrixElem{T}) where {T <: NCRingElement}
```

Return a new matrix over R by coercing each entry of A into R.

The input matrix is not modified.

**Examples**

```
julia> M = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> N = change_base_ring(QQ, M)
[1//1   2//1]
[3//1   4//1]

julia> base_ring(N)
Rationals
```

source

The same conversion of A can also be performed using the following constructor:

```
matrix(R::NCRing, A::MatElem)
```

#### Copying

An independent copy of an existing matrix can be created.

```
matrix(A::MatElem{T}) where {T<:NCRingElement}
```

#### Zero matrices

`Base.zero` – Method.

```
zero(A::MatElem{T}, R::NCRing, r::Int, c::Int) where {T <: NCRingElement}
zero(A::MatElem{T}, r::Int, c::Int) where {T <: NCRingElement}
zero(A::MatElem{T}, R::NCRing) where T <: {NCRingElement}
zero(A::MatElem{T}) where {T <: NCRingElement}
```

Create a zero matrix with the same implementation type as the given matrix A.

By default, the base ring and dimensions are inherited from A, but they can also be specified explicitly.

source

#### Identity matrices

`AbstractAlgebra.identity_matrix` – Method.

```
identity_matrix(A::MatElem{T}) where {T <: NCRingElement}
```

Return the identity matrix with the same base ring and dimensions as the given abstract matrix A. The matrix A must be square.

This is an alias for one(A).

**Examples**

```
julia> M = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> identity_matrix(M)
[1   0]
[0   1]
```

source

AbstractAlgebra.identity_matrix – Method.

```
identity_matrix(A::MatElem{T}, n::Int) where {T <: NCRingElement}
```

Return the identity $n \times n$ matrix over the same base ring as the given abstract matrix A.

source

Base.one – Method.

```
one(A::MatElem{T}) where {T <: NCRingElement}
```

Return the identity matrix with the same base ring and dimensions as A.

The matrix A must be square.

source

### Uninitialized matrices

Base.similar – Method.

```
similar(A::MatElem{T}, R::NCRing, r::Int, c::Int) where T <: NCRingElement
similar(A::MatElem{T}, R::NCRing) where T <: NCRingElement
similar(A::MatElem{T}, r::Int, c::Int) where T <: NCRingElement
similar(A::MatElem{T}) where T <: NCRingElement
```

Create an uninitialized matrix with the same implementation type as A.

By default, the base ring and dimensions are inherited from A, but they can also be specified explicitly.

This method is useful when implementing algorithms which create new matrices whose entries will be filled in later.

Despite the name, `similar` is not related to similarity transformations or similar matrices in the mathematical sense.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 4 5 6])
[1   2   3]
[4   5   6]

julia> similar(M)
[#undef   #undef   #undef]
[#undef   #undef   #undef]

julia> similar(M, 2, 2)
[#undef   #undef]
[#undef   #undef]
```

source

**Concatenation**

Matrices can be constructed from existing matrices by concatenating them horizontally or vertically.

`Base.hcat` – Method.

```
Base.hcat(A::MatElem...)
```

Return the horizontal concatenation of the matrices in $A$.

All component matrices must have the same base ring and the same number of rows.

**Examples**

```
julia> M = matrix(ZZ, BigInt[1 2 3; 2 3 4; 3 4 5])
[1   2   3]
[2   3   4]
[3   4   5]

julia> N = matrix(ZZ, BigInt[1 0 1; 0 1 0; 1 0 1])
[1   0   1]
[0   1   0]
[1   0   1]

julia> hcat(M, N)
[1   2   3   1   0   1]
[2   3   4   0   1   0]
[3   4   5   1   0   1]
```

source

`Base.vcat` – Method.

```
Base.vcat(A::MatElem...)
```

Return the vertical concatenation of the matrices in $A$.

All component matrices must have the same base ring and the same number of columns.

**Examples**

```
julia> M = matrix(ZZ, BigInt[1 2 3; 2 3 4; 3 4 5])
[1   2   3]
[2   3   4]
[3   4   5]

julia> N = matrix(ZZ, BigInt[1 0 1; 0 1 0; 1 0 1])
[1   0   1]
[0   1   0]
[1   0   1]

julia> vcat(M, N)
[1   2   3]
[2   3   4]
[3   4   5]
[1   0   1]
[0   1   0]
[1   0   1]
```

source

### Submatrices and views

Submatrices can be constructed using Julia indexing syntax. By default, this creates a new matrix containing the selected entries.

To avoid copying entries, one can instead construct a view using Julia's `@view` syntax. Views are particularly useful for working with small regions of large matrices, as modifications to a view also modify the original matrix.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 2 3 4; 3 4 5])
[1   2   3]
[2   3   4]
[3   4   5]

julia> M[1:2, 1:2]
[1   2]
[2   3]

julia> N = @view M[1:2, 1:2]
[1   2]
[2   3]

julia> N[1, 1] = 10
10
```

```
julia> M
[10   2   3]
[ 2   3   4]
[ 3   4   5]
```

### Special constructors

#### The zero matrix

AbstractAlgebra.zero_matrix – Method.

```
zero_matrix(R::NCRing, r::Int, c::Int)
```

Return the $r \times c$ matrix over the ring R whose entries are all zero.

**Examples**

```
julia> P = zero_matrix(ZZ, 3, 2)
[0   0]
[0   0]
[0   0]
```

source

#### The ones matrix

AbstractAlgebra.ones_matrix – Method.

```
ones_matrix(R::NCRing, r::Int, c::Int)
```

Return the $r \times c$ matrix over the ring R whose entries are all equal to the multiplicative identity of R.

**Examples**

```
julia> ones_matrix(ZZ, 3, 2)
[1   1]
[1   1]
[1   1]
```

source

#### The identity matrix

AbstractAlgebra.identity_matrix – Method.

```
identity_matrix(R::NCRing, n::Int)
```

Return the $n \times n$ identity matrix over the ring R.

**Examples**

```
julia> identity_matrix(ZZ, 2)
[1   0]
[0   1]
```

source

**Scalar matrices**

`AbstractAlgebra.scalar_matrix` – Method.

```
scalar_matrix(R::NCRing, n::Int, a::NCRingElement)
```

Return the $n \times n$ diagonal matrix over the ring R whose diagonal entries are all equal to the ring element a of R.

**Examples**

```
julia> scalar_matrix(QQ, 3, 1//2)
[1//2   0//1   0//1]
[0//1   1//2   0//1]
[0//1   0//1   1//2]
```

source

`AbstractAlgebra.scalar_matrix` – Method.

```
scalar_matrix(n::Int, a::NCRingElement)
```

Return the $n \times n$ diagonal matrix over `parent(a)` whose diagonal entries are all equal to the ring element a.

**Examples**

```
julia> scalar_matrix(3, ZZ(5))
[5   0   0]
[0   5   0]
[0   0   5]
```

source

### Diagonal matrices

`AbstractAlgebra.diagonal_matrix` – Function.

```
diagonal_matrix(x::NCRingElement, m::Int, n::Int = m)
```

Return the $m \times n$ matrix over the ring `parent(x)` with x along the main diagonal and zeros elsewhere. If n is omitted, return an $m \times m$ matrix.

**Examples**

```
julia> diagonal_matrix(ZZ(2), 2, 3)
[2   0   0]
[0   2   0]

julia> diagonal_matrix(QQ(-1), 3)
[-1//1    0//1    0//1]
[ 0//1   -1//1    0//1]
[ 0//1    0//1   -1//1]
```

source

`AbstractAlgebra.diagonal_matrix` – Method.

```
diagonal_matrix(R::NCRing, entries::AbstractVector{<:NCRingElement})
diagonal_matrix(entries::AbstractVector{<:NCRingElement})
diagonal_matrix(x::T, xs::T...) where {T<:NCRingElement}
```

Return a diagonal matrix with the given entries on the main diagonal.

For the vector forms, the diagonal entries are the elements of `entries`. For the vararg form, the diagonal entries are `x, xs...`.

If the ring R is given, the entries are coerced into R. Otherwise, the base ring is inferred from the entries.

**Examples**

```
julia> diagonal_matrix(ZZ(1), ZZ(2))
[1   0]
[0   2]

julia> diagonal_matrix(ZZ, [5, 6])
[5   0]
[0   6]

julia> diagonal_matrix([ZZ(3), ZZ(4)])
[3   0]
[0   4]
```

source

Diagonal matrices can also be constructed from matrix blocks. This is an alias for `block_diagonal_matrix`.

`AbstractAlgebra.diagonal_matrix` – Method.

```
diagonal_matrix(V::Vector{T}) where {T <: MatElem}
diagonal_matrix(R::NCRing, V::Vector{<:MatElem})
diagonal_matrix(x::T, xs::T...) where {T <: MatElem}
```

Return the block diagonal matrix whose diagonal blocks are the given matrices.

For the vector forms, the diagonal blocks are the elements of `V`. For the vararg form, the diagonal blocks are `x, xs...`.

If the ring `R` is given, the entries of the blocks are coerced into `R`.

These constructors use the corresponding `block_diagonal_matrix` functionality.

**Examples**

```
julia> A = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> B = matrix(ZZ, [5 6])
[5   6]

julia> diagonal_matrix([A, B])
[1   2   0   0]
[3   4   0   0]
[0   0   5   6]

julia> diagonal_matrix(QQ, [A, B])
[1//1   2//1   0//1   0//1]
[3//1   4//1   0//1   0//1]
[0//1   0//1   5//1   6//1]

julia> diagonal_matrix(B, A)
[5   6   0   0]
[0   0   1   2]
[0   0   3   4]
```

source

**Block diagonal matrix constructors**

`AbstractAlgebra.block_diagonal_matrix` – Method.

```
block_diagonal_matrix(V::Vector{<:MatElem{T}}) where {T <: NCRingElement}
```

Return the block diagonal matrix whose diagonal blocks are the matrices in `V`.

The vector `V` must be non-empty, since otherwise the base ring cannot be inferred.

**Examples**

```
julia> M = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> N = matrix(ZZ, [4 5 6; 7 8 9])
[4   5   6]
[7   8   9]

julia> block_diagonal_matrix([M, N])
[1   2   0   0   0]
[3   4   0   0   0]
[0   0   4   5   6]
[0   0   7   8   9]
```

source

AbstractAlgebra.block_diagonal_matrix – Method.

```
block_diagonal_matrix(R::NCRing, V::Vector{<:Matrix{T}}) where {T <: NCRingElement}
```

Return the block diagonal matrix over the ring R whose diagonal blocks are the matrices in V.

The entries of the blocks are coerced into R. If V is empty, the $0 \times 0$ zero matrix over R is returned.

**Examples**

```
julia> block_diagonal_matrix(ZZ, [[1 2; 3 4], [4 5 6; 7 8 9]])
[1   2   0   0   0]
[3   4   0   0   0]
[0   0   4   5   6]
[0   0   7   8   9]
```

source

### Triangular matrices

AbstractAlgebra.lower_triangular_matrix – Method.

```
lower_triangular_matrix(L::AbstractVector{T}) where {T <: NCRingElement}
```

Return the $n \times n$ lower triangular matrix whose entries on and below the main diagonal are given by the elements of L. The entries are filled row by row.

The size $n$ is determined by the condition that L has length $n(n+1)/2$. An exception is thrown if no such integer $n$ exists.

**Examples**

```
julia> lower_triangular_matrix([1, 2, 3])
[1   0]
[2   3]
```

source

AbstractAlgebra.upper_triangular_matrix – Method.

```
upper_triangular_matrix(L::AbstractVector{T}) where {T <: NCRingElement}
```

Return the $n \times n$ upper triangular matrix whose entries on and below the main diagonal are given by the elements of L. The entries are filled row by row.

The size $n$ is determined by the condition that L has length $n(n+1)/2$. An exception is thrown if no such integer $n$ exists.

**Examples**

```
julia> upper_triangular_matrix([1, 2, 3])
[1   2]
[0   3]
```

source

AbstractAlgebra.strictly_lower_triangular_matrix – Method.

```
strictly_lower_triangular_matrix(L::AbstractVector{T}) where {T <: NCRingElement}
```

Return the $n \times n$ strictly lower triangular matrix whose entries below the main diagonal are given by the elements of L. The entries are filled row by row.

The size $n$ is determined by the condition that L has length $n(n-1)/2$. An exception is thrown if no such integer $n$ exists.

**Examples**

```
julia> strictly_lower_triangular_matrix([1, 2, 3])
[0   0   0]
[1   0   0]
[2   3   0]
```

source

AbstractAlgebra.strictly_upper_triangular_matrix – Method.

```
strictly_upper_triangular_matrix(L::AbstractVector{T}) where {T <: NCRingElement}
```

Return the $n \times n$ strictly upper triangular matrix whose entries above the main diagonal are given by the elements of L. The entries are filled row by row.

The size $n$ is determined by the condition that L has length $n(n-1)/2$. An exception is thrown if no such integer $n$ exists.

**Examples**

```
julia> strictly_upper_triangular_matrix([1, 2, 3])
[0   1   2]
[0   0   3]
[0   0   0]
```

source

## 48.2 Manipulating matrices

This page describes functions for modifying or rearranging matrices. Many operations are available both as in-place and non-mutating variants.

### Entry-wise operations

`AbstractAlgebra.map_entries` – Method.

```
map_entries(f, A::MatElem{T}) where T <: NCRingElement
```

Return a new matrix obtained by applying `f` to each entry of the matrix `A`.

**Examples**

```
julia> M = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> M2 = map_entries(x -> x^2, M)
[1    4]
[9   16]
```

source

`AbstractAlgebra.map_entries!` – Method.

```
map_entries!(f, dst::MatElem{T}, src::MatElem{U}) where {T <: NCRingElement, U <: NCRingElement}
```

Apply `f` to each entry of `src`, store the result in the given matrix `dst` and return the modified matrix `dst`.

**Examples**

```
julia> M = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> N = zero_matrix(ZZ, 2, 2)
[0   0]
[0   0]

julia> map_entries!(x -> x^2, N, M)
```

```
[1    4]
[9   16]

julia> N
[1    4]
[9   16]
```

source

Base.map – Method.

```
map(f, A::MatrixElem{T}) where T <: NCRingElement
```

Return a new matrix obtained by applying f to each entry of the matrix A.

This is equivalent to map_entries(f, A), see map_entries.

source

Base.map! – Method.

```
map!(f, dst::MatrixElem{T}, src::MatrixElem{U}) where {T <: NCRingElement, U <: NCRingElement}
```

Apply f to each entry of src, store the result in the given matrix dst and return the modified matrix dst.

This is equivalent to map_entries!(f, dst, src), see map_entries!.

source

### Elementary row and column operations

AbstractAlgebra.add_column – Method.

```
add_column(A::MatrixElem{T}, s::RingElement, i::Int, j::Int, rows = 1:nrows(A)) where T <:
↪  RingElement
```

Return a new matrix obtained from A by adding s times the i-th column to the j-th column.

By default, this operation changes all entries of the j-th column in the returned matrix. An optional final argument restricts the operation to entries in the specified rows.

**Examples**

```
julia> M = ZZ[1 2 3; 2 3 4; 4 5 5]
[1   2   3]
[2   3   4]
[4   5   5]

julia> add_column(M, 2, 3, 1)
[ 7   2   3]
```

```
[10   3   4]
[14   5   5]

julia> M
[1   2   3]
[2   3   4]
[4   5   5]

julia> add_column(M, 2, 3, 1, 1:1)
[7   2   3]
[2   3   4]
[4   5   5]
```

source

AbstractAlgebra.add_column! – Method.

```
add_column!(A::MatrixElem{T}, s::RingElement, i::Int, j::Int, rows = 1:nrows(A)) where T <:
↪  RingElement
```

Add s times the i-th column to the j-th column of A and return the modified matrix A.

By default, this operation modifies all entries of the j-th column. An optional final argument restricts the operation to entries in the specified rows.

**Examples**

```
julia> M = ZZ[1 2 3; 2 3 4; 4 5 5]
[1   2   3]
[2   3   4]
[4   5   5]

julia> add_column!(M, 2, 3, 1)
[ 7   2   3]
[10   3   4]
[14   5   5]

julia> M
[ 7   2   3]
[10   3   4]
[14   5   5]

julia> add_column!(M, 2, 3, 1, 1:1)
[13   2   3]
[10   3   4]
[14   5   5]
```

source

AbstractAlgebra.add_row – Method.

```
add_row(A::MatrixElem{T}, s::RingElement, i::Int, j::Int, cols = 1:ncols(A)) where T <:
↪  RingElement
```

Return a new matrix obtained from A by adding s times the i-th row to the j-th row.

By default, this operation changes all entries of the j-th row in the returned matrix. An optional final argument restricts the operation to entries in the specified columns.

source

AbstractAlgebra.add_row! – Method.

```
add_row!(A::MatrixElem{T}, s::RingElement, i::Int, j::Int, cols = 1:ncols(A)) where T <:
↪  RingElement
```

Add s times the i-th row to the j-th row of A and return the modified matrix A.

By default, this operation modifies all entries of the j-th row. An optional final argument restricts the operation to entries in the specified columns.

source

AbstractAlgebra.multiply_column – Method.

```
multiply_column(A::MatrixElem{T}, s::RingElement, i::Int, rows = 1:nrows(A)) where T <:
↪  RingElement
```

Return a new matrix obtained from A by multiplying the i-th column by s.

By default, this operation changes all entries of the i-th column in the returned matrix. An optional final argument restricts the operation to entries in the specified rows.

source

AbstractAlgebra.multiply_column! – Method.

```
multiply_column!(A::MatrixElem{T}, s::RingElement, i::Int, rows = 1:nrows(A)) where T <:
↪  RingElement
```

Multiply the i-th column of A by s and return the modified matrix A.

By default, this operation modifies all entries of the i-th column. An optional final argument restricts the operation to entries in the specified rows.

source

AbstractAlgebra.multiply_row – Method.

```
multiply_row(A::MatrixElem{T}, s::RingElement, i::Int, cols = 1:ncols(A)) where T <: RingElement
```

Return a new matrix obtained from A by multiplying the i-th row by s.

By default, this operation changes all entries of the i-th row in the returned matrix. An optional final argument restricts the operation to entries in the specified columns.

**Examples**

```
julia> M = ZZ[1 2 3; 2 3 4; 4 5 5]
[1   2   3]
[2   3   4]
[4   5   5]

julia> multiply_row(M, 2, 3)
[1    2    3]
[2    3    4]
[8   10   10]

julia> M
[1   2   3]
[2   3   4]
[4   5   5]

julia> multiply_row(M, 2, 3, 2:2)
[1    2   3]
[2    3   4]
[4   10   5]
```

source

AbstractAlgebra.multiply_row! – Method.

```
multiply_row!(A::MatrixElem{T}, s::RingElement, i::Int, cols = 1:ncols(A)) where T <:
↪  RingElement
```

Multiply the i-th row of A by s and return the modified matrix A.

By default, this operation modifies all entries of the i-th row. An optional final argument restricts the operation to entries in the specified columns.

source

### Row and column permutations

Base.:* – Method.

```
*(P::Perm, A::MatrixElem{T}) where T <: NCRingElement
```

Return a new matrix obtained by applying the permutation P to the rows of A.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> G = SymmetricGroup(3)
Full symmetric group over 3 elements

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> P = G([1, 3, 2])
(2,3)

julia> P*A
[t + 1       t             1]
[   -2   t + 2   t^2 + t + 1]
[  t^2       t             t]
```

source

Base.:* – Method.

```
*(A::MatrixElem{T}, P::Perm) where T <: NCRingElement
```

Return a new matrix obtained by applying the permutation P to the columns of A.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> G = SymmetricGroup(3)
Full symmetric group over 3 elements

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> P = G([1, 3, 2])
(2,3)

julia> A*P
[t + 1             1       t]
```

```
[  t^2            t       t]
[   -2   t^2 + t + 1   t + 2]
```

source

AbstractAlgebra.swap_rows – Method.

```
swap_rows(A::MatElem{T}, i::Int, j::Int) where T <: NCRingElement
```

Return a new matrix obtained from A by swapping the i-th and j-th rows.

The original matrix A remains unchanged.

**Examples**

```
julia> M = identity_matrix(ZZ, 3)
[1   0   0]
[0   1   0]
[0   0   1]

julia> swap_rows(M, 1, 2)
[0   1   0]
[1   0   0]
[0   0   1]

julia> M
[1   0   0]
[0   1   0]
[0   0   1]
```

source

AbstractAlgebra.swap_rows! – Method.

```
swap_rows!(A::MatElem{T}, i::Int, j::Int) where T <: NCRingElement
```

Swap the i-th and j-th rows of A in place and return the modified matrix A.

No bounds checking is performed; the indices i and j must be in range.

**Examples**

```
julia> M = identity_matrix(ZZ, 3)
[1   0   0]
[0   1   0]
[0   0   1]

julia> swap_rows!(M, 1, 2)
[0   1   0]
[1   0   0]
[0   0   1]
```

```
julia> M
[0   1   0]
[1   0   0]
[0   0   1]
```

source

AbstractAlgebra.swap_cols – Method.

```
swap_cols(A::MatElem{T}, i::Int, j::Int) where T <: NCRingElement
```

Return a new matrix obtained from A by swapping the i-th and j-th columns.

source

AbstractAlgebra.swap_cols! – Method.

```
swap_cols!(A::MatElem{T}, i::Int, j::Int) where T <: NCRingElement
```

Swap the i-th and j-th columns of A in place and return the modified matrix A.

No bounds checking is performed; the indices i and j must be in range.

source

AbstractAlgebra.reverse_rows – Method.

```
reverse_rows(A::MatElem{T}) where T <: NCRingElement
```

Return a new matrix obtained from A by reversing the order of its rows.

source

AbstractAlgebra.reverse_rows! – Method.

```
reverse_rows!(A::MatElem{T}) where T <: NCRingElement
```

Reverse the order of the rows of A in place and return the modified matrix A.

source

AbstractAlgebra.reverse_cols – Method.

```
reverse_cols(A::MatElem{T}) where T <: NCRingElement
```

Return a new matrix obtained from A by reversing the order of its columns.

source

`AbstractAlgebra.reverse_cols!` – Method.

```
reverse_cols!(A::MatElem{T}) where T <: NCRingElement
```

Reverse the order of the columns of `A` in place and return the modified matrix `A`.

source

### Transposition

Matrices can be transposed either by creating a new matrix or by modifying an existing square matrix in place.

`Base.transpose` – Method.

```
transpose(A::MatElem)
```

Return a new matrix containing the transpose of `A`.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> A = matrix(R, [t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> transpose(A)
[t + 1   t^2            -2]
[    t     t         t + 2]
[    1     t   t^2 + t + 1]
```

source

`LinearAlgebra.transpose!` – Method.

```
transpose!(A::MatElem)
transpose!(N::T, A::T) where T <: MatElem
```

Return the transpose of `A`, storing the result in a pre-existing matrix.

The unary version stores the result in `A` itself and requires `A` to be square; an error is raised otherwise.

The binary version stores the transpose of `A` in `N` and returns `N`. The matrix `N` must have size `ncols(A)` by `nrows(A)`. No dimension checks are performed, and incorrect dimensions may result in undefined behaviour.

source

## 48.3 Matrix predicates

This page collects predicates, i.e. functions returning `true` or `false`, for testing whether matrices satisfy certain structural or normal-form conditions.

### Basic predicates

Matrices support `iszero` and `isone` for testing whether a matrix is the zero matrix or the identity matrix, respectively.

`Base.isempty` – Method.

```
isempty(A::MatrixElem{T}) where {T <: NCRingElement}
```

Return `true` if A has no entries, that is, if either the number of rows or the number of columns is zero. Otherwise, return `false`.

**Examples**

```
julia> A = zero_matrix(ZZ, 0, 3)
0 by 3 empty matrix

julia> isempty(A)
true

julia> B = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> isempty(B)
false
```

source

`Base.isassigned` – Method.

```
Base.isassigned(A::MatrixElem{T}, i::Int, j::Int) where {T <: NCRingElement}
```

Return `true` if the matrix A has an entry at position `(i, j)`, and `false` otherwise.

**Examples**

```
julia> M = matrix(ZZ, [3 1 2; 2 0 1])
[3   1   2]
[2   0   1]

julia> isassigned(M, 1, 2)
true

julia> isassigned(M, 4, 4)
false
```

source

AbstractAlgebra.is_zero_row – Method.

```
is_zero_row(A::Union{Matrix,MatrixElem}, i::Int)
```

Return `true` if the $i$-th row of the matrix $A$ is zero, and `false` otherwise.

This may be more efficient than checking all entries individually.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 0 0 0])
[1   2   3]
[0   0   0]

julia> is_zero_row(M, 1)
false

julia> is_zero_row(M, 2)
true
```

source

AbstractAlgebra.is_zero_column – Method.

```
is_zero_column(A::Union{Matrix,MatrixElem}, j::Int)
```

Return `true` if the $j$-th column of the matrix $A$ is zero, and `false` otherwise.

This may be more efficient than checking all entries individually.

**Examples**

```
julia> M = matrix(ZZ, [1 0; 2 0; 3 0])
[1   0]
[2   0]
[3   0]

julia> is_zero_column(M, 1)
false

julia> is_zero_column(M, 2)
true
```

source

### Triangular and diagonal matrices

AbstractAlgebra.is_lower_triangular – Method.

```
is_lower_triangular(A::MatElem)
```

Return `true` if $A$ is a lower triangular matrix, that is, all entries above the main diagonal are zero. Note that this definition also applies to non-square matrices.

Alias for `LinearAlgebra.istril`.

**Examples**

```
julia> is_lower_triangular(QQ[1 2 ; 0 4])
false

julia> is_lower_triangular(QQ[1 0 ; 3 4])
true

julia> is_lower_triangular(QQ[1 2 ;])
false

julia> is_lower_triangular(QQ[1 ; 2])
true
```

source

`AbstractAlgebra.is_upper_triangular` – Method.

```
is_upper_triangular(A::MatElem)
```

Return `true` if $A$ is an upper triangular matrix, that is, all entries below the main diagonal are zero. Note that this definition also applies to non-square matrices.

Alias for `LinearAlgebra.istriu`.

**Examples**

```
julia> is_upper_triangular(QQ[1 2 ; 0 4])
true

julia> is_upper_triangular(QQ[1 0 ; 3 4])
false

julia> is_upper_triangular(QQ[1 2 ;])
true

julia> is_upper_triangular(QQ[1 ; 2])
false
```

source

`AbstractAlgebra.is_diagonal` – Method.

```
is_diagonal(A::MatElem)
```

Return `true` if $A$ is a diagonal matrix, that is, if all entries off the main diagonal are zero. Note that this definition also applies to non-square matrices.

Alias for `LinearAlgebra.isdiag`.

**Examples**

```
julia> is_diagonal(QQ[1 0 ; 0 4])
true

julia> is_diagonal(QQ[1 2 ; 3 4])
false

julia> is_diagonal(QQ[1 0 ;])
true
```

source

`AbstractAlgebra.is_hessenberg` – Method.

```
is_hessenberg(A::MatElem{T}) where {T <: RingElement}
```

Return `true` if $A$ is in (upper) Hessenberg form, that is, if all entries below the first subdiagonal are zero, and `false` otherwise.

**Examples**

```
julia> A = matrix(ZZ, [1 2 3; 4 5 6; 0 7 8])
[1   2   3]
[4   5   6]
[0   7   8]

julia> is_hessenberg(A)
true

julia> B = matrix(ZZ, [1 2 3; 4 5 6; 7 8 9])
[1   2   3]
[4   5   6]
[7   8   9]

julia> is_hessenberg(B)
false
```

source

### Invertibility

`AbstractAlgebra.is_invertible_with_inverse` – Method.

```
is_invertible_with_inverse(A::MatrixElem{T}; side::Symbol = :left) where {T <: RingElement}
```

Return a tuple `(flag, B)` indicating whether the matrix $A$ has a one-sided inverse.

If $A$ is an $n \times m$ matrix and `side == :right`, then `flag` is `true` precisely if a right inverse exists. In this case, $B$ is an $m \times n$ matrix such that $AB$ is the $n \times n$ identity matrix.

If `side == :left`, then `flag` is `true` precisely if a left inverse exists. In this case, $B$ is an $m \times n$ matrix such that $BA$ is the $m \times m$ identity matrix.

If `flag` is `false`, then no inverse exists on the requested side.

To compute all one-sided inverses from one inverse, use the kernel: if $B$ and $C$ are both right inverses, then $A(B - C) = 0$, and similarly for left inverses.

**Examples**

```
julia> A = matrix(QQ, [1 2; 3 4])
[1//1   2//1]
[3//1   4//1]

julia> flag, B = is_invertible_with_inverse(A);

julia> flag
true

julia> B
[-2//1    1//1]
[ 3//2   -1//2]

julia> B*A == one(parent(A))
true
```

```
julia> A = matrix(QQ, [1 0 0; 0 1 0])
[1//1   0//1   0//1]
[0//1   1//1   0//1]

julia> flag, B = is_invertible_with_inverse(A; side = :right);

julia> flag
true

julia> A*B == one(parent(A*B))
true
```

source

AbstractAlgebra.is_invertible – Method.

```
is_invertible(A::MatElem{T}) where {T <: RingElement}
```

Return `true` if the square matrix $A$ is invertible, and `false` otherwise. To also compute an inverse, use `is_invertible_with_inverse`.

**Examples**

```
julia> A = matrix(ZZ, [1 2; 3 4])
[1   2]
[3   4]

julia> is_invertible(A)
false

julia> B = matrix(QQ, [1 2; 3 4])
[1//1   2//1]
[3//1   4//1]

julia> is_invertible(B)
true
```

source

### Symmetry

`AbstractAlgebra.is_symmetric` – Method.

```
is_symmetric(A::MatElem)
```

Return `true` if the given matrix is symmetric with respect to its main diagonal, i.e., `transpose(A) == A`, otherwise return `false`.

Alias for `LinearAlgebra.issymmetric`.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 2 4 5; 3 5 6])
[1   2   3]
[2   4   5]
[3   5   6]

julia> is_symmetric(M)
true

julia> N = matrix(ZZ, [1 2 3; 4 5 6; 7 8 9])
[1   2   3]
[4   5   6]
[7   8   9]

julia> is_symmetric(N)
false
```

source

`AbstractAlgebra.is_skew_symmetric` – Method.

```
is_skew_symmetric(A::MatElem)
```

Return `true` if the given matrix is skew symmetric with respect to its main diagonal, i.e., `transpose(A) == -A`, otherwise return `false`.

**Examples**

```
julia> M = matrix(ZZ, [0 -1 -2; 1 0 -3; 2 3 0])
[0   -1   -2]
[1    0   -3]
[2    3    0]

julia> is_skew_symmetric(M)
true
```

source

`AbstractAlgebra.is_alternating` – Method.

```
is_alternating(A::MatElem)
```

Return `true` if `A` is alternating, that is, if `A` is skew-symmetric and all entries on the main diagonal are zero. Return `false` otherwise.

Non-square matrices are not considered alternating.

**Examples**

```
julia> M = matrix(ZZ, [0 2 -3; -2 0 5; 3 -5 0])
[ 0    2   -3]
[-2    0    5]
[ 3   -5    0]

julia> is_alternating(M)
true

julia> N = matrix(ZZ, [1 2; -2 1])
[ 1   2]
[-2   1]

julia> is_alternating(N)
false
```

source

### Nilpotency

`AbstractAlgebra.is_nilpotent` – Method.

```
is_nilpotent(A::MatElem{T}) where {T <: RingElement}
```

Return `true` if `A` is nilpotent, that is, if there exists a positive integer $k$ such that $A^k = 0$. Return `false` otherwise.

If `A` is not square, an exception is raised. The test is only supported for matrices defined over integral domains.

**Examples**

```
julia> A = matrix(ZZ, [0 1 0; 0 0 1; 0 0 0])
[0   1   0]
[0   0   1]
[0   0   0]

julia> is_nilpotent(A)
true

julia> B = matrix(ZZ, [1 1; 0 1])
[1   1]
[0   1]

julia> is_nilpotent(B)
false
```

source

### Normal forms

`AbstractAlgebra.is_rref` – Method.

```
is_rref(A::MatrixElem{T}) where {T <: RingElement}
is_rref(A::MatrixElem{T}) where {T <: FieldElement}
```

Return `true` if $A$ is in reduced row echelon form, and `false` otherwise.

For matrices over fields, leading entries are required to be normalized to one.

**Examples**

```
julia> A = matrix(QQ, [1 0 2; 0 1 3; 0 0 0])
[1//1   0//1   2//1]
[0//1   1//1   3//1]
[0//1   0//1   0//1]

julia> is_rref(A)
true

julia> B = matrix(QQ, [2 0 4; 0 1 3; 0 0 0])
[2//1   0//1   4//1]
[0//1   1//1   3//1]
[0//1   0//1   0//1]
```

```
julia> is_rref(B)
false

julia> C = matrix(ZZ, [2 0 4; 0 3 6; 0 0 0])
[2   0   4]
[0   3   6]
[0   0   0]

julia> is_rref(C)
true
```

source

AbstractAlgebra.is_hnf – Method.

```
is_hnf(A::MatElem{T}) where {T <: RingElement}
```

Return `true` if the matrix $A$ is in Hermite normal form, and `false` otherwise.

**Examples**

```
julia> A = matrix(ZZ, [2 3 -1; 3 5 7; 11 1 12])
[ 2   3   -1]
[ 3   5    7]
[11   1   12]

julia> is_hnf(A)
false

julia> H = hnf(A)
[1   0   255]
[0   1    17]
[0   0   281]

julia> is_hnf(H)
true
```

source

AbstractAlgebra.is_snf – Method.

```
is_snf(A::MatElem{T}) where {T <: RingElement}
```

Return `true` if $A$ is in Smith normal form, and `false` otherwise.

**Examples**

```
julia> A = matrix(ZZ, [2 4 4; -6 6 12; 10 -4 -16])
[ 2    4     4]
[-6    6    12]
[10   -4   -16]

julia> S = snf(A)
[2   0    0]
[0   6    0]
[0   0   12]

julia> is_snf(S)
true
```

source

AbstractAlgebra.is_weak_popov – Method.

```
is_weak_popov(P::MatrixElem{T}, rank::Int) where {T <: PolyRingElem}
```

Return `true` if $P$ is in weak Popov form with the given rank, and `false` otherwise.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]));

julia> P = weak_popov(A)
[   1                        2                    3   x]
[   0                        0                    0   0]
[-x^3   -2*x^3 + x^2 - 2*x + 1   -2*x^3 + x^2 - 3*x   1]

julia> is_weak_popov(P, 2)
true

julia> is_weak_popov(P, 3)
false
```

source

AbstractAlgebra.is_popov – Method.

```
is_popov(P::MatrixElem{T}, rank::Int) where {T <: PolyRingElem}
```

Return `true` if $P$ is in Popov form with the given rank, and `false` otherwise.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]));

julia> P = popov(A)
[       0                           0                          0       0]
[       1                           2                          3       x]
[1//2*x^3   x^3 - 1//2*x^2 + x - 1//2   x^3 - 1//2*x^2 + 3//2*x   -1//2]

julia> is_popov(P, 1)
false

julia> is_popov(P, 2)
true
```

source

## 48.4 Matrix properties

This page collects functions which compute values, invariants, or associated objects from matrices, rather than testing whether matrices satisfy certain conditions. Examples include determinants, ranks, inverses, nullspaces, polynomials, and related constructions.

### Basic properties

`AbstractAlgebra.number_of_rows` – Method.

```
number_of_rows(A::MatElem)
```

Return the number of rows of the matrix `A`.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 4 5 6])
[1   2   3]
[4   5   6]

julia> number_of_rows(M)
2
```

source

`AbstractAlgebra.number_of_columns` – Method.

```
number_of_columns(A::MatElem)
```

Return the number of columns of the matrix `A`.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 4 5 6])
[1   2   3]
[4   5   6]

julia> number_of_columns(M)
3
```

source

Base.length – Method.

```
length(A::MatrixElem{T}) where T <: NCRingElement
```

Return the number of entries in the matrix A.

**Examples**

```
julia> M = matrix(ZZ, [1 2 3; 4 5 6])
[1   2   3]
[4   5   6]

julia> length(M)
6
```

source

### Trace, determinant and rank

LinearAlgebra.tr – Method.

```
tr(A::MatElem{T}) where T <: NCRingElement
```

Return the trace of the matrix $A$, i.e. the sum of its diagonal elements. The matrix is required to be square.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> b = tr(A)
t^2 + 3*t + 2
```

source

LinearAlgebra.det – Method.

```
det(A::MatElem{T}) where {T <: RingElement}
```

Return the determinant of the given matrix $A$. The matrix is required to be square.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x)
(Univariate polynomial ring in x over rationals, x)

julia> A = R[x 1; 1 x^2];

julia> det(A)
x^3 - 1
```

source

LinearAlgebra.rank – Method.

```
rank(A::MatElem{T}) where {T <: RingElement}
```

Return the rank of the given matrix $A$.

**Examples**

```
julia> A = QQ[1 2 3; 2 4 6; 1 1 1];

julia> rank(A)
2
```

source

### Inverses

Base.inv – Method.

```
inv(A::MatElem{T}) where {T <: RingElement}
```

Given an invertible $n \times n$ matrix $A$ over a ring, return the $n \times n$ matrix $X$ such that $MX = I_n$, where $I_n$ is the $n \times n$ identity matrix.

If $A$ is not invertible over the base ring, an exception is raised.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> inv(M)
[-5//3    2//3    1//1]
[ 4//3   -1//3   -2//1]
[ 0//1    0//1    1//1]
```

source

AbstractAlgebra.pseudo_inv – Method.

```
pseudo_inv(A::MatElem{T}) where {T <: RingElement}
```

Given a non-singular $n \times n$ matrix $A$ over a ring, return a tuple $X, d$ consisting of an $n \times n$ matrix $X$ and a denominator $d$ such that $MX = dI_n$, where $I_n$ is the $n \times n$ identity matrix.

The denominator $d$ is the determinant of $A$ up to sign.

If $A$ is not invertible over the base ring, an exception is raised.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3;4 5 6;0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> pseudo_inv(M)
([5 -2 -3; -4 1 6; 0 0 -3], -3//1)
```

source

### Characteristic and minimal polynomials

AbstractAlgebra.charpoly – Method.

```
charpoly(A::MatElem{T}) where {T <: RingElement}
charpoly(S::PolyRing{T}, A::MatElem{T}) where {T <: RingElement}
```

Return the characteristic polynomial $p$ of the square matrix $A$. If a polynomial ring $S$ over the same base ring as $A$ is supplied, the resulting polynomial is an element of it.

**Examples**

```
julia> R, = residue_ring(ZZ, 7);

julia> S = matrix_space(R, 4, 4)
```

```
Matrix space of 4 rows and 4 columns
  over residue ring of integers modulo 7

julia> T, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> M = S([R(1) R(2) R(4) R(3); R(2) R(5) R(1) R(0);
              R(6) R(1) R(3) R(2); R(1) R(1) R(3) R(5)])
[1   2   4   3]
[2   5   1   0]
[6   1   3   2]
[1   1   3   5]

julia> A = charpoly(T, M)
y^4 + 2*y^2 + 6*y + 2

julia> A = charpoly(M)
x^4 + 2*x^2 + 6*x + 2
```

source

AbstractAlgebra.minpoly – Method.

```
minpoly(A::MatElem{T}) where {T <: RingElement}
minpoly(S::PolyRing{T}, A::MatElem{T}) where {T <: RingElement}
```

Return the minimal polynomial $p$ of the square matrix $A$. If a polynomial ring $S$ over the same base ring as $A$ is supplied, the resulting polynomial is an element of it.

**Examples**

```
julia> R = GF(13)
Finite field F_13

julia> S, y = polynomial_ring(R, :y)
(Univariate polynomial ring in y over R, y)

julia> M = R[7 6 1;
             7 7 5;
             8 12 5]
[7    6   1]
[7    7   5]
[8   12   5]

julia> A = minpoly(S, M)
y^2 + 10*y

julia> A = minpoly(M)
x^2 + 10*x
```

source

### Powers

AbstractAlgebra.powers – Method.

```
powers(a::Union{NCRingElement, MatElem}, d::Int)
```

Return an array $M$ of "powers" of a where $M[i+1] = a^i$ for $i = 0..d$.

**Examples**

```
julia> M = ZZ[1 2 3; 2 3 4; 4 5 5]
[1   2   3]
[2   3   4]
[4   5   5]

julia> A = powers(M, 4)
5-element Vector{AbstractAlgebra.Generic.MatSpaceElem{BigInt}}:
 [1 0 0; 0 1 0; 0 0 1]
 [1 2 3; 2 3 4; 4 5 5]
 [17 23 26; 24 33 38; 34 48 57]
 [167 233 273; 242 337 394; 358 497 579]
 [1725 2398 2798; 2492 3465 4044; 3668 5102 5957]
```

source

### Gram matrices

AbstractAlgebra.gram – Method.

```
gram(A::MatElem)
```

Return the Gram matrix of $A$, i.e. if $A$ is an $r \times c$ matrix, return the $r \times r$ matrix whose $(i, j)$-th entry is the dot product of the $i$-th and $j$-th rows of $A$.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> B = gram(A)
[2*t^2 + 2*t + 2   t^3 + 2*t^2 + t                   2*t^2 + t - 1]
[t^3 + 2*t^2 + t       t^4 + 2*t^2                       t^3 + 3*t]
[  2*t^2 + t - 1         t^3 + 3*t   t^4 + 2*t^3 + 4*t^2 + 6*t + 9]
```

source

### Content

AbstractAlgebra.content – Method.

```
content(A::MatrixElem{T}) where T <: RingElement
```

Return the greatest common divisor of all entries of the matrix $A$, assuming such a greatest common divisor exists.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> b = content(A)
1
```

source

### Minors and exterior powers

AbstractAlgebra.minors – Method.

```
minors(A::MatElem, k::Int)
```

Return an array consisting of all $k$-minors of the given matrix $A$, i.e. the determinants of all $k \times k$ submatrices of $A$.

**Examples**

```
julia> M = ZZ[1 2 3; 4 5 6]
[1   2   3]
[4   5   6]

julia> minors(M, 2)
3-element Vector{BigInt}:
 -3
 -6
 -3
```

source

AbstractAlgebra.minors_with_position – Method.

```
minors_with_position(A::MatElem, k::Int)
```

Return an array consisting of all $k$-minors of $A$, together with the row and column indices defining the corresponding submatrices.

**Examples**

```
julia> M = ZZ[1 2 3; 4 5 6]
[1   2   3]
[4   5   6]

julia> minors_with_position(M, 2)
3-element Vector{Tuple{BigInt, Vector{Int64}, Vector{Int64}}}:
 (-3, [1, 2], [1, 2])
 (-6, [1, 2], [1, 3])
 (-3, [1, 2], [2, 3])
```

source

AbstractAlgebra.minors_iterator – Method.

```
minors_iterator(A::MatElem, k::Int)
```

Return an iterator computing all $k$-minors of $A$, i.e. the determinants of all $k \times k$ submatrices of $A$.

**Examples**

```
julia> M = ZZ[1 2 3; 4 5 6]
[1   2   3]
[4   5   6]

julia> first(minors_iterator(M, 2))
-3

julia> collect(minors_iterator(M, 2))
3-element Vector{BigInt}:
 -3
 -6
 -3
```

source

AbstractAlgebra.minors_iterator_with_position – Method.

```
minors_iterator_with_position(A::MatElem, k::Int)
```

Return an iterator computing all $k$-minors of $A$, together with the row and column indices defining the corresponding submatrices.

**Examples**

```
julia> M = ZZ[1 2 3; 4 5 6]
[1   2   3]
[4   5   6]

julia> first(minors_iterator_with_position(M, 2))
(-3, [1, 2], [1, 2])
```

source

`AbstractAlgebra.exterior_power` – Method.

```
exterior_power(A::MatElem, k::Int) -> MatElem
```

Return the matrix of the induced map on the k-th exterior power. Its entries are the determinants of the $k \times k$ submatrices of $A$.

**Examples**

```
julia> M = matrix(ZZ, 3, 3, [1, 2, 3, 4, 5, 6, 7, 8, 9]);

julia> exterior_power(M, 2)
[-3    -6   -3]
[-6   -12   -6]
[-3    -6   -3]
```

source

### Pfaffians

`AbstractAlgebra.pfaffian` – Method.

```
pfaffian(A::MatElem)
```

Return the Pfaffian of the skew-symmetric matrix $A$.

**Examples**

```
julia> R, x = polynomial_ring(QQ, ["x$i" for i in 1:6]);

julia> M = R[0 x[1] x[2] x[3]; -x[1] 0 x[4] x[5]; -x[2] -x[4] 0 x[6]; -x[3] -x[5] -x[6] 0]
[  0    x1    x2   x3]
```

```
[-x1     0    x4   x5]
[-x2   -x4     0   x6]
[-x3   -x5   -x6    0]

julia> pfaffian(M)
x1*x6 - x2*x5 + x3*x4
```

source

AbstractAlgebra.pfaffians – Method.

```
pfaffians(A::MatElem, k::Int)
```

Return a vector consisting of the Pfaffians of all $k \times k$ principal submatrices of the skew-symmetric matrix $A$.

**Examples**

```
julia> R, x = polynomial_ring(QQ, ["x$i" for i in 1:6]);

julia> M = R[0 x[1] x[2] x[3]; -x[1] 0 x[4] x[5]; -x[2] -x[4] 0 x[6]; -x[3] -x[5] -x[6] 0]
[  0    x1    x2   x3]
[-x1     0    x4   x5]
[-x2   -x4     0   x6]
[-x3   -x5   -x6    0]

julia> pfaffians(M, 2)
6-element Vector{AbstractAlgebra.Generic.MPoly{Rational{BigInt}}}:
 x1
 x2
 x4
 x3
 x5
 x6
```

source

## 48.5 Matrix normal forms

This page collects functionality for transforming matrices into normal forms or related decompositions. Some algorithms also return transformation matrices which certify the result.

### LU factorisation

LinearAlgebra.lu – Method.

```
lu(A::MatrixElem{T}, P = SymmetricGroup(nrows(A))) where {T <: FieldElement}
```

Return the LU decomposition of $A$.

More precisely, return a tuple $r, p, L, U$ consisting of the rank $r$ of $A$, a permutation $p$ belonging to $P$, a lower triangular matrix $L$ and an upper triangular matrix $U$ such that $p(A) = LU$. Here $p(A)$ denotes the matrix obtained by applying the permutation $p$ to the rows of $A$.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> r, p, L, U = lu(M)
(3, (), [1 0 0; 4 1 0; 0 0 1], [1 2 3; 0 -3 -6; 0 0 1])
```

source

`AbstractAlgebra.fflu` – Method.

```
fflu(A::MatrixElem{T}, P = SymmetricGroup(nrows(A))) where {T <: RingElement}
```

Return the fraction-free LU decomposition of $A$.

More precisely, return a tuple $r, d, p, L, U$ consisting of the rank $r$ of $A$, a denominator $d$, a permutation $p$ belonging to $P$, a lower triangular matrix $L$ and an upper triangular matrix $U$ such that $p(A) = LDU$. Here $p(A)$ denotes the matrix obtained by applying the permutation $p$ to the rows of $A$.

The matrix $D$ is the diagonal matrix $\mathrm{diag}(p_1, p_1p_2, \ldots, p_{n-2}p_{n-1}, p_{n-1}p_n)$, where the $p_i$ are the inverses of the diagonal entries of $L$.

The denominator $d$ is set to $\pm\det(S)$, where $S$ is an appropriate submatrix of $A$; if $A$ is square and nonsingular, then $S = A$. The sign is determined by the parity of the permutation.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> r, d, p, L, U = fflu(M)
(3, -3//1, (), [1 0 0; 4 -3 0; 0 0 -3], [1 2 3; 0 -3 -6; 0 0 -3])
```

source

### Reduced row-echelon form

`AbstractAlgebra.rref_rational` – Method.

```
rref_rational(A::MatrixElem{T}) where {T <: RingElement}
```

Return the reduced row echelon form of $A$ using fraction-free arithmetic.

More precisely, return a tuple $r, B, d$ consisting of the rank $r$ of $A$, a matrix $B$ and a denominator $d$ in the base ring of $A$ such that $B/d$ is the reduced row echelon form of $A$.

Note that the denominator $d$ is not necessarily minimal.

**Examples**

```
julia> M = matrix(ZZ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1   2   3]
[4   5   6]
[0   0   1]

julia> r, A, d = rref_rational(M)
(3, [-3 0 0; 0 -3 0; 0 0 -3], -3)

julia> is_rref(A)
true

julia> A/d
[1   0   0]
[0   1   0]
[0   0   1]
```

source

`AbstractAlgebra.rref` – Method.

```
rref(A::MatrixElem{T}) where {T <: FieldElement}
```

Return the reduced row echelon form of $A$.

More precisely, return a tuple $r, B$ consisting of the rank $r$ of $A$ and the reduced row echelon form $B$ of $A$.

**Examples**

```
julia> M = matrix(QQ, 3, 3, [1 2 3; 4 5 6; 0 0 1])
[1//1   2//1   3//1]
[4//1   5//1   6//1]
[0//1   0//1   1//1]

julia> r, A = rref(M)
(3, [1 0 0; 0 1 0; 0 0 1])

julia> is_rref(A)
true
```

source

### Hessenberg form and similarity transformations

`LinearAlgebra.hessenberg` – Method.

```
hessenberg(A::MatElem{T}) where {T <: RingElement}
```

Return the Hessenberg form of $A$, i.e. an upper Hessenberg matrix which is similar to $A$.

An upper Hessenberg matrix has zero entries below the first subdiagonal.

**Examples**

```
julia> R, = residue_ring(ZZ, 7);

julia> M = matrix(R, 4, 4, [1 2 4 3; 2 5 1 0; 6 1 3 2; 1 1 3 5])
[1   2   4   3]
[2   5   1   0]
[6   1   3   2]
[1   1   3   5]

julia> H = hessenberg(M)
[1   5   5   3]
[2   1   1   0]
[0   1   3   2]
[0   0   2   2]

julia> is_hessenberg(H)
true
```

source

AbstractAlgebra.similarity! – Method.

```
similarity!(A::MatrixElem{T}, r::Int, d::T) where {T <: RingElement}
```

Apply a similarity transformation to the square matrix $A$ in-place.

Let $P$ be the identity matrix with all off-diagonal entries in row $r$ replaced by $d$. This function replaces $A$ by $P^{-1}AP$.

Similarity transformations preserve the minimal and characteristic polynomials of a matrix.

**Examples**

```
julia> R, = residue_ring(ZZ, 7);

julia> S = matrix_space(R, 4, 4)
Matrix space of 4 rows and 4 columns
  over residue ring of integers modulo 7

julia> M = S([R(1) R(2) R(4) R(3); R(2) R(5) R(1) R(0);
              R(6) R(1) R(3) R(2); R(1) R(1) R(3) R(5)])
[1   2   4   3]
[2   5   1   0]
[6   1   3   2]
[1   1   3   5]

julia> similarity!(M, 1, R(3))
```

source

### Hermite normal form

AbstractAlgebra.hnf – Method.

```
hnf(A::MatElem{T}) where {T <: RingElement}
```

Return the upper right row Hermite normal form of $A$.

The Hermite normal form is a canonical form for matrices. It is obtained by employing elementary row operations to produce an upper triangular matrix.

**Examples**

```
julia> A = matrix(ZZ, [2 3 -1; 3 5 7; 11 1 12])
[ 2   3   -1]
[ 3   5    7]
[11   1   12]

julia> H = hnf(A)
[1   0   255]
[0   1    17]
[0   0   281]

julia> is_hnf(H)
true
```

source

AbstractAlgebra.hnf_with_transform – Method.

```
hnf_with_transform(A::MatElem{T}) where {T <: RingElement}
```

Return the upper right row Hermite normal form of $A$, together with a transformation matrix.

More precisely, return a tuple $H, U$ where $H$ is the upper right row Hermite normal form of $A$ and $U$ is an invertible matrix such that $UA = H$.

**Examples**

```
julia> A = matrix(ZZ, [2 3 -1; 3 5 7; 11 1 12])
[ 2   3   -1]
[ 3   5    7]
[11   1   12]

julia> H, U = hnf_with_transform(A)
([1 0 255; 0 1 17; 0 0 281], [-47 28 1; -3 2 0; -52 31 1])

julia> U*A == H
true
```

source

### Smith normal form

`AbstractAlgebra.snf` – Method.

```
snf(A::MatElem{T}) where {T <: RingElement}
```

Return the Smith normal form of $A$.

The Smith normal form is a canonical diagonal form obtained by applying invertible row and column transformations.

**Examples**

```
julia> A = matrix(ZZ, [2 3 -1; 3 5 7; 11 1 12])
[ 2   3   -1]
[ 3   5    7]
[11   1   12]

julia> S = snf(A)
[1   0     0]
[0   1     0]
[0   0   281]

julia> is_snf(S)
true
```

source

`AbstractAlgebra.snf_with_transform` – Method.

```
snf_with_transform(A::MatElem{T}) where {T <: RingElement}
```

Return the Smith normal form of $A$, together with transformation matrices.

More precisely, return a tuple $S, T, U$ where $S$ is the Smith normal form of $A$ and $T$ and $U$ are invertible matrices such that $TAU = S$.

**Examples**

```
julia> A = matrix(ZZ, [2 3 -1; 3 5 7; 11 1 12])
[ 2   3   -1]
[ 3   5    7]
[11   1   12]

julia> S, T, U = snf_with_transform(A)
([1 0 0; 0 1 0; 0 0 281], [1 0 0; 7 1 0; 229 31 1], [0 -3 26; 0 2 -17; -1 0 1])

julia> T*A*U == S
true
```

source

### Popov forms

AbstractAlgebra.weak_popov – Method.

```
weak_popov(A::MatElem{T}) where {T <: PolyRingElem}
```

Return the weak Popov form of $A$.

The matrix $A$ must have entries in a univariate polynomial ring over a field. The weak Popov form is a row-reduced matrix, in which the nonzero rows have distinct leading positions with respect to their degrees.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]))
[1         2           3               x]
[x       2*x         3*x             x^2]
[x   x^2 + 1   x^3 + x^2   x^4 + x^2 + 1]

julia> P = weak_popov(A)
[   1                        2                    3   x]
[   0                        0                    0   0]
[-x^3   -2*x^3 + x^2 - 2*x + 1   -2*x^3 + x^2 - 3*x   1]
```

source

AbstractAlgebra.weak_popov_with_transform – Method.

```
weak_popov_with_transform(A::MatElem{T}) where {T <: PolyRingElem}
```

Return the weak Popov form of $A$, together with a transformation matrix.

The matrix $A$ must have entries in a univariate polynomial ring over a field. The weak Popov form is a row-reduced form in which the nonzero rows have distinct leading positions determined by their degrees.

More precisely, return a tuple $P, U$ where $P$ is the weak Popov form of $A$ and $U$ is a transformation matrix such that $P = UA$.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]))
[1         2           3               x]
[x       2*x         3*x             x^2]
[x   x^2 + 1   x^3 + x^2   x^4 + x^2 + 1]

julia> P, U = weak_popov_with_transform(A)
([1 2 3 x; 0 0 0 0; -x^3 -2*x^3+x^2-2*x+1 -2*x^3+x^2-3*x 1], [1 0 0; -x 1 0; -x^3-x 0 1])

julia> U*A == P
true
```

source

AbstractAlgebra.popov – Method.

```
popov(A::MatElem{T}) where {T <: PolyRingElem}
```

Return the Popov form of $A$.

The matrix $A$ must have entries in a univariate polynomial ring over a field. The Popov form is a canonical row-reduced form. It is a weak Popov form satisfying additional normalization conditions on the leading entries.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]))
[1         2           3              x]
[x       2*x         3*x            x^2]
[x   x^2 + 1   x^3 + x^2   x^4 + x^2 + 1]

julia> P = popov(A)
[       0                          0                         0       0]
[       1                          2                         3       x]
[1//2*x^3   x^3 - 1//2*x^2 + x - 1//2   x^3 - 1//2*x^2 + 3//2*x   -1//2]
```

source

AbstractAlgebra.popov_with_transform – Method.

```
popov_with_transform(A::MatElem{T}) where {T <: PolyRingElem}
```

Return the Popov form of $A$, together with a transformation matrix.

The matrix $A$ must have entries in a univariate polynomial ring over a field. The Popov form is a canonical row-reduced form. It is a weak Popov form satisfying additional normalization conditions on the leading entries.

More precisely, return a tuple $P, U$ where $P$ is the Popov form of $A$ and $U$ is a transformation matrix such that $P = UA$.

**Examples**

```
julia> R, x = polynomial_ring(QQ, :x);

julia> A = matrix(R, map(R, Any[1 2 3 x; x 2*x 3*x x^2; x x^2+1 x^3+x^2 x^4+x^2+1]))
[1         2           3              x]
[x       2*x         3*x            x^2]
[x   x^2 + 1   x^3 + x^2   x^4 + x^2 + 1]

julia> P, U = popov_with_transform(A)
([0 0 0 0; 1 2 3 x; 1//2*x^3 x^3-1//2*x^2+x-1//2 x^3-1//2*x^2+3//2*x -1//2], [-x 1 0; 1 0 0;
↪  1//2*x^3+1//2*x 0 -1//2])
```

```
julia> U*A == P
true
```

source

## 48.6 Solving Linear Systems

### Introduction

This page explains how to solve systems of linear equations. It introduces the basic solving routines, the conventions used for left and right systems, and related functionality such as kernel computations.

### Solving a left linear system

To solve a system of linear equations, use the function `solve`.

**Note**

By default, `solve` computes a solution x of a left system $xA = b$. Right linear systems are discussed in the next section.

```
julia> A = matrix(QQ, [2 0 0; 0 0 0; 0 3 0])
[2//1   0//1   0//1]
[0//1   0//1   0//1]
[0//1   3//1   0//1]

julia> b = matrix(QQ, 1, 3, [1, 3//2, 0])
[1//1   3//2   0//1]

julia> x = solve(A, b)
[1//2   0//1   1//2]

julia> x * A == b
true
```

If the system is solvable, `solve` returns a particular solution. Otherwise, it throws an error.

Since the matrix A in the above example does not have full rank, the system may have infinitely many solutions. The full set of solutions is obtained by adding arbitrary elements of the *left* kernel of A to the particular solution returned by `solve`.

```
julia> A = matrix(QQ, [2 0 0; 0 0 0; 0 3 0]);

julia> rank(A)
2

julia> kernel(A)
[0//1   1//1   0//1]
```

See the documentation of `kernel` for more information on computing kernels.

### Left and right linear systems

The function `solve` supports both left systems ($xA = b$) and right systems ($Ax = b$). The keyword argument `side` determines which convention is used.

- `side = :left` (the default) solves the system `xA = b`.
- `side = :right` solves the system `Ax = b`.

The following example solves a right linear system:

```
julia> A = matrix(QQ, [2 0 0; 0 0 0; 0 3 0])
[2//1   0//1   0//1]
[0//1   0//1   0//1]
[0//1   3//1   0//1]

julia> b = matrix(QQ, 3, 1, [1, 0, 3//2])
[1//1]
[0//1]
[3//2]

julia> x = solve(A, b, side = :right)
[1//2]
[1//2]
[0//1]

julia> A * x == b
true

julia> kernel(A, side = :right)
[0//1]
[0//1]
[1//1]
```

### Testing whether a system is solvable

The function `solve` throws an error if the system has no solution. If it is not known in advance whether a solution exists, one of the following functions may be more appropriate:

| Function | Description |
|---|---|
| `can_solve` | Return whether the system is solvable. |
| `can_solve_with_solution` | Return whether the system is solvable and, if so, a particular solution. |
| `can_solve_with_solution_and_kernel` | Return whether the system is solvable and, if so, a particular solution together with a basis of the corresponding kernel. |

For example:

```
julia> A = matrix(QQ, [2 0 0; 0 0 0; 0 3 0]);

julia> b = matrix(QQ, 1, 3, [1, 3//2, 1]);

julia> can_solve(A, b)
```

```
false

julia> b2 = matrix(QQ, 1, 3, [1, 3//2, 0]);

julia> can_solve_with_solution(A, b2)
(true, [1//2 0 1//2])

julia> can_solve_with_solution_and_kernel(A, b2)
(true, [1//2 0 1//2], [0 1 0])
```

### Solving multiple linear systems simultaneously

The functions discussed above also accept a matrix as the right-hand side. In this case, each row (respectively column) represents a separate right-hand side, and all systems are solved simultaneously.

If several systems with the same coefficient matrix but different right-hand sides need to be solved one after another, it is more efficient to construct a solving context first. To construct such a solving context, use the function `solve_init`. The resulting context object can be used in place of the coefficient matrix in calls to `solve`, `can_solve`, and the related functions. This avoids recomputing the reduced form of the coefficient matrix each time a new system is solved.

### Reference documentation

For a complete description of the available solving functions and solving contexts, see the Linear solving interface.

# Chapter 49

# Matrix spaces

A matrix space represents the collection of all matrices with a fixed number of rows and columns over a fixed base ring. Matrix spaces are parent objects; their elements are the corresponding matrices.

## 49.1 Creating matrix spaces

AbstractAlgebra.matrix_space – Method.

```
matrix_space(R::NCRing, r::Int, c::Int)
```

Return the space of $r \times c$ matrices over the ring R.

The returned object is the parent object for matrices with r rows and c columns whose entries belong to R.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 2, 3)
Matrix space of 2 rows and 3 columns
  over univariate polynomial ring in t over rationals
```

source

## 49.2 Properties of matrix spaces

AbstractAlgebra.number_of_rows – Method.

```
number_of_rows(s::MatSpace)
```

Return the number of rows of the matrices in the matrix space s.

**Examples**

```
julia> S = matrix_space(QQ, 2, 3)
Matrix space of 2 rows and 3 columns
  over rationals

julia> number_of_rows(S)
2
```

source

AbstractAlgebra.number_of_columns – Method.

```
number_of_columns(s::MatSpace)
```

Return the number of columns of the matrices in the matrix space s.

**Examples**

```
julia> S = matrix_space(QQ, 2, 3)
Matrix space of 2 rows and 3 columns
  over rationals

julia> number_of_columns(S)
3
```

source

The base ring and the vector space dimension of the matrix space can be queried with base_space and vector_space_dim, respectively.

## 49.3 Creating elements of a matrix space

### Calling a matrix space

AbstractAlgebra.MatSpace – Type.

```
(S::MatSpace{T})() where {T <: NCRingElement}
(S::MatSpace)(a::NCRingElement)
(S::MatSpace{T})(a::MatrixElem{T}) where {T <: NCRingElement}
(S::MatSpace{T})(a::AbstractVecOrMat) where {T <: NCRingElement}
```

Construct an element of the matrix space S.

The call S() returns the zero matrix in s.

If a is a ring element coercible into the base ring of S, then S(a) returns the diagonal matrix in S whose diagonal entries are a.

If a is a matrix whose dimensions and base ring agree with those of S, then S(a) returns the corresponding element of S. If necessary, a new matrix with the appropriate implementation type is constructed.

If a is a Julia vector or matrix, then `S(a)` constructs an element of S whose entries are obtained by coercing the entries of a into the base ring of S. The entries are interpreted in row-major order and must have length `nrows(S) * ncols(S)`.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_space(R, 3, 3)
Matrix space of 3 rows and 3 columns
  over univariate polynomial ring in t over rationals

julia> S()
[0   0   0]
[0   0   0]
[0   0   0]

julia> S(12)
[12    0    0]
[ 0   12    0]
[ 0    0   12]

julia> S(zero_matrix(R, 3, 3))
[0   0   0]
[0   0   0]
[0   0   0]

julia> S(BigInt[2 3 1; 1 0 4; 0 0 1])
[2   3   1]
[1   0   4]
[0   0   1]
```

source

### Special elements

Base.zero – Method.

```
zero(s::MatSpace)
```

Return the zero matrix in the matrix space s.

**Examples**

```
julia> S = matrix_space(QQ, 2, 3)
Matrix space of 2 rows and 3 columns
  over rationals

julia> zero(S)
[0//1   0//1   0//1]
[0//1   0//1   0//1]
```

source

Base.one – Method.

```
one(s::MatSpace)
```

Return the identity matrix in the matrix space s.

The matrix space s must contain square matrices.

**Examples**

```
julia> S = matrix_space(QQ, 3, 3)
Matrix space of 3 rows and 3 columns
  over rationals

julia> one(S)
[1//1   0//1   0//1]
[0//1   1//1   0//1]
[0//1   0//1   1//1]
```

source

### Recovering the parent object

Base.parent – Method.

```
parent(A::MatElem)
```

Return the matrix space over the base ring of A with the same dimensions as A.

**Examples**

```
julia> M = matrix(QQ, [1 2 3; 4 5 6])
[1//1   2//1   3//1]
[4//1   5//1   6//1]

julia> parent(M)
Matrix space of 2 rows and 3 columns
  over rationals
```

source

# Chapter 50

# Matrix algebras

A matrix algebra (or matrix ring) represents the collection of all square matrices of a fixed number of rows, referred to as the degree of the matrix algebra, over a fixed base ring. Matrix algebras are parent objects; their elements are the corresponding matrices.

Matrix algebras are noncommutative rings. They support the usual ring operations, such as addition and multiplication, and can be used in constructions accepting noncommutative base rings.

Although elements of matrix algebras mostly behave like matrices, their parent object is a matrix algebra rather than a matrix space. Therefore, some functionality available for matrices in matrix spaces is not available for matrix algebra elements, in particular functions which do not preserve square matrices (e.g. `kernel`).

## 50.1 Creating matrix algebras

`AbstractAlgebra.matrix_ring` – Method.

```
matrix_ring(R::NCRing, n::Int)
```

Return the matrix algebra (or matrix ring) of degree $n$ over the base ring $R$.

The returned parent object represents the ring of all $n \times n$ matrices over $R$.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_ring(R, 3)
Matrix ring of degree 3
  over univariate polynomial ring in t over rationals
```

source

## 50.2 Properties of matrix algebras

`AbstractAlgebra.degree` – Method.

```
degree(R::MatRing)
```

Return the degree $n$ of the given matrix algebra.

The degree is the number of rows of the square matrices belonging to R.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_ring(R, 3)
Matrix ring of degree 3
  over univariate polynomial ring in t over rationals

julia> degree(S)
3
```

source

The base ring can be queried using base_ring.

## 50.3 Creating elements of a matrix algebra

### Calling a matrix algebra

AbstractAlgebra.Generic.MatRing – Type.

```
(a::MatRing{T})() where {T <: NCRingElement}
(a::MatRing{T})(b::S) where {S <: NCRingElement, T <: NCRingElement}
(a::MatRing{T})(b::T) where {S <: NCRingElement, T <: MatRingElem{S}}
(a::MatRing{T})(b::MatRingElem{T}) where {T <: NCRingElement}
(a::MatRing{T})(b::MatrixElem{S}) where {S <: NCRingElement, T <: NCRingElement}
(a::MatRing{T})(b::Matrix{S}) where {S <: NCRingElement, T <: NCRingElement}
(a::MatRing{T})(b::Vector{S}) where {S <: NCRingElement, T <: NCRingElement}
```

Construct an element of the matrix algebra a.

The call a() returns the zero matrix in a.

If b is a ring element coercible into the base ring of a, then a(b) returns the scalar matrix in a whose diagonal entries are b.

If b is a matrix algebra element whose parent is a, then a(b) returns b unchanged.

If b is a matrix whose dimensions and base ring agree with those of a, then a(b) returns the corresponding element of a. If necessary, a new matrix with the appropriate implementation type is constructed.

If b is a Julia vector or matrix, then a(b) constructs an element of a whose entries are obtained by coercing the entries of b into the base ring of a. The dimensions of the Julia object must be compatible with the degree of the matrix algebra.

**Examples**

```
julia> R, = residue_ring(ZZ, 7);

julia> A = matrix_ring(R, 3);

julia> A()
[0   0   0]
[0   0   0]
[0   0   0]

julia> A(R(3))
[3   0   0]
[0   3   0]
[0   0   3]

julia> M = A([R(1) R(2) R(3); R(4) R(5) R(6); R(0) R(1) R(2)])
[1   2   3]
[4   5   6]
[0   1   2]

julia> A(M) === M
true

julia> A([R(1), R(2), R(3), R(4), R(5), R(6), R(0), R(1), R(2)])
[1   2   3]
[4   5   6]
[0   1   2]

julia> S = matrix_space(R, 3, 3);

julia> N = S([R(1) R(0) R(0); R(0) R(2) R(0); R(0) R(0) R(3)]);

julia> A(N)
[1   0   0]
[0   2   0]
[0   0   3]
```

source

### Special elements

As for other rings, the additive and multiplicative identities of a matrix algebra can be constructed using `zero` and one.

```
julia> S = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> zero(S)
[0   0]
[0   0]

julia> one(S)
[1   0]
[0   1]
```

## 50.4 Properties of matrix algebra elements

`Base.parent` – Method.

```
parent(A::MatRingElem{T}) where T <: NCRingElement
```

Return the matrix algebra containing `A`.

This is the matrix algebra over the base ring of `A` whose degree is the number of rows (equivalently columns) of `A`.

**Examples**

```
julia> S = matrix_ring(ZZ, 2)
Matrix ring of degree 2
  over integers

julia> A = S([1 2; 3 4])
[1   2]
[3   4]

julia> parent(A) == S
true
```

source

`AbstractAlgebra.degree` – Method.

```
degree(A::MatRingElem{T}) where T <: NCRingElement
```

Return the degree $n$ of the parent matrix algebra of `A`.

**Examples**

```
julia> R, t = polynomial_ring(QQ, :t)
(Univariate polynomial ring in t over rationals, t)

julia> S = matrix_ring(R, 3)
Matrix ring of degree 3
  over univariate polynomial ring in t over rationals

julia> A = S([t + 1 t R(1); t^2 t t; R(-2) t + 2 t^2 + t + 1])
[t + 1       t             1]
[  t^2       t             t]
[   -2   t + 2   t^2 + t + 1]

julia> degree(A)
3
```

source

## 50.5 Converting matrix algebra elements to matrices

`AbstractAlgebra.matrix` – Method.

```
matrix(R::NCRing, A::MatRingElem)
```

Return the matrix over the ring R obtained by coercing the entries of the matrix algebra element A into R. The result belongs to a matrix space rather than a matrix algebra.

source

# Chapter 51

# Developer documentation

## 51.1 Matrix implementation

This page gives an overview of how matrices are implemented in AbstractAlgebra.jl. It is intended for contributors who want to understand the source layout and the relation between the matrix interface, the generic dense matrix implementation, and matrix algebras.

For the list of functions that a new matrix type has to provide, see the matrix interface documentation.

### Source layout

The matrix code is split into interface-level functionality and concrete generic implementations.

The file `src/Matrix.jl` contains functionality for matrices belonging to matrix spaces. This includes parent and element methods, constructors, indexing, iteration, views, arithmetic, basic predicates, block constructions, concatenation, linear algebra routines, and generic fallback algorithms.

The file `src/MatRing.jl` contains functionality for square matrices viewed as elements of a matrix algebra. Matrix algebras are rings, and this file provides the corresponding ring operations and matrix-algebra-specific functionality.

The file `src/generic/Matrix.jl` contains the dense generic implementation for matrices in matrix spaces. These matrices store their entries in a Julia two-dimensional array internally, but they are not themselves Julia arrays.

The file `src/generic/MatRing.jl` contains the dense generic implementation for matrix algebras. A generic matrix algebra element wraps a generic dense matrix and equips it with the parent and ring structure of a matrix algebra.

Additional algorithms are implemented in separate files. For example, `src/Matrix-Strassen.jl` contains Strassen-style and block matrix algorithms, while `src/MatrixNormalForms.jl` contains normal form functionality such as echelon, Hermite, and Howell forms.

### Matrix spaces vs matrix algebras

AbstractAlgebra distinguishes between matrices in matrix spaces and matrices in matrix algebras.

A matrix space contains matrices of fixed size $m \times n$ over a base ring. Its elements belong to the abstract type `MatElem{T}`, where T is the type of the matrix entries. The parent object has type `MatSpace{T}`.

A matrix algebra contains square matrices of fixed degree $n$ over a base ring, with multiplication as the ring multiplication. Its elements belong to the abstract type `MatRingElem{T}`. Matrix algebra parent objects be-

long to `MatRing{T}`. Since matrix multiplication is generally noncommutative, matrix algebras belong to the noncommutative ring hierarchy.

### Implementation of generic matrix spaces

The generic dense matrix implementation is provided by the `Generic` module.

Matrices in generic matrix spaces have type `Generic.MatSpaceElem{T}`. Views of such matrices have type `Generic.MatSpaceView{T}`. Both are subtypes of the abstract type `Generic.Mat{T}`, so generic code can dispatch on `Generic.Mat` when it should accept both ordinary dense matrices and views.

Generic matrix-space elements and views store their entries in Julia array-like storage internally. The base ring is stored with the matrix object, and the number of rows and columns is obtained from the internal storage.

### Implementation of generic matrix algebras

Generic matrix algebra elements have type `Generic.MatRingElem{T}`. Internally, such an element stores a generic matrix and uses it for entry access, mutation, arithmetic, and linear algebra operations.

The parent of a generic matrix algebra element has type `Generic.MatRing{T}`. It stores the base ring and the degree of the algebra.

Generic matrix algebras reuse much of the matrix functionality from `src/Matrix.jl` and `src/generic/Matrix.jl`, but they additionally satisfy the ring interface. In particular, they provide ring operations such as `zero`, `one`, multiplication, promotion, exact division where available, and parent construction through `matrix_ring`.

### Parents

For generic matrices, the parent is not stored as a direct reference in every matrix element. Instead, enough data is stored on the element to reconstruct the parent on demand.

For a matrix in a matrix space, the parent is reconstructed from the base ring and the number of rows and columns. Calling `parent(M)` returns the corresponding matrix space.

For a matrix algebra element, the parent is reconstructed from the base ring and the degree of the underlying square matrix. Calling `parent(A)` returns the corresponding matrix algebra.

This design allows matrices to be constructed directly, without first explicitly constructing the parent object, while still supporting the parent-based model used throughout AbstractAlgebra.

### Relation to the matrix interface

The matrix interface documentation describes what a matrix implementation must provide: required constructors, entry access, mutation, dimensions, views, and optional methods that can be implemented for performance.

This page instead describes the implementation supplied by AbstractAlgebra itself and how the source files fit together. In particular:

- use the matrix interface page when implementing a new matrix type;
- use this page when navigating the AbstractAlgebra matrix source code;
- use the user-facing matrix pages for construction, manipulation, properties, transformations, matrix spaces, and matrix algebras.

# Part X

# Maps

# Chapter 52

# Introduction

Maps in AbstractAlgebra model maps on sets $f : D \to C$ for some domain $D$ and codomain $C$, which have no real limitations except that elements of the codomain and domain be represented by element objects in the system.

Maps $f : D \to C$ in AbstractAlgebra are modeled by Julia objects that are able to be called on a single element $d \in D$ of the domain to yield an element $f(d) \in C$ of the codomain. We say that the map is being applied.

Maps can be constructed from Julia functions, or they can be represented by some other kind of data, e.g. a matrix, or built up from other maps.

Maps in AbstractAlgebra have a domain and codomain, can be applied, composed with other maps. Various special kinds of map provide more functionality.

For details please refer to the Map Interface documentation.

For example, there are functional maps which wrap a Julia function, cached maps which cache values so they do not have to be recomputed each time they are applied to the same inputs and various kinds of maps with inverses, e.g. maps with sections, retractions and full inverses.

The map system uses a complex four parameter `Map` type, however various helper functions are provided to make it easier to work with.

# Chapter 53

# Functional maps

A functional map in AbstractAlgebra is a map which can be applied by evaluating a Julia function or closure. It is represented by a map object that contains such a function/closure, usually in a field called `image_fn`.

All functional maps belong to the map class `FunctionalMap`.

A generic concrete type `Generic.FunctionalMap` is provided by the Generic module to implement a generic functional map type. This allows for functional maps that contain no extra data, other than a Julia function/closure.

Custom map types can also be defined which have map class `FunctionalMap`.

## 53.1 Functional map interface

All functional map types must define their supertypes as in the following example:

```
mutable struct MyFunctionalMap{D, C} <: Map{D, C, FunctionalMap, MyFunctionalMap}
   # some fields
   image_fn::Function
end
```

Of course `MyFunctionalMap` need not be parameterised if the types `D` and `C` of the domain and codomain objects are known.

### Required functions for functional maps

The following functions must be defined for all functional map types or classes:

`AbstractAlgebra.image_fn` – Function.

```
image_fn(f::Map)
```

Return the Julia function or closure that evaluates `f`. Only maps of class `FunctionalMap` are guaranteed to provide it.

source

This function only needs to be provided if the Julia function is not stored in an `image_fn` field of the `MyFunctionalMap` type.

## 53.2 Generic functional maps

The Generic module provides a concrete type `FunctionalMap` which merely keeps track of a Julia function/closure implementing the map.

Such maps can be constructed using the following function:

`AbstractAlgebra.map_from_func` – Method.

```
map_from_func(D, C, image_fn, [inverse_fn])
```

Creates the map `D -> C, x -> image_fn(x)` of type `MapFromFunc` given the callable object `image_fn`. If `inverse_fn` is provided, it is assumed to satisfy `image_fn(inverse_fn(x)) = x` and will be used as the preimage function.

**Examples**

```
julia> f = map_from_func(ZZ, ZZ, x -> x + 1)
Map defined by a Julia function
  from integers
  to integers

julia> f(ZZ(2))
3

julia> F = GF(2);

julia> g = map_from_func(QQ, F, x -> F(numerator(x)) * inv(F(denominator(x))), y -> QQ(lift(y)))
Map defined by a Julia function with inverse
  from rationals
  to finite field F_2

julia> preimage(g, F(1))
1//1
```

source

# Chapter 54

# Cached maps

All basic map (i.e. those not built up from other maps) in AbstractAlgebra can be cached.

A cache is a dictionary that can be switched on and off at run time that keeps a cache of previous evaluations of the map. This can be useful if the map is extremely difficult to evaluate, e.g. a discrete logarithm map. Rather than evaluate the map afresh each time, the map first looks up the dictionary of previous known values of the map.

To facilitate caching of maps, the Generic module provides a type `Generic.MapCache`, which can be used to wrap any existing map object with a dictionary.

Importantly, the supertype of the resulting `Generic.MapCache` object is identical to that of the map being cached. This means that any functions that would accept the original map will also accept the cached version.

**Note**

Caching of maps only works for maps that correctly abstract access to their fields using accessor functions, as described in the map interface.

## 54.1 Cached map constructors

To construct a cached map from an existing map object, we have the following function:

`AbstractAlgebra.cached` – Function.

```
cached(M::Map; limit::Int=100, enabled::Bool=true)
```

Return a cached version of the map `M`, having the same supertype as `M`, that memoises up to `limit` values of `M` in a dictionary. Setting `enabled` to `false` creates the map with its cache switched off; it can be switched on later with `enable_cache!`.

**Examples**

```
julia> f = map_from_func(ZZ, ZZ, x -> x + 1);

julia> g = cached(f);

julia> f(ZZ(1)) == g(ZZ(1))
true
```

source

Constructing a map with its cache disabled allows the user to quickly go through code and completely disable caches of maps that were previously enabled, for testing purposes, etc.

Caches can also be turned on and off at run time (see below).

## 54.2 Functionality for cached maps

The following functions are provided for cached maps.

`AbstractAlgebra.Generic.enable_cache!` – Function.

```
enable_cache!(M::Generic.MapCache)
```

Switch the cache of `M` on. Values stored in the cache while it was disabled are kept.

**Examples**

```
julia> f = cached(map_from_func(ZZ, ZZ, x -> x + 1); enabled=false);

julia> enable_cache!(f)

julia> f(ZZ(1))
2
```

source

`AbstractAlgebra.Generic.disable_cache!` – Function.

```
disable_cache!(M::Generic.MapCache)
```

Switch the cache of `M` off, keeping the values it already stores. See `enable_cache!`.

**Examples**

```
julia> f = cached(map_from_func(ZZ, ZZ, x -> x + 1));

julia> disable_cache!(f)

julia> f(ZZ(1))
2
```

source

`AbstractAlgebra.Generic.set_limit!` – Function.

```
set_limit!(M::Generic.MapCache, limit::Int)
```

Set the number of further values that may be stored in the cache of `M` to `limit`. Setting it to `0` prevents any further values from being cached.

**Examples**

```
julia> f = cached(map_from_func(ZZ, ZZ, x -> x + 1));

julia> set_limit!(f, 200)
200
```

source

# Chapter 55

# Map with inverse

It is not possible to provide generic functionality to invert a map. However, sometimes one knows an inverse map explicitly and would like to keep track of this.

Recall that as map composition is not commutative, there is a notion of a left inverse and a right inverse for maps.

To keep track of such inverse maps, AbstractAlgebra provides data types `Generic.MapWithRetraction` and `Generic.MapWithSection`.

Given a map $f : X \to Y$, a retraction of $f$ is a map $g : Y \to X$ such that $g(f(x)) = x$ for all $x \in X$.

Given a map $f : X \to Y$, a section of $f$ is a map $g : Y \to X$ such that $f(g(x)) = x$ for all $y \in Y$.

In AbstractAlgebra, a map with retraction/section is an object containing a pair of maps, the second of which is a retraction/section of the first.

Maps with retraction/section can be composed, and we also define the inverse of such a pair to be the map with the pair swapped. Thus the inverse of a map with retraction is a map with section.

## 55.1 Map with inverse constructors

To construct a map with retraction/section from a pair of maps, we have the following functions:

`AbstractAlgebra.map_with_retraction` – Function.

```
map_with_retraction(f::Map{D, C}, r::Map{C, D}) where {D, C}
```

Return the map `f` together with a known retraction `r` of it, i.e. a map with $r(f(x)) = x$ for all $x$ in the domain of `f`.

source

`AbstractAlgebra.map_with_section` – Function.

```
map_with_section(f::Map{D, C}, s::Map{C, D}) where {D, C}
```

Return the map `f` together with a known section `s` of it, i.e. a map with $f(s(y)) = y$ for all $y$ in the codomain of `f`.

source

For convenience we allow construction of maps with retraction/section from a pair of Julia functions/closures.

`AbstractAlgebra.map_with_retraction_from_func` – Function.

```
map_with_retraction_from_func(R, S, f::Function, r::Function)
```

Return the map from R to S given by the Julia function `f`, together with the retraction given by the Julia function `r`. See `map_with_retraction`.

**Examples**

```
julia> f = map_with_retraction_from_func(ZZ, ZZ, x -> x + 1, x -> x - 1)
Map with retraction
  from integers
  to integers

julia> f(ZZ(1))
2
```

source

`AbstractAlgebra.map_with_section_from_func` – Function.

```
map_with_section_from_func(R, S, f::Function, s::Function)
```

Return the map from R to S given by the Julia function `f`, together with the section given by the Julia function `s`. See `map_with_section`.

source

## 55.2 Functionality for maps with inverses

The following functionality is provided for maps with inverses.

```
inv(M::Generic.MapWithRetraction)
inv(M::Generic.MapWithSection)
```

Return the map with the two maps contained in $M$ swapped. In the first case, a `MapWithSection` is returned. In the second case a `MapWithRetraction` is returned.

**Examples**

```
julia> f = map_with_retraction_from_func(ZZ, ZZ, x -> x + 1, x -> x - 1)
Map with retraction
  from integers
  to integers

julia> g = inv(f)
Map with section
  from integers
```

```
  to integers

julia> h = f*g
Composite map
  from integers
  to integers
which is the composite of
  Map: integers -> integers
  Map: integers -> integers

julia> h(ZZ(1))
1
```

To access the two maps stored in a map with retraction/section, we have the following:

`AbstractAlgebra.Generic.image_map` – Function.

```
image_map(f::Union{Generic.MapWithSection, Generic.MapWithRetraction})
```

Return the underlying map of `f`, i.e. `f` stripped of its section resp. retraction.

source

`AbstractAlgebra.Generic.retraction_map` – Function.

```
retraction_map(f::Generic.MapWithRetraction)
```

Return the retraction stored in `f`. See `map_with_retraction`.

source

`AbstractAlgebra.Generic.section_map` – Function.

```
section_map(f::Generic.MapWithSection)
```

Return the section stored in `f`. See `map_with_section`.

source

# Part XI

# Miscellaneous

# Chapter 56

# Miscellaneous

## 56.1 Printing options

AbstractAlgebra supports printing to LaTeX using the MIME type "text/latex". To enable LaTeX rendering in Jupyter notebooks and query for the current state, use the following functions:

`AbstractAlgebra.PrettyPrinting.set_html_as_latex` – Function.

```
set_html_as_latex(fl::Bool)
```

Toggles whether MIME type `text/html` should be printed as `text/latex`. Note that this is a global option. The return value is the old value.

source

`AbstractAlgebra.PrettyPrinting.get_html_as_latex` – Function.

```
get_html_as_latex()
```

Returns whether MIME type `text/html` is printed as `text/latex`.

source

## 56.2 Updating the type diagrams

Updating the diagrams of the documentation can be done by modifying and running the script `docs/create_type_diagrams.jl`. Note that this requires the package `Kroki`.

## 56.3 Attributes

Often it is desirable to have a flexible way to attach additional data to mathematical structures such as groups, rings, fields, etc. beyond what the original implementation covers. To facilitate this, we provide an *attributes* system: for objects of suitable types, one may use `set_attribute!` to attach key-value pairs to the object, and query them using `has_attribute`, `get_attribute` and `get_attribute!`.

Attributes are supported for all singletons (i.e., instances of an empty `struct` type), as well as for instances of mutable struct type for which attribute storage was enabled. There are two ways to enable attribute storage for such types:

1. By applying `@attributes` to a mutable struct declaration, storage is reserved inside that struct type itself (this increases the size of each struct by 8 bytes if no attributes are set).

2. By applying `@attributes` to the name of a mutable struct type, methods are installed which store attributes to instances of the type in a `WeakKeyDict` outside the struct.

`AbstractAlgebra.@attributes` – Macro.

```
@attributes typedef
```

This is a helper macro that ensures that there is storage for attributes in the type declared in the expression `typedef`, which must be either a `mutable struct` definition expression, or the name of a `mutable struct` type.

The latter variant is useful to enable attribute storage for types defined in other packages. Note that `@attributes` is idempotent: when applied to a type for which attribute storage is already available, it does nothing.

For singleton types, attribute storage is also supported, and in fact always enabled. Thus it is not necessary to apply this macro to such a type.

**Note**

When applied to a struct definition this macro adds a new field to the struct. For structs without constructor, this will change the signature of the default inner constructor, which requires explicit values for every field, including the attribute storage field this macro adds. Usually it is thus preferable to add an explicit default constructor, as in the example below.

**Examples**

Applying the macro to a struct definition results in internal storage of the attributes:

```
julia> @attributes mutable struct MyGroup
           order::Int
           MyGroup(order::Int) = new(order)
       end

julia> G = MyGroup(5)
MyGroup(5, #undef)

julia> set_attribute!(G, :isfinite, :true)

julia> get_attribute(G, :isfinite)
true
```

Applying the macro to a typename results in external storage of the attributes:

```
julia> mutable struct MyOtherGroup
           order::Int
           MyOtherGroup(order::Int) = new(order)
       end

julia> @attributes MyOtherGroup
```

```
julia> G = MyOtherGroup(5)
MyOtherGroup(5)

julia> set_attribute!(G, :isfinite, :true)

julia> get_attribute(G, :isfinite)
true
```

source

AbstractAlgebra.@attr – Macro.

```
@attr RetType funcdef
```

This macro is applied to the definition of a unary function, and enables caching ("memoization") of its return values based on the argument. This assumes the argument supports attribute storing (see @attributes) via get_attribute!.

The name of the function is used as name for the underlying attribute.

The macro works the same for unary functions with keyword arguments, but ignores the keyword arguments when caching the result, i.e. different calls with different keyword arguments will return the identical (cached) result. In case that there is no result cached yet, the function is called with the given keyword arguments.

Effectively, this turns code like this:

```
@attr RetType function myattr(obj::Foo)
   # ... expensive computation
   return result
end
```

into something essentially equivalent to this:

```
function myattr(obj::Foo)
  return get_attribute!(obj, :myattr) do
    # ... expensive computation
    return result
  end::RetType
end
```

**Examples**

```
julia> @attributes mutable struct Foo
           x::Int
           Foo(x::Int) = new(x)
       end;

julia> @attr Int function myattr(obj::Foo)
```

```
                println("Performing expensive computation")
                return factorial(obj.x)
            end;

julia> obj = Foo(5);

julia> myattr(obj)
Performing expensive computation
120

julia> myattr(obj) # second time uses the cached result
120
```

source

AbstractAlgebra.has_attribute – Function.

```
has_attribute(G::Any, attr::Symbol)
```

Return a boolean indicating whether G has a value stored for the attribute attr.

source

AbstractAlgebra.get_attribute – Function.

```
get_attribute(f::Function, G::Any, attr::Symbol)
```

Return the value stored for the attribute attr, or if no value has been set, return f().

This is intended to be called using do block syntax.

```
get_attribute(obj, attr) do
    # default value calculated here if needed
    ...
end
```

source

```
get_attribute(G::Any, attr::Symbol, default::Any = nothing)
```

Return the value stored for the attribute attr, or if no value has been set, return default.

source

AbstractAlgebra.get_attribute! – Function.

```
get_attribute!(f::Function, G::Any, attr::Symbol)
```

Return the value stored for the attribute `attr` of `G`, or if no value has been set, store `key => f()` and return `f()`.

This is intended to be called using `do` block syntax.

```
get_attribute!(obj, attr) do
    # default value calculated here if needed
    ...
end
```

source

```
get_attribute!(G::Any, attr::Symbol, default::Any)
```

Return the value stored for the attribute `attr` of `G`, or if no value has been set, store `key => default`, and return `default`.

source

`AbstractAlgebra.set_attribute!` – Function.

```
set_attribute!(G::Any, data::Pair{Symbol, <:Any}...)
```

Attach the given sequence of `key=>value` pairs as attributes of `G`.

source

```
set_attribute!(G::Any, attr::Symbol, value::Any)
```

Attach the given `value` as attribute `attr` of `G`.

source

`AbstractAlgebra.is_attribute_storing` – Function.

```
is_attribute_storing(G::Any)
```

Return a boolean indicating whether `G` has the ability to store attributes.

source

`AbstractAlgebra.is_attribute_storing_type` – Function.

```
is_attribute_storing_type(T::Type)
```

Return a boolean indicating whether instances of type `T` have the ability to store attributes.

source

## 56.4 Advanced printing

### Self-given names

We provide macros `@show_name`, `@show_special` and `@show_special_elem` to change the way certain objects are printed.

In compact and terse printing mode, `@show_name` tries to determine a suitable name to print instead of the object (see `AbstractAlgebra.get_name`).

`@show_special` checks if an attribute `:show` is present. If so, it has to be a function taking `IO`, optionally a MIME-type, and the object. This is then called instead of the usual `show` function.

Similarly, `@show_special_elem` checks if an attribute `:show_elem` is present in the object's parent. The semantics are the same as for `@show_special`.

All are supposed to be used within the usual `show` function, where `@show_special_elem` is only relevant for element types of algebraic structures.

```
@attributes MyObj

function show(io::IO, A::MyObj)
   @show_name(io, A)
   @show_special(io, A)

   # ... usual stuff
end

function show(io::IO, mime::MIME"text/plain", A::MyObj)
   @show_name(io, A)
   @show_special(io, mime, A)

   # ... usual stuff
end

function show(io::IO, A::MyObjElem)
   @show_name(io, A)
   @show_special_elem(io, A)

   # ... usual stuff
end

function show(io::IO, mime::MIME"text/plain", A::MyObjElem)
   @show_name(io, A)
   @show_special_elem(io, mime, A)

   # ... usual stuff
end
```

#### Documentation

`AbstractAlgebra.PrettyPrinting.@show_special` – Macro.

```
@show_special(io::IO, obj)
```

If the `obj` has a `show` attribute, this gets called with `io` and `obj` and returns from the current scope. Otherwise, does nothing.

If `obj` does not have attribute storage available, this macro does nothing.

It is supposed to be used at the start of show methods as shown in the documentation.

**Examples**

```
julia> R = @polynomial_ring(QQ, :x; cached=false)
Univariate polynomial ring in x over rationals

julia> AbstractAlgebra.@show_special(stdout, R)

julia> set_attribute!(R, :show, (i,o) -> print(i, "=> The One True Ring <="))

julia> AbstractAlgebra.@show_special(stdout, R)
=> The One True Ring <=

julia> R   # show for R uses @show_special, so we can observe the effect directly
=> The One True Ring <=
```

source

```
@show_special(io::IO, mime, obj)
```

If the `obj` has a `show` attribute, this gets called with `io`, `mime` and `obj` (if applicable) and `io` and `obj` otherwise, and returns from the current scope. Otherwise, does nothing.

If `obj` does not have attribute storage available, this macro does nothing.

It is supposed to be used at the start of show methods as shown in the documentation.

**Examples**

```
julia> R = @polynomial_ring(QQ, :x; cached=false)
Univariate polynomial ring in x over rationals

julia> AbstractAlgebra.@show_special(stdout, MIME"text/plain"(), R)

julia> myshow(i,o) = print(i, "=> The One True Ring <=");

julia> myshow(i,m,o) = print(i, "=> The One True Ring with mime type $m <=");

julia> set_attribute!(R, :show, myshow)

julia> AbstractAlgebra.@show_special(stdout, MIME"text/plain"(), R)
=> The One True Ring with mime type text/plain <=

julia> R   # show for R uses @show_special, so we can observe the effect directly
=> The One True Ring <=
```

source

`AbstractAlgebra.PrettyPrinting.@show_special_elem` – Macro.

```
@show_special_elem(io::IO, obj)
```

If the `parent` of `obj` has a `show_elem` attribute, this gets called with `io` and `obj` and returns from the current scope. Otherwise, does nothing.

If `parent(obj)` does not have attribute storage available, this macro does nothing.

It is supposed to be used at the start of `show` methods as shown in the documentation.

**Examples**

```
julia> R = @polynomial_ring(QQ, :x; cached=false)
Univariate polynomial ring in x over rationals

julia> AbstractAlgebra.@show_special_elem(stdout, x)

julia> set_attribute!(R, :show_elem, (i,o) -> print(i, "=> $o <="))

julia> AbstractAlgebra.@show_special_elem(stdout, x)
=> x <=

julia> x   # show for x does not uses @show_special_elem, so x prints as before
x
```

source

```
@show_special_elem(io::IO, mime, obj)
```

If the `parent` of `obj` has a `show_elem` attribute, this gets called with `io`, `mime` and `obj` (if applicable) and `io` and `obj` otherwise, and returns from the current scope. Otherwise, does nothing.

If `parent(obj)` does not have attribute storage available, this macro does nothing.

It is supposed to be used at the start of `show` methods as shown in the documentation.

**Examples**

```
julia> R = @polynomial_ring(QQ, :x; cached=false)
Univariate polynomial ring in x over rationals

julia> AbstractAlgebra.@show_special_elem(stdout, MIME"text/plain"(), x)

julia> set_attribute!(R, :show_elem, (i,m,o) -> print(i, "=> $o with mime type $m <="))

julia> AbstractAlgebra.@show_special_elem(stdout, MIME"text/plain"(), x)
=> x with mime type text/plain <=

julia> x   # show for x does not uses @show_special_elem, so x prints as before
x
```

source

`AbstractAlgebra.PrettyPrinting.@show_name` – Macro.

```
@show_name(io::IO, obj)
```

If either `is_terse(io)` is true or property `:compact` is set to `true` for `io` (see `IOContext`), print the name `get_name(obj)` of the object `obj` to the `io` stream, then return from the current scope. Otherwise, do nothing.

It is supposed to be used at the start of show methods as shown in the documentation.

source

`AbstractAlgebra.PrettyPrinting.get_name` – Function.

```
get_name(obj) -> Union{String,Nothing}
```

Returns the name of the object `obj` if it is set, or `nothing` otherwise. This function tries to find a name in the following order:

1. The name set by `AbstractAlgebra.set_name!`.
2. The name of a variable in global (`Main` module) namespace with value bound to the object `obj` (see `AbstractAlgebra.PrettyPrinting.find_name`).
3. The name returned by `AbstractAlgebra.extra_name`.

source

`AbstractAlgebra.PrettyPrinting.set_name!` – Function.

```
set_name!(obj, name::String; override::Bool=true)
```

Sets the name of the object `obj` to `name`. This name is used for printing using `AbstractAlgebra.@show_name`. If `override` is `false`, the name is only set if there is no name already set.

This function errors if `obj` does not support attribute storage.

source

```
set_name!(obj; override::Bool=true)
```

Sets the name of the object `obj` to the name of a variable in global (`Main` module) namespace with value bound to the object `obj`, if such a variable exists (see `AbstractAlgebra.PrettyPrinting.find_name`). This name is used for printing using `AbstractAlgebra.@show_name`. If `override` is `false`, the name is only set if there is no name already set.

This function errors if `obj` does not support attribute storage.

source

`AbstractAlgebra.PrettyPrinting.extra_name` – Function.

```
extra_name(obj) -> Union{String,Nothing}
```

May be overloaded to provide a fallback name for the object `obj` in `AbstractAlgebra.get_name`. The default implementation returns `nothing`.

source

`AbstractAlgebra.PrettyPrinting.find_name` – Function.

```
find_name(obj, M = Main; all::Bool = false) -> Union{String,Nothing}
```

Return name of a variable in `M`'s namespace with value bound to the object `obj`, or `nothing` if no such variable exists. If `all` is `true`, private and non-exported variables are also searched.

**Note**

If the object is stored in several variables, the first one will be used, but a name returned once is kept until the variable no longer contains this object.

For this to work in doctests, one should call `AbstractAlgebra.set_current_module(@__MODULE__)` in the `value` argument of `Documenter.DocMeta.setdocmeta!` and keep the default value of `M = Main` here.

**Warning**

This function should not be used directly, but rather through `AbstractAlgebra.get_name`.

source

### Indentation and Decapitalization

To facilitate printing of nested mathematical structures, we provide a modified `IOCustom` object, that supports indentation and decapitalization.

#### Example

We illustrate this with an example

```
struct A{T}
  x::T
end

function Base.show(io::IO, a::A)
  io = AbstractAlgebra.pretty(io)
  println(io, "Something of type A")
  print(io, AbstractAlgebra.Indent(), "over ", AbstractAlgebra.Lowercase(), a.x)
  print(io, AbstractAlgebra.Dedent()) # don't forget to undo the indentation!
end

struct B
end

function Base.show(io::IO, b::B)
```

```
  io = AbstractAlgebra.pretty(io)
  print(io, LowercaseOff(), "Hilbert thing")
end
```

At the REPL, this will then be printed as follows:

```
julia> A(2)
Something of type A
  over 2

julia> A(A(2))
Something of type A
  over something of type A
    over 2

julia> A(B())
Something of type A
  over Hilbert thing
```

**Documentation**

`AbstractAlgebra.PrettyPrinting.pretty` – Function.

```
pretty(io::IO) -> IOCustom
```

Wrap `io` into an `IOCustom` object.

**Examples**

```
julia> io = AbstractAlgebra.pretty(stdout);
```

source

`AbstractAlgebra.PrettyPrinting.Indent` – Type.

```
Indent
```

When printed to an `IOCustom` object, increases the indentation level by one.

**Examples**

```
julia> io = AbstractAlgebra.pretty(stdout);

julia> print(io, AbstractAlgebra.Indent(), "This is indented")
  This is indented
```

source

`AbstractAlgebra.PrettyPrinting.Dedent` – Type.

```
Dedent
```

When printed to an `IOCustom` object, decreases the indentation level by one.

**Examples**

```
julia> io = AbstractAlgebra.pretty(stdout);

julia> print(io, AbstractAlgebra.Indent(), AbstractAlgebra.Dedent(), "This is indented")
This is indented
```

source

`AbstractAlgebra.PrettyPrinting.Lowercase` – Type.

```
Lowercase
```

When printed to an `IOCustom` object, the next letter printed will be lowercase.

**Examples**

```
julia> io = AbstractAlgebra.pretty(stdout);

julia> print(io, AbstractAlgebra.Lowercase(), "Foo")
foo
```

source

`AbstractAlgebra.PrettyPrinting.LowercaseOff` – Type.

```
LowercaseOff
```

When printed to an `IOCustom` object, the case of the next letter will not be changed when printed.

**Examples**

```
julia> io = AbstractAlgebra.pretty(stdout);

julia> print(io, AbstractAlgebra.Lowercase(), AbstractAlgebra.LowercaseOff(), "Foo")
Foo
```

source

`AbstractAlgebra.PrettyPrinting.terse` – Function.

```
terse(io::IO) -> IO
```

Return a new IO objects derived from `io` for which "terse" printing mode has been enabled.

See https://docs.oscar-system.org/stable/DeveloperDocumentation/printing_details/ for details.

**Examples**

```
julia> AbstractAlgebra.is_terse(stdout)
false

julia> io = AbstractAlgebra.terse(stdout);

julia> AbstractAlgebra.is_terse(io)
true
```

source

`AbstractAlgebra.PrettyPrinting.is_terse` – Function.

```
is_terse(io::IO) -> Bool
```

Test whether "terse" printing mode is enabled for `io`.

See https://docs.oscar-system.org/stable/DeveloperDocumentation/printing_details/ for details.

**Examples**

```
julia> AbstractAlgebra.is_terse(stdout)
false

julia> io = AbstractAlgebra.terse(stdout);

julia> AbstractAlgebra.is_terse(io)
true
```

source

## 56.5 Linear solving interface for developers

AbstractAlgebra has a generic interface for linear solving and we describe here how one may extend this interface. For the user-facing functionality of linear solving, see Linear Solving.

Notice that the functionality is implemented in the module `AbstractAlgebra.Solve` and the internal functions are not exported from there.

### Matrix normal forms

To distinguish between different algorithms, we use type traits of abstract type `MatrixNormalFormTrait` which usually correspond to a certain matrix normal form. The available algorithms/normal forms are

- `HowellFormTrait`: uses a Howell form;
- `HermiteFormTrait`: uses a Hermite normal form;
- `RREFTrait`: uses a row-reduced echelon form over fields;
- `LUTrait`: uses a LU factoring of the matrix;
- `FFLUTrait`: uses a "fraction-free" LU factoring of the matrix over fraction fields;
- `MatrixInterpolateTrait`: uses interpolation of polynomials for fraction fields of polynomial rings.

To select a normal form type for rings of type `NewRing`, implement the function

```
Solve.matrix_normal_form_type(::NewRing) = Bla()
```

where `Bla <: MatrixNormalFormTrait`. A new type trait can be added via

```
struct NewTrait <: Solve.MatrixNormalFormTrait end
```

### Internal solving functionality

If a new ring type `NewRing` can make use of one of the available `MatrixNormalFormTraits`, then it suffices to specify this normal form as described above to use the generic solving functionality. (However, for example `HermiteFormTrait` requires that the function `hermite_form_with_transformation` is implemented.)

For a new trait `NewTrait <: MatrixNormalFormTrait`, one needs to implement the function

```
Solve._can_solve_internal_no_check(
  ::NewTrait, A::MatElem{T}, b::MatElem{T}, task::Symbol; side::Symbol = :left
  ) where T
```

Inside this function, one can assume that `A` and `b` have the same base ring and have compatible dimensions. Further, `task` and `side` are set to "legal" options. (All this is checked in `Solve._can_solve_internal`.) This function should then (try to) solve `Ax = b` (`side == :right`) or `xA = b` (`side == :left`) possibly with kernel. The function must always return a tuple `(::Bool, ::MatElem{T}, ::MatElem{T})` consisting of:

- `true`/`false` whether a solution exists or not
- the solution (or a placeholder if no solution exists or a solution is not requested)
- the kernel (or a placeholder if the kernel is not requested)

The input `task` may be:

- `:only_check`: Only test whether there is a solution, the second and third return value are only for type stability;
- `:with_solution`: Compute a solution, if it exists, the last return value is only for type stability;
- `:with_kernel`: Compute a solution and a kernel.

One should further implement the function

```
kernel(::NewTrait, A::MatElem; side::Symbol = :left)
```

which computes a left (or right) kernel of A.

### Internal solve context functionality

To efficiently solve several linear systems with the same matrix A, we provide the "solve contexts objects" of type `Solve.SolveCtx`. These can be extended for a ring of type `NewRing` as follows.

#### Solve context type

For a new ring type, one may have to define the type parameters of a `Solve.SolveCtx` object. First of all, one needs to implement the function

```
function Solve.solve_context_type(::NewRing)
  return Solve.solve_context_type(::NormalFormTrait, elem_type(NewRing))
end
```

to pick a `MatrixNormalFormTrait`.

Usually, nothing else should be necessary. However, if for example the normal form of a matrix does not live over the same ring as the matrix itself, one might also need to implement

```
function Solve.solve_context_type(NF::NormalFormTrait, T::Type{NewRingElem})
  return Solve.SolveCtx{T, typeof(NF), MatType, RedMatType, TranspMatType}
end
```

where `MatType` is the dense matrix type over `NewRing`, `RedMatType` the type of a matrix in reduced/normal form and `TranspMatType` the type of the reduced/normal form of the transposed matrix.

#### Initialization

To initialize the solve context functionality for a new normal form `NewTrait`, one needs to implement the functions

```
Solve._init_reduce(C::Solve.SolveCtx{T, NewTrait}) where T
Solve._init_reduce_transpose(C::Solve.SolveCtx{T, NewTrait}) where T
```

These should fill the corresponding fields of the solve context `C` with a "reduced matrix" (that is, a matrix in normal form) of `matrix(C)`, respectively `transpose(matrix(C))`, and other information necessary to solve a linear system. The fields can be accessed via `reduced_matrix`, `reduced_matrix_of_transpose`, etc. New fields may also be added via attributes.

#### Internal solving functionality

As above, one finally needs to implement the functions

```
Solve._can_solve_internal_no_check(
  ::NewTrait, C::Solve.SolveCtx{T, NewTrait}, b::MatElem{T}, task::Symbol;
  side::Symbol = :left
  ) where T
```

and

```
kernel(::NewTrait, C::Solve.SolveCtx{T, NewTrait}; side::Symbol = :left)
```

## 56.6 Miscellaneous

`AbstractAlgebra.is_pairwise` – Function.

```
is_pairwise(f, a::AbstractVector)
```

Return whether `f(a[i], a[j])` is true for all `i != j`. It is assumed that `f` is symmetric.

**Examples**

```
julia> is_pairwise(!is_associated, [2, -3, 5])
true

julia> is_pairwise(!is_associated, [2, -3, -2])
false
```

source

# Chapter 57

# Assertion and Verbosity Macros

We describe here various macros provided by AbstractAlgebra.

## 57.1 Verbosity macros

There is a (global) list of symbols called *verbosity scopes* which represent keywords used to trigger some verbosity macros within the code. Each of these verbosity scopes has its own integer *verbosity level*, which is set to $0$ by default. A verbosity macro call must specify the verbosity scope S and optionally the trigger level k (defaulting to $1$) such that, if the current verbosity level `l` of S is bigger than or equal to k, then the macro triggers a given action. Inside a module, the function `add_verbosity_scope` must be called in the `__init__` function of that module.

`AbstractAlgebra.add_verbosity_scope` – Method.

```
AbstractAlgebra.add_verbosity_scope(s::Symbol) -> Nothing
```

Add the symbol s to the list of (global) verbosity scopes. For use inside a module this function must be called in the `__init__` function of that module; the verbosity scope is then available to all functions in the module.

**Examples**

```
julia> AbstractAlgebra.add_verbosity_scope(:MyScope)
```

source

`AbstractAlgebra.set_verbosity_level` – Method.

```
AbstractAlgebra.set_verbosity_level(s::Symbol, l::Int) -> Int
```

If s represents a known verbosity scope, set the current verbosity level of s to `l`.

One can access the current verbosity level of s by calling the function `get_verbosity_level`.

If s is not yet known as a verbosity scope, the function raises an `ErrorException` showing the error message "Not a valid symbol". One can add s to the list of verbosity scopes by calling the function `add_verbosity_scope`.

**Examples**

```
julia> AbstractAlgebra.add_verbosity_scope(:MyScope)

julia> AbstractAlgebra.set_verbosity_level(:MyScope, 4)
4

julia> AbstractAlgebra.set_verbosity_level(:MyScope, 0)
0
```

source

`AbstractAlgebra.get_verbosity_level` – Method.

```
AbstractAlgebra.get_verbosity_level(s::Symbol) -> Int
```

If s represents a known verbosity scope, return the current verbosity level of s.

One can modify the current verbosity level of s by calling the function `set_verbosity_level`.

If s is not yet known as a verbosity scope, the function raises an `ErrorException` showing the error message "Not a valid symbol". One can add s to the list of verbosity scopes by calling the function `add_verbosity_scope`.

**Examples**

```
julia> AbstractAlgebra.add_verbosity_scope(:MyScope)

julia> AbstractAlgebra.get_verbosity_level(:MyScope)
0

julia> AbstractAlgebra.set_verbosity_level(:MyScope, 4)
4

julia> AbstractAlgebra.get_verbosity_level(:MyScope)
4

julia> AbstractAlgebra.set_verbosity_level(:MyScope, 0)
0

julia> AbstractAlgebra.get_verbosity_level(:MyScope)
0
```

source

## Printings

`AbstractAlgebra.@vprintln` – Macro.

```
@vprintln(S::Symbol, k::Int, msg::String)
@vprintln S k msg
```

```
@vprintln(S::Symbol, msg::String)
@vprintln S msg
```

This macro can be used to control printings inside the code.

The macro `@vprintln` takes two or three arguments: a symbol S specifying a *verbosity scope*, an optional integer k and a string `msg`. If k is not specified, it is set by default to $1$.

To each verbosity scope S is associated a *verbosity level* `l` which is cached. If the verbosity level $l$ of S is bigger than or equal to $k$, the macro `@vprintln` triggers the printing of the associated string `msg` followed by a newline.

One can add a new verbosity scope by calling the function `add_verbosity_scope`.

When starting a new instance, all the verbosity levels are set to $0$. One can adjust the verbosity level of a verbosity scope by calling the function `set_verbosity_level`.

One can access the current verbosity level of a verbosity scope by calling the function `get_verbosity_level`.

**Examples**

We will set up different verbosity scopes with different verbosity levels in a custom function to show how to use this macro.

```
julia> AbstractAlgebra.add_verbosity_scope(:Test1);

julia> AbstractAlgebra.add_verbosity_scope(:Test2);

julia> AbstractAlgebra.add_verbosity_scope(:Test3);

julia> AbstractAlgebra.set_verbosity_level(:Test1, 1);

julia> AbstractAlgebra.set_verbosity_level(:Test2, 3);

julia> function vprint_example()
       @vprintln :Test1 "Triggered"
       @vprintln :Test2 2 "Triggered"
       @vprintln :Test3 "Not triggered"
       @vprintln :Test2 4 "Not triggered"
       end
vprint_example (generic function with 1 method)

julia> vprint_example()
Triggered
Triggered
```

If one does not setup in advance a verbosity scope, the macro will raise an `ExceptionError` showing the error message "Not a valid symbol".

source

`AbstractAlgebra.@vprint` – Macro.

```
@vprint(S::Symbol, k::Int, msg::String)
@vprint S k msg

@vprint(S::Symbol, msg::String)
@vprint S msg
```

The same as `@vprintln`, but without the final newline.

source

### Actions

`AbstractAlgebra.@v_do` – Macro.

```
@v_do(S::Symbol, k::Int, act::Expr)
@v_do S k act

@v_do(S::Symbol, act::Expr)
@v_do S act
```

This macro can be used to control actions inside the code.

The macro `@v_do` takes two or three arguments: a symbol `S` specifying a *verbosity scope*, an optional integer `k` and an action `act`. If `k` is not specified, it is set by default to $1$.

To each verbosity scope `S` is associated a *verbosity level* `l`. If the verbosity level $l$ of `S` is bigger than or equal to $k$, the macro `@v_do` triggers the action `act`.

One can add a new verbosity scope by calling the function `add_verbosity_scope`.

When starting a new instance, all the verbosity levels are set to $0$. One can adjust the verbosity level of a verbosity scope by calling the function `set_verbosity_level`.

One can access the current verbosity level of a verbosity scope by calling the function `get_verbosity_level`.

#### Examples

We will set up different verbosity scopes with different verbosity levels in a custom function to show how to use this macro.

```
julia> AbstractAlgebra.add_verbosity_scope(:Test1);

julia> AbstractAlgebra.add_verbosity_scope(:Test2);

julia> AbstractAlgebra.add_verbosity_scope(:Test3);

julia> AbstractAlgebra.set_verbosity_level(:Test1, 1);

julia> AbstractAlgebra.set_verbosity_level(:Test2, 3);

julia> function v_do_example(a::Int, b::Int, c::Int, d::Int)
       @v_do :Test1 a = 2*a
       @v_do :Test2 2 b = 3*b
       @v_do :Test3 c = 4*c
```

```
        @v_do :Test2 4 d = 5*d
        return (a, b, c, d)
        end
v_do_example (generic function with 1 method)

julia> v_do_example(1,1,1,1)
(2, 3, 1, 1)
```

If one does not setup in advance a verbosity scope, the macro will raise an `ExceptionError` showing the error message "Not a valid symbol".

source

## 57.2 Assertion macros

There is a list of symbols called *assertion scopes* which represent keywords used to trigger some particular macros within the codes. Each of these assertion scopes is associated with an *assertion level*, being set to $0$ by default. An assertion macro is joined to an assertion scope S and a value k (set to $1$ by default) such that, if the current assertion level `l` of S is bigger than or equal to k, then the macro triggers an action on the given assertion

`AbstractAlgebra.add_assertion_scope` – Method.

```
AbstractAlgebra.add_assertion_scope(s::Symbol) -> Nothing
```

Add the symbol s to the list of (global) assertion scopes.

**Examples**

```
julia> AbstractAlgebra.add_assertion_scope(:MyScope)
```

source

`AbstractAlgebra.set_assertion_level` – Method.

```
AbstractAlgebra.set_assertion_level(s::Symbol, l::Int) -> Int
```

If s represents a known assertion scope, set the current assertion level of s to `l`.

One can access the current assertion level of s by calling the function `get_assertion_level`.

If s is not yet known as an assertion scope, the function raises an `ErrorException` showing the error message "Not a valid symbol". One can add s to the list of assertion scopes by calling the function `add_assertion_scope`.

**Examples**

```
julia> AbstractAlgebra.add_assertion_scope(:MyScope)

julia> AbstractAlgebra.set_assertion_level(:MyScope, 4)
4

julia> AbstractAlgebra.set_assertion_level(:MyScope, 0)
0
```

source

AbstractAlgebra.get_assertion_level – Method.

```
AbstractAlgebra.get_assertion_level(s::Symbol) -> Int
```

If s represents a symbol of a known assertion scope, return the current assertion level of s.

One can modify the current assertion level of s by calling the function set_assertion_level.

If s is not yet known as an assertion scope, the function raises an ErrorException showing the error message "Not a valid symbol". One can add s to the list of assertion scopes by calling the function add_assertion_scope.

**Examples**

```
julia> AbstractAlgebra.add_assertion_scope(:MyScope)

julia> AbstractAlgebra.get_assertion_level(:MyScope)
0

julia> AbstractAlgebra.set_assertion_level(:MyScope, 1)
1

julia> AbstractAlgebra.get_assertion_level(:MyScope)
1

julia> AbstractAlgebra.set_assertion_level(:MyScope, 0)
0

julia> AbstractAlgebra.get_assertion_level(:MyScope)
0
```

source

### Check

AbstractAlgebra.@hassert – Macro.

```
@hassert(S::Symbol, k::Int, assert::Expr)
@hassert S k assert

@hassert(S::Symbol, assert::Expr)
@hassert S assert
```

This macro can be used to control assertion checks inside the code.

The macro @hassert takes two or three arguments: a symbol S specifying an *assertion scope*, an optional integer k and an assertion assert. If k is not specified, it is set by default to $1$.

To each assertion scope S is associated an *assertion level* l which is cached. If the assertion level $l$ of S is bigger than or equal to $k$, the macro @hassert triggers the check of the assertion assert. If assert is wrong, an AssertionError is thrown.

One can add a new assertion scope by calling the function add_assertion_scope.

When starting a new instance, all the assertion levels are set to $0$. One can adjust the assertion level of an assertion scope by calling the function set_assertion_level.

One can access the current assertion level of an assertion scope by calling the function get_assertion_level.

**Examples**

We will set up different assertion scopes with different assertion levels in a custom function to show how to use this macro.

```
julia> AbstractAlgebra.add_assertion_scope(:MyScope)

julia> AbstractAlgebra.get_assertion_level(:MyScope)
0

julia> function hassert_test(x::Int)
       @hassert :MyScope 700 mod(x, 3) == 0
       return div(x, 3)
       end
hassert_test (generic function with 1 method)

julia> hassert_test(2)
0

julia> AbstractAlgebra.set_assertion_level(:MyScope, 701);

julia> try hassert_test(2)
       catch e e
       end
AssertionError("\$(Expr(:escape, :(mod(x, 3) == 0)))")

julia> hassert_test(3)
1

julia> AbstractAlgebra.set_assertion_level(:MyScope, 0)
0
```

If one does not setup in advance an assertion scope, the macro will raise an ExceptionError showing the error message "Not a valid symbol".

source

## 57.3 Miscellaneous

AbstractAlgebra.@req – Macro.

```
@req(assert, msg)
@req assert msg
```

Check whether the assertion `assert` is true. If not, throw an `ArgumentError` with error message `msg`.

The macro `@req` takes two arguments: the first one is an assertion `assert` (an expression which returns a boolean) and a string `msg` corresponding to the desired error message to be returned whenever `assert` is false.

If the number of arguments is not 2, an `AssertionError` is raised.

**Examples**

```
julia> function req_test(x::Int)
       @req iseven(x) "x must be even"
       return div(x,2)
       end
req_test (generic function with 1 method)

julia> try req_test(3)
       catch e e
       end
ArgumentError("x must be even")

julia> try req_test(2)
       catch e e
       end
1
```

source

# Part XII

# Interfaces

# Chapter 58

# Introduction

AbstractAlgebra defines a series of interfaces that can be extended with new types that implement those interfaces. For example, if one were implementing a new polynomial ring type, one would implement all of the required functionality described in this chapter for the relevant AbstractAlgebra interfaces. This would include the Ring Interface and the Univariate Polynomial Ring Interface.

Once a new type implements all the required functionality, all the corresponding generic functionality would then function automatically for the new type.

One may then go on to implement some of the optional functionality for performance if the provided generic functionality is insufficient.

AbstractAlgebra tries to provide all generic constructions recursively so that one can have towers of generic constructions. This means that new interfaces should generally only be added if they cooperate with all the existing interfaces, at least so far as the theory exists to do so.

# Chapter 59

# Ring Interface

AbstractAlgebra.jl generic code makes use of a standardised set of functions which it expects to be implemented for all rings. Here we document this interface. All libraries which want to make use of the generic capabilities of AbstractAlgebra.jl must supply all of the required functionality for their rings.

In addition to the required functions, there are also optional functions which can be provided for certain types of rings, e.g. GCD domains or fields, etc. If implemented, these allow the generic code to provide additional functionality for those rings, or in some cases, to select more efficient algorithms.

## 59.1 Types

Most rings must supply two types:

- a type for the parent object (representing the ring itself)
- a type for elements of that ring

For example, the generic univariate polynomial type in AbstractAlgebra.jl provides two types in generic/GenericTypes.jl:

- `Generic.PolyRing{T}` for the parent objects
- `Generic.Poly{T}` for the actual polynomials

The parent type must belong to `Ring` and the element type must belong to `RingElem`. Of course, the types may belong to these abstract types transitively, e.g. `Poly{T}` actually belongs to `PolyRingElem{T}` which in turn belongs to `RingElem`.

For parameterised rings, we advise that the types of both the parent objects and element objects to be parameterised by the types of the elements of the base ring (see the function `base_ring` below for a definition).

There can be variations on this theme: e.g. in some areas of mathematics there is a notion of a coefficient domain, in which case it may make sense to parameterise all types by the type of elements of this coefficient domain. But note that this may have implications for the ad hoc operators one might like to explicitly implement.

## 59.2 RingElement type union

Because of its lack of multiple inheritance, Julia does not allow Julia Base types to belong to `RingElem`. To allow us to work equally with AbstractAlgebra and Julia types that represent elements of rings we define a union type `RingElement` in `src/julia/JuliaTypes`.

So far, in addition to `RingElem` the union type `RingElement` includes the Julia types `Integer`, `Rational` and `AbstractFloat`.

Most of the generic code in AbstractAlgebra makes use of the union type `RingElement` instead of `RingElem` so that the generic functions also accept the Julia Base ring types.

**Note**

One must be careful when defining ad hoc binary operations for ring element types. It is often necessary to define separate versions of the functions for `RingElem` then for each of the Julia types separately in order to avoid ambiguity warnings.

Note that even though `RingElement` is a union type we still have the following inclusion

```
RingElement <: NCRingElement
```

## 59.3 Parent object caches

In many cases, it is desirable to have only one object in the system to represent each ring. This means that if the same ring is constructed twice, elements of the two rings will be compatible as far as arithmetic is concerned.

In order to facilitate this, global caches of rings are stored in AbstractAlgebra.jl, usually implemented using dictionaries. For example, the `Generic.PolyRing` parent objects are looked up in a dictionary `PolyID` to see if they have been previously defined.

Whether these global caches are provided or not, depends on both mathematical and algorithmic considerations. E.g. in the case of number fields, it isn't desirable to identify all number fields with the same defining polynomial, as they may be considered with distinct embeddings into one another. In other cases, identifying whether two rings are the same may be prohibitively expensive. Generally, it may only make sense algorithmically to identify two rings if they were constructed from identical data.

If a global cache is provided, it must be optionally possible to construct the parent objects without caching. This is done by passing a boolean value cached to the inner constructor of the parent object. See `src/generic/GenericTypes.jl` for examples of how to construct and handle such caches.

## 59.4 Required functions for all rings

In the following, we list all the functions that are required to be provided for rings in AbstractAlgebra.jl or by external libraries wanting to use AbstractAlgebra.jl.

We give this interface for fictitious types `MyParent` for the type of the ring parent object `R` and `MyElem` for the type of the elements of the ring.

**Note**

Generic functions in AbstractAlgebra.jl may not rely on the existence of functions that are not documented here. If they do, those functions will only be available for rings that implement that additional functionality, and should be documented as such.

### Data type and parent object methods

```
parent_type(::Type{MyElem})
```

Return the type of the corresponding parent object for the given element type. For example, `parent_type(Generic.Poly{T})` will return `Generic.PolyRing{T}`.

```
elem_type(::Type{MyParent})
```

Return the type of the elements of the ring whose parent object has the given type. This is the inverse of the `parent_type` function, i.e. `elem_type(Generic.PolyRing{T})` will return `Generic.Poly{T}`.

```
base_ring_type(::Type{MyParent})
```

Return the type of the of base rings for parent objects with the given parent type. For example, `base_ring_type(Generic.PolyRi`
will return `parent_type(T)`.

If the rings of this type are not parameterised by another ring, this function must return `Union{}`.

```
base_ring(R::MyParent)
```

Given a parent object R, representing a ring, this function returns the parent object of any base ring that parameterises this ring. For example, the base ring of the ring of polynomials over the integers would be the integer ring.

If the ring is not parameterised by another ring, calling this function should rais an error.

**Note**

There is a distinction between a base ring and other kinds of parameters. For example, in the ring $\mathbb{Z}/n\mathbb{Z}$, the modulus $n$ is a parameter, but the only base ring is $\mathbb{Z}$. We consider the ring $\mathbb{Z}/n\mathbb{Z}$ to have been constructed from the base ring $\mathbb{Z}$ by taking its quotient by a (principal) ideal.

There is no general mathematical definition for base ring, so to know what `base_ring` does for any given type of ring, you need to consult its documentation.

```
parent(f::MyElem)
```

Return the parent object of the given element, i.e. return the ring to which the given element belongs.

This is usually stored in a field `parent` in each ring element. (If the parent objects have `mutable struct` types, the internal overhead here is just an additional machine pointer stored in each element of the ring.)

For some element types it isn't necessary to append the parent object as a field of every element. This is the case when the parent object can be reconstructed just given the type of the elements. For example, this is the case for the ring of integers and in fact for any ring element type that isn't parameterised or generic in any way.

```
is_domain_type(::Type{MyElem})
```

Return `true` if every element of the given element type (which may be parameterised or an abstract type) necessarily has a parent that is an integral domain, otherwise if this cannot be guaranteed, the function returns `false`.

For example, if `MyElem` was the type of elements of generic residue rings of a polynomial ring, the answer to the question would depend on the modulus of the residue ring. Therefore `is_domain_type` would have to return `false`, since we cannot guarantee that we are dealing with elements of an integral domain in general. But if the given element type was for rational integers, the answer would be `true`, since every rational integer has as parent the ring of rational integers, which is an integral domain.

Note that this function depends only on the type of an element and cannot access information about the object itself, or its parent.

```
is_exact_type(::Type{MyElem})
```

Return `true` if every element of the given type is represented exactly. For example, $p$-adic numbers, real and complex floating point numbers and power series are not exact, as we can only represent them in general with finite truncations. Similarly polynomials and matrices over inexact element types are themselves inexact.

Integers, rationals, finite fields and polynomials and matrices over them are always exact.

Note that `MyElem` may be parameterised or an abstract type, in which case every element of every type represented by `MyElem` must be exact, otherwise the function must return `false`.

```
Base.hash(f::MyElem, h::UInt)
```

Return a hash for the object $f$ of type `UInt`. This is used as a hopefully cheap way to distinguish objects that differ arithmetically.

If the object has components, e.g. the coefficients of a polynomial or elements of a matrix, these should be hashed recursively, passing the same parameter `h` to all levels. Each component should then be xor'd with `h` before combining the individual component hashes to give the final hash.

The hash functions in AbstractAlgebra.jl usually start from some fixed 64 bit hexadecimal value that has been picked at random by the library author for that type. That is then truncated to fit a `UInt` (in case the latter is not 64 bits). This ensures that objects that are the same arithmetically (or that have the same components), but have different types (or structures), are unlikely to hash to the same value.

```
deepcopy_internal(f::MyElem, dict::IdDict)
```

Return a copy of the given element, recursively copying all components of the object.

Obviously the parent, if it is stored in the element, should not be copied. The new element should have precisely the same parent as the old object.

For types that cannot self-reference themselves anywhere internally, the `dict` argument may be ignored.

In the case that internal self-references are possible, please consult the Julia documentation on how to implement `deepcopy_internal`.

### Constructors

Outer constructors for most AbstractAlgebra types are provided by overloading the call syntax for parent objects.

If R is a parent object for a given ring we require the following constructors.

```
(R::MyParent)()
```

Return the zero object of the given ring.

```
(R::MyParent)(a::Integer)
```

Coerce the given integer into the given ring.

```
(R::MyParent)(a::MyElem)
```

If $a$ belongs to the given ring, the function returns it (without making a copy). Otherwise an error is thrown.

For parameterised rings we also require a function to coerce from the base ring into the parent ring.

```
(R::MyParent{T})(a::T) where T <: RingElem
```

Coerce $a$ into the ring $R$ if $a$ belongs to the base ring of $R$.

### Basic manipulation of rings and elements

```
zero(R::MyParent)
```

Return the zero element of the given ring.

```
one(R::MyParent)
```

Return the multiplicative identity of the given ring.

```
iszero(f::MyElem)
```

Return `true` if the given element is the zero element of the ring it belongs to.

```
isone(f::MyElem)
```

Return `true` if the given element is the multiplicative identity of the ring it belongs to.

### Canonicalisation

```
canonical_unit(f::MyElem)
```

When fractions are created with two elements of the given type, it is nice to be able to represent them in some kind of canonical form. This is of course not always possible. But for example, fractions of integers can be canonicalised by first removing any common factors of the numerator and denominator, then making the denominator positive.

In AbstractAlgebra.jl, the denominator would be made positive by dividing both the numerator and denominator by the canonical unit of the denominator. For a negative denominator, this would be $-1$.

For non-zero elements of a field, `canonical_unit` simply returns the element itself. In general, `canonical_unit` of an invertible element should be that element. Finally, if $a = bc$, then `canonical_unit(a)*a = canonical_unit(b)*canonical` holds. Thus if $a$ is not a zero-divisor, then we even have `canonical_unit(a) = canonical_unit(b)*canonical_unit(c)`.

For some rings, it is completely impractical to implement this function, in which case it may return $1$ in the given ring. The function must however always exist, and always return an element of the ring.

### String I/O

```
show(io::IO, R::MyParent)
```

This should print an English description of the parent ring (to the given IO object). If the ring is parameterised, it can call the corresponding `show` function for any rings it depends on.

```
show(io::IO, f::MyElem)
```

This should print a human readable, textual representation of the object (to the given IO object). It can recursively call the corresponding `show` functions for any of its components.

### Expressions

To obtain best results when printing composed types derived from other types, e.g., polynomials, the following method should be implemented.

`AbstractAlgebra.PrettyPrinting.expressify` – Function.

```
expressify(a; context = nothing)
```

Return an `Expr`, `Symbol`, `Integer` or `String` describing a, from which plain text, LaTeX and html output are derived. The fallback simply prints a.

Implementing this for a new element type, and then applying `@enable_all_show_via_expressify` to that type, gives correctly bracketed output when values of the type appear inside composed types such as polynomials.

source

`AbstractAlgebra.PrettyPrinting.@enable_all_show_via_expressify` – Macro.

```
@enable_all_show_via_expressify(T)
```

Define Base.show for the type T for plain text, LaTeX and html output, all derived from expressify, which must already be implemented for T.

source

@enable_all_show_via_expressify MyElem defines the following show methods for the specified type MyElem:

```
function Base.show(io::IO, a::MyElem)
  show_via_expressify(io, a)
end

function Base.show(io::IO, mi::MIME"text/plain", a::MyElem)
  show_via_expressify(io, mi, a)
end

function Base.show(io::IO, mi::MIME"text/latex", a::MyElem)
  show_via_expressify(io, mi, a)
end

function Base.show(io::IO, mi::MIME"text/html", a::MyElem)
  show_via_expressify(io, mi, a)
end
```

As an example, assume that an object f of type MyElem has two components f.a and f.b of integer type, which should be printed as a^b, this can be implemented as

```
expressify(f::MyElem; context = nothing) = Expr(:call, :^, f.a, f.b)
```

If f.a and f.b themselves are objects that can be expressified, this can be implemented as

```
function expressify(f::MyElem; context = nothing)
  return Expr(:call, :^, expressify(f.a, context = context),
                         expressify(f.b, context = context))
end
```

As noted above, expressify should return an Expr, Symbol, Integer or String. The rendering of such expressions with a particular MIME type to an output context is controlled by the following rules which are subject to change slightly in future versions of AbstracAlgebra.

Integer: The printing of integers is straightforward and automatically includes transformations such as 1 + (-2)*x => 1 - 2*x as this is cumbersome to implement per-type.

Symbol: Since variable names are stored as mere symbols in AbstractAlgebra, some transformations related to subscripts are applied to symbols automatically in latex output. The \operatorname{ in the following table is actually replaced with the more portable \mathop{\mathrm{.

Expr: These are the most versatile as the Expr objects themselves contain a symbolic head and any number of arguments. What looks like f(a,b) in textual output is Expr(:call, :f, :a, :b) under the hood. AbstractAlgebra currently contains the following printing rules for such expressions.

String: Strings are printed verbatim and should only be used as a last resort as they provide absolutely no precedence information on their contents.

| expressify | latex output |
|---|---|
| `Symbol("a")` | `a` |
| `Symbol("α")` | `{\alpha}` |
| `Symbol("x1")` | `\operatorname{x1}` |
| `Symbol("xy_1")` | `\operatorname{xy}_{1}` |
| `Symbol("sin")` | `\operatorname{sin}` |
| `Symbol("sin_cos")` | `\operatorname{sin\_cos}` |
| `Symbol("sin_1")` | `\operatorname{sin}_{1}` |
| `Symbol("sin_cos_1")` | `\operatorname{sin\_cos}_{1}` |
| `Symbol("αaβb_1_2")` | `\operatorname{{\alpha}a{\beta}b}_{1,2}` |

| expressify | output | latex notes |
|---|---|---|
| `Expr(:call, :+, a, b)` | `a + b` | |
| `Expr(:call, :*, a, b)` | `a*b` | one space for implied multiplication |
| `Expr(:call, :cdot, a, b)` | `a * b` | a real `\cdot` is used |
| `Expr(:call, :^, a, b)` | `a^b` | may include some courtesy parentheses |
| `Expr(:call, ://, a, b)` | `a//b` | will create a fraction box |
| `Expr(:call, :/, a, b)` | `a/b` | will not create a fraction box |
| `Expr(:call, a, b, c)` | `a(b, c)` | |
| `Expr(:ref, a, b, c)` | `a[b, c]` | |
| `Expr(:vcat, a, b)` | `[a; b]` | actually vertical |
| `Expr(:vect, a, b)` | `[a, b]` | |
| `Expr(:tuple, a, b)` | `(a, b)` | |
| `Expr(:list, a, b)` | `{a, b}` | |
| `Expr(:series, a, b)` | `a, b` | |
| `Expr(:sequence, a, b)` | `ab` | |
| `Expr(:row, a, b)` | `a b` | combine with `:vcat` to make matrices |
| `Expr(:hcat, a, b)` | `a b` | |

### Unary operations

```
-(f::MyElem)
```

Return $-f$.

### Binary operations

```
+(f::MyElem, g::MyElem)
-(f::MyElem, g::MyElem)
*(f::MyElem, g::MyElem)
```

Return $f + g$, $f - g$ or $fg$, respectively.

### Comparison

```
==(f::MyElem, g::MyElem)
```

Return `true` if $f$ and $g$ are arithmetically equal. In the case where the two elements are inexact, the function returns `true` if they agree to the minimum precision of the two.

```
isequal(f::MyElem, g::MyElem)
```

For exact rings, this should return the same thing as == above. For inexact rings, this returns `true` only if the two elements are arithmetically equal and have the same precision.

### Powering

```
^(f::MyElem, e::Int)
```

Return $f^e$. The function should throw a `DomainError()` if negative exponents don't make sense but are passed to the function.

### Exact division

```
divexact(f::MyElem, g::MyElem; check::Bool=true)
```

Return $f/g$, though note that Julia uses `/` for floating point division. Here we mean exact division in the ring, i.e. return $q$ such that $f = gq$. A `DivideError()` should be thrown if $g$ is zero.

If `check=true` the function should check that the division is exact and throw an exception if not.

If `check=false` the check may be omitted for performance reasons. The behaviour is then undefined if a division is performed that is not exact. This may include throwing an exception, returning meaningless results, hanging or crashing. The function should only be called with `check=false` if it is already known that the division will be exact.

### Inverse

```
inv(f::MyElem)
```

Return the inverse of $f$, i.e. $1/f$, though note that Julia uses `/` for floating point division. Here we mean exact division in the ring.

A fallback for this function is provided in terms of `divexact` so an implementation can be omitted if preferred.

### Random generation

The random functions are only used for test code to generate test data. They therefore don't need to provide any guarantees on uniformity, and in fact, test values that are known to be a good source of corner cases can be supplied.

```
rand(R::MyParent, v...)
```

Return a random element in the given ring of the specified size.

There can be as many arguments as is necessary to specify the size of the test example which is being produced.

### Promotion rules

AbstractAlgebra currently has a very simple coercion model. With few exceptions only simple coercions are supported. For example if $x \in \mathbb{Z}$ and $y \in \mathbb{Z}[x]$ then $x + y$ can be computed by coercing $x$ into the same ring as $y$ and then adding in that ring.

Complex coercions such as adding elements of $\mathbb{Q}$ and $\mathbb{Z}[x]$ are not supported, as this would require finding and creating a common overring in which the elements could be added.

AbstractAlgebra supports simple coercions by overloading parent object call syntax `R(x)` to coerce the object x into the ring R. However, to coerce elements up a tower of rings, one needs to also have a promotion system similar to Julia's type promotion system.

As for Julia, AbstractAlgebra's promotion system only specifies what happens to types. It is the coercions themselves that must deal with the mathematical situation at the level of rings, including checking that the object can even be coerced into the given ring.

Note that `AbstractAlgebra.promote_rule` is a separate function from `Base.promote_rule` and is the one that must be extended.

We now describe the required AbstractAlgebra type promotion rules.

For every ring, one wants to be able to coerce integers into the ring. And for any ring constructed over a base ring, one would like to be able to coerce from the base ring into the ring.

The required promotion rules to support this look a bit different depending on whether the element type is parameterised or not and whether it is built on a base ring.

For ring element types `MyElem` that are neither parameterised nor built over a base ring, the promotion rules can be defined as follows:

```
AbstractAlgebra.promote_rule(::Type{MyElem}, ::Type{T}) where {T <: Integer} = MyElem
```

For ring element types `MyElem` that aren't parameterised, but which have a base ring with concrete element type T the promotion rules can be defined as follows:

```
AbstractAlgebra.promote_rule(::Type{MyElem}, ::Type{U}) where U <: Integer = MyElem
```

```
AbstractAlgebra.promote_rule(::Type{MyElem}, ::Type{T}) = MyElem
```

For ring element types `MyElem{T}` that are parameterised by the type of elements of the base ring, the promotion rules can be defined as follows:

```
AbstractAlgebra.promote_rule(::Type{MyElem{T}}, ::Type{MyElem{T}}) where T <: RingElement =
↪  MyElem{T}
```

```
function AbstractAlgebra.promote_rule(::Type{MyElem{T}}, ::Type{U}) where {T <: RingElement, U <:
↪  RingElement}
   AbstractAlgebra.promote_rule(T, U) == T ? MyElem{T} : Union{}
end
```

## 59.5 Required functionality for inexact rings

### Approximation (floating point and ball arithmetic only)

```
isapprox(f::MyElem, g::MyElem; atol::Real=sqrt(eps()))
```

This is used by test code that uses rings involving floating point or ball arithmetic. The function should return `true` if all components of $f$ and $g$ are equal to within the square root of the Julia epsilon, since numerical noise may make an exact comparison impossible.

For parameterised rings over an inexact ring, we also require the following ad hoc approximation functionality.

```
isapprox(f::MyElem{T}, g::T; atol::Real=sqrt(eps())) where T <: RingElem
```

```
isapprox(f::T, g::MyElem{T}; atol::Real=sqrt(eps())) where T <: RingElem
```

These notionally coerce the element of the base ring into the parameterised ring and do a full comparison.

## 59.6 Optional functionality

Some functionality is difficult or impossible to implement for all rings in the system. If it is provided, additional functionality or performance may become available. Here is a list of all functions that are considered optional and can't be relied on by generic functions in the AbstractAlgebra Ring interface.

It may be that no algorithm, or no efficient algorithm is known to implement these functions. As these functions are optional, they do not need to exist. Julia will already inform the user that the function has not been implemented if it is called but doesn't exist.

### Optional unsafe operators

The various operators described in Unsafe ring operators such as `add!` and `mul!` have default implementations which are not faster than their regular safe counterparts. Implementors may wish to implement some or all of them for their rings. Note that in general only the variants with the most arguments needs to be implemented. E.g. for `add!` only `add(z,a,b)` has to be implemented for any new ring type, as `add!(a,b)` delegates to `add!(a,a,b)`.

### Optional basic manipulation functionality

```
is_unit(f::MyElem)
```

Return `true` if the given element is a unit in the ring it belongs to.

```
is_zero_divisor(f::MyElem)
```

Return `true` if the given element is a zero divisor in the ring it belongs to. When this function does not exist for a given ring then the total ring of fractions may not be usable over that ring. All fields in the system have a fallback defined for this function.

```
characteristic(R::MyParent)
```

Return the characteristic of the ring. The function should not be defined if it is not possible to unconditionally give the characteristic. AbstractAlgebra will raise an exception is such cases.

### Optional binary ad hoc operators

By default, ad hoc operations are handled by AbstractAlgebra.jl if they are not defined explicitly, by coercing both operands into the same ring and then performing the required operation.

In some cases, e.g. for matrices, this leads to very inefficient behaviour. In such cases, it is advised to implement some of these operators explicitly.

It can occasionally be worth adding a separate set of ad hoc binary operators for the type `Int`, if this can be done more efficiently than for arbitrary Julia Integer types.

```
+(f::MyElem, c::Integer)
-(f::MyElem, c::Integer)
*(f::MyElem, c::Integer)
```

```
+(c::Integer, f::MyElem)
-(c::Integer, f::MyElem)
*(c::Integer, f::MyElem)
```

For parameterised types, it is also sometimes more performant to provide explicit ad hoc operators with elements of the base ring.

```
+(f::MyElem{T}, c::T) where T <: RingElem
-(f::MyElem{T}, c::T) where T <: RingElem
*(f::MyElem{T}, c::T) where T <: RingElem
```

```
+(c::T, f::MyElem{T}) where T <: RingElem
-(c::T, f::MyElem{T}) where T <: RingElem
*(c::T, f::MyElem{T}) where T <: RingElem
```

### Optional ad hoc comparisons

```
==(f::MyElem, c::Integer)
```

```
==(c::Integer, f::MyElem)
```

```
==(f::MyElem{T}, c:T) where T <: RingElem
```

```
==(c::T, f::MyElem{T}) where T <: RingElem
```

### Optional ad hoc exact division functions

```
divexact(a::MyElem{T}, b::T) where T <: RingElem
```

```
divexact(a::MyElem, b::Integer)
```

### Optional powering functions

```
^(f::MyElem, e::BigInt)
```

In case $f$ cannot explode in size when powered by a very large integer, and it is practical to do so, one may provide this function to support powering with `BigInt` exponents (or for external modules, any other big integer type).

### Optional unsafe operators

```
addmul!(c::MyElem, a::MyElem, b::MyElem, t::MyElem)
```

Set $c = c + ab$ in-place. Return the mutated value. The value $t$ should be a temporary of the same type as $a$, $b$ and $c$, which can be used arbitrarily by the implementation to speed up the computation. Aliasing between $a$, $b$ and $c$ is permitted.

## 59.7 Minimal example of ring implementation

Here is a minimal example of implementing the Ring Interface for a constant polynomial type (i.e. polynomials of degree less than one).

```
# ConstPoly.jl : Implements constant polynomials

using AbstractAlgebra

using Random: Random, SamplerTrivial
using AbstractAlgebra.RandomExtensions: RandomExtensions, Make2, AbstractRNG

import AbstractAlgebra: parent_type, elem_type, base_ring, base_ring_type, parent, is_domain_type,
       is_exact_type, canonical_unit, isequal, divexact, zero!, mul!, add!,
       get_cached!, is_unit, characteristic, Ring, RingElem, expressify,
       @show_name, @show_special, is_terse, pretty, terse, Lowercase,
       promote_rule

import Base: show, +, -, *, ^, ==, inv, isone, iszero, one, zero, rand,
             deepcopy_internal, hash

@attributes mutable struct ConstPolyRing{T <: RingElement} <: Ring
   base_ring::Ring

   function ConstPolyRing{T}(R::Ring, cached::Bool) where T <: RingElement
      return get_cached!(ConstPolyID, R, cached) do
         new{T}(R)
      end::ConstPolyRing{T}
   end
end

const ConstPolyID = AbstractAlgebra.CacheDictType{Ring, ConstPolyRing}()

mutable struct ConstPoly{T <: RingElement} <: RingElem
   c::T
   parent::ConstPolyRing{T}

   function ConstPoly{T}(c::T) where T <: RingElement
      return new(c)
   end
end

# Data type and parent object methods

parent_type(::Type{ConstPoly{T}}) where T <: RingElement = ConstPolyRing{T}

elem_type(::Type{ConstPolyRing{T}}) where T <: RingElement = ConstPoly{T}

base_ring_type(::Type{ConstPolyRing{T}}) where T <: RingElement = parent_type(T)

base_ring(R::ConstPolyRing) = R.base_ring::base_ring_type(R)

parent(f::ConstPoly) = f.parent

is_domain_type(::Type{ConstPoly{T}}) where T <: RingElement = is_domain_type(T)

is_exact_type(::Type{ConstPoly{T}}) where T <: RingElement = is_exact_type(T)

function hash(f::ConstPoly, h::UInt)
   r = 0x65125ab8e0cd44ca
```

```
   return xor(r, hash(f.c, h))
end

function deepcopy_internal(f::ConstPoly{T}, dict::IdDict) where T <: RingElement
   r = ConstPoly{T}(deepcopy_internal(f.c, dict))
   r.parent = f.parent # parent should not be deepcopied
   return r
end

# Basic manipulation

zero(R::ConstPolyRing) = R()

one(R::ConstPolyRing) = R(1)

iszero(f::ConstPoly) = iszero(f.c)

isone(f::ConstPoly) = isone(f.c)

is_unit(f::ConstPoly) = is_unit(f.c)

characteristic(R::ConstPolyRing) = characteristic(base_ring(R))

# Canonical unit

canonical_unit(f::ConstPoly) = canonical_unit(f.c)

# String I/O

function show(io::IO, R::ConstPolyRing)
   @show_name(io, R)
   @show_special(io, R)
   print(io, "Constant polynomials")
   if !is_terse(io)
     io = pretty(io)
     print(terse(io), " over ", Lowercase(), base_ring(R))
   end
end

function show(io::IO, f::ConstPoly)
   print(io, f.c)
end

# Expressification (optional)

function expressify(R::ConstPolyRing; context = nothing)
   return Expr(:sequence, Expr(:text, "Constant polynomials over "),
                          expressify(base_ring(R), context = context))
end

function expressify(f::ConstPoly; context = nothing)
   return expressify(f.c, context = context)
end

# Unary operations
```

```
function -(f::ConstPoly)
   R = parent(f)
   return R(-f.c)
end

# Binary operations

function +(f::ConstPoly{T}, g::ConstPoly{T}) where T <: RingElement
   check_parent(f, g)
   R = parent(f)
   return R(f.c + g.c)
end

function -(f::ConstPoly{T}, g::ConstPoly{T}) where T <: RingElement
   check_parent(f, g)
   R = parent(f)
   return R(f.c - g.c)
end

function *(f::ConstPoly{T}, g::ConstPoly{T}) where T <: RingElement
   check_parent(f, g)
   R = parent(f)
   return R(f.c*g.c)
end

# Comparison

function ==(f::ConstPoly{T}, g::ConstPoly{T}) where T <: RingElement
   check_parent(f, g)
   return f.c == g.c
end

function isequal(f::ConstPoly{T}, g::ConstPoly{T}) where T <: RingElement
   check_parent(f, g)
   return isequal(f.c, g.c)
end

# Powering need not be implemented if * is

# Exact division

function divexact(f::ConstPoly{T}, g::ConstPoly{T}; check::Bool = true) where T <: RingElement
   check_parent(f, g)
   R = parent(f)
   return R(divexact(f.c, g.c, check = check))
end

# Inverse

function inv(f::ConstPoly)
   R = parent(f)
   return R(AbstractAlgebra.inv(f.c))
end
```

```
# Unsafe operators

function zero!(f::ConstPoly)
   f.c = zero(base_ring(parent(f)))
   return f
end

function one!(f::ConstPoly)
   f.c = one(base_ring(parent(f)))
   return f
end

function mul!(f::ConstPoly{T}, g::ConstPoly{T}, h::ConstPoly{T}) where T <: RingElement
   f.c = g.c*h.c
   return f
end

function add!(f::ConstPoly{T}, g::ConstPoly{T}, h::ConstPoly{T}) where T <: RingElement
   f.c = g.c + h.c
   return f
end

# Random generation

RandomExtensions.maketype(R::ConstPolyRing, _) = elem_type(R)

rand(rng::AbstractRNG, sp::SamplerTrivial{<:Make2{ConstPoly,ConstPolyRing}}) =
        sp[][1](rand(rng, sp[][2]))

rand(rng::AbstractRNG, R::ConstPolyRing, n::AbstractUnitRange{Int}) = R(rand(rng, n))

rand(R::ConstPolyRing, n::AbstractUnitRange{Int}) = rand(Random.default_rng(), R, n)

# Promotion rules

AbstractAlgebra.promote_rule(::Type{ConstPoly{T}}, ::Type{ConstPoly{T}}) where T <: RingElement =
↪  ConstPoly{T}

function AbstractAlgebra.promote_rule(::Type{ConstPoly{T}}, ::Type{U}) where {T <: RingElement, U
↪  <: RingElement}
   AbstractAlgebra.promote_rule(T, U) == T ? ConstPoly{T} : Union{}
end

# Constructors

function (R::ConstPolyRing{T})() where T <: RingElement
   r = ConstPoly{T}(base_ring(R)(0))
   r.parent = R
   return r
end

function (R::ConstPolyRing{T})(c::Integer) where T <: RingElement
   r = ConstPoly{T}(base_ring(R)(c))
   r.parent = R
   return r
```

```
end

# Needed to prevent ambiguity
function (R::ConstPolyRing{T})(c::T) where T <: Integer
   r = ConstPoly{T}(base_ring(R)(c))
   r.parent = R
   return r
end

function (R::ConstPolyRing{T})(c::T) where T <: RingElement
   base_ring(R) != parent(c) && error("Unable to coerce element")
   r = ConstPoly{T}(c)
   r.parent = R
   return r
end

function (R::ConstPolyRing{T})(f::ConstPoly{T}) where T <: RingElement
   R != parent(f) && error("Unable to coerce element")
   return f
end

# Parent constructor

function constant_polynomial_ring(R::Ring, cached::Bool=true)
   T = elem_type(R)
   return ConstPolyRing{T}(R, cached)
end

# output

constant_polynomial_ring (generic function with 2 methods)
```

The above implementation of `constant_polynomial_ring` may be tested as follows.

```
using Test

function ConformanceTests.generate_element(R::ConstPolyRing{elem_type(ZZ)})
   n = rand(1:999)
   return R(rand(-n:n))
end

test_Ring_interface(constant_polynomial_ring(ZZ))

# output
Test Summary:                                                                | Pass  Total
↪  Time
Ring interface for Constant polynomials over integers of type ConstPolyRing{BigInt} | 13844  13844
↪  0.9s
```

Note that we only showed a minimal implementation of the ring interface. Additional interfaces exists, e.g. for Euclidean rings. Additional interface usually require implementing additional methods, and in some cases we also provide additional conformance tests. In this case, just one necessary method is missing.

```
function Base.divrem(a::ConstPoly{elem_type(ZZ)}, b::ConstPoly{elem_type(ZZ)})
   check_parent(a, b)
   q, r = AbstractAlgebra.divrem(a.c, b.c)
   return parent(a)(q), parent(a)(r)
end

# output
```

We can test it like this.

```
test_EuclideanRing_interface(constant_polynomial_ring(ZZ))

# output
Test Summary:                                                                                  |
↪  Pass  Total  Time
Euclidean Ring interface for Constant polynomials over integers of type ConstPolyRing{BigInt} |
↪  2212   2212  0.1s
```

# Chapter 60

# Euclidean Ring Interface

If a ring provides a meaningful Euclidean structure such that a useful Euclidean remainder can be computed practically, various additional functionality is provided by AbstractAlgebra.jl for those rings. This functionality depends on the following functions existing. An implementation must provide `divrem`, and the remaining are optional as generic fallbacks exist.

`Base.divrem` – Method.

```
divrem(f::T, g::T) where T <: RingElem
```

Return a pair `q, r` consisting of the Euclidean quotient and remainder of $f$ by $g$. A `DivideError` should be thrown if $g$ is zero.

source

`Base.mod` – Method.

```
mod(f::T, g::T) where T <: RingElem
```

Return the Euclidean remainder of $f$ by $g$. A `DivideError` should be thrown if $g$ is zero.

**Note**

For best compatibility with the internal assumptions made by AbstractAlgebra, the Euclidean remainder function should provide unique representatives for the residue classes; the `mod` function should satisfy

1. `mod(a_1, b) = mod(a_2, b)` if and only if $b$ divides $a_1 - a_2$, and
2. `mod(0, b) = 0`.

source

`Base.div` – Method.

```
div(f::T, g::T) where T <: RingElem
```

Return the Euclidean quotient of $f$ by $g$. A `DivideError` should be thrown if $g$ is zero.

source

`AbstractAlgebra.mulmod` – Method.

```
mulmod(f::T, g::T, m::T) where T <: RingElem
```

Return `mod(f*g, m)` but possibly computed more efficiently.

source

`Base.powermod` – Method.

```
powermod(f::T, e::Int, m::T) where T <: RingElem
```

Return `mod(f^e, m)` but possibly computed more efficiently.

source

`Base.invmod` – Method.

```
invmod(f::T, m::T) where T <: RingElem
```

Return an inverse of $f$ modulo $m$, meaning that `isone(mod(invmod(f,m)*f,m))` returns `true`.

If such an inverse doesn't exist, a `NotInvertibleError` should be thrown.

source

`AbstractAlgebra.divides` – Method.

```
divides(f::T, g::T) where T <: RingElem
```

Return a pair, `flag, q`, where `flag` is set to `true` if $g$ divides $f$, in which case `q` is set to the quotient, or `flag` is set to `false` and `q` is undefined.

source

`AbstractAlgebra.is_divisible_by` – Method.

```
is_divisible_by(x::T, y::T) where T <: RingElem
```

Check if `x` is divisible by `y`, i.e. if $x = zy$ for some $z$.

source

`AbstractAlgebra.is_associated` – Method.

```
is_associated(x::T, y::T) where T <: RingElem
```

Check if x and y are associated, i.e. if x is a unit times y.

source

`AbstractAlgebra.remove` – Method.

```
remove(f::T, p::T) where T <: RingElem
```

Return a pair `v, q` where $p^v$ is the highest power of $p$ dividing $f$ and $q$ is the cofactor after $f$ is divided by this power.

See also `valuation`, which only returns the valuation.

source

`AbstractAlgebra.valuation` – Method.

```
valuation(f::T, p::T) where T <: RingElem
```

Return v where $p^v$ is the highest power of $p$ dividing $f$.

See also `remove`.

source

`Base.gcd` – Method.

```
gcd(a::T, b::T) where T <: RingElem
```

Return a greatest common divisor of $a$ and $b$, i.e., an element $g$ which is a common divisor of $a$ and $b$, and with the property that any other common divisor of $a$ and $b$ divides $g$.

**Note**

For best compatibility with the internal assumptions made by AbstractAlgebra, the return is expected to be unit-normalized in such a way that if the return is a unit, that unit should be one.

source

`Base.gcd` – Method.

```
gcd(f::T, g::T, hs::T...) where T <: RingElem
```

Return a greatest common divisor of $f$, $g$ and the elements in `hs`.

source

`Base.gcd` – Method.

```
gcd(fs::AbstractArray{<:T}) where T <: RingElem
```

Return a greatest common divisor of the elements in `fs`. Requires that `fs` is not empty.

source

`Base.lcm` – Method.

```
lcm(f::T, g::T) where T <: RingElem
```

Return a least common multiple of $f$ and $g$, i.e., an element $d$ which is a common multiple of $f$ and $g$, and with the property that any other common multiple of $f$ and $g$ is a multiple of $d$.

source

`Base.lcm` – Method.

```
lcm(f::T, g::T, hs::T...) where T <: RingElem
```

Return a least common multiple of $f$, $g$ and the elements in `hs`.

source

`Base.lcm` – Method.

```
lcm(fs::AbstractArray{<:T}) where T <: RingElem
```

Return a least common multiple of the elements in `fs`. Requires that `fs` is not empty.

source

`Base.gcdx` – Method.

```
gcdx(f::T, g::T) where T <: RingElem
```

Return a triple `d, s, t` such that $d = gcd(f, g)$ and $d = sf + tg$, with $s$ loosely reduced modulo $g/d$ and $t$ loosely reduced modulo $f/d$.

source

`AbstractAlgebra.gcdinv` – Method.

```
gcdinv(f::T, g::T) where T <: RingElem
```

Return a tuple `d, s` such that $d = gcd(f, g)$ and $s = (f/d)^{-1} \pmod{g/d}$. Note that $d = 1$ iff $f$ is invertible modulo $g$, in which case $s = f^{-1} \pmod{g}$.

source

`AbstractAlgebra.crt` – Method.

```
crt(r1::T, m1::T, r2::T, m2::T; check::Bool=true) where T <: RingElement
```

Return an element congruent to $r_1$ modulo $m_1$ and $r_2$ modulo $m_2$. If `check = true` and no solution exists, an error is thrown.

If T is a fixed precision integer type (like `Int`), the result will be correct if `abs(ri) <= abs(mi)` and `abs(m1 * m2) < typemax(T)`.

source

`AbstractAlgebra.crt` – Method.

```
crt(r::Vector{T}, m::Vector{T}; check::Bool=true) where T <: RingElement
```

Return an element congruent to $r_i$ modulo $m_i$ for each $i$.

source

`AbstractAlgebra.crt_with_lcm` – Method.

```
crt_with_lcm(r1::T, m1::T, r2::T, m2::T; check::Bool=true) where T <: RingElement
```

Return a tuple consisting of an element congruent to $r_1$ modulo $m_1$ and $r_2$ modulo $m_2$ and the least common multiple of $m_1$ and $m_2$. If `check = true` and no solution exists, an error is thrown.

source

`AbstractAlgebra.crt_with_lcm` – Method.

```
crt_with_lcm(r::Vector{T}, m::Vector{T}; check::Bool=true) where T <: RingElement
```

Return a tuple consisting of an element congruent to $r_i$ modulo $m_i$ for each $i$ and the least common multiple of the $m_i$.

source

`AbstractAlgebra.coprime_base` – Function.

```
coprime_base(S::Vector{RingElement}) -> Vector{RingElement}
```

Returns a coprime base for $S$, i.e. the resulting array contains pairwise coprime objects that multiplicatively generate the same set as the input array.

source

`AbstractAlgebra.coprime_base_push!` – Function.

```
coprime_base_push!(S::Vector{RingElem}, a::RingElem) -> Vector{RingElem}
```

Given an array $S$ of coprime elements, insert a new element, that is, find a coprime base for `push(S, a)`.

source

# Chapter 61

# Univariate Polynomial Ring Interface

Univariate polynomial rings are supported in AbstractAlgebra, and in addition to the standard Ring interface, numerous additional functions are required to be present for univariate polynomial rings.

Univariate polynomial rings can be built over both commutative and noncommutative rings.

Univariate polynomial rings over a field are also Euclidean and therefore such rings must implement the Euclidean interface.

Since a sparse distributed multivariate format can generally also handle sparse univariate polynomials, the univariate polynomial interface is designed around the assumption that they are dense. This is not a requirement, but it may be easier to use the multivariate interface for sparse univariate types.

## 61.1 Types and parents

AbstractAlgebra provides two abstract types for polynomial rings and their elements over a commutative ring:

- `PolyRing{T}` is the abstract type for univariate polynomial ring parent types
- `PolyRingElem{T}` is the abstract type for univariate polynomial types

Similarly there are two abstract types for polynomial rings and their elements over a noncommutative ring:

- `NCPolyRing{T}` is the abstract type for univariate polynomial ring parent types
- `NCPolyRingElem{T}` is the abstract type for univariate polynomial types

We have that `PolyRing{T} <: Ring` and `PolyRingElem{T} <: RingElem`. Similarly we have that `NCPolyRing{T} <: NCRing` and `NCPolyRingElem{T} <: NCRingElem`.

Note that the abstract types are parameterised. The type T should usually be the type of elements of the coefficient ring of the polynomial ring. For example, in the case of $\mathbb{Z}[x]$ the type T would be the type of an integer, e.g. `BigInt`.

If the parent object for such a ring has type `MyZX` and polynomials in that ring have type `MyZXPoly` then one would have:

- `MyZX <: PolyRing{BigInt}`
- `MyZXPoly <: PolyRingElem{BigInt}`

Polynomial rings should be made unique on the system by caching parent objects (unless an optional cache parameter is set to `false`). Polynomial rings should at least be distinguished based on their base (coefficient) ring. But if they have the same base ring and symbol (for their variable/generator), they should certainly have the same parent object.

See `src/generic/GenericTypes.jl` for an example of how to implement such a cache (which usually makes use of a dictionary).

## 61.2 Required functionality for univariate polynomials

In addition to the required functionality for the Ring/NCRing interface (and in the case of polynomials over a field, the Euclidean Ring interface), the Polynomial Ring interface has the following required functions.

We suppose that R is a fictitious base ring (coefficient ring) and that S is a univariate polynomial ring over R (i.e. $S = R[x]$) with parent object S of type `MyPolyRing{T}`. We also assume the polynomials in the ring have type `MyPoly{T}`, where T is the type of elements of the base (coefficient) ring.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type `RingElem` or `NCRingElem`.

We describe the functionality below for polynomials over commutative rings, i.e. with element type belonging to `RingElem`, however similar constructors should be available for element types belonging to `NCRingElem` instead, if the coefficient ring is noncommutative.

### Constructors

In addition to the standard constructors, the following constructors, taking an array of coefficients, must be available.

```
(S::MyPolyRing{T})(A::Vector{T}) where T <: RingElem
(S::MyPolyRing{T})(A::Vector{U}) where T <: RingElem, U <: RingElem
(S::MyPolyRing{T})(A::Vector{U}) where T <: RingElem, U <: Integer
```

Create the polynomial in the given ring whose degree $i$ coefficient is given by `A[1 + i]`. The elements of the array are assumed to be able to be coerced into the base ring R. If the argument is an empty vector, the zero polynomial shall be returned.

It may be desirable to have a additional version of the function that accepts an array of Julia `Int` values if this can be done more efficiently.

It is also possible to create polynomials directly without first creating the corresponding polynomial ring.

```
polynomial(R::Ring, arr::Vector{T}, var::VarName=:x; cached::Bool=true)
```

Given an array of coefficients construct the polynomial with those coefficients over the given ring and with the given variable.

**Note**

If cached is set to `false` then the parent ring of the created polynomial is not cached. However, this means that subsequent polynomials created in the same way will not be compatible. Instead, one should use the parent object of the first polynomial to create subsequent polynomials instead of calling this function repeatedly with `cached=false`.

### Data type and parent object methods

```
var(S::MyPolyRing{T}) where T <: RingElem
```

Return a `Symbol` representing the variable (generator) of the polynomial ring. Note that this is a `Symbol` not a `String`, though its string value will usually be used when printing polynomials.

```
symbols(S::MyPolyRing{T}) where T <: RingElem
```

Return the array `[s]` where `s` is a `Symbol` representing the variable of the given polynomial ring. This is provided for uniformity with the multivariate interface, where there is more than one variable and hence an array of symbols.

Given a type for the coefficients, we can obtain the corresponding type of a univariate polynomial having that type of coefficient:

`AbstractAlgebra.poly_type` – Function.

```
poly_type(::Type{T}) where T<:NCRingElement
poly_type(::T) where T<:NCRingElement
poly_type(::Type{S}) where S<:NCRing
poly_type(::S) where S<:NCRing
```

Return the type of a (univariate) polynomial whose coefficients have type `T` or type `elem_type(S)`. The type of the corresponding polynomial ring can be found via `poly_ring_type`.

For multivariate polynomials see `mpoly_ring_type`.

**Implementation**

This function is already defined for generic univariate polynomials (namely `Generic.NCPoly{T}` or `Generic.Poly{T}`), so *needs to be defined only for* special polynomial rings, *e.g.* those defined by a C implementation.

**Examples**

```
julia> poly_type(AbstractAlgebra.ZZ(1))
AbstractAlgebra.Generic.Poly{BigInt}

julia> poly_type(elem_type(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.Poly{BigInt}

julia> poly_type(AbstractAlgebra.ZZ)
AbstractAlgebra.Generic.Poly{BigInt}

julia> poly_type(typeof(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.Poly{BigInt}
```

source

Analogously, given a type for the coefficients, we can obtain the corresponding type of the ring of univariate polynomials having that type of coefficient:

`AbstractAlgebra.poly_ring_type` – Function.

```
poly_ring_type(::Type{T}) where T<:NCRingElement
poly_ring_type(::T) where T<:NCRingElement
poly_ring_type(::Type{S}) where S<:NCRing
poly_ring_type(::S) where S<:NCRing
```

The type of univariate polynomial rings with coefficients of type T respectively `elem_type(S)`. Implemented via `poly_type`.

See also `mpoly_type` and `mpoly_ring_type`.

**Examples**

```
julia> poly_ring_type(AbstractAlgebra.ZZ(1))
AbstractAlgebra.Generic.PolyRing{BigInt}

julia> poly_ring_type(elem_type(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.PolyRing{BigInt}

julia> poly_ring_type(AbstractAlgebra.ZZ)
AbstractAlgebra.Generic.PolyRing{BigInt}

julia> poly_ring_type(typeof(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.PolyRing{BigInt}
```

source

`AbstractAlgebra.poly_ring` – Method.

```
poly_ring(R::NCRing, s::Symbol; cached::Bool=false)
```

Like `polynomial_ring(R::NCRing, s::Symbol)` but return only the polynomial ring.

source

The default implementation figures out the appropriate polynomial ring type via `poly_type` and calls its constructor with `R, s, cached` as arguments. In our example, this would be

```
MyPolyRing{T}(R, s, cached)
```

Accordingly, a `poly_ring` method only needs to be defined if such a constructor does not exist for `poly_type(T)` or if other behaviour is wanted.

### Basic manipulation of rings and elements

```
length(f::MyPoly{T}) where T <: RingElem
```

Return the length of the given polynomial. The length of the zero polynomial is defined to be $0$, otherwise the length is the degree plus $1$. The return value should be of type `Int`.

```
set_length!(f::MyPoly{T}, n::Int) where T <: RingElem
```

This function must zero any coefficients beyond the requested length $n$ and then set the length of the polynomial to $n$. This function does not need to normalise the polynomial and is not useful to the user, but is used extensively by the AbstractAlgebra generic functionality.

This function returns the resulting polynomial.

```
coeff(f::MyPoly{T}, n::Int) where T <: RingElem
```

Return the coefficient of the polynomial f of degree n. If n is larger than the degree of the polynomial, it should return zero in the coefficient ring.

```
setcoeff!(f::MyPoly{T}, n::Int, a::T) where T <: RingElem
```

Set the degree $n$ coefficient of $f$ to $a$. This mutates the polynomial in-place if possible and returns the mutated polynomial (so that immutable types can also be supported). The function must not assume that the polynomial already has space for $n + 1$ coefficients. The polynomial must be resized if this is not the case.

Note that this function is not required to normalise the polynomial and is not necessarily useful to the user, but is used extensively by the generic functionality in AbstractAlgebra.jl. It is for setting raw coefficients in the representation.

```
normalise(f::MyPoly{T}, n::Int) where T <: RingElem
```

Given a polynomial whose length is currently $n$, including any leading zero coefficients, return the length of the normalised polynomial (either zero or the length of the polynomial with nonzero leading coefficient). Note that the function does not actually perform the normalisation.

```
fit!(f::MyPoly{T}, n::Int) where T <: RingElem
```

Ensure that the polynomial $f$ internally has space for $n$ coefficients. This function must mutate the function in-place if it is mutable. It does not return the mutated polynomial. Immutable types can still be supported by defining this function to do nothing.

Some interfaces for C polynomial types automatically manage the internal allocation of polynomials in every function that can be called on them. Explicit adjustment by the generic code in AbstractAlgebra.jl is not required. In such cases, this function can also be defined to do nothing.

## 61.3 Optional functionality for polynomial rings

Sometimes parts of the Euclidean Ring interface can and should be implemented for polynomials over a ring that is not necessarily a field.

When divisibility testing can be implemented for a polynomial ring over a field, it should be possible to implement the following functions from the Euclidean Ring interface:

- divides

- remove
- valuation

When the given polynomial ring is a GCD domain, with an effective GCD algorithm, it may be possible to implement the following functions:

- gcd
- lcm

Polynomial rings can optionally implement any part of the generic univariate polynomial functionality provided by AbstractAlgebra.jl, using the same interface.

Obviously additional functionality can also be added to that provided by AbstractAlgebra.jl on an ad hoc basis.

### Similar

The `similar` function is available for all univariate polynomial types, but new polynomial rings can define a specialised version of it if required.

```
similar(x::MyPoly{T}, R::Ring=base_ring(x), var::VarName=var(parent(x))) where T <: RingElem
```

Construct the zero polynomial with the given variable and coefficients in the given ring, if specified, and with the defaults shown if not.

Custom polynomial rings may choose which polynomial type is best-suited to return for any given arguments. If they don't specialise the function the default polynomial type returned is a `Generic.Poly`.

# Chapter 62

# Multivariate Polynomial Ring Interface

Multivariate polynomial rings are supported in AbstractAlgebra.jl, and in addition to the standard Ring interface, numerous additional functions are provided.

Unlike other kinds of rings, even complex operations such as GCD depend heavily on the multivariate representation. Therefore AbstractAlgebra.jl cannot provide much in the way of additional functionality to external multivariate implementations.

This means that external libraries must be able to implement their multivariate formats in whatever way they see fit. The required interface here should be implemented, even if it is not optimal. But it can be extended, either by implementing one of the optional interfaces, or by extending the required interface in some other way.

Naturally, any multivariate polynomial ring implementation provides the full Ring interface, in order to be treated as a ring for the sake of AbstractAlgebra.jl.

Considerations which make it impossible for AbstractAlgebra.jl to provide generic functionality on top of an arbitrary multivariate module include:

- orderings (lexical, degree, weighted, block, arbitrary)
- sparse or dense representation
- distributed or recursive representation
- packed or unpacked exponents
- exponent bounds (and whether adaptive or not)
- random access or iterators
- whether monomials and polynomials have the same type
- whether special cache aware data structures such as Geobuckets are used

## 62.1 Types and parents

AbstractAlgebra.jl provides two abstract types for multivariate polynomial rings and their elements:

- `MPolyRing{T}` is the abstract type for multivariate polynomial ring parent types
- `MPolyRingElem{T}` is the abstract type for multivariate polynomial types

We have that `MPolyRing{T} <: Ring` and `MPolyRingElem{T} <: RingElem`.

Note that both abstract types are parameterised. The type T should usually be the type of elements of the coefficient ring of the polynomial ring. For example, in the case of $\mathbb{Z}[x, y]$ the type T would be the type of an integer, e.g. `BigInt`.

Given a type for the coefficients, we can obtain the corresponding type of a multivariate polynomial having that type of coefficient:

`AbstractAlgebra.mpoly_type` – Function.

```
mpoly_type(::Type{T}) where T<:RingElement
mpoly_type(::T) where T<:RingElement
mpoly_type(::Type{S}) where S<:Ring
mpoly_type(::S) where S<:Ring
```

Return the type of a (multivariate) polynomial whose coefficients have type T or type `elem_type(S)`. The type of the corresponding polynomial ring can be found via `mpoly_ring_type`.

For univariate polynomials see `poly_type`.

**Implementation**

This function is already defined for generic multivariate polynomials (namely `Generic.MPoly{T}`), so *needs to be defined only for* special polynomial rings, *e.g.* those defined by a C implementation.

**Examples**

```
julia> mpoly_type(AbstractAlgebra.ZZ(1))
AbstractAlgebra.Generic.MPoly{BigInt}

julia> mpoly_type(elem_type(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.MPoly{BigInt}

julia> mpoly_type(AbstractAlgebra.ZZ)
AbstractAlgebra.Generic.MPoly{BigInt}

julia> mpoly_type(typeof(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.MPoly{BigInt}
```

source

Analogously, given a type for the coefficients, we can obtain the corresponding type of the ring of multivariate polynomials having that type of coefficient:

`AbstractAlgebra.mpoly_ring_type` – Function.

```
mpoly_ring_type(::Type{T}) where T<:RingElement
mpoly_ring_type(::T) where T<:RingElement
mpoly_ring_type(::Type{S}) where S<:Ring
mpoly_ring_type(::S) where S<:Ring
```

Return the type of the parent of a (multivariate) polynomial whose coefficients have type T or type `elem_type(S)`. The type of the polynomials themselves can be found via `mpoly_type`.

For univariate polynomials see poly_ring_type.

**Implementation**

This function is already defined for generic multivariate polynomials (namely Generic.MPoly{T}), so *needs to be defined only for* special polynomial rings, *e.g.* those defined by a C implementation.

**Examples**

```
julia> mpoly_ring_type(AbstractAlgebra.ZZ(1))
AbstractAlgebra.Generic.MPolyRing{BigInt}

julia> mpoly_ring_type(elem_type(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.MPolyRing{BigInt}

julia> mpoly_ring_type(AbstractAlgebra.ZZ)
AbstractAlgebra.Generic.MPolyRing{BigInt}

julia> mpoly_ring_type(typeof(AbstractAlgebra.ZZ))
AbstractAlgebra.Generic.MPolyRing{BigInt}
```

source

### Identical polynomial rings

Multivariate polynomial rings should be made unique on the system by caching parent objects (unless an optional cache parameter is set to false). Multivariate polynomial rings should at least be distinguished based on their base (coefficient) ring and number of variables. But if they have the same base ring, symbols (for their variables/generators) and ordering, they should certainly have the same parent object.

Implementaors of a new type of polynomial ring should see src/generic/GenericTypes.jl for an example of how to implement such a cache (which usually makes use of a dictionary). In particular look at MPolyID.

## 62.2 Required functionality for multivariate polynomials

In addition to the required functionality for the Ring interface, the Multivariate Polynomial interface has the following required functions.

We suppose that R is a fictitious base ring (coefficient ring) and that S is a multivariate polynomial ring over R (i.e. $S = R[x, y, \ldots]$) with parent object S of type MyMPolyRing{T}. We also assume the polynomials in the ring have type MyMPoly{T}, where T is the type of elements of the base (coefficient) ring.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type RingElem or more generally the union type RingElement which includes the Julia integer, rational and floating point types.

### Constructors

To construct a multivariate polynomial ring, there is the following constructor.

AbstractAlgebra.polynomial_ring – Method.

```
polynomial_ring(R::Ring, varnames::Vector{Symbol}; cached=true, internal_ordering=:lex)
```

Given a coefficient ring R and variable names, say `varnames = [:x1, :x2, ...]`, return a tuple `S, [x1, x2, ...]` of the polynomial ring $S = R[x1, x2, \ldots]$ and its generators $x1, x2, \ldots$.

By default (`cached=true`), the output S will be cached, i.e. if `polynomial_ring` is invoked again with the same arguments, the same (*identical*) ring is returned. Setting `cached` to `false` ensures a distinct new ring is returned, and will also prevent it from being cached.

The monomial ordering used for the internal storage of polynomials in S can be set with `internal_ordering` and must be one of `:lex`, `:deglex` or `:degrevlex`.

See also: `polynomial_ring(::Ring, ::Vararg)`, `@polynomial_ring`.

**Examples**

```
julia> S, generators = polynomial_ring(ZZ, [:x, :y, :z])
(Multivariate polynomial ring in 3 variables over integers,
↪  AbstractAlgebra.Generic.MPoly{BigInt}[x, y, z])
```

source

Polynomials in a given ring can be constructed using the generators and basic polynomial arithmetic. However, this is inefficient and the following build context is provided for building polynomials term-by-term. It assumes the polynomial data type is random access, and so the constructor functions must be reimplemented for all other types of polynomials.

```
MPolyBuildCtx(R::MPolyRing)
```

Return a build context for creating polynomials in the given polynomial ring.

```
push_term!(M::MPolyBuildCtx, c::RingElem, v::Vector{Int})
```

Add the term with coefficient $c$ and exponent vector $v$ to the polynomial under construction in the build context $M$.

```
finish(M::MPolyBuildCtx)
```

Finish construction of the polynomial, sort the terms, remove duplicate and zero terms and return the created polynomial.

### Data type and parent object methods

```
symbols(S::MyMPolyRing{T}) where T <: RingElem
```

Return an array of `Symbols` representing the variables (generators) of the polynomial ring. Note that these are `Symbols` not `Strings`, though their string values will usually be used when printing polynomials.

```
number_of_variables(f::MyMPolyRing{T}) where T <: RingElem
```

Return the number of variables of the polynomial ring.

```
gens(S::MyMPolyRing{T}) where T <: RingElem
```

Return an array of all the generators (variables) of the given polynomial ring (as polynomials).

The first entry in the array will be the variable with most significance with respect to the ordering.

```
gen(S::MyMPolyRing{T}, i::Int) where T <: RingElem
```

Return the $i$-th generator (variable) of the given polynomial ring (as a polynomial).

```
internal_ordering(S::MyMPolyRing{T})
```

Return the ordering of the given polynomial ring as a symbol. Supported values currently include `:lex`, `:deglex` and `:degrevlex`.

### Basic manipulation of rings and elements

```
length(f::MyMPoly{T}) where T <: RingElem
```

Return the number of nonzero terms of the given polynomial. The length of the zero polynomial is defined to be $0$. The return value should be of type `Int`.

```
degrees(f::MyMPoly{T}) where T <: RingElem
```

Return an array of the degrees of the polynomial $f$ in each of the variables.

```
total_degree(f::MyMPoly{T}) where T <: RingElem
```

Return the total degree of the polynomial $f$, i.e. the highest sum of exponents occurring in any term of $f$.

```
is_gen(x::MyMPoly{T}) where T <: RingElem
```

Return `true` if $x$ is a generator of the polynomial ring.

```
coefficients(p::MyMPoly{T}) where T <: RingElem
```

Return an iterator for the coefficients of the polynomial $p$, starting with the coefficient of the most significant term with respect to the ordering. Generic code will provide this function automatically for random access polynomials that implement the `coeff` function.

```
monomials(p::MyMPoly{T}) where T <: RingElem
```

Return an iterator for the monomials of the polynomial $p$, starting with the monomial of the most significant term with respect to the ordering. Monomials in AbstractAlgebra are defined to have coefficient $1$. See the function `terms` if you also require the coefficients, however note that only monomials can be compared. Generic code will provide this function automatically for random access polynomials that implement the `monomial` function.

```
terms(p::MyMPoly{T}) where T <: RingElem
```

Return an iterator for the terms of the polynomial $p$, starting with the most significant term with respect to the ordering. Terms in AbstractAlgebra include the coefficient. Generic code will provide this function automatically for random access polynomials that implement the `term` function.

```
exponent_vectors(a::MyMPoly{T}) where T <: RingElement
```

Return an iterator for the exponent vectors for each of the terms of the polynomial starting with the most significant term with respect to the ordering. Each exponent vector is an array of `Int`s, one for each variable, in the order given when the polynomial ring was created. Generic code will provide this function automatically for random access polynomials that implement the `exponent_vector` function.

### Exact division

For any ring that implements exact division, the following can be implemented.

```
divexact(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return the exact quotient of $f$ by $g$ if it exists, otherwise throw an error.

```
divides(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return a tuple `(flag, q)` where `flag` is `true` if $g$ divides $f$, in which case $q$ will be the exact quotient, or `flag` is false and $q$ is undefined.

```
remove(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return a tuple $(v, q)$ such that the highest power of $g$ that divides $f$ is $g^v$ and the cofactor is $q$.

```
valuation(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return $v$ such that the highest power of $g$ that divides $f$ is $g^v$.

### Ad hoc exact division

For any ring that implements exact division, the following can be implemented.

```
divexact(f::MyMPoly{T}, c::Integer) where T <: RingElem
divexact(f::MyMPoly{T}, c::Rational) where T <: RingElem
divexact(f::MyMPoly{T}, c::T) where T <: RingElem
```

Divide the polynomial exactly by the constant $c$.

### Euclidean division

Although multivariate polynomial rings are not in general Euclidean, it is possible to define a quotient with remainder function that depends on the polynomial ordering in the case that the quotient ring is a field or a Euclidean domain. In the case that a polynomial $g$ divides a polynomial $f$, the result no longer depends on the ordering and the remainder is zero, with the quotient agreeing with the exact quotient.

```
divrem(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return a tuple $(q, r)$ such that $f = qg + r$, where the coefficients of terms of $r$ whose monomials are divisible by the leading monomial of $g$ are reduced modulo the leading coefficient of $g$ (according to the Euclidean function on the coefficients).

Note that the result of this function depends on the ordering of the polynomial ring.

```
div(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

As per the `divrem` function, but returning the quotient only. Especially when the quotient happens to be exact, this function can be exceedingly fast.

### GCD

In cases where there is a meaningful Euclidean structure on the coefficient ring, it is possible to compute the GCD of multivariate polynomials.

```
gcd(f::MyMPoly{T}, g::MyMPoly{T}) where T <: RingElem
```

Return a greatest common divisor of $f$ and $g$.

### Square root

Over rings for which an exact square root is available, it is possible to take the square root of a polynomial or test whether it is a square.

```
sqrt(f::MyMPoly{T}, check::Bool=true) where T <: RingElem
```

Return the square root of the polynomial $f$ and raise an exception if it is not a square. If `check` is set to `false`, the input is assumed to be a perfect square and this assumption is not fully checked. This can be significantly faster.

```
is_square(::MyMPoly{T}) where T <: RingElem
```

Return `true` if $f$ is a square.

## 62.3 Interface for sparse distributed, random access multivariates

The following additional functions should be implemented by libraries that provide a sparse distributed polynomial format, stored in a representation for which terms can be accessed in constant time (e.g. where arrays are used to store coefficients and exponent vectors).

### Sparse distributed, random access constructors

In addition to the standard constructors, the following constructor, taking arrays of coefficients and exponent vectors, should be provided.

```
(S::MyMPolyRing{T})(A::Vector{T}, m::Vector{Vector{Int}}) where T <: RingElem
```

Create the polynomial in the given ring with nonzero coefficients specified by the elements of $A$ and corresponding exponent vectors given by the elements of $m$.

There is no assumption about coefficients being nonzero or terms being in order or unique. Zero terms are removed by the function, duplicate terms are combined (added) and the terms are sorted so that they are in the correct order.

Each exponent vector uses a separate integer for each exponent field, the first of which should be the exponent for the most significant variable with respect to the ordering. All exponents must be non-negative.

A library may also optionally provide an interface that makes use of `BigInt` (or any other big integer type) for exponents instead of `Int`.

### Sparse distributed, random access basic manipulation

```
coeff(f::MyMPoly{T}, n::Int) where T <: RingElem
```

Return the coefficient of the $n$-th term of $f$. The first term should be the most significant term with respect to the ordering.

```
coeff(a::MyMPoly{T}, exps::Vector{Int}) where T <: RingElement
```

Return the coefficient of the term with the given exponent vector, or zero if there is no such term.

```
monomial(f::MyMPoly{T}, n::Int) where T <: RingElem
monomial!(m::MyMPoly{T}, f::MyMPoly{T}, n::Int) where T <: RingElem
```

Return the $n$-th monomial of $f$ or set $m$ to the $n$-th monomial of $f$, respectively. The first monomial should be the most significant term with respect to the ordering. Monomials have coefficient $1$ in AbstractAlgebra. See the function `term` if you also require the coefficient, however, note that only monomials can be compared.

```
term(f::MyMPoly{T}, n::Int) where T <: RingElem
```

Return the $n$-th term of $f$. The first term should be the one whose monomial is most significant with respect to the ordering.

```
exponent(f::MyMPoly{T}, i::Int, j::Int) where T <: RingElem
```

Return the exponent of the $j$-th variable in the $i$-th term of the polynomial $f$. The first term is the one with whose monomial is most significant with respect to the ordering.

```
exponent_vector(a::MyMPoly{T}, i::Int) where T <: RingElement
```

Return a vector of exponents, corresponding to the exponent vector of the i-th term of the polynomial. Term numbering begins at $1$ and the exponents are given in the order of the variables for the ring, as supplied when the ring was created.

```
setcoeff!(a::MyMPoly, exps::Vector{Int}, c::S) where S <: RingElement
```

Set the coefficient of the term with the given exponent vector to the given value $c$. If no such term exists (and $c \neq 0$), one will be inserted. This function takes $O(\log n)$ operations if a term with the given exponent already exists and $c \neq 0$, or if the term is inserted at the end of the polynomial. Otherwise it can take $O(n)$ operations in the worst case. This function must return the modified polynomial.

### Unsafe functions

The following functions must be provided, but are considered unsafe, as they may leave the polynomials in an inconsistent state and they mutate their inputs. As usual, such functions should only be applied on polynomials that have no references elsewhere in the system and are mainly intended to be used in carefully written library code, rather than by users.

Users should instead build polynomials using the constructors described above.

```
fit!(f::MyMPoly{T}, n::Int) where T <: RingElem
```

Ensure that the polynomial $f$ internally has space for $n$ nonzero terms. This function must mutate the function in-place if it is mutable. It does not return the mutated polynomial. Immutable types can still be supported by defining this function to do nothing.

```
setcoeff!(a::MyMPoly{T}, i::Int, c::T) where T <: RingElement
setcoeff!(a::MyMPoly{T}, i::Int, c::U) where {T <: RingElement, U <: Integer}
```

Set the $i$-th coefficient of the polynomial $a$ to $c$. No check is performed on the index $i$ or for $c = 0$. It may be necessary to call `combine_like_terms` after calls to this function, to remove zero terms. The function must return the modified polynomial.

```
combine_like_terms!(a::MyMPoly{T}) where T <: RingElement
```

Remove zero terms and combine any adjacent terms with the same exponent vector (by adding them). It is assumed that all the exponent vectors are already in the correct order with respect to the ordering. The function must return the resulting polynomial.

```
set_exponent_vector!(a::MyMPoly{T}, i::Int, exps::Vector{Int}) where T <: RingElement
```

Set the $i$-th exponent vector to the given exponent vector. No check is performed on the index $i$, which is assumed to be valid (or that the polynomial has enough space allocated). No sorting of exponents is performed by this function. To sort the terms after setting any number of exponents with this function, run the `sort_terms!` function. The function must return the modified polynomial.

```
sort_terms!(a::MyMPoly{T}) where {T <: RingElement}
```

Sort the terms of the given polynomial according to the polynomial ring ordering. Zero terms and duplicate exponents are ignored. To deal with those call `combine_like_terms`. The sorted polynomial must be returned by the function.

## 62.4 Optional functionality for multivariate polynomials

The following functions can optionally be implemented for multivariate polynomial types.

### Reduction by an ideal

```
divrem(f::MyMPoly{T}, G::Vector{MyMPoly{T}}) where T <: RingElem
```

As per the `divrem` function above, except that each term of $r$ starting with the most significant term, is reduced modulo the leading terms of each of the polynomials in the array $G$ for which the leading monomial is a divisor.

A tuple $(Q, r)$ is returned from the function, where $Q$ is an array of polynomials of the same length as $G$, and such that $f = r + \sum Q[i]G[i]$.

The result is again dependent on the ordering in general, but if the polynomials in $G$ are over a field and the reduced generators of a Groebner basis, then the result is unique.

### Evaluation

```
evaluate(a::MyMPoly{T}, A::Vector{T}) where T <: RingElem
```

Evaluate the polynomial at the given values in the coefficient ring of the polynomial. The result should be an element of the coefficient ring.

```
evaluate(f::MyMPoly{T}, A::Vector{U}) where {T <: RingElem, U <: Integer}
```

Evaluate the polynomial $f$ at the values specified by the entries of the array $A$.

```
(a::MyMPoly{T})(vals::NCRingElement...) where T <: RingElement
```

Evaluate the polynomial at the given arguments. This provides functional notation for polynomial evaluation, i.e. $f(a, b, c)$. It must be defined for each supported polynomial type (Julia does not allow functional notation to be defined for an abstract type).

The code for this function in MPoly.jl can be used when implementing this as it provides the most general possible evaluation, which is much more general than the case of evaluation at elements of the same ring.

The evaluation should succeed for any set of values for which a multiplication is defined with the product of a coefficient and all the values before it.

**Note**

The values at which a polynomial is evaluated may be in non-commutative rings. Products are performed in the order of the variables in the polynomial ring that the polynomial belongs to, preceded by a multiplication by the coefficient on the left.

### Derivations

The following function allows to compute derivations of multivariate polynomials of type MPoly.

```
derivative(f::MyMPoly{T}, j::Int) where T <: RingElem
```

Compute the derivative of $f$ with respect to the $j$-th variable of the polynomial ring.

# Chapter 63

# Series Ring Interface

Univariate power series rings are supported in AbstractAlgebra in a variety of different forms, including absolute and relative precision models and Laurent series.

In addition to the standard Ring interface, numerous additional functions are required to be present for power series rings.

## 63.1 Types and parents

AbstractAlgebra provides two abstract types for power series rings and their elements:

- `SeriesRing{T}` is the abstract type for all power series ring parent types
- `SeriesElem{T}` is the abstract type for all power series types

We have that `SeriesRing{T} <: Ring` and `SeriesElem{T} <: RingElem`.

Note that both abstract types are parameterised. The type T should usually be the type of elements of the coefficient ring of the power series ring. For example, in the case of $\mathbb{Z}[[x]]$ the type T would be the type of an integer, e.g. `BigInt`.

Within the `SeriesElem{T}` abstract type is the abstract type `RelPowerSeriesRingElem{T}` for relative power series, and `AbsPowerSeriesRingElem{T}` for absolute power series.

Relative series are typically stored with a valuation and a series that is either zero or that has nonzero constant term. Absolute series are stored starting from the constant term, even if it is zero.

If the parent object for a relative series ring over the bignum integers has type `MySeriesRing` and series in that ring have type `MySeries` then one would have:

- `MySeriesRing <: SeriesRing{BigInt}`
- `MySeries <: RelPowerSeriesRingElem{BigInt}`

Series rings should be made unique on the system by caching parent objects (unless an optional cache parameter is set to `false`). Series rings should at least be distinguished based on their base (coefficient) ring. But if they have the same base ring and symbol (for their variable/generator) and same default precision, they should certainly have the same parent object.

See `src/generic/GenericTypes.jl` for an example of how to implement such a cache (which usually makes use of a dictionary).

## 63.2 Required functionality for series

In addition to the required functionality for the Ring interface the Series Ring interface has the following required functions.

We suppose that R is a fictitious base ring (coefficient ring) and that S is a series ring over R (e.g. $S = R[[x]]$) with parent object S of type `MySeriesRing{T}`. We also assume the series in the ring have type `MySeries{T}`, where T is the type of elements of the base (coefficient) ring.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type `RingElem`.

### Constructors

In addition to the standard constructors, the following constructors, taking an array of coefficients, must be available.

For relative power series and Laurent series we have:

```
(S::MySeriesRing{T})(A::Vector{T}, len::Int, prec::Int, val::Int) where T <: RingElem
```

Create the series in the given ring whose valuation is `val`, whose absolute precision is given by `prec` and the coefficients of which are given by A, starting from the first nonzero term. Only `len` terms of the array are used, the remaining terms being ignored. The value `len` cannot exceed the length of the supplied array.

It is permitted to have trailing zeros in the array, but it is not needed, even if the precision minus the valuation is bigger than the length of the array.

```
(S::MySeriesRing{T})(A::Vector{U}, len::Int, prec::Int, val::Int) where {T <: RingElem, U <:
↪  RingElem}
```

As above, but where the array is an array of coefficient that can be coerced into the base ring of the series ring.

```
(S::MySeriesRing{T})(A::Vector{U}, len::Int, prec::Int, val::Int) where {T <: RingElem, U <:
↪  Integer}
```

As above, but where the array is an array of integers that can be coerced into the base ring of the series ring.

It may be desirable to implement an addition version which accepts an array of Julia `Int` values if this can be done more efficiently.

For absolute power series we have:

```
(S::MySeriesRing{T})(A::Vector{T}, len::Int, prec::Int) where T <: RingElem
```

Create the series in the given ring whose absolute precision is given by `prec` and the coefficients of which are given by A, starting from the constant term. Only `len` terms of the array are used, the remaining terms being ignored.

Note that `len` is usually maintained separately of any polynomial that is underlying the power series. This allows for easy truncation of a power series without actually modifying the polynomial underlying it.

It is permitted to have trailing zeros in the array, but it is not needed, even if the precision is bigger than the length of the array.

It is also possible to create series directly without having to create the corresponding series ring.

```
abs_series(R::Ring, arr::Vector{T}, len::Int, prec::Int, var::VarName=:x; max_precision::Int=prec,
↪  cached::Bool=true) where T
rel_series(R::Ring, arr::Vector{T}, len::Int, prec::Int, val::Int, var::VarName=:x;
↪  max_precision::Int=prec, cached::Bool=true) where T
```

Create the power series over the given base ring R with coefficients specified by `arr` with the given absolute precision `prec` and in the case of relative series with the given valuation `val`.

Note that more coefficients may be specified than are actually used. Only the first `len` coefficients are made part of the series, the remainder being stored internally but ignored.

In the case of absolute series one must have `prec >= len` and in the case of relative series one must have `prec >= len + val`.

By default the series are created in a ring with variable `x` and `max_precision` equal to `prec`, however one may specify these directly to override the defaults. Note that series are only compatible if they have the same coefficient ring R, `max_precision` and variable name `var`.

Also by default any parent ring created is cached. If this behaviour is not desired, set `cached=false`. However, this means that subsequent series created in the same way will not be compatible. Instead, one should use the parent object of the first series to create subsequent series instead of calling this function repeatedly with cached=false.

### Data type and parent object methods

```
var(S::MySeriesRing{T}) where T <: RingElem
```

Return a `Symbol` representing the variable (generator) of the series ring. Note that this is a `Symbol` not a `String`, though its string value will usually be used when printing series.

Custom series types over a given ring should define one of the following functions which return the type of an absolute or relative series object over that ring.

```
abs_series_type(::Type{T}) where T <: RingElement
rel_series_type(::Type{T}) where T <: RingElement
```

Return the type of a series whose coefficients have the given type.

This function is defined for generic series and only needs to be defined for custom series rings, e.g. ones defined by a C implementation.

```
max_precision(S::MySeriesRing{T}) where T <: RingElem
```

Return the (default) maximum precision of the power series ring. This is the precision that the output of an operation will be if it cannot be represented to full precision (e.g. because it mathematically has infinite precision).

This value is usually supplied upon creation of the series ring and stored in the ring. It is independent of the precision which each series in the ring actually has. Those are stored on a per element basis in the actual series elements.

### Basic manipulation of rings and elements

```
pol_length(f::MySeries{T}) where T <: RingElem
```

Return the length of the polynomial underlying the given power series. This is not generally useful to the user, but is used internally.

```
set_length!(f::MySeries{T}, n::Int) where T <: RingElem
```

This function sets the effective length of the polynomial underlying the given series. The function doesn't modify the actual polynomial, but simply changes the number of terms of the polynomial which are considered to belong to the power series. The remaining terms are ignored.

This function cannot set the length to a value greater than the length of any underlying polynomial.

The function mutates the series in-place but does not return the mutated series.

```
precision(f::MySeries{T})
```

Return the absolute precision of $f$.

```
set_precision!(f::MySeries{T}, prec::Int)
```

Set the absolute precision of the given series to the given value.

This return the updated series.

```
valuation(f::MySeries{T})
```

Return the valuation of the given series.

```
set_valuation!(f::MySeries{T}, val::Int)
```

For relative series and Laurent series only, this function alters the valuation of the given series to the given value.

This function returns the updated series.

```
polcoeff(f::MySeries{T}, n::Int)
```

Return the coefficient of degree n of the polynomial underlying the series. If n is larger than the degree of this polynomial, zero is returned. This function is not generally of use to the user but is used internally.

```
setcoeff!(f::MySeries{T}, n::Int, a::T) where T <: RingElem
```

Set the degree $n$ coefficient of the polynomial underlying $f$ to $a$. This mutates the polynomial in-place if possible and returns the mutated series (so that immutable types can also be supported). The function must not assume that the polynomial already has space for $n + 1$ coefficients. The polynomial must be resized if this is not the case.

**Note**

This function is not required to normalise the polynomial and is not necessarily useful to the user, but is used extensively by the generic functionality in AbstractAlgebra.jl. It is for setting raw coefficients in the representation.

```
normalise(f::MySeries{T}, n::Int)
```

Given a series $f$ represented by a polynomial of at least the given length, return the normalised length of the underlying polynomial assuming it has length at most $n$. This function does not actually normalise the polynomial and is not particularly useful to the user. It is used internally.

```
renormalize!(f::MySeries{T}) where T <: RingElem
```

Given a relative series or Laurent series whose underlying polynomial has zero constant term, say as the result of some internal computation, renormalise the series so that the polynomial has nonzero constant term. The precision and valuation of the series are adjusted to compensate. This function is not intended to be useful to the user, but is used internally.

```
fit!(f::MySeries{T}, n::Int) where T <: RingElem
```

Ensure that the polynomial underlying $f$ internally has space for $n$ coefficients. This function must mutate the series in-place if it is mutable. It does not return the mutated series. Immutable types can still be supported by defining this function to do nothing.

Some interfaces for C polynomial types automatically manage the internal allocation of polynomials in every function that can be called on them. Explicit adjustment by the generic code in AbstractAlgebra.jl is not required. In such cases, this function can also be defined to do nothing.

```
gen(R::MySeriesRing{T}) where T <: RingElem
```

Return the generator x of the series ring.

## 63.3 Optional functionality for series

### Similar and zero

The following functions are available for all absolute and relative series types. The functions `similar` and `zero` do the same thing, but are provided for uniformity with other parts of the interface.

```
similar(x::MySeries, R::Ring, max_prec::Int, var::VarName=var(parent(x)); cached::Bool=true)
zero(a::MySeries, R::Ring, max_prec::Int, var::VarName=var(parent(a)); cached::Bool=true)
```

Construct the zero series with the given variable (if specified), coefficients in the specified coefficient ring and with relative/absolute precision cap on its parent ring as given by `max_prec`.

```
similar(x::MySeries, R::Ring, var::VarName=var(parent(x)); cached::Bool=true)
similar(x::MySeries, max_prec::Int, var::VarName=var(parent(x)); cached::Bool=true)
similar(x::MySeries, var::VarName=var(parent(x)); cached::Bool=true)
similar(x::MySeries, R::Ring, max_prec::Int, var::VarName; cached::Bool=true)
similar(x::MySeries, R::Ring, var::VarName; cached::Bool=true)
similar(x::MySeries, max_prec::Int, var::VarName; cached::Bool=true)
similar(x::MySeries, var::VarName; cached::Bool=true)
zero(x::MySeries, R::Ring, var::VarName=var(parent(x)); cached::Bool=true)
zero(x::MySeries, max_prec::Int, var::VarName=var(parent(x)); cached::Bool=true)
zero(x::MySeries, var::VarName=var(parent(x)); cached::Bool=true)
zero(x::MySeries, R::Ring, max_prec::Int, var::VarName; cached::Bool=true)
zero(x::MySeries, R::Ring, var::VarName; cached::Bool=true)
zero(x::MySeries, max_prec::Int, var::VarName; cached::Bool=true)
zero(x::MySeries, var::VarName; cached::Bool=true)
```

As above, but use the precision cap of the parent ring of `x` and the `base_ring` of `x` if these are not specified.

Custom series rings may choose which series type is best-suited to return for the given coefficient ring, precision cap and variable, however they should return a series with the same model as `x`, i.e. relative or series.

If custom implementations don't specialise these function the default return type is a `Generic.AbsSeries` or `Generic.RelSeries`.

The default implementation of zero calls out to similar, so it's generally sufficient to specialise only similar. For both similar and zero only the most general method has to be implemented as all other methods call out to this more general method.

# Chapter 64

# Residue Ring Interface

Residue rings (currently a quotient ring modulo a principal ideal) are supported in AbstractAlgebra.jl, at least for Euclidean base rings. There is also partial support for residue rings of polynomial rings where the modulus has invertible leading coefficient.

In addition to the standard Ring interface, some additional functions are required to be present for residue rings.

## 64.1 Types and parents

AbstractAlgebra provides four abstract types for residue rings and their elements:

- `ResidueRing{T}` is the abstract type for residue ring parent types
- `ResidueField{T}` is the abstract type for residue rings known to be fields
- `ResElem{T}` is the abstract type for types of elements of residue rings (residues)
- `ResFieldElem{T}` is the abstract type for types of elements of residue fields

We have that `ResidueRing{T} <: AbstractAlgebra.Ring` and `ResElem{T} <: AbstractAlgebra.RingElem`.

Note that these abstract types are parameterised. The type T should usually be the type of elements of the base ring of the residue ring/field.

If the parent object for a residue ring has type `MyResRing` and residues in that ring have type `MyRes` then one would have:

- `MyResRing <: ResidueRing{BigInt}`
- `MyRes <: ResElem{BigInt}`

Residue rings should be made unique on the system by caching parent objects (unless an optional cache parameter is set to `false`). Residue rings should at least be distinguished based on their base ring and modulus (the principal ideal one is taking a quotient of the base ring by).

See `src/generic/GenericTypes.jl` for an example of how to implement such a cache (which usually makes use of a dictionary).

## 64.2 Required functionality for residue rings

In addition to the required functionality for the Ring interface the Residue Ring interface has the following required functions.

We suppose that R is a fictitious base ring, $m$ is an element of that ring, and that S is the residue ring (quotient ring) $R/(m)$ with parent object S of type `MyResRing{T}`. We also assume the residues $r \pmod m$ in the residue ring have type `MyRes{T}`, where T is the type of elements of the base ring.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type `RingElem`.

### Data type and parent object methods

```
modulus(S::MyResRing{T}) where T <: AbstractAlgebra.RingElem
```

Return the modulus of the given residue ring, i.e. if the residue ring $S$ was specified to be $R/(m)$, return $m$.

### Basic manipulation of rings and elements

```
data(f::MyRes{T}) where T <: RingElem
lift(f::MyRes{T}) where T <: RingElem
```

Given a residue $r \pmod m$, represented as such, return $r$. In the special case where machine integers are used to represent the residue, `data` will return the machine integer, whereas `lift` will return a multiprecision integer. Otherwise `lift` falls back to `data` by default.

# Chapter 65

# Field Interface

AbstractAlgebra.jl generic code makes use of a standardised set of functions which it expects to be implemented for all fields. Here we document this interface. All libraries which want to make use of the generic capabilities of AbstractAlgebra.jl must supply all of the required functionality for their fields.

## 65.1 Types

Most fields must supply two types:

- a type for the parent object (representing the field itself)
- a type for elements of that field

For example, the generic fraction field type in AbstractAlgebra.jl provides two types in `generic/GenericTypes.jl`:

- `Generic.FracField{T}` for the parent objects
- `Generic.FracFieldElem{T}` for the actual fractions

The parent type must belong to `Field` and the element type must belong to `FieldElem`. Of course, the types may belong to these abstract types transitively.

For parameterised fields, we advise parameterising both the type of parent objects and the type of the element objects by the type of the elements of the base ring.

There can be variations on this theme: e.g. in some areas of mathematics there is a notion of a coefficient domain, in which case it may make sense to parameterise all types by the type of elements of this coefficient domain. But note that this may have implications for the ad hoc operators one might like to explicitly implement.

## 65.2 FieldElement type union

Because of its lack of multiple inheritance, Julia does not allow Julia Base types to belong to `FieldElem`. To allow us to work equally with AbstractAlgebra and Julia types that represent elements of fields we define a union type `FieldElement` in `src/julia/JuliaTypes`.

So far, in addition to `FieldElem` the union type `FieldElement` includes the Julia types `Rational` and `AbstractFloat`.

Most of the generic code in AbstractAlgebra makes use of the union type `FieldElement` instead of `FieldElem` so that the generic functions also accept the Julia Base field types.

**Note**

One must be careful when defining ad hoc binary operations for field element types. It is often necessary to define separate versions of the functions for `FieldElem` then for each of the Julia types separately in order to avoid ambiguity warnings.

Note that even though `FieldElement` is a union type we still have the following inclusion

```
FieldElement <: RingElement
```

## 65.3 Parent object caches

In many cases, it is desirable to have only one object in the system to represent each field. This means that if the same field is constructed twice, elements of the two fields will be compatible as far as arithmetic is concerned.

In order to facilitate this, global caches of fields are stored in AbstractAlgebra.jl, usually implemented using dictionaries. For example, the `Generic.FracField` parent objects are looked up in a dictionary `FracDict` to see if they have been previously defined.

Whether these global caches are provided or not, depends on both mathematical and algorithmic considerations. E.g. in the case of number fields, it isn't desirable to identify all number fields with the same defining polynomial, as they may be considered with distinct embeddings into one another. In other cases, identifying whether two fields are the same may be prohibitively expensive. Generally, it may only make sense algorithmically to identify two fields if they were constructed from identical data.

If a global cache is provided, it must be optionally possible to construct the parent objects without caching. This is done by passing a boolean value `cached` to the inner constructor of the parent object. See `generic/GenericTypes.jl` for examples of how to construct and handle such caches.

## 65.4 Required functions for all fields

In the following, we list all the functions that are required to be provided for fields in AbstractAlgebra.jl or by external libraries wanting to use AbstractAlgebra.jl.

We give this interface for fictitious types `MyParent` for the type of the field parent object `R` and `MyElem` for the type of the elements of the field.

**Note**

Generic functions in AbstractAlgebra.jl may not rely on the existence of functions that are not documented here. If they do, those functions will only be available for fields that implement that additional functionality, and should be documented as such.

In the first place, all fields are rings and therefore any field type must implement all of the Ring interface. The functionality below is in addition to this basic functionality.

### Data type and parent object methods

```
characteristic(R::MyParent)
```

Return the characteristic of the field. If the characteristic is not known, an exception is raised.

### Basic manipulation of rings and elements

```
is_unit(f::MyElem)
```

Return `true` if the given element is invertible, i.e. nonzero in the field.

# Chapter 66

# Fraction Field Interface

Fraction fields are supported in AbstractAlgebra.jl, at least for gcd domains. In addition to the standard Ring interface, some additional functions are required to be present for fraction fields.

## 66.1 Types and parents

AbstractAlgebra provides two abstract types for fraction fields and their elements:

- `FracField{T}` is the abstract type for fraction field parent types
- `FracElem{T}` is the abstract type for types of fractions

We have that `FracField{T} <: Field` and `FracElem{T} <: FieldElem`.

Note that both abstract types are parameterised. The type T should usually be the type of elements of the base ring of the fraction field.

Fraction fields should be made unique on the system by caching parent objects (unless an optional cache parameter is set to `false`). Fraction fields should at least be distinguished based on their base ring.

See `src/generic/GenericTypes.jl` for an example of how to implement such a cache (which usually makes use of a dictionary).

## 66.2 Required functionality for fraction fields

In addition to the required functionality for the Field interface the Fraction Field interface has the following required functions.

We suppose that R is a fictitious base ring, and that S is the fraction field with parent object S of type `MyFracField{T}`. We also assume the fractions in the field have type `MyFrac{T}`, where T is the type of elements of the base ring.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type `RingElem`.

### Constructors

The following constructors create fractions. Note that these constructors don't require construction of the parent object first. This is easier to achieve if the fraction element type doesn't contain a reference to the

parent object, but merely contains a reference to the base ring. The parent object can then be constructed on demand.

```
//(x::T, y::T) where T <: RingElem
```

Return the fraction $x/y$.

```
//(x::T, y::FracElem{T}) where T <: RingElem
```

Return $x/y$ where $x$ is in the base ring of $y$.

```
//(x::FracElem{T}, y::T) where T <: RingElem
```

Return $x/y$ where $y$ is in the base ring of $x$.

### Basic manipulation of fields and elements

```
numerator(d::MyFrac{T}) where T <: RingElem
```

Given a fraction $d = a/b$ return $a$, where $a/b$ is in lowest terms with respect to the `canonical_unit` and `gcd` functions on the base ring.

```
denominator(d::MyFrac{T}) where T <: RingElem
```

Given a fraction $d = a/b$ return $b$, where $a/b$ is in lowest terms with respect to the `canonical_unit` and `gcd` functions on the base ring.

# Chapter 67

# Module Interface

**Note**

The module infrastructure in AbstractAlgebra should be considered experimental at this stage. This means that the interface may change in the future.

AbstractAlgebra allows the construction of finitely presented modules (i.e. with finitely many generators and relations), starting from free modules. The generic code provided by AbstractAlgebra will only work for modules over euclidean domains, however there is nothing preventing a library from implementing more general modules using the same interface.

All finitely presented module types in AbstractAlgebra follow the following interface which is a loose interface of functions, without much generic infrastructure built on top.

Free modules can be built over both commutative and noncommutative rings. Other types of module are restricted to fields and euclidean rings.

## 67.1 Abstract types

AbstractAlgebra provides two abstract types for finitely presented modules and their elements:

- `FPModule{T}` is the abstract type for finitely presented module parent

types

- `FPModuleElem{T}` is the abstract type for finitely presented module

element types

Note that the abstract types are parameterised. The type T should usually be the type of elements of the ring the module is over.

## 67.2 Required functionality for modules

We suppose that R is a fictitious base ring and that S is a module over R with parent object S of type `MyModule{T}`. We also assume the elements in the module have type `MyModuleElem{T}`, where T is the type of elements of the ring the module is over.

Of course, in practice these types may not be parameterised, but we use parameterised types here to make the interface clearer.

Note that the type T must (transitively) belong to the abstract type `RingElement` or `NCRingElem`.

We describe the functionality below for modules over commutative rings, i.e. with element type belonging to `RingElement`, however similar constructors should be available for element types belonging to `NCRingElem` instead, for free modules over a noncommutative ring.

Although not part of the module interface, implementations of modules that wish to follow our interface should use the same function names for submodules, quotient modules, direct sums and module homomorphisms if they wish to remain compatible with our module generics in the future.

### Basic manipulation

```
iszero(m::MyModuleElem{T}) where T <: RingElement
```

Return `true` if the given module element is zero.

```
number_of_generators(M::MyModule{T}) where T <: RingElement
```

Return the number of generators of the module $M$ in its current representation.

```
gen(M::MyModule{T}, i::Int) where T <: RingElement
```

Return the $i$-th generator (indexed from $1$) of the module $M$.

```
gens(M::MyModule{T}) where T <: RingElement
```

Return a Julia array of the generators of the module $M$.

```
relations(M::MyModule{T}) where T <: RingElement
```

Return a Julia vector of all the relations between the generators of `M`. Each relation is given as an AbstractAlgebra row matrix.

### Element constructors

We can construct elements of a module $M$ by specifying linear combinations of the generators of $M$. This is done by passing a vector of ring elements.

```
(M::Module{T})(v::Vector{T}) where T <: RingElement
```

Construct the element of the module $M$ corresponding to $\sum_i g[i]v[i]$ where $g[i]$ are the generators of the module $M$. The resulting element will lie in the module $M$.

### Coercions

Given a module $M$ and an element $n$ of a module $N$, it is possible to coerce $n$ into $M$ using the notation $M(n)$ in certain circumstances.

In particular the element $n$ will be automatically coerced along any canonical injection of a submodule map and along any canonical projection of a quotient map. There must be a path from $N$ to $M$ along such maps.

### Arithmetic operators

Elements of a module can be added, subtracted or multiplied by an element of the ring the module is defined over and compared for equality.

In the case of a noncommutative ring, both left and right scalar multiplication are defined.

# Chapter 68

# Ideal Interface

AbstractAlgebra.jl generic code makes use of a standardised set of functions which it expects to be implemented by anyone implementing ideals for commutative rings. Here we document this interface. All libraries which want to make use of the generic capabilities of AbstractAlgebra.jl must supply all of the required functionality for their ideals. There are already many helper methods in AbstractAlgebra.jl for the methods mentioned below.

In addition to the required functions, there are also optional functions which can be provided for certain types of ideals e.g., for ideals of polynomial rings. If implemented, these allow the generic code to provide additional functionality for those ideals, or in some cases, to select more efficient algorithms.

## 68.1 Types and parents

Below we describe this interface for a fictitious type `NewIdeal` representing ideals over a base ring of type `NewRing`, with element type `NewRingElem`. To make use of the functionality described on this page, `NewIdeal` must be a subtype of `Ideal{NewRingElem}`. To inform the system about this relationship, it is necessary to provide the following method:

```
ideal_type(::Type{NewRing}) = NewIdeal
```

julia The system automatically provides the following reverse method:

```
base_ring_type(::Type{NewIdeal}) = NewRing
```

For ideals of a Euclidean domain, it may also be possibly to opt into using the existing functionality which is implemented in `src/generic/Ideal.jl`. In that case you would essentially defined `const NewIdeal = Generic.Ideal{NewRingElem}`. For more information about implementing new rings, see the Ring interface.

## 68.2 Required functionality for ideals

In the following, we list all the functions that are required to be provided for ideals in AbstractAlgebra.jl or by external libraries wanting to use AbstractAlgebra.jl.

To facilitate construction of new ideals, implementations must provide a method with signature

```
ideal(R::NewRing, xs::Vector{NewRingElem})
```

Here `xs` is a list of generators, and `NewRingElem === elem_type(NewRing)` holds.

With this in place, the following additional ideal constructors will automatically work via generic implementations:

```
ideal(R::NewRing, x::RingElement...) = ideal(R, [x...])
ideal(x::RingElement, y::RingElement...) = ideal(parent(x), x, y...)
ideal(xs::Vector{NewRingElem}) = ideal(parent(xs[1]), xs)
*(x::NewRingElem, R::NewRing) = ideal(R, x)
*(R::NewRing, x::NewRingElem) = ideal(R, x)
```

In addition sums and products of ideals can be formed:

```
+(I::T, J::T) where {T <: NewIdeal}
*(I::T, J::T) where {T <: NewIdeal}
```

An implementation of an `Ideal` subtype must also provide the following methods:

```
base_ring(I::NewIdeal)
gen(I::NewIdeal, k::Int)
gens(I::NewIdeal)
ngens(I::NewIdeal)
```

## 68.3 Optional functionality for ideals

Some functionality is difficult or impossible to implement for all ideals. If it is provided, additional functionality or performance may become available. Here is a list of all functions that are considered optional and can't be relied on by generic functions in the AbstractAlgebra Ideal interface.

It may be that no algorithm, or no efficient algorithm is known to implement these functions. As these functions are optional, they do not need to exist. Julia will already inform the user that the function has not been implemented if it is called but doesn't exist.

The following method have no generic implementation and only work when explicitly implemented.

```
in(v::NewRingElem, I::NewIdeal)
intersect(I::T, J::T) where {T <: NewIdeal}
```

If a method for `in` as above is provided, then the following automatically works:

```
issubset(I::NewIdeal, J::NewIdeal)
```

If a method for `in` as above is provided (e.g. indirectly by providing method for `in`), then the following automatically works:

```
==(I::T, J::T) where {T <: NewIdeal}
```

Note that implementing `==` for a Julia type means that we have to provide a matching `hash` method which preserves the invariant that `I == J` implies `hash(I) == hash(J)`. We provide such a method but by necessity

it is very conservative and hence does not provide good hashing. You may wish to implement a better hash methods.

The following method is implemented generically via the ideal generators.

```
iszero(I::Ideal) = all(iszero, gens(I))
```

# Chapter 69

# Matrix interface

This page describes the interface that a matrix implementation must satisfy in order to use the generic matrix functionality provided by AbstractAlgebra.jl.

It is intended for developers implementing new matrix types. For details on the generic dense matrix implementation provided by AbstractAlgebra itself, see the matrix implementation documentation.

AbstractAlgebra supports two kinds of matrix objects:

- matrices in matrix spaces,
- matrices in matrix algebras.

Matrices in a matrix space form an additive group but are generally not themselves a ring. Matrix algebras consist of square matrices and implement the noncommutative ring interface.

All matrix objects are assumed to be mutable. This is important for performance and for compatibility with in-place algorithms.

**Note**

AbstractAlgebra matrices are not Julia `Matrix` objects. They store additional algebraic information, such as the base ring, and support arithmetic over arbitrary rings.

## 69.1 Types and parents

Matrix space implementations use the following abstract types:

- `MatSpace{T}`: parent type for matrix spaces,
- `MatElem{T}`: abstract type for elements of matrix spaces.

Matrix algebra implementations use:

- `MatRing{T}`: abstract parent type for matrix algebras,
- `MatRingElem{T}`: abstract type for elements of matrix algebras.

The parameter T is the type of the entries of the matrix.

Matrix spaces and matrix algebras should be uniquely determined by their defining data, in particular their base ring and dimensions.

## 69.2 Required functionality

Suppose that:

- R is the base ring,
- S is a matrix space or matrix algebra over R,
- `MyMat{T}` is the matrix element type,
- `MyMatParent{T}` the parent type of the matrix space or matrix algebra.

The entry type T must belong to the `RingElem` hierarchy.

### Required constructors

Implementations must provide constructors from Julia arrays of entries:

```
(P::MyMatParent{T})(A::Matrix{U}) where {T <: NCRingElem, U <: NCRingElem}
```

Implementations must provide constructors from Julia arrays of entries:

```
(P::MyMatParent{T})(A::Matrix{U}) where {T <: NCRingElem, U <: NCRingElem}
```

and constructors allowing coercion into the base ring:

```
(P::MyMatParent{T})(A::Matrix{U}) where {T <: NCRingElem, U <: NCRingElem}
```

as well as construction from flat vectors:

```
(P::MyMatParent{T})(A::Vector{U}) where {T <: NCRingElem, U <: NCRingElem}
```

### Dimensions and entry access

Matrix implementations must provide:

```
number_of_rows(M::MyMat)
number_of_columns(M::MyMat)
```

returning the dimensions of the matrix.

Entry access and mutation are provided through:

```
getindex(M::MyMat, i::Int, j::Int)
setindex!(M::MyMat, x, i::Int, j::Int)
```

Matrices are mutable, and `setindex!` modifies the given matrix in place.

### Transpose

Implementations must provide:

```
transpose(M::MyMat)
```

returning the transpose of the matrix.

### Views

Matrix types must support views of submatrices. A view behaves like a matrix, but modifying entries of the view modifies the original matrix.

The following behaviour is required:

- deepcopy of a view returns a view into a copy of the original matrix;
- `similar` applied to a view returns an ordinary matrix, not another view.

## 69.3 Dense matrix type selection

Implementations can specify the preferred dense matrix representation for a given coefficient type by defining:

```
dense_matrix_type(::Type{T}) where T <: NCRingElem
```

The remaining methods dispatch to this one:

```
dense_matrix_type(::T) where T <: NCRingElem
dense_matrix_type(::Type{S}) where S <: NCRing
dense_matrix_type(::S) where S <: NCRing
```

For example, another package may specify that matrices over a particular ring should use a specialised representation instead of the generic dense matrix implementation.

## 69.4 Optional functionality

The generic matrix code provides fallback implementations for many operations. Custom matrix implementations may override these methods for improved performance.

### Submatrices

```
Base.getindex(M::MyMat, rows::AbstractVector{Int}, cols::AbstractVector{Int})
```

Return the corresponding submatrix.

### Row operations

```
swap_rows!(M::MyMat, i::Int, j::Int)
```

Swap two rows of `M` in place.

### Concatenation

```
hcat(M::MyMat, N::MyMat)
vcat(M::MyMat, N::MyMat)
```

Return horizontal and vertical concatenations.

### Zero tests

```
is_zero_entry(M::MyMat, i::Int, j::Int)
is_zero_row(M::MyMat, i::Int)
is_zero_column(M::MyMat, j::Int)
```

### Similar and zero matrices

Implementations may specialise:

```
similar(M::MyMat, R::Ring, r::Int, c::Int)
zero(M::MyMat, R::Ring, r::Int, c::Int)
```

The function `similar` constructs a matrix of the requested type, base ring and dimensions, but does not initialise its entries. It is unrelated to similarity transformations of matrices.

The function `zero` constructs the corresponding zero matrix, i.e. a matrix whose entries are all initialised to zero.

### Assigned entries

```
Base.isassigned(M::MyMat, i, j)
```

Tests whether an entry exists at the given position.

### Symmetry

```
is_symmetric(M::MyMat)
```

Return whether the matrix is symmetric.

# Chapter 70

# Map Interface

Maps in AbstractAlgebra can be constructed from Julia functions, or they can be represented by some other kind of data, e.g. a matrix, or built up from other maps.

In the following, we will always use the word "function" to mean a Julia function, and reserve the word "map" for a map on sets, whether mathematically, or as an object in the system.

## 70.1 Parent objects

Maps in AbstractAlgebra currently don't have parents. This will change later when AbstractAlgebra has a category system, so that the parent of a map can be some sort of Hom set.

## 70.2 Map classes

All maps in AbstractAlgebra belong to a class of maps. The classes are modeled as abstract types that lie in a hierarchy, inheriting from `SetMap` at the top of the hierarchy. Other classes that inherit from `SetMap` are `FunctionalMap` for maps that are constructed from a Julia function (or closure), and `IdentityMap` for the class of the identity maps within the system.

One might naturally assume that map types belong directly to these classes in the way that types of other objects in the system belong to abstract types in the AbstractAlgebra type hierarchy. However, in order to provide an extensible system, this is not the case.

Instead, a map type `MyMap` will belong to an abstract type of the form `Map{D, C, T, MyMap}`, where `D` is the type of the object representing the domain of the map type (this can also be an abstract type, such as `Group`), `C` is the type of the object representing the codomain of the map type and `T` is the map class that `MyMap` belongs to, e.g. `SetMap` or `FunctionalMap`.

Because a four parameter type system becomes quite cumbersome to use, we provide a number of functions for referring to collections of map types.

If writing a function that accepts any map type, one makes the type of its argument belong to `Map`. For example `f(M::Map) = 1`.

If writing a function that accepts any map from a domain of type `D` to a codomain of type `C`, one makes writes for example `f(M::Map{D, C}) = 2`. Note that `D` and `C` can be abstract types, such as `Group`, but otherwise must be the types of the parent objects representing the domain and codomain.

A function that accepts any map belonging to a given map class might be written as `f(M::Map(FunctionalMap)) = 3` or `f(M::Map(FunctionalMap){D, C}) = 4` for example, where `D` and `C` are the types of the parent objects for the domain and codomain.

Finally, if a function should only work for a map of a given map type MyMap, say, one writes this f(M::Map(MyMap)) or f(M::Map(MyMap){D, C}, where as usual D and C are the types of the domain and codomain parent objects.

## 70.3 Implementing new map types

There are two common kinds of map type that developers will need to write. The first has a fixed domain and codomain, and the second is a type parameterised by the types of the domain and codomain. We give two simple examples here of how this might look.

In the case of fixed domain and codomain, e.g. Integers{BigInt}, we would write it as follows:

```
mutable struct MyMap <: Map{Integers{BigInt}, Integers{BigInt}, SetMap, MyMap}
   # some data fields
end
```

In the case of parameterisation by the type of the domain and codomain:

```
mutable struct MyMap{D, C} <: Map{D, C, SetMap, MyMap}
   # some data fields
end
```

As mentioned above, to write a function that only accepts maps of type MyMap, one writes the functions as follows:

```
function my_fun(M::Map(MyMap))
```

The Map function then computes the correct type to use, which is actually not MyMap if all features of the generic Map infrastructure are required. It is bad practice to write functions for MyMap directly instead of Map(MyMap), since other users will be unable to use generic constructions over the map type MyMap.

## 70.4 Required functionality for maps

All map types must implement a standard interface, which we specify here.

We will define this interface for a custom map type MyMap belonging to Map(SetMap), SetMap being the map class that all maps types belong to.

Note that map types do not need to contain any specific fields, but must provide accessor functions (getters and setters) in the manner described above.

The required accessors for map types of class SetMap are as follows.

AbstractAlgebra.domain – Function.

```
domain(f::Map)
```

Return the parent object f maps from.

source

AbstractAlgebra.codomain – Function.

```
codomain(f::Map)
```

Return the parent object `f` maps to.

source

It is only necessary to define these functions if the map type `MyMap` does not contain fields `domain` and `codomain` containing these parent objects.

It is also necessary to be able to apply a map. This amounts to overloading the call method for objects belonging to `Map(MyMap)`.

```
(M::Map(MyMap)(a))
```

Apply the map `M` to the element `a` of the domain of `M`. Note that it is usual to add a type assertion to the return value of this function, asserting that the return value has type `elem_type(C)` where `C` is the type of the codomain parent object.

## 70.5 Optional functionality for maps

The Generic module in AbstractAlgebra automatically provides certain functionality for map types, assuming that they satisfy the full interface described above.

However, certain map types or map classes might like to provide their own implementation of this functionality, overriding the generic functionality.

We describe this optional functionality here.

### Show method

Custom map types may like to provide a custom `show` method if the default of displaying the domain and codomain of the map is not sufficient.

```
show(io::IO, M::Map(MyMap))
```

### Identity maps

There is a concrete map type `Generic.IdentityMap{D}` for the identity map on a given domain. Here `D` is the type of the object representing that domain.

`Generic.IdentityMap` belongs to the supertype `Map{D, C, AbstractAlgebra.IdentityMap, IdentityMap}`.

Note that the map class is also called `IdentityMap`. It is an abstract type, whereas `Generic.IdentityMap` is a concrete type in the Generic module.

An identity map has the property that when composed with any map whose domain or codomain is compatible, that map will be returned as the composition. Identity maps can therefore serve as a starting point when building up a composition of maps, starting an identity map.

We do not cached identity maps in the system, so that if more than one is created on the same domain, there will be more than one such map in the system. This underscores the fact that there is in general no way for

the system to know if two maps compose to give an identity map, and therefore the only two maps that can be composed to give an identity map are identity maps on the same domain.

To construct an identity map for a given domain, specified by a parent object R, say, we have the following function.

`AbstractAlgebra.identity_map` – Method.

```
identity_map(R::D) where D <: AbstractAlgebra.Set
```

Return an identity map on the domain $R$.

**Examples**

```
julia> R, t = ZZ[:t]
(Univariate polynomial ring in t over integers, t)

julia> f = identity_map(R)
Identity map
  of univariate polynomial ring in t over integers

julia> f(t)
t
```

source

Return an identity map on the domain $R$.

Of course there is nothing stopping a map type or class from implementing its own identity map type, and defining composition of maps of the same kind with such an identity map. In such a case, the class of such an identity map type must belong to `IdentityMap` so that composition with other map types still works.

### Composition of maps

Any two compatible maps in AbstractAlgebra can be composed and any composition can be applied.

In order to facilitate this, the Generic module provides a type `Generic.CompositeMap{D, C}`, which contains two maps `map1` and `map2`, corresponding to the two maps to be applied in a composition, in the order they should be applied.

To construct a composition map from two existing maps, we have the following function:

`AbstractAlgebra.compose` – Method.

```
compose(f::Map, g::Map)
```

Compose the two maps $f$ and $g$, i.e. return the map $h$ such that $h(x) = g(f(x))$.

**Examples**

```
julia> f = map_from_func(ZZ, ZZ, x -> x + 1);

julia> g = map_from_func(ZZ, QQ, x -> QQ(x));
```

```
julia> h = compose(f, g)
Composite map
  from integers
  to rationals
which is the composite of
  Map: integers -> integers
  Map: integers -> rationals
```

source

As a shortcut for this function we have the following operator:

```
*(f::Map{D, U}, g::Map{U, C}) where {D, U, C} = compose(f, g)
```

Note the order of composition. If we have maps $f : X \to Y$, $g : Y \to Z$ the correct order of the maps in this operator is `f*g`, so that `(f*g)(x) = g(f(x))`.

This is chosen so that for left $R$-module morphisms represented by a matrix, the order of matrix multiplication will match the order of composition of the corresponding morphisms.

Of course, a custom map type or class of maps can implement its own composition type and compose function.

This is the case with the `FunctionalMap` class for example, which caches the Julia function/closure corresponding to the composition of two functional maps. As this cached function needs to be stored inside the composition, a special type is necessary for the composition of two functional maps.

By default, `compose` will check that the two maps are composable, i.e. the codomain of the first map matches the domain of the second map. This is implemented by the following function:

`AbstractAlgebra.check_composable` – Function.

```
check_composable(f::Map, g::Map)
```

Raise an exception unless the codomain of `f` equals the domain of `g`, i.e. unless the composition `f*g` is defined.

source

Note that composite maps should keep track of the two maps they were constructed from. To access these maps, the following functions are provided:

`AbstractAlgebra.Generic.map1` – Function.

```
map1(f::Map)
```

Return the first of the two maps a composite map was built from, i.e. the one applied first. See also `map2`.

source

`AbstractAlgebra.Generic.map2` – Function.

```
map2(f::Map)
```

Return the second of the two maps a composite map was built from, i.e. the one applied last. See also `map1`.

source

Any custom composite map type must also provide these functions for that map type, even if there exist fields with those names. This is because there is no common map class for all composite map types. Therefore the Generic system cannot provide fallbacks for all such composite map types.

# Chapter 71

# Random interface

AbstractAlgebra makes use of the Julia Random interface for random generation.

In addition we make use of an experimental package RandomExtensions.jl for extending the random interface in Julia.

The latter is required because some of our types require more than one argument to specify how to randomise them.

The usual way of generating random values that Julia and these extensions provide would look as follows:

```
julia> using AbstractAlgebra

julia> using Random

julia> using RandomExtensions

julia> S, x = polynomial_ring(ZZ, :x)
(Univariate Polynomial Ring in x over Integers, x)

julia> rand(Random.default_rng(), make(S, 1:3, -10:10))
-5*x + 4
```

This example generates a polynomial over the integers with degree in the range 1 to 3 and with coefficients in the range -10 to 10.

In addition we implement shortened versions for ease of use which don't require creating a make instance or passing in the standard RNG.

```
julia> using AbstractAlgebra

julia> S, x = polynomial_ring(ZZ, :x)
(Univariate Polynomial Ring in x over Integers, x)

julia> rand(S, 1:3, -10:10)
-5*x + 4
```

Because rings can be constructed over other rings in a tower, all of this is supported by defining `RandomExtensions.make` instances that break the various levels of the ring down into separate make instances.

For example, here is the implementation of `make` for polynomial rings such as the above:

```
function RandomExtensions.make(S::PolyRing, deg_range::AbstractUnitRange{Int}, vs...)
   R = base_ring(S)
   if length(vs) == 1 && elem_type(R) == Random.gentype(vs[1])
      Make(S, deg_range, vs[1]) # forward to default Make constructor
   else
      Make(S, deg_range, make(R, vs...))
   end
end
```

As you can see, it has two cases. The first is where this invocation of make is already at the bottom of the tower of rings, in which case it just forwards to the default Make constructor.

The second case expects that we are higher up in the tower of rings and that make needs to be broken up (recursively) into the part that deals with the ring level we are at and the level that deals with the base ring.

To help make we tell it the type of object we are hoping to randomly generate.

```
RandomExtensions.maketype(S::PolyRing, dr::AbstractUnitRange{Int}, _) = elem_type(S)
```

Finally we implement the actual random generation itself.

```
# define rand for make(S, deg_range, v)
function rand(rng::AbstractRNG, sp::SamplerTrivial{<:Make3{<:RingElement, <:PolyRing,
↪  <:AbstractUnitRange{Int}}})
   S, deg_range, v = sp[][1:end]
   R = base_ring(S)
   f = S()
   x = gen(S)
   # degree -1 is zero polynomial
   deg = rand(rng, deg_range)
   if deg == -1
      return f
   end
   for i = 0:deg - 1
      f += rand(rng, v)*x^i
   end
   # ensure leading coefficient is nonzero
   c = R()
   while iszero(c)
      c = rand(rng, v)
   end
   f += c*x^deg
   return f
end
```

Note that when generating random elements of the base ring for example, one should use the random number generator rng that is passed in.

As mentioned above, we define a simplified random generator that saves the user having to create make instances.

```
rand(rng::AbstractRNG, S::PolyRing, deg_range::AbstractUnitRange{Int}, v...) =
   rand(rng, make(S, deg_range, v...))

rand(S::PolyRing, degs::Union{Int, AbstractUnitRange{Int}}, v...) =
   rand(Random.default_rng(), S, degs, v...)
```

To test whether a random generator is working properly, the `test_rand` function exists in the AbstractAlgebra test submodule in the file `test/runtests.jl`. For example, in AbstractAlgebra test code:

```
using Test

R, x = polynomial_ring(ZZ, :x)

test_rand(R, -1:10, -10:10)
```

In general, we try to use `UnitRange`'s to specify how 'big' we want the random instance to be, e.g. the range of degrees a polynomial could take, the range random integers could lie in, etc. The objective is to make it easy for the user to control the 'size' of random values in test code.

Give those parameters concrete types, as in the `rand` method above. `Random` and `RandomExtensions` both provide methods of the form `rand(X, args...)` with `X` untyped – `rand(X, ::Type{<:Array}, ::Integer...)`, `rand(X, ::Integer, ::Integer...)` and so on – so an untyped parameter makes the two sets of methods ambiguous for calls such as `rand(R, 3)`.

# Chapter 72

# Linear solving interface

## 72.1 Overview of the functionality

The module `AbstractAlgebra.Solve` provides the following four functions for solving linear systems:

- `solve`
- `can_solve`
- `can_solve_with_solution`
- `can_solve_with_solution_and_kernel`

All of these take the same set of arguments, namely:

- a matrix $A$ of type `MatElem`;
- a vector or matrix $B$ of type `Vector` or `MatElem`;
- a keyword argument `side` which can be either `:left` (default) or `:right`.

If `side` is `:left`, the system $xA = B$ is solved, otherwise the system $Ax = B$ is solved.

The functionality of the functions can be summarized as follows.

- `solve`: return a solution, if it exists, otherwise throw an error.
- `can_solve`: return `true`, if a solution exists, `false` otherwise.
- `can_solve_with_solution`: return `true` and a solution, if this exists, and `false` and an empty vector or matrix otherwise.
- `can_solve_with_solution_and_kernel`: like `can_solve_with_solution` and additionally return a matrix whose rows (respectively columns) give a basis of the kernel of $A$.

Furthermore, there is a function `kernel` which computes the kernel of a matrix $A$.

## 72.2 Solving with several right hand sides

Systems $xA = b_1, \ldots, xA = b_k$ with the same matrix $A$, but several right hand sides $b_i$ can be solved more efficiently, by first initializing a "context object" `C`.

`AbstractAlgebra.Solve.solve_init` – Method.

```
solve_init(A::MatElem)
```

Return a context object `C` that allows to efficiently solve linear systems $Ax = b$ or $xA = b$ for different $b$.

**Examples**

```
julia> A = QQ[1 2 3; 0 3 0; 5 0 0];

julia> C = solve_init(A)
Linear solving context of matrix
  [1//1   2//1   3//1]
  [0//1   3//1   0//1]
  [5//1   0//1   0//1]

julia> solve(C, [QQ(1), QQ(1), QQ(1)]; side = :left)
3-element Vector{Rational{BigInt}}:
 1//3
 1//9
 2//15

julia> solve(C, [QQ(1), QQ(1), QQ(1)]; side = :right)
3-element Vector{Rational{BigInt}}:
 1//5
 1//3
 2//45
```

source

Now the functions `solve`, `can_solve`, etc. can be used with `C` in place of $A$. This way the time-consuming part of the solving (i.e. computing a reduced form of $A$) is only done once and the result cached in `C` to be reused.

## 72.3 Detailed documentation

`AbstractAlgebra.Solve.solve` – Method.

```
solve(A::MatElem{T}, b::Vector{T}; side::Symbol = :left) where T
solve(A::MatElem{T}, b::MatElem{T}; side::Symbol = :left) where T
solve(C::SolveCtx{T}, b::Vector{T}; side::Symbol = :left) where T
solve(C::SolveCtx{T}, b::MatElem{T}; side::Symbol = :left) where T
```

Return $x$ of same type as $b$ solving the linear system $xA = b$, if `side == :left` (default), or $Ax = b$, if `side == :right`.

If no solution exists, an error is raised.

If a context object C is supplied, then the above applies for `A = matrix(C)`.

See also `can_solve_with_solution`.

**Examples**

```
julia> A = QQ[2 0 0;0 3 0;0 0 5]
[2//1   0//1   0//1]
[0//1   3//1   0//1]
[0//1   0//1   5//1]

julia> solve(A, one(A))
[1//2   0//1   0//1]
[0//1   1//3   0//1]
[0//1   0//1   1//5]
```

source

`AbstractAlgebra.Solve.can_solve` – Method.

```
can_solve(A::MatElem{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve(A::MatElem{T}, b::MatElem{T}; side::Symbol = :left) where T
can_solve(C::SolveCtx{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve(C::SolveCtx{T}, b::MatElem{T}; side::Symbol = :left) where T
```

Return `true` if the linear system $xA = b$ or $Ax = b$ with `side == :left` (default) or `side == :right`, respectively, has a solution and `false` otherwise.

If a context object C is supplied, then the above applies for `A = matrix(C)`.

See also `can_solve_with_solution`.

**Examples**

```
julia> A = QQ[2 0 0;0 3 0;0 0 5]
[2//1   0//1   0//1]
[0//1   3//1   0//1]
[0//1   0//1   5//1]

julia> can_solve(A,one(A))
true

julia> A = ZZ[2 0 0;0 3 0;0 0 5]
[2   0   0]
[0   3   0]
[0   0   5]

julia> can_solve(A,one(A))
false
```

source

`AbstractAlgebra.can_solve_with_solution` – Method.

```
can_solve_with_solution(A::MatElem{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve_with_solution(A::MatElem{T}, b::MatElem{T}; side::Symbol = :left) where T
can_solve_with_solution(C::SolveCtx{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve_with_solution(C::SolveCtx{T}, b::MatElem{T}; side::Symbol = :left) where T
```

Return `true` and $x$ of same type as $b$ solving the linear system $xA = b$, if such a solution exists. Return `false` and an empty vector or matrix, if the system has no solution.

If `side == :right`, the system $Ax = b$ is solved.

If a context object `C` is supplied, then the above applies for `A = matrix(C)`.

See also `solve`.

source

`AbstractAlgebra.kernel` – Method.

```
kernel(A::MatElem; side::Symbol = :left)
kernel(C::SolveCtx; side::Symbol = :left)
```

Return a matrix $K$ whose rows generate the left kernel of $A$, that is, $KA$ is the zero matrix.

If `side == :right`, the columns of $K$ generate the right kernel of $A$, that is, $AK$ is the zero matrix.

If the base ring is a principal ideal domain, the rows or columns respectively of $K$ are a basis of the respective kernel.

If a context object `C` is supplied, then the above applies for `A = matrix(C)`.

#Example

```
julia> A = QQ[2 6 0 0;1 3 0 0;0 0 5 0];

julia> kernel(A, side=:right)
[-3//1   0//1]
[ 1//1   0//1]
[ 0//1   0//1]
[ 0//1   1//1]

julia> kernel(A)
[-1//2   1//1   0//1]
```

source

`AbstractAlgebra.Solve.can_solve_with_solution_and_kernel` – Method.

```
can_solve_with_solution_and_kernel(A::MatElem{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve_with_solution_and_kernel(A::MatElem{T}, b::MatElem{T}; side::Symbol = :left) where T
can_solve_with_solution_and_kernel(C::SolveCtx{T}, b::Vector{T}; side::Symbol = :left) where T
can_solve_with_solution_and_kernel(C::SolveCtx{T}, b::MatElem{T}; side::Symbol = :left) where T
```

Return `true`, $x$ of same type as $b$ solving the linear system $xA = b$, together with a matrix $K$ giving the kernel of $A$ (i.e. $KA = 0$), if such a solution exists. Return `false`, an empty vector or matrix and an empty matrix, if the system has no solution.

If `side == :right`, the system $Ax = b$ is solved.

If a context object `C` is supplied, then the above applies for `A = matrix(C)`.

See also `solve` and `kernel`.

source